# Deep Reinforcement Learning

슬라이드별 학습 가이드

순차 의사결정부터 오프라인 강화학습까지

2026년 2학기

# Contents


**1 01-1. 강화학습이 필요한 이유** **11**

1.1 슬라이드 01 · Deep Reinforcement Learning . . . 11
1.2 슬라이드 02 · 학습의 전제 . . . 11
1.3 슬라이드 03 · 학습 경로 . . . 11
1.4 슬라이드 04 · 강화학습의 첫 질문 . . . 11
1.5 슬라이드 05 · 상황을 행동에 대응시킨다는 뜻 . . . 12
1.6 슬라이드 06 · 에이전트와 환경의 닫힌 고리 . . . 12
1.7 슬라이드 07 · 문제와 해결 방법을 구분하기 . . . 12
1.8 슬라이드 08 · 학습의 초점 . . . 13
1.9 슬라이드 09 · 여러 주제의 연결 . . . 13
1.10 슬라이드 10 · 주제별 공부 순서 . . . 13
1.11 슬라이드 11 · 필요한 수학과 구현 도구 . . . 14
1.12 슬라이드 12 · 평가 요소와 학습 전략 . . . 14
1.13 슬라이드 13 · 과제가 따라가는 알고리즘 . . . 14
1.14 슬라이드 14 · 실험을 일찍 시작해야 하는 이유 . . . 14
1.15 슬라이드 15 · 시험에서 설명해야 할 것 . . . 15
1.16 슬라이드 16 · 출석과 실습의 시간 관리 . . . 15
1.17 슬라이드 17 · 토론으로 점검할 질문 . . . 15
1.18 슬라이드 18 · 정보와 참고 자료를 찾는 순서 . . . 16
1.19 슬라이드 19 · 세 가지 기본 참고 축 . . . 16
1.20 슬라이드 20 · 응용 사례를 읽는 기준 . . . 16
1.21 슬라이드 21 · AGI라는 질문의 범위 . . . 16
1.22 슬라이드 22 · SVM에서 AlphaGo까지 . . . 17
1.23 슬라이드 23 · 생성형 AI의 서로 다른 학습 신호 . . . 17
1.24 슬라이드 24 · 지도학습이 충분하지 않을 때 . . . 17
1.25 슬라이드 25 · 회로 설계: PrefixRL . . . 18
1.26 슬라이드 26 · 언어모델 정렬에서의 RLHF . . . 18
1.27 슬라이드 27 · 확산 모델에서의 보상 최적화 . . . 19
1.28 슬라이드 28 · 코드 에이전트의 순차 결정 . . . 19
1.29 슬라이드 29 · 다족 로봇의 보행 . . . 19
1.30 슬라이드 30 · 로봇 조작과 접촉 . . . 20
1.31 슬라이드 31 · 왜 강화학습을 배우는가 . . . 20
1.32 슬라이드 32 · 다음 단계: 모방학습 . . . 20

**2 01-2. 모방학습과 로봇 정책** **21**

2.1 슬라이드 01 · Imitation Learning . . . 21

2.2 슬라이드 02 · 강화학습과 모방학습의 위치 . . . . . 21
2.3 슬라이드 03 · 에이전트–환경 고리 다시 보기 . . . . . 21
2.4 슬라이드 04 · 상태, 행동, 정책 . . . . . 21
2.5 슬라이드 05 · 궤적과 전이의 표기 . . . . . 22
2.6 슬라이드 06 · 정답 행동과 보상 . . . . . 22
2.7 슬라이드 07 · 이 장의 질문 . . . . . 22
2.8 슬라이드 08 · Behavioral Cloning의 목적함수 . . . . . 23
2.9 슬라이드 09 · 시연의 품질과 범위 . . . . . 23
2.10 슬라이드 10 · 시연을 수집하는 방식 . . . . . 23
2.11 슬라이드 11 · 실패 원인으로 넘어가기 . . . . . 24
2.12 슬라이드 12 · 실패를 분해하는 틀 . . . . . 24
2.13 슬라이드 13 · 첫째 실패: 오류의 누적 . . . . . 24
2.14 슬라이드 14 · 오류 누적을 줄이는 두 경로 . . . . . 25
2.15 슬라이드 15 · 둘째 실패: 여러 정답 행동 . . . . . 25
2.16 슬라이드 16 · 셋째 실패: 전문가와 에이전트의 관측 차이 . . . . . 26
2.17 슬라이드 17 · 세 실패의 진단 순서 . . . . . 26
2.18 슬라이드 18 · 온라인 교정으로 넘어가기 . . . . . 26
2.19 슬라이드 19 · DAgger의 네 단계 . . . . . 26
2.20 슬라이드 20 · 사람이 개입하는 DAgger . . . . . 27
2.21 슬라이드 21 · 로봇 사례를 읽는 틀 . . . . . 28
2.22 슬라이드 22 · ALOHA와 ACT의 결합 . . . . . 28
2.23 슬라이드 23 · 저비용 하드웨어가 바꾸는 것 . . . . . 28
2.24 슬라이드 24 · ACT가 마주한 두 문제 . . . . . 28
2.25 슬라이드 25 · 행동을 묶어 예측하는 이유 . . . . . 29
2.26 슬라이드 26 · 절대 관절 목표와 잠재변수 . . . . . 29
2.27 슬라이드 27 · ACT 구조를 입력과 출력으로 읽기 . . . . . 30
2.28 슬라이드 28 · 50개 시연으로 만든 조작 정책 . . . . . 30
2.29 슬라이드 29 · 배터리 삽입의 정밀도 . . . . . 30
2.30 슬라이드 30 · Mobile ALOHA로 확장하기 . . . . . 31
2.31 슬라이드 31 · 원격 조작 시연의 역할 . . . . . 31
2.32 슬라이드 32 · 자율 실행의 평가 . . . . . 31
2.33 슬라이드 33 · Diffusion Policy로 넘어가기 . . . . . 31
2.34 슬라이드 34 · 세 실패 원인을 다시 연결하기 . . . . . 32
2.35 슬라이드 35 · 노이즈에서 미래 행동열로 . . . . . 32
2.36 슬라이드 36 · 결과 영상의 해석 . . . . . 32
2.37 슬라이드 37 · 로봇 파운데이션 모델의 조건부 정책 . . . . . 33
2.38 슬라이드 38 · DROID와 환경 다양성 . . . . . 33

2.39 슬라이드 39 · Open X-Embodiment와 로봇 몸체의 차이 . . . . . 33
2.40 슬라이드 40 · $\pi_0$의 자율 수행 장면 . . . . . 34
2.41 슬라이드 41 · 사전학습과 과업별 후속 학습 . . . . . 34
2.42 슬라이드 42 · VLM과 행동 전문가의 연결 . . . . . 34
2.43 슬라이드 43 · 로봇 데이터와 텍스트 데이터의 규모 비교 . . . . . 35
2.44 슬라이드 44 · 데이터 수집 규모의 현실 . . . . . 35
2.45 슬라이드 45 · 가정 환경의 긴 작업 . . . . . 35
2.46 슬라이드 46 · ACT-1이라는 이름을 해석할 때 . . . . . 35
2.47 슬라이드 47 · 다양성을 시각화한 데이터 지도 . . . . . 36
2.48 슬라이드 48 · 새로운 로봇 플랫폼에서의 조작 . . . . . 36
2.49 슬라이드 49 · 모방학습의 도달 범위 . . . . . 36
2.50 슬라이드 50 · 보상으로 배우는 단계 . . . . . 37

**3 02-1. 강화학습의 수학적 기초 38**
3.1 슬라이드 01 · 강화학습의 수학적 기초 . . . . . 38
3.2 슬라이드 02 · 복습에서 형식화로 . . . . . 38
3.3 슬라이드 03 · 평가 요소와 개념 학습 . . . . . 38
3.4 슬라이드 04 · 시험에서 MDP를 설명하는 법 . . . . . 38
3.5 슬라이드 05 · 출석과 연속적인 학습 . . . . . 39
3.6 슬라이드 06 · 질문으로 확인할 수 있는 가정 . . . . . 39
3.7 슬라이드 07 · 자료를 사용할 때의 순서 . . . . . 39
3.8 슬라이드 08 · 교재와 현대 알고리즘 사이 . . . . . 39
3.9 슬라이드 09 · 모방학습을 되짚는 이유 . . . . . 40
3.10 슬라이드 10 · 결정 문제의 최소 구성 . . . . . 40
3.11 슬라이드 11 · 행동 복제의 기준선 . . . . . 40
3.12 슬라이드 12 · 모방학습의 세 실패 . . . . . 40
3.13 슬라이드 13 · DAgger의 상태 분포 수정 . . . . . 41
3.14 슬라이드 14 · 사람의 개입과 보상의 차이 . . . . . 41
3.15 슬라이드 15 · RL 문제로 넘어가는 연결 . . . . . 41
3.16 슬라이드 16 · 누적 보상의 의미 . . . . . 41
3.17 슬라이드 17 · RL 문제를 확률로 쓰기 . . . . . 42
3.18 슬라이드 18 · MDP가 환경과 과업을 함께 담는 이유 . . . . . 42
3.19 슬라이드 19 · 확률 과정의 기초 . . . . . 42
3.20 슬라이드 20 · Markov 성질의 정확한 뜻 . . . . . 43
3.21 슬라이드 21 · Markov chain의 두 요소 . . . . . 43
3.22 슬라이드 22 · 공부와 과제의 Markov chain . . . . . 43
3.23 슬라이드 23 · Markov Reward Process . . . . . 44

3.24 슬라이드 24 · 즉시 비용과 최종 보상 . . . 44
3.25 슬라이드 25 · 선택을 더한 MDP . . . 44
3.26 슬라이드 26 · 공부 예시에 행동을 넣기 . . . 45
3.27 슬라이드 27 · 환경을 정의하는 식과 학습 목표 . . . 45
3.28 슬라이드 28 · 정책이 정의하는 행동 . . . 45
3.29 슬라이드 29 · 초기 상태 분포까지 포함하기 . . . 46
3.30 슬라이드 30 · 시작 상태가 여러 개일 때 . . . 46
3.31 슬라이드 31 · 상태와 행동은 과업마다 다르다 . . . 46
3.32 슬라이드 32 · 환경을 완전히 정의해도 모를 수 있다 . . . 46
3.33 슬라이드 33 · 실제 던지기의 MDP . . . 47
3.34 슬라이드 34 · 환경 API와 수학 기호의 대응 . . . 47
3.35 슬라이드 35 · 완전 관측의 한계 . . . 47
3.36 슬라이드 36 · Ant 환경을 MDP로 쓰기 . . . 48
3.37 슬라이드 37 · 게임에서의 상태와 보상 . . . 48
3.38 슬라이드 38 · 실제 로봇에서의 상태와 보상 . . . 48
3.39 슬라이드 39 · 대화의 상태와 보상 . . . 49
3.40 슬라이드 40 · 용어를 한 궤적에 모으기 . . . 49
3.41 슬라이드 41 · 정책경사로의 연결 . . . 49

**4 02-2. 정책경사와 REINFORCE 50**

4.1 슬라이드 01 · 정책경사의 질문 . . . 50
4.2 슬라이드 02 · 반환과 기울기 . . . 50
4.3 슬라이드 03 · 한 궤적의 기호 . . . 50
4.4 슬라이드 04 · MDP와 정책의 역할 . . . 50
4.5 슬라이드 05 · 확률적 정책의 의미 . . . 51
4.6 슬라이드 06 · 지도학습과 다른 손실 . . . 51
4.7 슬라이드 07 · 정책 손실을 바로 쓸 수 없는 이유 . . . 51
4.8 슬라이드 08 · 궤적 확률의 곱 . . . 51
4.9 슬라이드 09 · 로그미분 항등식 . . . 52
4.10 슬라이드 10 · 환경 전이 항이 사라지는 이유 . . . 52
4.11 슬라이드 11 · Monte Carlo 정책경사 . . . 52
4.12 슬라이드 12 · REINFORCE의 반복 . . . 53
4.13 슬라이드 13 · 유도의 네 단계 복기 . . . 53
4.14 슬라이드 14 · 장점과 두 한계 . . . 53
4.15 슬라이드 15 · On-policy 데이터 조건 . . . 54
4.16 슬라이드 16 · 표본 기울기의 분산 . . . 54
4.17 슬라이드 17 · 분산을 줄이는 두 방향 . . . 54

4.18 슬라이드 18 · 표본 수만 늘리기 어려운 이유 . . . . . 55
4.19 슬라이드 19 · 더 나은 가중치의 계보 . . . . . 55
4.20 슬라이드 20 · Actor–Critic으로의 연결 . . . . . 55

**5 03-1. 가치함수와 Actor–Critic 56**

5.1 슬라이드 01 · Actor–Critic의 출발점 . . . . . 56
5.2 슬라이드 02 · 분산 감소의 세 단계 . . . . . 56
5.3 슬라이드 03 · 로그미분 유도 다시 보기 . . . . . 56
5.4 슬라이드 04 · 정책 로그확률만 남기기 . . . . . 56
5.5 슬라이드 05 · 표본 정책경사의 구조 . . . . . 57
5.6 슬라이드 06 · On-policy와 분산의 문제 . . . . . 57
5.7 슬라이드 07 · 가중치를 단계적으로 바꾸기 . . . . . 57
5.8 슬라이드 08 · 먼저 필요한 점수 함수의 성질 . . . . . 57
5.9 슬라이드 09 · 점수 함수의 기대값이 0인 이유 . . . . . 58
5.10 슬라이드 10 · 과거 보상은 현재 행동의 결과가 아니다 . . . . . 58
5.11 슬라이드 11 · 전체 반환을 시점별로 나누기 . . . . . 58
5.12 슬라이드 12 · 과거 보상 항의 기대값 . . . . . 59
5.13 슬라이드 13 · Reward-to-go가 남긴 것 . . . . . 59
5.14 슬라이드 14 · 인과성의 직관 . . . . . 59
5.15 슬라이드 15 · 가치함수가 필요한 순간 . . . . . 59
5.16 슬라이드 16 · $V^\pi$와 $Q^\pi$의 차이 . . . . . 60
5.17 슬라이드 17 · 가치와 advantage를 한 표기로 . . . . . 60
5.18 슬라이드 18 · 상태가치의 학습 목표 . . . . . 60
5.19 슬라이드 19 · 할인율의 두 역할 . . . . . 61
5.20 슬라이드 20 · Bellman 식에 할인율 넣기 . . . . . 61
5.21 슬라이드 21 · Reward-to-go를 더 안정적으로 추정하기 . . . . . 61
5.22 슬라이드 22 · $Q$가 행동의 공로를 평가한다 . . . . . 61
5.23 슬라이드 23 · 진짜 $Q$를 얻는 두 방법 . . . . . 62
5.24 슬라이드 24 · 상태 baseline을 생각하기 . . . . . 62
5.25 슬라이드 25 · Baseline을 빼도 기대값이 같은 증명 . . . . . 62
5.26 슬라이드 26 · 기대값 보존과 분산 감소는 별개 . . . . . 62
5.27 슬라이드 27 · 분산을 실제로 비교하는 식 . . . . . 63
5.28 슬라이드 28 · 왜 $V^\pi(s)$를 선택하는가 . . . . . 63
5.29 슬라이드 29 · 평균보다 좋은 행동 강화하기 . . . . . 63
5.30 슬라이드 30 · Advantage와 Actor–Critic . . . . . 63
5.31 슬라이드 31 · 네 가지 가중치의 비교 . . . . . 64
5.32 슬라이드 32 · REINFORCE의 데이터 요구 . . . . . 64

5.33 슬라이드 33 · Actor와 critic의 반복 . . . 64
5.34 슬라이드 34 · 표본 효율의 정확한 비교 . . . 64
5.35 슬라이드 35 · 다음 단계: 길이와 재사용 . . . 65

**6 03-2. GAE와 PPO 66**

6.1 슬라이드 01 · PPO가 해결하려는 문제 . . . 66
6.2 슬라이드 02 · 두 개선 방향 . . . 66
6.3 슬라이드 03 · 정책경사 가중치의 계보 . . . 66
6.4 슬라이드 04 · REINFORCE가 기다리는 것 . . . 66
6.5 슬라이드 05 · Actor–Critic의 한 단계 . . . 67
6.6 슬라이드 06 · 자주 갱신할 수 있다는 뜻 . . . 67
6.7 슬라이드 07 · 남은 두 약점 . . . 67
6.8 슬라이드 08 · $n$단계 반환 . . . 67
6.9 슬라이드 09 · GAE의 가중 평균과 재귀 . . . 68
6.10 슬라이드 10 · Advantage 추정과 데이터 재사용은 별개 . . . 69
6.11 슬라이드 11 · 왜 현재 정책 데이터가 필요한가 . . . 69
6.12 슬라이드 12 · 중요도 샘플링의 기본식 . . . 69
6.13 슬라이드 13 · 궤적 비율에서 행동 비율로 . . . 69
6.14 슬라이드 14 · 여러 갱신에서 이전 정책 고정하기 . . . 70
6.15 슬라이드 15 · 큰 중요도 비율과 정책 변화 . . . 70
6.16 슬라이드 16 · PPO를 구성하는 네 조각 . . . 70
6.17 슬라이드 17 · Advantage 부호에 따른 clipping 직관 . . . 70
6.18 슬라이드 18 · 하나의 PPO-Clip 목적함수로 합치기 . . . 71
6.19 슬라이드 19 · PPO 학습 반복의 순서 . . . 71
6.20 슬라이드 20 · PPO가 널리 쓰이는 이유와 한계 . . . 72
6.21 슬라이드 21 · 접촉이 많은 로봇 조립 . . . 72
6.22 슬라이드 22 · SAPG와 병렬 학습 . . . 72
6.23 슬라이드 23 · 사족 보행의 실제 로봇 시연 . . . 72
6.24 슬라이드 24 · 보행을 넘어서는 전신 제어 . . . 73
6.25 슬라이드 25 · 조작 정책 시연을 읽는 법 . . . 73
6.26 슬라이드 26 · 사람 피드백과 언어모델 . . . 73
6.27 슬라이드 27 · 왜 다음은 off-policy인가 . . . 73

**7 04-1 Q-learning과 DQN 75**

7.1 슬라이드 01 · Q-learning의 출발 . . . 75
7.2 슬라이드 02 · 지금까지의 정책경사 . . . 75
7.3 슬라이드 03 · 정책 반복에서 가치 반복으로 . . . 75
7.4 슬라이드 04 · $V$와 $Q$의 관계 . . . 75

7.5 슬라이드 05 · 모델을 요구하는 fitted value iteration . . . 75
7.6 슬라이드 06 · 최적 Bellman 방정식 . . . 76
7.7 슬라이드 07 · Fitted Q-iteration . . . 76
7.8 슬라이드 08 · 세 가지 반복을 구분하기 . . . 77
7.9 슬라이드 09 · Q-learning의 장점과 위험 . . . 77
7.10 슬라이드 10 · Deep Q-Network . . . 77
7.11 슬라이드 11 · DQN을 지탱하는 세 장치 . . . 77
7.12 슬라이드 12 · 연속 transition의 상관 . . . 78
7.13 슬라이드 13 · Replay buffer의 역할 . . . 78
7.14 슬라이드 14 · 움직이는 목표 문제 . . . 78
7.15 슬라이드 15 · 자기 자신을 쫓는 회귀 . . . 78
7.16 슬라이드 16 · Target network . . . 78
7.17 슬라이드 17 · 탐색이 남는다 . . . 79
7.18 슬라이드 18 · $\epsilon$-greedy의 확률 . . . 79
7.19 슬라이드 19 · Frame skip과 frame stack . . . 79
7.20 슬라이드 20 · 시각 자료: Atari 화면 . . . 79
7.21 슬라이드 21 · DQN 구조를 다시 묶기 . . . 80
7.22 슬라이드 22 · DQN을 개선하는 질문 . . . 80
7.23 슬라이드 23 · 네 가지 개선 축 . . . 80
7.24 슬라이드 24 · 큰 Q와 큰 실제 반환은 같은가 . . . 80
7.25 슬라이드 25 · 과대평가가 드러나는 경우 . . . 80
7.26 슬라이드 26 · 최대값 편향의 간단한 유도 . . . 81
7.27 슬라이드 27 · Double DQN의 분리 . . . 81
7.28 슬라이드 28 · 다음 개선: 목표의 지평 . . . 81
7.29 슬라이드 29 · $n$단계 Q 목표 . . . 81
7.30 슬라이드 30 · $n$단계의 off-policy 주의점 . . . 82
7.31 슬라이드 31 · 자료를 뽑는 방법도 중요하다 . . . 82
7.32 슬라이드 32 · Prioritized experience replay . . . 82
7.33 슬라이드 33 · 평균 가치만으로 충분한가 . . . 83
7.34 슬라이드 34 · 반환 분포와 평균의 차이 . . . 83
7.35 슬라이드 35 · 불확실성의 출처 . . . 83
7.36 슬라이드 36 · 분포적 Bellman 관계 . . . 83
7.37 슬라이드 37 · C51의 고정 지지점 . . . 84
7.38 슬라이드 38 · C51 목표 분포의 투영 . . . 84
7.39 슬라이드 39 · 분포적 학습이 바꾸는 것 . . . 84
7.40 슬라이드 40 · 개선들의 상호작용 . . . 85
7.41 슬라이드 41 · Rainbow . . . 85

7.42 슬라이드 42 · 연속 행동으로 넘어가기 . . . . . 85

**8 05-1 Off-policy Actor–Critic 86**

8.1 슬라이드 01 · 연속 행동을 향하여 . . . . . 86
8.2 슬라이드 02 · 다룰 알고리즘의 관계 . . . . . 86
8.3 슬라이드 03 · 정책 개선의 두 경로 . . . . . 86
8.4 슬라이드 04 · Bellman 식의 재확인 . . . . . 86
8.5 슬라이드 05 · DQN의 안전장치 . . . . . 86
8.6 슬라이드 06 · DQN 한 사이클 . . . . . 87
8.7 슬라이드 07 · DQN 개선의 역할 . . . . . 87
8.8 슬라이드 08 · 최댓값은 오차를 고른다 . . . . . 87
8.9 슬라이드 09 · Double DQN의 기준 . . . . . 87
8.10 슬라이드 10 · 개선책의 한계도 이어진다 . . . . . 88
8.11 슬라이드 11 · Q-learning의 정책은 암묵적이다 . . . . . 88
8.12 슬라이드 12 · DDPG로 넘어가는 단계 . . . . . 88
8.13 슬라이드 13 · 연속 행동에서 전수 비교가 막힌다 . . . . . 88
8.14 슬라이드 14 · 결정적 actor로 최대화를 근사 . . . . . 88
8.15 슬라이드 15 · 결정적 정책경사의 연쇄법칙 . . . . . 88
8.16 슬라이드 16 · 두 target 망의 소프트 갱신 . . . . . 89
8.17 슬라이드 17 · DDPG의 전체 업데이트 . . . . . 89
8.18 슬라이드 18 · DQN에서 DDPG로 바뀐 자리 . . . . . 89
8.19 슬라이드 19 · DDPG의 실용적 한계 . . . . . 90
8.20 슬라이드 20 · TD3가 해결할 세 문제 . . . . . 90
8.21 슬라이드 21 · TD3의 세 안정화 장치 . . . . . 90
8.22 슬라이드 22 · 두 critic이 주는 보수성 . . . . . 90
8.23 슬라이드 23 · TD3 학습 흐름 . . . . . 91
8.24 슬라이드 24 · 성능 그래프를 해석할 때 . . . . . 91
8.25 슬라이드 25 · TD3 요약 . . . . . 91
8.26 슬라이드 26 · 확률적 actor라는 다른 선택 . . . . . 91
8.27 슬라이드 27 · 평균과 분산을 모두 배우기 . . . . . 91
8.28 슬라이드 28 · 최대 엔트로피 목적 . . . . . 92
8.29 슬라이드 29 · 무작위성의 효용과 비용 . . . . . 92
8.30 슬라이드 30 · Soft Bellman 식 . . . . . 93
8.31 슬라이드 31 · SAC의 critic과 actor 목표 . . . . . 93
8.32 슬라이드 32 · 온도 $\alpha$를 자동 조절 . . . . . 93
8.33 슬라이드 33 · SAC와 TD3의 공통 기반 . . . . . 94
8.34 슬라이드 34 · SAC가 유용한 조건 . . . . . 94

8.35 슬라이드 35 · TD3와 SAC 실험 비교 . . . 94
8.36 슬라이드 36 · 지금까지의 학습 지형 . . . 94
8.37 슬라이드 37 · 벤치마크로 이어지는 질문 . . . 94

**9 05-2 강화학습 벤치마크 96**
9.1 슬라이드 01 · 벤치마크를 읽는 관점 . . . 96
9.2 슬라이드 02 · 최종 성능과 표본 효율 . . . 96
9.3 슬라이드 03 · 벤치마크의 지도 . . . 96
9.4 슬라이드 04 · Gymnasium MuJoCo와 DMC . . . 97
9.5 슬라이드 05 · Atari의 관측과 예산 . . . 97
9.6 슬라이드 06 · 로봇 벤치마크의 폭 . . . 97
9.7 슬라이드 07 · LIBERO와 평생학습 . . . 97
9.8 슬라이드 08 · RoboTwin 2.0의 양팔 과제 . . . 98
9.9 슬라이드 09 · SIMPLER: 현실 정책을 시뮬레이션에서 평가 . . . 98
9.10 슬라이드 10 · RoboArena의 분산형 실기 평가 . . . 98
9.11 슬라이드 11 · FurnitureBench의 재현 가능한 실물 조립 . . . 98
9.12 슬라이드 12 · HumanoidBench의 전신 제어 . . . 99
9.13 슬라이드 13 · 과제 그림에서 평가 설계로 . . . 99

**10 06 오프라인 강화학습 100**
10.1 슬라이드 01 · 데이터가 고정되면 달라지는 문제 . . . 100
10.2 슬라이드 02 · 이 장의 방법 지도 . . . 100
10.3 슬라이드 03 · Offline과 online의 자료 흐름 . . . 100
10.4 슬라이드 04 · 행동 정책과 목표 정책 . . . 100
10.5 슬라이드 05 · BC와 궤적 연결 . . . 100
10.6 슬라이드 06 · 보지 못한 행동의 함정 . . . 101
10.7 슬라이드 07 · 언제 필요한가 . . . 101
10.8 슬라이드 08 · 보통 off-policy Q-learning을 그대로 쓰면 . . . 101
10.9 슬라이드 09 · 예측 Q와 실제 반환의 분리 . . . 101
10.10슬라이드 10 · Offline RL의 핵심 긴장 . . . 102
10.11슬라이드 11 · 두 가지 주요 통제 위치 . . . 102
10.12슬라이드 12 · TD3+BC의 목적 . . . 102
10.13슬라이드 13 · 정책 제약과 가치 보수화 . . . 103
10.14슬라이드 14 · 데이터 밖 Q를 낮추는 이유 . . . 103
10.15슬라이드 15 · CQL의 첫 목적식 . . . 103
10.16슬라이드 16 · 데이터 행동과의 상대적 보수성 . . . 103
10.17슬라이드 17 · Log-sum-exp가 나오는 이유 . . . 104
10.18슬라이드 18 · CQL 학습과 조절 계수 . . . 104

10.19슬라이드 19 · Offline RL의 평가는 왜 까다로운가 . . . 105
10.20슬라이드 20 · D4RL 데이터셋 . . . 105
10.21슬라이드 21 · 데이터의 질과 다양성 . . . 105
10.22슬라이드 22 · OGBench와 목표 조건부 학습 . . . 105
10.23슬라이드 23 · TD3+BC와 CQL을 비교하기 . . . 106
10.24슬라이드 24 · 다른 개선 경로의 질문 . . . 106
10.25슬라이드 25 · OOD 평가가 남는 자리 . . . 106
10.26슬라이드 26 · 보상을 쓰는 행동복제 . . . 106
10.27슬라이드 27 · Filtered BC . . . 107
10.28슬라이드 28 · AWR의 연속적 가중치 . . . 107
10.29슬라이드 29 · KL 제약에서 지수 가중치가 나오는 직관 . . . 107
10.30슬라이드 30 · AWR의 장점과 경계 . . . 108
10.31슬라이드 31 · SARSA식 데이터 안 부트스트랩 . . . 108
10.32슬라이드 32 · 데이터 행동 중 좋은 쪽을 고르기 . . . 108
10.33슬라이드 33 · Expectile 가치의 의미 . . . 109
10.34슬라이드 34 · 비대칭 제곱손실 예 . . . 109
10.35슬라이드 35 · IQL의 세 단계 . . . 109
10.36슬라이드 36 · 방법들의 공통 기준 . . . 110
10.37슬라이드 37 · 고정 자료 학습의 응용 . . . 110
10.38슬라이드 38 · 로봇에서 offline 사전학습과 online 미세조정 . . . 110
10.39슬라이드 39 · 핵융합 플라스마 제어 . . . 111
10.40슬라이드 40 · 건물 냉각 제어 . . . 111
10.41슬라이드 41 · 칩 회로 설계와 PrefixRL . . . 111
10.42슬라이드 42 · 전체 정리 . . . 111

**11 과제 가이드** **113**

11.1 과제 1 · 모방학습 . . . 113
11.2 과제 2 · 정책경사와 PPO . . . 116

# 1 01-1. 강화학습이 필요한 이유

강화학습의 출발점은 **행동이 다음 상황을 바꾼다**는 사실이다. 지금 유리해 보이는 행동이 나중에도 좋으리라는 보장은 없다. 이 장에서는 상태와 행동, 결과와 보상을 구분하고, 정답 예측만으로는 순차적 의사결정을 모두 설명할 수 없는 이유를 여러 응용에서 확인한다.

## 1.1 슬라이드 01 · Deep Reinforcement Learning

Deep Reinforcement Learning은 상태나 관측을 입력받아 행동을 고르는 **정책(policy)**을 신경망으로 표현하고, 행동 뒤에 생긴 결과를 이용해 그 정책을 개선한다. 로봇이 물체를 집을 때 현재 영상을 이해하는 것만으로는 부족하다. 팔을 어디로 움직이면 다음 영상과 물체의 위치가 어떻게 바뀔지, 몇 단계 뒤에 집기에 성공할지까지 고려해야 한다.

앞으로 등장하는 알고리즘은 무엇을 상태로 볼지, 가능한 행동은 무엇인지, 결과를 어떤 보상으로 측정할지, 아직 해 보지 않은 행동의 가치를 어떻게 추정할지라는 공통 질문에 답한다.

같은 영상을 입력받는 모델이라도 출력이 물체의 이름이면 인식 문제이고, 다음 로봇 동작이면 제어 문제다. 제어에서는 현재 출력이 다음 입력 영상을 바꾸므로 훈련 자료의 분포까지 정책에 따라 달라진다. 이 차이가 이후에 나오는 시연 데이터의 한계와 환경 상호작용의 필요성을 연결한다.

## 1.2 슬라이드 02 · 학습의 전제

개념과 구현이 빠르게 이어지는 과목에서는 수식의 정의와 실험의 조건을 함께 이해해야 한다. 강화학습은 같은 알고리즘이라도 환경, 난수 시드, 표본 수, 보상 척도에 따라 학습 곡선이 달라질 수 있다. 한 번 실행해서 점수가 나왔다고 알고리즘을 이해했다고 볼 수 없는 이유다.

구현을 읽을 때 정책이 행동을 출력하고, 환경이 전이를 반환하며, 버퍼가 데이터를 저장하고, 목적함수로 파라미터를 갱신하는 흐름을 추적하자. 이후 과제는 이 고리의 서로 다른 부분을 직접 만드는 연습이 된다.

실험 노트에는 반환의 평균만 쓰지 말고 환경 단계 수, 평가 episode 수, 시드, 학습 중 탐색 행동을 썼는지까지 적자. 예를 들어 두 정책의 점수가 같아도 하나는 환경을 1만 번, 다른 하나는 100만 번 호출했다면 **sample efficiency**가 다르다. 여러 시드에서 한쪽만 가끔 성공한다면 평균 수치만으로 안정성을 판단하기도 어렵다.

## 1.3 슬라이드 03 · 학습 경로

먼저 문제의 범위와 필요한 도구를 확인하고, 다음에는 지도학습이 이미 강력한데도 왜 강화학습을 배우는지 따진다. 이 질문을 놓치면 정책경사나 Q-learning이 서로 연결되지 않은 최적화 기법 목록처럼 보이기 쉽다.

판별 기준은 **한 번의 예측으로 끝나는가, 그 예측이 다음 입력을 바꾸는가**다. 사진 분류는 대체로 전자에 가깝다. 로봇 제어·게임·대화 에이전트는 한 행동이 뒤의 관측과 선택지를 바꾸므로 후자에 가깝다.

자율주행의 차선 변경을 생각해 보자. 주변 차량 위치를 인식하는 단계는 정적 예측에 가깝지만, 실제로 차선을 바꾸면 다른 차량도 속도를 바꾸고 다음 관측이 달라진다. 따라서 좋은 인식기가 곧 좋은 운전 정책은 아니다. 관측의 정확도와 행동의 장기 결과를 따로 평가해야 한다.

## 1.4 슬라이드 04 · 강화학습의 첫 질문

에이전트는 매 시점 환경을 관측하고 행동을 정한다. 환경은 다음 상태와 보상을 돌려준다. 간단히 $s_t \xrightarrow{a_t} (r_t, s_{t+1})$라고 쓴다. $t$는 시간, $s_t$는 의사결정에 쓰는 상태, $a_t$는 행동, $r_t$는 행동 뒤에 관측한

평가 신호다.

지금 보상을 크게 받는 것과 장기적으로 좋은 결과를 얻는 것은 다르다. 보상이 작아도 다음 상태가 유리해지면 좋은 행동일 수 있다. 뒤에서 정의할 반환 $G_t$는 여러 시점의 보상을 하나의 평가로 묶는다.

**보상이 직접 주어지지 않으면 강화학습 문제를 만들 수 없을까?** 수치 보상이 바로 없어도 성공·실패, 비용, 제약 위반, 사람의 선호 비교 같은 평가에서 보상을 설계하거나 추정할 수 있다. 그 값이 실제 목표를 제대로 나타내는지는 별도로 검증해야 한다.

두 단계짜리 예를 들면 A는 지금 5점을 받고 다음에 0점, B는 지금 0점을 받고 다음에 10점을 받는다. 미래 보상의 할인율이 $\gamma = 0.9$이면 반환은 각각 $G(A) = 5, G(B) = 0 + 0.9 \cdot 10 = 9$다. 즉각적 보상만 보는 기준과 장기 반환을 보는 기준은 B에 대해 정반대의 결론을 내릴 수 있다.

**미래 보상을 왜 할인할까?** 먼 미래 결과의 불확실성이나 빨리 얻는 보상의 선호를 표현할 수 있고, 무한한 시간의 보상 합이 수렴하도록 만드는 역할도 한다. 다만 할인율은 과업의 시간 규모를 바꾼다. 가까운 보상만 중요하게 만드는 작은 값이 언제나 좋은 설정은 아니다.

## 1.5 슬라이드 05 · 상황을 행동에 대응시킨다는 뜻

Sutton과 Barto의 정의에서 중요한 말은 "상황을 행동에 대응시킨다"와 "수치 보상을 최대화한다"이다. 대응 규칙이 정책 $\pi(a \mid s)$다. 결정론적 정책은 상태마다 한 행동을 내고, 확률적 정책은 각 행동의 확률을 지정한다.

매 시점에 **정답 행동**이 제공되는 것은 아니다. 결과가 좋았는지에 대한 신호만 주어질 수 있다. 멀리 떨어진 성공을 앞선 행동의 공로로 돌리는 신용 할당 문제가 생기는 이유다. 바둑의 최종 승패로 초반 수를 평가해야 하는 경우가 대표적이다.

같은 "왼쪽으로 이동"도 장애물이 왼편에 있을 때와 목표가 왼편에 있을 때 성과가 다르다. 확률적 정책의 무작위 선택과 행동 뒤에 생기는 환경의 무작위 결과는 이후의 기대값에서 별도로 다룬다.

**보상이 하나인데 행동의 정답을 바로 알 수 없는 이유는?** 결과 하나는 여러 행동과 환경의 우연성이 함께 만든다. 성공한 궤적의 모든 행동이 최선이었다고 할 수도 없고, 실패한 궤적의 모든 행동이 잘못되었다고 할 수도 없다. 각각의 행동이 결과를 얼마나 바꾸었는지 추정하는 과정이 필요하다.

## 1.6 슬라이드 06 · 에이전트와 환경의 닫힌 고리

상태 $s_t$를 받은 에이전트가 $a_t$를 내면 환경은 전이 확률 $P(s_{t+1} \mid s_t, a_t)$에 따라 움직이고 보상 $r_t$를 준다. 정책과 환경이 맞물리면 하나의 **trajectory** $\tau = (s_0, a_0, r_0, s_1, \ldots)$가 만들어진다. 같은 행동도 환경의 우연성 때문에 늘 같은 결과를 내지는 않는다.

로봇이 컵을 향해 팔을 뻗어도 마찰과 위치 오차가 다르면 컵이 잡히거나 미끄러진다. 학습 목표는 한 번의 성공 경로를 암기하는 일이 아니라 이런 변동을 고려한 기대 장기 성과를 높이는 것이다.

여기서 상태는 환경의 완전한 물리 상태와 같지 않을 수 있다. 카메라에 컵이 팔 뒤로 가려지면 로봇이 받는 영상은 일부 정보만 담는다. 같은 관측에서도 실제 컵의 위치가 다를 수 있으므로 과거 관측을 기억하거나 불확실성을 추정해야 한다. 뒤에서 MDP의 상태와 실제 센서 관측을 구분하는 이유다.

## 1.7 슬라이드 07 · 문제와 해결 방법을 구분하기

**순차적 의사결정 문제**는 여러 번 행동하고 그때마다 새로운 정보를 받는 문제 자체다. **모방학습**은 전문가의 상태–행동 쌍에서 배우는 한 해결 방법이다. **강화학습**은 보상과 상호작용으로 행동을 개선하는 방법들의 묶음이다. 따라서 모든 순차 결정 문제를 반드시 RL로 풀 필요는 없다.

온라인 학습은 새 상호작용을 수집하며 갱신하고, 오프라인 학습은 이미 모아 둔 고정 데이터에서 학습한다. 모델 기반 방법은 환경 전이를 추정해 계획하고, 모델 프리 방법은 전이 모델을 명시적으로

만들지 않고 정책이나 가치를 학습한다. 같은 문제라도 데이터 접근성과 실험 비용에 따라 다른 선택이 가능하다.

로봇 팔의 충돌이 비싸다면 먼저 사람의 시연으로 BC 정책을 만들 수 있다. 안전한 시뮬레이터가 있다면 많은 시도를 통해 RL로 개선할 수 있다. 이미 수집된 작동 기록만 접근 가능하다면 offline RL이나 보수적인 모방이 출발점이다. **문제의 종류**와 **데이터를 얻는 방법**을 먼저 구분하면 알고리즘 이름을 외우지 않아도 선택의 이유가 보인다.

**off-policy와 offline은 같은 말일까?** 둘 다 현재 학습 정책과 다른 행동으로 모은 데이터를 활용할 수 있지만 동일하지 않다. off-policy 학습은 replay buffer에 새 상호작용을 계속 넣을 수 있다. offline 학습은 학습 자료가 고정되어, 추정이 불확실한 행동을 실행해 확인할 수 없다.

## 1.8 슬라이드 08 · 학습의 초점

이후의 흐름은 문제의 정의, 신경망을 쓰는 주요 알고리즘, 구현에서 생기는 오차, 로봇과 제어 응용에 집중한다. 이름을 암기하기보다 각 방법을 공통 기준으로 비교하면 새로운 논문도 읽기 쉬워진다.

정책을 직접 바꾸는지 가치를 거쳐 바꾸는지, 현재 정책에서 모은 데이터가 필요한지, 보상이 언제 알려지는지, 추정 오차가 어디서 증폭되는지, 새 상호작용이 얼마나 필요한지를 확인하자. 이 다섯 질문이 뒤 장들의 안내선이 된다.

예를 들어 PPO는 현재 정책의 rollout으로 얻은 advantage를 써 정책을 바꾸므로 새 데이터가 자주 필요하다. DQN은 과거 전이를 다시 사용하면서 Q값의 Bellman 목표를 맞추지만, 목표값 자체가 잘못 커질 수 있다. 두 방법을 비교할 때 "어느 쪽이 더 최신인가"보다 **데이터 사용 방식과 오차의 위치**를 설명하는 편이 정확하다.

## 1.9 슬라이드 09 · 여러 주제의 연결

모방학습은 좋은 행동의 예시가 있을 때 시작한다. 정책경사는 보상으로 정책을 직접 개선한다. Q-learning과 연속 행동 제어는 행동의 가치를 추정해 선택을 바꾼다. 오프라인 RL은 추가 실험 없이 고정된 데이터에서 보상을 활용해야 한다.

RLHF는 선호를 간접적 보상으로 바꾸는 사례이고, 계층적 RL은 긴 과제를 하위 목표로 나눈다. 서로 별개인 목록이라기보다 **어떤 피드백을 받을 수 있는가**와 **어느 데이터에서 학습하는가**에 따라 이어지는 방법들이다.

예를 들어 사람이 로봇의 정확한 관절 명령을 매 순간 쓰기는 어렵지만, 두 실행 영상 중 어느 쪽이 더 좋은지는 고를 수 있다. 이 경우 시연 행동을 직접 따라 하는 모방학습보다 선호에서 보상을 추정하는 방법이 맞을 수 있다. 반대로 시연이 풍부하고 성공 보상을 자동으로 얻기 어렵다면 BC가 더 간단한 출발점이다.

## 1.10 슬라이드 10 · 주제별 공부 순서

초반의 모방학습·정책경사·Q-learning은 뒤 알고리즘의 언어를 만든다. 모방학습은 데이터셋의 행동을 재현하고, 정책경사는 기대 보상을 미분하고, Q-learning은 Bellman 목표에 맞게 가치를 갱신한다. 모델 기반 RL, 보상 학습, 인간 피드백과 고급 로봇 제어는 이 축을 결합하거나 확장한다.

과제 이름이 알고리즘 이름과 같아도 구현 목표는 더 구체적이다. PPO 과제는 확률비 계산뿐 아니라 rollout, advantage 추정, 이전 정책의 로그확률 보존, 여러 번의 minibatch 갱신이 함께 맞아야 한다.

구현을 비교할 때에는 "학습 신호가 어디서 오는가?"를 먼저 확인하자. BC에는 전문가 행동 $a^*$, 정책경사에는 실행 궤적의 반환, Q-learning에는 관측 보상과 다음 상태 가치로 만든 Bellman 목표가 쓰인다.

## 1.11 슬라이드 11 · 필요한 수학과 구현 도구

확률과 기대값은 여러 행동·전이가 만드는 평균 성과를 정의한다. 미분은 정책 파라미터 $\theta$를 어느 방향으로 바꿀지 결정하고, 선형대수는 관측·행동·신경망 층을 벡터와 행렬로 다룬다. 역전파는 계산 그래프를 미분하지만 환경의 전이까지 자동으로 미분해 주지는 않는다. 정책경사의 로그미분은 바로 이 차이를 이용한다.

구현에서는 PyTorch 텐서의 모양과 gradient 흐름, 환경 API에서 초기화와 한 단계 진행이 반환하는 값, 난수 시드와 데이터 저장 위치를 확인한다. 같은 수식이어도 차원이나 종료 처리 한 줄이 틀리면 학습 곡선은 크게 달라질 수 있다.

특히 기대값 $E[X]$는 단순히 관측치 하나가 아니라 가능한 결과에 확률을 곱해 합한 값이다. 시뮬레이터에서는 모든 미래를 열거하기 어려워 여러 궤적의 평균으로 이를 근사한다. 따라서 수식의 기대값, 코드의 batch 평균, 그래프의 여러 시드 평균은 서로 다른 평균임을 구분해야 한다.

**자동미분이 있는데 왜 정책경사를 따로 유도할까?** 신경망의 출력까지는 자동미분이 가능해도, 환경이 뽑은 다음 상태와 보상은 대개 미분 가능한 계산 그래프로 연결되지 않는다. 로그미분 항등식은 환경의 내부 미분 없이도 표본 행동의 확률을 통해 기대 반환의 기울기를 추정하게 한다.

## 1.12 슬라이드 12 · 평가 요소와 학습 전략

구현 과제, 시험, 출석과 참여는 서로 다른 능력을 확인한다. 공식의 기호만 외우는 것으로는 코드를 실행할 수 없고, 코드가 돌아간다는 사실만으로 알고리즘의 가정이 맞는지도 알 수 없다. 수식은 결과보다 유도 단계를 재현하고, 실험은 조건과 로그를 남겨 비교 가능하게 만드는 것이 중요하다.

운영 세부 사항은 학기 중 조정될 수 있다. 실제 제출·응시 판단은 현재 공지를 기준으로 하되, 학습 계획은 개념 이해와 구현 검증을 함께 배치하자.

개념을 읽을 때는 기호마다 "무엇을 입력받고 무엇을 평균 내는가"를 적고, 구현을 읽을 때는 그 기호에 대응하는 배열과 축을 찾자. 가령 $Q(s,a)$를 배웠다면 실제 코드에서 상태 batch와 행동 batch가 같은 전이에서 왔는지 확인한다. 작은 일치 검사가 공식 암기보다 많은 오류를 막는다.

## 1.13 슬라이드 13 · 과제가 따라가는 알고리즘

과제 순서는 모방학습에서 PPO, DQN, SAC, 오프라인 RL, 모델 기반 RL과 RLHF로 이어진다. 같은 정책 인터페이스를 사용하더라도 학습 신호는 다르다. 모방학습은 전문가 행동의 손실, PPO는 표본 advantage로 가중한 정책 개선, DQN은 Bellman 오차를 사용한다.

제공된 첫 두 과제는 이 차이를 직접 비교하는 출발점이다. 첫 번째는 시연 데이터가 있을 때의 BC·DAgger, 두 번째는 환경 보상만 있을 때의 정책경사·baseline·GAE·PPO를 구현한다. 관련 슬라이드의 끝에서 과제 안내로 이동할 수 있다.

두 과제를 같은 질문으로 읽으면 연결이 선명해진다. 학습할 상태는 누가 방문했는가? 목표 행동이나 보상은 누가 제공했는가? 갱신 전후 정책을 어떻게 평가했는가? 특히 DAgger의 전문가 라벨은 **현재 정책이 방문한 상태**에 붙고, PPO의 advantage는 **현재 정책이 만든 rollout의 보상**에서 계산된다.

## 1.14 슬라이드 14 · 실험을 일찍 시작해야 하는 이유

신경망 학습 자체가 빨리 끝나도 강화학습 실험에는 환경 상호작용, rollout 평가, 여러 시드, 하이퍼파라미터 비교가 쌓인다. 실패한 실행에서 로그가 없으면 손실, 데이터 수집, 평가 설정 중 어디가 문제인지 구분하기 어렵다.

작은 표본으로 입출력 차원을 확인하고, 한 환경에서 짧게 실행한 뒤, 실제 행동을 시각화하고, 마지막에 긴 실험을 실행하는 순서가 유용하다. 결과 평균과 표준편차, 사용한 환경 단계 수를 함께 기록해야 비교가 가능하다.

예를 들어 초기 관측 10개를 정책에 넣어 행동의 범위와 NaN 여부를 먼저 확인할 수 있다. 짧은 rollout의 전이를 몇 개 출력하면 행동이 환경에 반영되는지, 보상과 종료 신호가 올바르게 저장되는지 알 수 있다. 여기서 틀린 코드를 긴 학습으로 돌리면 계산 자원만 낭비하고 원인은 더 찾기 어려워진다.

**학습 손실이 내려가는데 평가 반환이 그대로면 어디를 볼까?** 손실의 목표와 평가 지표가 다른지, 행동 범위의 clip이 심한지, 훈련에는 탐색 잡음을 쓰고 평가에는 제거하는지, 종료 처리와 보상 척도가 맞는지 확인한다. 손실 감소는 구현한 목적함수를 맞춘다는 뜻일 뿐 실제 과업 성공의 증거는 아니다.

## 1.15 슬라이드 15 · 시험에서 설명해야 할 것

확률적 정책 $\pi_\theta(a_t \mid s_t)$는 시점 $t$의 상태 $s_t$에서 행동 $a_t$를 고를 확률이며, $\theta$는 그 정책을 표현하는 신경망 파라미터다. 정책경사의 기본 성분은 선택된 행동의 로그확률을 $\theta$로 미분한 다음 벡터다.

$$\nabla_\theta \log \pi_\theta(a_t \mid s_t)$$

각 성분은 해당 파라미터가 조금 변할 때 그 행동의 로그확률이 어느 방향으로 변하는지 알려 준다. 환경 전이 확률 $P(s_{t+1} \mid s_t, a_t)$는 행동 뒤의 다음 상태 $s_{t+1}$을 정한다. 정책경사는 이 전이를 직접 미분하지 않고 표본 궤적의 보상으로 로그확률 기울기를 가중한다.

baseline과 Bellman 방정식도 최종식보다 각 항의 의미와 적용 조건을 설명할 수 있어야 한다. 구현 과제의 실패 원인을 말할 수 있으면 공식을 더 정확히 이해했는지 확인할 수 있다.

유도의 핵심은 $\nabla_\theta \pi_\theta(a \mid s) = \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s)$라는 로그미분 항등식이다. 뒤 장에서는 이 항등식을 궤적의 기대 반환에 적용하고, 얻은 기대값을 rollout 평균으로 근사한다.

## 1.16 슬라이드 16 · 출석과 실습의 시간 관리

실험에는 예상보다 긴 시간이 걸릴 수 있으므로 강의일에 질문할 코드 경로와 그래프를 준비하면 문제를 빨리 좁힐 수 있다. "학습이 안 된다"보다 "손실은 내려가는데 평가 반환이 그대로이고, 평가 행동이 모두 상한에 붙는다"라는 관찰이 훨씬 유용하다.

출석과 인정 절차는 현재 공지에 따르되, 학습 측면에서는 각 실험의 환경 버전·시드·실행 시각을 남겨 두는 습관이 중요하다. 이런 기록이 이후 연구 실험의 기본이 된다.

질문을 준비할 때에는 비교 가능한 두 관찰을 가져가자. 예를 들어 "같은 시드에서 행동 표준편차를 줄이자 학습 반환은 올랐지만 평가 반환은 떨어졌다"처럼 변경 전후를 말하면 탐색 부족인지 평가 코드 차이인지 논의할 수 있다. 그래프에는 반환뿐 아니라 환경 단계 수를 가로축으로 적는 편이 학습량을 비교하기 쉽다.

## 1.17 슬라이드 17 · 토론으로 점검할 질문

강화학습에서는 직관과 실제 업데이트가 자주 어긋난다. "baseline을 빼면 평균 정책경사가 바뀌지 않는가?", "PPO의 clipping이 정책 이동을 보장하는가?", "Q-learning에서 같은 네트워크를 목표에 쓰면 왜 불안정한가?"처럼 가정 하나를 확인하는 질문이 유익하다.

좋은 답은 이름 풀이보다 작은 계산이나 반례를 포함한다. 한 상태에서 행동 확률을 합하면 1이므로, 행동과 무관한 baseline을 곱한 로그확률 기울기의 기대값은 0이라는 식으로 검증할 수 있다.

예를 들어 행동이 두 개이고 $\pi_\theta(a_1 \mid s) = p$, $\pi_\theta(a_2 \mid s) = 1 - p$라면 $\sum_a \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s) = \nabla_\theta [p + (1 - p)] = 0$이다. 이 계산 하나로 왜 상태에만 의존하는 baseline은 기대 기울기를 바꾸지 않는지 확인할 수 있다. 행동마다 다른 값을 아무 조건 없이 빼면 같은 논리가 성립하지 않는다.

### 1.18 슬라이드 18 · 정보와 참고 자료를 찾는 순서

운영 공지, 과제 질의, 교재와 논문은 역할이 다르다. 일정·제출 형식은 현재 강좌 공지에서 확인하고, 정의와 표기는 교재로 정리하며, 특정 알고리즘의 가정과 실험은 원 논문으로 검증한다. 연구 코드가 있어도 환경 버전과 전처리가 다르면 결과가 달라질 수 있다.

로봇·언어모델 사례는 영상이나 데모만으로 학습 방법을 단정하기 어렵다. 어떤 데이터로 무엇을 최적화했는지 확인해야 "순차적으로 행동한다"와 "RL로 학습했다"를 구별할 수 있다.

논문이나 프로젝트 페이지를 볼 때는 환경과 데이터 수집 방법, 보상 정의, baseline, 평가 횟수를 먼저 찾자. 예를 들어 성공률 90%라고 해도 훈련 때 본 물체만 썼는지, 새로운 물체에서도 되는지에 따라 해석이 달라진다. 데모 영상은 어떤 행동이 가능한지 보여 주지만 실패율이나 평균 성능을 알려 주지 않는다.

### 1.19 슬라이드 19 · 세 가지 기본 참고 축

Sutton과 Barto의 *Reinforcement Learning: An Introduction*은 MDP, 가치함수, Bellman 방정식의 표준적인 언어를 제공한다. CS 224R과 CS 285 자료는 신경망 정책·가치함수 및 현대 알고리즘을 구현과 연결한다. 자료마다 정의가 달라 보이면 상태와 관측, 종료 처리, 할인 보상, on-policy 여부를 먼저 비교하자.

원 논문의 성능 수치는 대개 특정 과업과 예산에서 나온다. 숫자만 옮기지 말고 학습 단계 수, 평가 episode와 시드, 관측 전처리까지 함께 읽어야 한다.

교재는 정의를 확인할 때, 강의 자료는 알고리즘의 연결을 따라갈 때, 원 논문은 실제 가정과 실험 설정을 볼 때 각각 유용하다. 같은 "return"이라도 할인된 학습 목표인지, 할인하지 않은 평가 episode 합계인지 다를 수 있다. 문헌 사이에 숫자가 안 맞을 때는 먼저 용어와 지표의 정의를 맞춰야 한다.

### 1.20 슬라이드 20 · 응용 사례를 읽는 기준

다음의 AI 사례는 모두 신경망을 사용하지만 피드백이 다르다. SVM·AlexNet은 정답 라벨을, GAN은 판별기의 신호를, AlphaGo는 게임 결과와 탐색을 활용한다. 대화모델 정렬은 시연·선호 비교·보상 모델을 조합할 수 있다.

기술의 이름보다 **데이터, 행동, 피드백, 평가 기준**을 나눠 보자. 그래야 RL이 기여한 부분과 지도학습·탐색이 맡은 부분을 분리할 수 있다.

같은 바둑 문제를 예로 들면, 기보의 다음 수를 맞히는 학습은 지도학습이다. 완성된 대국의 승패를 보고 앞으로의 수 선택을 개선하면 RL이 들어간다. 현재 국면에서 후보 수를 가상으로 펼쳐 보는 과정은 탐색이다. 세 요소가 하나의 시스템에 함께 들어갈 수 있으므로, 최종 성능만 보고 어느 요소가 핵심이었다고 단정할 수 없다.

**판별기가 점수를 주는 GAN도 강화학습일까?** GAN의 판별기 출력은 생성자에 대한 학습 신호지만, 보통은 고정된 입력에서 표본을 만드는 생성 손실을 최적화한다. 환경에 행동하고 그 결과로 다음 상태와 선택지가 바뀌는 순차 결정 구조를 별도로 정의하지 않았다면 판별 점수만으로 RL이라 부르지 않는다.

### 1.21 슬라이드 21 · AGI라는 질문의 범위

AGI는 과업을 바꿔도 폭넓게 적응하는 능력을 가리키는 넓은 표현이다. 한 벤치마크의 높은 점수만으로 그런 능력을 증명하기 어렵다. 강화학습 관점에서는 새로운 목표와 환경에서 정보를 모으고 시행착오를 통해 장기 성과를 높일 수 있는지가 중요하다.

바둑을 잘 두는 정책이 로봇의 컵 잡기까지 할 수는 없다. 관측, 가능한 행동, 보상의 의미가 달라지기

때문이다. 일반성을 논할 때는 전이 가능한 표현, 데이터 효율, 안전한 탐색을 구체적으로 따져야 한다.

검증 질문도 구체화해야 한다. 훈련하지 않은 규칙이나 새로운 물체를 만났을 때, 설명만 듣고 행동을 바꿀 수 있는가? 첫 시도에 실패하면 다음 시도에 필요한 정보를 얻는가? 한 과업의 평균 점수보다 **새 과업에서 필요한 상호작용 횟수**와 실패 비용이 일반화 능력을 더 잘 드러낼 때가 있다.

**높은 벤치마크 점수가 일반 지능을 증명할까?** 정해진 과업에서는 강해도 목표와 관측이 달라지면 성능이 떨어질 수 있다. 학습하지 않은 과업에 얼마나 빨리 적응하는지와 실패 뒤에 어떻게 회복하는지도 함께 살펴야 한다.

## 1.22 슬라이드 22 · SVM에서 AlphaGo까지

SVM은 라벨로 분류 경계를 찾고 AlexNet은 깊은 신경망으로 이미지 분류 성능을 높였다. GAN은 생성자와 판별자의 경쟁을 통해 데이터와 비슷한 표본을 만든다. 이 흐름은 표현 학습의 발전을 보여 주지만, 환경 속 행동의 장기 결과를 중심에 놓는 것은 AlphaGo 사례다.

AlphaGo는 사람의 기보로 출발해 자기 대국을 통한 강화학습과 탐색을 결합했다. 정책망은 후보 수를 제안하고 가치망은 국면의 결과를 예측한다. 이세돌과의 대국은 사람의 수를 그대로 복제하는 것을 넘어서는 경로를 보여 준다. 다만 성능을 RL 하나만의 효과로 돌리면 탐색과 지도학습의 기여를 놓친다. AlphaGo의 방법

2016년 서울에서 열린 이세돌과의 대국 결과는 4승 1패였다. 이 국내 사례가 보여 주는 학습상의 핵심은 "한 수의 정답"과 "그 수 이후의 승률"이 서로 다른 목표라는 점이다. 초반 수는 바로 점수를 주지 않으므로 정책은 자기 대국의 최종 승패를 앞선 선택에 연결해야 한다. 동시에 실제 착수 전의 탐색이 성능을 높였다는 사실도 함께 읽어야 한다.

**정책망과 가치망이 모두 있으면 둘 다 행동을 고를까?** 정책망은 수의 후보와 확률을 제안한다. 가치망은 특정 국면이 얼마나 유리한지 평가한다. 탐색은 두 예측을 결합해 후보 수의 뒤를 살펴보고 실제 수를 결정한다. 따라서 모델의 출력과 최종 착수를 같은 것으로 보면 안 된다.

## 1.23 슬라이드 23 · 생성형 AI의 서로 다른 학습 신호

대화 모델, 코드 도우미, 이미지·영상 생성기는 출력을 만들어 내지만 그 학습이 전부 RL은 아니다. 대규모 사전학습은 주로 다음 토큰이나 노이즈 예측처럼 관측된 데이터를 맞히는 손실을 사용한다. 그 뒤 사람의 시연을 따라 배우거나 선호를 보상으로 바꾸어 출력을 조정할 수 있다.

어떤 답변이 더 유용한지 두 후보를 비교하는 선호 데이터는 한 개의 정답 문자열보다 풍부한 평가 기준을 담을 수 있다. 반면 그 비교로 배운 보상 모델은 편향될 수 있으므로 최종 사용자 평가와 따로 대조해야 한다.

예를 들어 코드 설명을 요청했을 때 사전학습은 다음에 올 법한 문장을 잘 예측하게 하고, 시연 학습은 좋은 설명의 형식을 따라 하게 한다. 두 답변 중 하나가 정확하면서도 질문에 더 맞는다는 비교는 선호 학습에 쓰일 수 있다. 이 세 학습 신호를 모두 "모델이 정답을 배웠다"로 묶으면 무엇을 최적화했는지 놓친다.

## 1.24 슬라이드 24 · 지도학습이 충분하지 않을 때

입력 $x$와 목표 $y$가 고정되어 있으면 지도학습은 $f_\theta(x)$와 $y$의 차이를 줄이는 간결한 방법이다. 순차적 행동에서는 예측한 $a_t$가 다음 입력 $s_{t+1}$을 바꾼다. 훈련 중에는 전문가가 방문한 상태만 보았는데 실제 정책은 자기 실수로 낯선 상태에 들어갈 수 있다.

또한 최종 목표가 "물체 집기 성공"이나 "대화가 도움이 되었는가"라면 각 프레임의 행동 오차나 다음 토큰 확률만으로 목표를 완전히 표현하기 어렵다. 환경의 결과를 학습 신호로 삼는 방법이 필요한

이유다. 다음 장의 모방학습은 이 문제를 구체적으로 드러낸다.

직관을 위한 단순 계산을 해 보자. 복제 정책이 전문가 경로의 각 단계에서 독립적으로 90% 확률로 올바르게 행동하고, 다섯 단계 모두 맞아야 성공한다고 가정하면 성공 확률은 $0.9^5 \approx 0.59$다. 실제로는 첫 실수 뒤에 낯선 상태가 생겨 이후 오류 확률이 더 커질 수도 있다. 숫자 자체는 장난감 가정의 결과지만, **한 단계의 높은 정확도와 전체 rollout의 성공률이 다르다**는 점을 보여 준다.

**정답 행동이 있는 경우에도 RL이 필요한가?** 항상 그렇지는 않다. 충분한 시연으로 목표를 달성하면 지도학습이 더 단순하다. RL은 행동의 장기 결과가 중요하거나 시연보다 나은 성과를 목표로 하거나 보상은 있지만 정답 행동을 쓰기 어려울 때 유용하다.

### 1.25 슬라이드 25 · 회로 설계: PrefixRL

병렬 prefix 회로는 덧셈기 같은 디지털 회로의 지연과 면적을 좌우한다. 연결을 한 번에 완성하기보다 부분 구조를 차례로 정하면 앞선 선택이 뒤의 가능한 배선을 제한한다. PrefixRL은 회로 합성 결과의 면적·지연을 평가 신호로 삼아 큰 설계 공간을 탐색한다.

상태는 현재까지 구성된 회로, 행동은 합법적인 다음 연결 선택, 보상은 설계 품질과 연관된다. 신경망이 좋은 회로의 정답 그림을 받는 것이 아니라 합성 도구로 확인한 결과를 통해 설계 전략을 개선한다. 원 논문

여기서 면적과 지연은 동시에 작을수록 좋지만 한 수치만 최소화하면 다른 수치가 악화될 수 있다. 예를 들어 회로 A가 더 빠르고 회로 B가 더 작다면 어느 쪽이 좋은지는 설계 제약에 달려 있다. PrefixRL 연구는 이런 **Pareto trade-off**에서 같은 지연의 기존 회로보다 면적이 작은 설계를 보고했다. 회로의 품질을 단일 숫자로 만들 때는 어떤 제약과 가중치를 썼는지까지 확인해야 한다.

**왜 합성 도구를 학습 고리에 넣을까?** 그림만 보고 회로의 실제 면적과 지연을 정확히 알 수 없기 때문이다. 합성은 선택한 구조를 구현 가능한 회로로 바꾸어 평가한다. 이 피드백은 신뢰성을 높이지만 계산 비용이 커져, 보상 한 번을 얻는 데 드는 비용도 학습 설계의 일부가 된다.

### 1.26 슬라이드 26 · 언어모델 정렬에서의 RLHF

언어모델은 먼저 텍스트의 패턴을 배우고, 좋은 답변의 시연으로 지시 수행을 익힌 뒤, 여러 답변 중 사람이 선호하는 것을 맞히는 보상 모델을 학습할 수 있다. 그다음 정책을 이 보상이 높아지는 쪽으로 갱신한다. 공개된 InstructGPT 절차에서는 마지막 단계에 PPO를 사용했다. 훈련 절차

토큰마다 정답을 받는 것과 달리 전체 답변에 대한 선호가 뒤늦게 주어지므로 어떤 선택이 평가에 기여했는지 추정해야 한다. 다만 모델마다 실제 학습 파이프라인은 다르다. 모든 대화 모델이 같은 RLHF 절차를 쓴다고 일반화하지 않는다.

가령 같은 질문에 두 답변을 생성해 사람이 A를 선호했다고 하자. 보상 모델은 $r_\phi(x, A) > r_\phi(x, B)$가 되도록 학습하고, 이후 정책은 높은 예측 보상을 받는 답변을 더 자주 만들도록 바뀐다. 그러나 보상 모델은 사람의 판단을 근사한 함수일 뿐이다. 정책이 그 근사의 빈틈을 이용할 수 있으므로 실제 사람 평가와 원래 언어 능력의 유지 여부를 별도로 확인한다.

선호 비교가 숫자 보상으로 바뀌는 한 방법은 두 답변의 점수 차이에 sigmoid를 적용하는 것이다. 선호된 답변을 $y_w$, 다른 답변을 $y_l$라 하면 보상 모델의 학습 손실은 $-\log \sigma(r_\phi(x, y_w) - r_\phi(x, y_l))$이다. 점수 차이가 2라면 선호 예측 확률은 $\sigma(2) \approx 0.881$이다. 두 점수 모두에 같은 상수를 더해도 차이와 확률은 변하지 않으므로, 이 학습은 **절대적인 행복 점수**보다 답변 간 상대 순서를 배운다. InstructGPT 원논문

**보상 모델의 점수가 높으면 사람이 반드시 선호할까?** 아니다. 비교 데이터에 드물었던 형식이나 지나치게 긴 답변에 대해 보상 모델이 잘못 일반화할 수 있다. 정책이 보상 모델만 반복적으로 최적화할수록 그 오류를 이용하는 reward hacking 위험이 생긴다.

## 1.27 슬라이드 27 · 확산 모델에서의 보상 최적화

확산 모델은 노이즈가 큰 표본을 여러 단계에서 정제해 이미지를 만든다. 보통의 확산 학습 목표는 데이터 분포를 재현하도록 설계된다. 그러나 사용자가 원하는 최종 품질은 미적 선호, 프롬프트 정합성, 압축성처럼 다른 기준일 수 있다.

DDPO는 각 정제 단계를 행동으로 보고 최종 이미지의 평가 점수를 보상으로 삼아 생성 과정을 조정한다. 보상은 마지막에 나오므로 중간 정제 단계에도 결과를 연결해야 한다. 보상 모델의 점수가 올라가도 사람이 보기에 더 좋아졌는지는 따로 확인해야 한다. DDPO 논문

구체적으로 이미지 파일의 압축 용량을 보상으로 삼으면 모델은 압축하기 쉬운 패턴을 더 만들 수 있다. 이것은 측정 가능한 목표를 최적화했다는 사례이지, 곧바로 사진의 미적 품질이 좋아졌다는 뜻은 아니다. 논문은 미적 점수와 프롬프트–이미지 정합성 같은 다른 평가도 다룬다. **무엇을 보상으로 정의했는가**를 먼저 물어야 결과를 해석할 수 있다.

**확산 모델 자체가 RL인가?** 일반적인 확산 모델은 노이즈 제거 손실로 학습한다. DDPO처럼 최종 평가를 보상으로 삼아 여러 정제 단계를 정책 갱신의 대상으로 볼 때 강화학습을 적용한다.

## 1.28 슬라이드 28 · 코드 에이전트의 순차 결정

코드 에이전트는 파일을 읽고 수정하고 도구를 실행하고 실패 로그를 본 뒤 다음 행동을 고른다. 첫 패치가 테스트 결과를 바꾸고 그 결과가 다음 탐색 경로를 바꾸므로 순차적 의사결정의 예다. "이 요청에 맞는 파일 하나를 예측하라"는 정적 문제와 다르다.

제품이 에이전트처럼 행동한다는 사실만으로 내부 정책이 RL로 훈련되었다고 결론 내릴 수는 없다. 여기서는 상태(코드와 로그), 행동(편집·도구 호출), 피드백(검사 결과·사람의 평가)으로 문제를 나누는 방법을 배운다.

예를 들어 버그 수정에서 첫 행동이 오류를 재현하는 명령 실행이라면 즉각적인 코드 변경은 없다. 그러나 얻은 stack trace가 원인을 좁혀 다음 편집의 성공 확률을 높인다. 반대로 성급한 패치가 한 검사는 통과시켜도 다른 경로를 깨뜨릴 수 있다. 도구 출력은 현재 문제에 대한 **관측**이고, 검사의 통과 여부는 여러 가능한 피드백 가운데 하나다.

**검사 통과를 보상으로 주면 충분할까?** 검사는 명세의 일부만 확인한다. 보상을 검사 결과에만 맞추면 검사에 드러나지 않는 요구를 놓칠 수 있다. 패치의 정확성, 회귀 여부, 사용자 요청 충족 등을 함께 평가해야 한다.

## 1.29 슬라이드 29 · 다족 로봇의 보행

다족 로봇은 관절의 힘이나 목표 위치를 정하고 몸의 기울기·속도·접촉 상태를 관측하며 균형을 유지한다. 보상은 이동 거리, 넘어짐, 에너지 사용, 발 미끄러짐을 조합할 수 있다. 이동 거리만 크게 주면 불안정한 자세나 과도한 에너지 소비가 생길 수 있다.

보행 정책은 매 순간 동작을 따로 맞히기보다 여러 걸음에 걸쳐 안정성을 유지해야 한다. 시뮬레이션에서 배운 동작을 실제 로봇으로 옮길 때는 마찰·질량·센서 지연의 차이도 문제다. 성공 영상을 해석할 때 배포 조건까지 확인해야 한다.

국내 KAIST HOUND 연구는 모터가 낼 수 있는 토크와 속도의 범위를 학습에 반영해 실제 로봇에서 불가능한 명령을 줄였다. 해외 ANYmal 연구는 많은 시뮬레이션 로봇을 병렬로 학습시킨 뒤 실제 기체에 정책을 옮겼다. 둘 다 "시뮬레이터에서 보상이 높다"와 "실제 하드웨어가 그 행동을 수행할 수 있다" 사이에 간격이 있음을 보여 준다. HOUND 연구 · ANYmal 연구

**넘어지지 않는 보행과 빠른 보행은 같은 목표일까?** 안정성, 속도, 에너지 사용은 서로 충돌할 수 있다. 속도 보상만 키우면 위험한 자세나 모터 한계에 가까운 동작이 나올 수 있다. 목적함수의 항과 실제 하드웨어 제약을 함께 설계해야 한다.

## 1.30 슬라이드 30 · 로봇 조작과 접촉

물체를 집거나 밀 때는 접촉 순간에 작은 위치 차이가 결과를 크게 바꾼다. 접근 경로가 좋아도 마지막 몇 밀리미터의 오차로 놓칠 수 있다. 로봇이 접촉력을 보고 행동을 수정하면 폐회로 제어이고, 긴 행동열을 그대로 실행하면 열린 고리 제어의 비중이 크다.

조작 보상은 성공·실패처럼 드물게 올 수 있다. 그러면 어떤 단계가 실패 원인인지 알기 어렵다. 시연으로 초기 정책을 만들고 RL로 개선하거나 거리·자세 같은 중간 신호를 조심스럽게 설계하는 이유가 여기에 있다.

정육면체를 집어 다른 물체 위에 쌓는 일을 생각해 보자. 집기 전 손끝 위치, 접촉 순간의 힘, 들어 올린 뒤의 미끄러짐이 모두 최종 성공에 영향을 준다. 영상만 입력받는다면 가려진 면과 접촉력을 직접 볼 수 없어 상태 추정도 필요하다. 실제 시각 기반 cube stacking 연구는 시뮬레이션에서 배운 표현을 실제 로봇의 라벨 없는 영상으로 적응시키는 문제까지 다뤘다.

## 1.31 슬라이드 31 · 왜 강화학습을 배우는가

강화학습이 필요한 대표 조건은 **정답 행동을 알기 어렵고, 현재 행동이 이후 상태를 바꾸며, 목표가 여러 행동의 결과로 드러난다**는 것이다. 바둑에서는 승패가 마지막에 나타나고 로봇에서는 접촉이 다음 조작 가능성을 바꾸며 대화에서는 앞선 답변이 다음 질문을 바꾼다.

상호작용이 위험하거나 비싸면 충분한 시연이나 오프라인 데이터부터 활용할 수 있다. 뒤의 각 방법은 이러한 제약 아래서 새로운 정보를 얻고 기존 경험을 이용하는 방식을 다르게 정한다.

문제를 고를 때는 먼저 "전문가의 행동 기록이 충분한가", "시도 한 번의 비용이 얼마인가", "성공을 자동으로 측정할 수 있는가"를 묻자. 시연이 풍부하고 시행착오가 위험하면 BC가 실용적인 기준선이다. 시뮬레이터에서 수많은 시도를 할 수 있고 장기 결과가 중요하면 RL의 장점이 커진다. 고정된 기록만 쓸 수 있다면 offline RL의 분포 이동 문제가 중심이 된다.

**보상을 정의했는데도 학습한 행동이 이상할 수 있나?** 에이전트는 실제로 계산되는 보상을 높인다. 이동 거리만 주면 충돌을 감수하고 빠르게 움직일 수 있다. 의도한 목표와 수치 보상이 일치하는지 별도의 성공·안전 지표로 평가해야 한다.

## 1.32 슬라이드 32 · 다음 단계: 모방학습

이제 에이전트–환경 고리를 더 엄밀하게 정의하고 전문가의 행동을 배우는 가장 직관적인 방법을 살펴본다. 모방학습은 보상 함수를 쓰지 않고도 출발하지만 전문가가 방문한 상태 밖에서는 정책이 취약해질 수 있다.

그 한계를 이해하면 왜 환경과 상호작용하며 결과를 평가해야 하는지 자연스럽게 이어진다. 다음 장에서는 행동 복제의 수식, 오류 누적, 여러 유효 행동의 공존, 전문가와 에이전트의 정보 차이, DAgger와 로봇 사례를 차례로 연결한다.

이어 읽을 때는 작은 표를 머릿속에 두면 좋다. BC는 **전문가가 방문한 상태**에서 **전문가 행동**을 목표로 학습한다. DAgger는 **현재 정책이 방문한 상태**에 전문가 행동을 새로 붙인다. RL은 행동의 정답 대신 **환경에서 얻은 결과**로 정책을 개선한다. 셋 모두 행동을 바꾸지만, 어떤 상태를 보고 어떤 피드백을 받는지가 다르다.

# 2 01-2. 모방학습과 로봇 정책

전문가가 이미 좋은 행동을 보여 주었다면 먼저 그 행동을 배울 수 있다. 모방학습은 정책을 시작하는 가장 직접적인 방법이지만, 실제로 움직일 때는 훈련 때 보지 못한 상태를 만들고 여러 유효한 행동 중 하나를 골라야 한다. 이 장은 행동 복제의 목적함수에서 출발해 분포 이동, DAgger, 현대 로봇 정책까지 연결한다.

## 2.1 슬라이드 01 · Imitation Learning

모방학습은 전문가의 행동을 재현하도록 정책을 학습한다. 데이터 한 항목은 관측 또는 상태 $s$와 그때 전문가가 택한 행동 $a^*$로 이루어진다. 정책 $\pi_\theta(a \mid s)$가 이 행동에 높은 확률을 주도록 만들면 새로운 상태에서도 비슷하게 행동하기를 기대할 수 있다.

하지만 정책이 실제 환경에 들어가면 자신의 행동이 다음 상태를 만든다. 데이터셋에 있는 상황을 잘 맞힌다는 사실과 긴 과제를 성공적으로 수행한다는 사실 사이에는 간격이 있다. 이 장의 실패 사례는 그 간격이 어디서 생기는지 분해한다.

예를 들어 컵을 집는 시연에서 로봇이 본 입력이 손목 카메라 영상인데, 학습 때는 외부 카메라 영상만 준다면 상태–행동 쌍의 의미가 달라진다. 모방학습에서 데이터의 양보다 먼저 확인할 것은 **전문가가 행동을 고를 때 쓴 정보와 정책이 받을 정보가 일치하는가**다.

## 2.2 슬라이드 02 · 강화학습과 모방학습의 위치

강화학습의 일반적인 목표는 여러 행동 뒤에 얻는 누적 보상을 높이는 것이다. 모방학습은 그 목표로 가기 위한 출발점일 수 있다. 예를 들어 사람이 로봇 팔을 직접 조종해 집기 동작을 보여 주면, 로봇은 성공·실패 보상을 수천 번 시험하기 전에 기본적인 접근 경로를 배운다.

둘은 학습 신호가 다르다. 모방학습은 전문가가 선택한 행동을 알고, RL은 행동의 결과를 평가하는 보상을 받는다. 좋은 시연이 많으면 전자가 간결하지만 전문가의 행동이 불완전하거나 더 나은 전략을 찾아야 한다면 결과 신호가 중요해진다.

같은 로봇 집기라도 전문가 시연은 "이 상태에서 손을 어디로 옮겼는가"를 알려 주고, 성공 보상은 "완성된 시도가 성공했는가"만 알려 줄 수 있다. 시연에는 매 단계의 감독 신호가 있지만 전문가의 품질을 넘어서는 동작을 찾기 어렵다. 보상은 행동의 정답을 주지 않지만 다른 경로의 가치를 평가할 수 있다.

## 2.3 슬라이드 03 · 에이전트–환경 고리 다시 보기

에이전트가 현재 정보 $s_t$를 보고 행동 $a_t$를 내면 환경은 보상 $r_t$와 다음 상태 $s_{t+1}$을 반환한다. 시연 데이터는 이 고리 안에서 전문가 정책이 만든 궤적이다. 따라서 시연의 상태는 임의의 상태 모음이 아니라 **전문가가 방문한 상태의 분포**를 따른다.

모방 정책이 전문가와 조금만 달라져도 다른 상태에 들어갈 수 있다. 정책의 평가를 한 시점의 행동 정답률만으로 끝내지 않고, 실제로 굴려서 얻는 궤적과 최종 성과까지 봐야 하는 이유다.

전문가가 늘 컵의 왼쪽에서 접근했다면 데이터의 컵 위치와 팔 자세도 그 경로에 집중된다. 학습 정책이 한 번 오른쪽으로 틀어지면 다음 영상은 전문가 시연에 드물거나 없을 수 있다. 그래서 시연 데이터는 독립적으로 수집된 정답 표가 아니라 특정 정책이 만든 **방문 분포**의 표본이다.

## 2.4 슬라이드 04 · 상태, 행동, 정책

정책은 입력 상태 $s$에서 행동 $a$를 고르는 규칙이다. 확률적 정책은 $\pi_\theta(a \mid s)$, 결정론적 정책은 $a = \mu_\theta(s)$로 쓴다. 로봇의 상태에는 카메라 영상, 관절 각도, 그리퍼 상태가 포함될 수 있고 행동은 목표 관절

위치나 모터 토크일 수 있다.

"최적 행동열을 찾는다"는 말은 매 시점의 행동을 따로 맞히는 것보다 어렵다. 물체를 집기 전의 접근 방향이 이후의 잡기 가능성을 바꾸기 때문이다. 여기서 정책의 입력에 필요한 정보가 빠지면 관측만으로 좋은 행동을 결정할 수 없게 된다.

행동의 단위도 명시해야 한다. 같은 숫자 0.2가 관절의 절대 목표 각도인지, 현재 각도에서 더할 변화량인지, 모터 토크인지에 따라 결과가 전혀 다르다. 정책을 비교할 때 관측과 행동의 차원뿐 아니라 좌표계, 단위, 제어 주기까지 맞춰야 하는 이유다.

## 2.5 슬라이드 05 · 궤적과 전이의 표기

시간 $t = 0, 1, \dots$에서 상태 $s_t$, 행동 $a_t$, 전이 $P(s_{t+1} \mid s_t, a_t)$를 구분하자. 관측 $o_t$는 센서가 실제로 보여 주는 값이고, 상태 $s_t$는 미래를 예측하기에 충분하다고 가정하는 정보다. 둘이 같을 수도 있지만 카메라 한 장에 속도나 가려진 물체 위치가 없으면 다르다.

정책을 환경에서 실행해 얻은 $\tau = (s_0, a_0, s_1, a_1, \dots, s_T)$를 rollout 또는 episode라고 한다. 시연 하나도 하나의 궤적이고, 그 안의 상태–행동 쌍을 잘라 행동 복제의 훈련 데이터로 사용할 수 있다.

궤적을 쌍으로 자를 때 $(s_t, a_t)$의 시각이 정확히 맞아야 한다. 카메라 프레임이 100ms 늦었다면 데이터에는 행동의 결과 영상과 이전 행동이 잘못 짝지어질 수 있다. 정책은 "컵이 이미 왼쪽으로 움직인 뒤에 오른쪽으로 밀라" 같은 뒤늦은 규칙을 배울 수 있다.

**episode와 rollout은 항상 같은 길이일까?** episode는 보통 환경의 시작부터 종료까지를 뜻한다. rollout은 학습에 필요한 고정 길이만 수집해 중간에 끊을 수도 있다. 시간 제한으로 끊긴 표본을 진짜 실패 종료와 혼동하면 가치 학습의 목표도 달라진다.

## 2.6 슬라이드 06 · 정답 행동과 보상

지도학습은 입력마다 따라야 할 정답을 알고 있다고 가정한다. 행동 복제에서는 전문가 행동 $a^*$가 그 정답 역할을 한다. 강화학습에서는 정답 행동 없이 선택의 결과를 나타내는 보상만 받을 수 있다. 로봇이 컵을 떨어뜨렸다는 피드백은 "정확히 어느 관절 각도를 골랐어야 하는지"를 알려 주지 않는다.

두 방식은 서로 대체 관계로만 볼 필요가 없다. 시연으로 초기 정책을 만든 뒤, 실제 성공 보상으로 세부 행동을 개선할 수 있다. 다만 보상을 얻기 위한 실제 상호작용이 비싸거나 위험하면 시연의 가치가 커진다.

예를 들어 장난감 환경에서 성공하면 마지막에 1점만 받는다면, 실패한 로봇에게는 어느 관절 명령이 잘못되었는지 알 길이 없다. 전문가 시연의 각 시점 행동은 그 공백을 메운다. 반대로 시연에서 손목이 매번 조금 흔들린다면 지도학습은 그 흔들림도 목표로 보지만, 보상 기반 평가는 최종 성공에 무해한 차이를 구분할 수 있다.

## 2.7 슬라이드 07 · 이 장의 질문

모방학습을 이해하려면 네 단계가 필요하다. 시연을 어떻게 모으는지, 행동 복제가 어떤 손실을 줄이는지, 실제 실행에서 왜 실패하는지, 그리고 분포 이동을 줄이는 방법이 무엇인지다. 후반의 로봇 사례는 같은 문제를 실제 장치와 대규모 데이터에서 다시 만난다.

한 방법의 성능을 읽을 때 성공률 외에 시연 시간, 관측 구성, 행동 주기, 환경 변경, 사람이 개입한 정도를 함께 기록하자. 이 조건이 다르면 숫자만 나란히 놓아 비교하기 어렵다.

이 네 단계는 원인과 해결책을 맞추는 지도이기도 하다. 시연이 부족한지, 손실이 행동 분포와 맞지 않는지, rollout에서 새로운 상태를 만나는지, 사람의 복구가 필요한지를 순서대로 확인한다. 실험표에는 훈련 손실과 평가 성공률을 나란히 적어 **한 단계 예측과 전체 행동의 차이**를 드러내자.

## 2.8 슬라이드 08 · Behavioral Cloning의 목적함수

전문가 데이터 $\mathcal{D} = \{(s_i, a_i^*)\}_{i=1}^{N}$는 $N$개의 상태 $s_i$와 전문가 행동 $a_i^*$의 쌍이다. $\theta$를 정책 신경망의 파라미터, $\pi_\theta(a_i^* \mid s_i)$를 그 상태에서 전문가 행동에 부여한 확률이라고 하자. 행동 복제(BC)는 다음 평균 음의 로그우도 $\mathcal{L}_{\mathrm{BC}}$를 최소화한다.

$$\mathcal{L}_{\mathrm{BC}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log \pi_\theta(a_i^* \mid s_i).$$

행동이 범주형이면 이 식은 정답 클래스의 교차 엔트로피가 된다. 연속 행동에 고정 분산 Gaussian 정책을 가정하면 음의 로그우도는 상수항을 제외하고 평균 행동의 제곱오차에 비례한다. 그래서 같은 "BC" 라도 선택한 행동 분포에 따라 학습 손실이 다르게 보인다. 초기 자율주행 연구인 ALVINN은 카메라 입력에서 조향을 배우는 고전적인 예다.

**연속 행동에서 L2 손실과 최대우도 학습이 언제 같은가?** 정책을 평균 $\mu_\theta(s)$와 고정 분산 $\sigma^2 I$를 가진 Gaussian으로 두면 $-\log \pi_\theta(a \mid s) = \|a - \mu_\theta(s)\|^2/(2\sigma^2) + C$다. 분산까지 학습하거나 다봉 분포를 쓰면 단순 L2 회귀와 같지 않다.

예를 들어 전문가 행동에 부여한 확률이 0.8이면 한 표본 손실은 $-\log 0.8 \approx 0.22$, 0.2이면 $-\log 0.2 \approx 1.61$이다. 이 손실은 전문가가 방문한 상태에서의 행동 예측을 측정한다. 배포 시 정책 스스로 방문하는 상태의 성공률은 rollout으로 평가한다.

## 2.9 슬라이드 09 · 시연의 품질과 범위

BC의 손실을 계산하려면 행동뿐 아니라 **그 행동을 고른 당시의 관측**이 필요하다. 나중에 성공한 비디오만 모아도 정확한 관절 명령과 관측 시각이 없으면 같은 감독 신호로 쓰기 어렵다. 시간 동기화와 좌표계가 중요하다.

또한 시연의 수보다 다양성이 중요할 수 있다. 물체가 늘 같은 위치에 있으면 그 위치만 잘 배우고 조금 옮겨 놓은 시험에서 실패한다. 다양한 초기 상태, 조명, 물체 형태, 실수 뒤의 복구 시연이 배포 범위를 넓힌다.

예를 들어 물체가 탁자 중앙에만 있는 1000개 시연과, 위치·크기·조명·실패 후 복구가 다양한 200개 시연은 서로 다른 정보를 준다. 더 많은 데이터가 언제나 넓은 상태 범위를 보장하지는 않는다. 시연의 **coverage**는 개수뿐 아니라 어떤 조건을 실제로 포함하는지로 확인한다.

## 2.10 슬라이드 10 · 시연을 수집하는 방식

텍스트나 운전 기록처럼 사람이 평소 하는 행동을 로그로 남길 수도 있다. 로봇에서는 팔을 직접 움직이는 kinesthetic teaching, 원격 조종기, 사람의 조작을 로봇의 다른 팔에 전달하는 puppeteering을 쓴다. 각각 장비 비용, 조작 편의, 영상에 사람이 가리는 정도, 행동의 정밀도가 다르다.

사람이 그대로 시연할 수 없는 행동도 있다. 네 발로 걷는 로봇의 각 관절 토크를 사람이 매 순간 정답으로 제시하기는 어렵다. 이 경우 시뮬레이션이나 보상 기반 RL이 필요할 수 있다.

**시연을 많이 모으면 어떤 데이터 문제가 남을까?** 관측과 행동의 시간 지연, 서로 다른 사람의 행동 방식, 센서 캘리브레이션, 실패 시연의 혼입이 남는다. 양을 늘리기 전에 시연이 실제 배포 정책이 볼 상태와 같은 표현으로 저장되는지 확인해야 한다.

시연 장치가 다르면 학습 가능한 행동의 정밀도도 바뀐다. 원격 조작자가 큰 버튼으로 그리퍼를 여닫는다면 세밀한 손가락 힘 조절을 라벨로 얻을 수 없다. 반대로 고정밀 장치를 쓰더라도 조작자의 지연이 크면 빠른 접촉 순간의 타이밍이 흐려질 수 있다. 따라서 데이터 수집 인터페이스 자체가 정책의 성능 상한에 영향을 준다.

### 2.11 슬라이드 11 · 실패 원인으로 넘어가기

훈련 손실이 낮다고 긴 궤적에서 성공하는 것은 아니다. BC는 전문가가 방문한 상태에서의 행동 정확도를 측정한다. 실제로 정책을 실행하면 자신의 행동 때문에 다른 상태를 만들고, 그 상태에서는 학습 데이터가 거의 없을 수 있다.

다음의 세 실패 원인은 서로 다르다. 분포 이동은 **어느 상태를 보느냐**의 문제, 다봉 시연은 **같은 상태에서 어느 행동을 고르느냐**의 문제, 관측 불일치는 **행동 결정에 필요한 정보가 입력에 있느냐**의 문제다.

훈련 정확도가 99%인 정책이 100단계 과업을 실패할 수 있다. 각 단계가 독립이고 성공에 전 단계가 필요하다는 단순 가정만으로도 완주 확률은 $0.99^{100} \approx 0.37$이다. 실제 행동은 독립적이지 않아서 한 번의 실수가 이후 상태를 더 어렵게 만들 수 있다. 높은 정확도와 낮은 성공률이 모순이 아닌 이유다.

### 2.12 슬라이드 12 · 실패를 분해하는 틀

모방학습 문제를 세 개의 질문으로 점검하자. 첫째, 실행 중 정책이 방문하는 상태가 전문가 데이터에 있나? 둘째, 같은 관측에서 서로 다른 유효 행동이 나올 수 있나? 셋째, 전문가가 보았지만 정책은 보지 못한 정보가 있나?

세 문제의 처방도 다르다. 분포 이동에는 정책이 방문한 상태의 교정 데이터를, 다봉 행동에는 여러 모드를 표현하는 정책 분포를, 관측 불일치에는 입력 정보의 보강이나 시연 수집 조건의 조정이 필요하다.

세 실패는 증상이 비슷해 보여도 해결법이 다르다. 낯선 상태에서만 넘어지면 데이터를 그 상태로 넓혀야 한다. 익숙한 상태에서도 좌우 동작의 평균을 내면 분포 모델을 바꿔야 한다. 관측에 물체가 가려져 정답 자체를 알 수 없으면 카메라 위치나 기억 상태를 바꿔야 한다. 무작정 네트워크만 키우면 셋 중 어느 문제도 풀리지 않을 수 있다.

### 2.13 슬라이드 13 · 첫째 실패: 오류의 누적

조향 정책이 한 번 차선 가장자리로 틀어지면 다음 영상은 전문가 시연에서 드문 상태가 된다. 정책이 만든 상태를 다시 입력받는 이 구조에서, 전문가의 방문분포와 학습 정책의 방문분포가 달라지는 현상을 **정책 유발 분포 이동**이라고 한다.

오류 누적을 수량화해 보자. $\pi^*$는 전문가 정책, $\pi$는 학습 정책, $T$는 행동 횟수다. $d^t_{\pi^*}$와 $d^t_{\pi}$는 각각 시점 $t$에 두 정책이 방문하는 상태의 확률분포다. 분석을 간단히 하기 위해 두 정책은 결정론적이라고 두고, 전문가 상태에서의 시점별 행동 오류율 $e_t$와 그 평균 $\epsilon$을 정의한다.

$$e_t = \Pr_{s \sim d^t_{\pi^*}}\big[\pi(s) \neq \pi^*(s)\big], \qquad \epsilon = \frac{1}{T}\sum_{t=1}^{T} e_t.$$

두 정책을 같은 초기 상태와 환경의 무작위 결과에 놓으면, 첫 행동 오류 전까지는 같은 상태를 지나간다. $D_t$를 시점 $t$까지 두 경로가 갈라진 사건이라고 하자. $i \le t$에서 첫 오류가 일어날 사건의 확률은 전문가 경로에서 시점 $i$에 오류가 날 확률 $e_i$ 이하이므로, 사건들의 합집합에 대한 **union bound**로 $\Pr(D_t) \le \sum_{i=1}^{t} e_i$를 얻는다. 단계별 오류가 독립일 필요는 없다.

이제 한 단계 비용을 $c_t \in [0,1]$, 정책의 총기대비용을 $J(\pi) = E_\pi[\sum_{t=1}^{T} c_t]$라고 정의하자. 두 경로가 아직 같으면 비용 차이는 0이고, 갈라진 뒤의 한 단계 비용 차이는 최대 1이다. 따라서 경로가 갈라질 확률을 시점마다 더하면

$$
\begin{aligned}
J(\pi) - J(\pi^*) &\le \sum_{t=1}^{T} \Pr(D_t) \\
&\le \sum_{t=1}^{T} \sum_{i=1}^{t} e_i = \sum_{i=1}^{T} (T - i + 1) e_i \\
&\le T \sum_{i=1}^{T} e_i = T^2 \epsilon.
\end{aligned}
$$

$i$번째 오류가 남은 최대 $T - i + 1$단계에 영향을 줄 수 있어 $T^2\epsilon$이 나타난다. 각 시점의 오류율이 동일한 $e_i = \epsilon$인 예에서는 위 식의 중간 합이 $\epsilon T(T+1)/2$다. $T = 20$, $\epsilon = 0.01$이면 추가 비용의 상한은 2.1이다. 실제 비용은 복구 가능성에 따라 달라진다. DAgger 원논문

**시연 데이터에서 행동 정확도가 높은데 실행은 왜 실패할까?** 시연 기반 시험은 $d^t_{\pi^*}$에서 오류율을 재지만, 실제 실행은 $d^t_{\pi}$의 상태를 만든다. 한 번 이탈한 뒤의 상태가 시연에 거의 없다면 시험 정확도는 복구 능력을 보여 주지 않는다. 정책을 실행한 rollout에서 방문 상태와 전체 과제 성공률을 함께 확인해야 한다.

## 2.14 슬라이드 14 · 오류 누적을 줄이는 두 경로

하나는 매우 넓은 시연 데이터를 모아 배포 중 만날 상태를 미리 덮는 것이다. 그러나 가능한 상태 공간이 커서 모든 이탈 상황을 수동으로 수집하기 어렵다. 다른 하나는 정책이 실제로 만든 실수 상태에서 **어떻게 복구할지** 전문가의 교정 행동을 받는 것이다.

예를 들어 조향 정책이 차선 왼쪽으로 치우친 영상을 만든다면, 중앙에서 직진하는 시연을 더 모으기보다 왼쪽으로 치우친 상태에서 오른쪽으로 복구하는 행동을 추가해야 한다. 다음 DAgger가 바로 이 원리를 반복적인 데이터 수집 절차로 만든다.

핵심은 단순히 "실패 데이터를 더 모으자"가 아니다. 잘못 조향한 후의 영상에 **그 상태에서 취해야 할 올바른 복구 행동**이 있어야 한다. 실패 장면만 쌓고 라벨이 없으면 BC 손실에 바로 넣을 수 없다. 이 점에서 전문가의 개입이 필요한 DAgger와 결과 보상만 받는 RL이 갈린다.

## 2.15 슬라이드 15 · 둘째 실패: 여러 정답 행동

같은 물체를 왼쪽으로 돌아 잡거나 오른쪽으로 돌아 잡는 두 경로가 모두 성공할 수 있다. 이때 제곱오차를 최소화하는 결정론적 예측은 조건부 평균 $E[a \mid s]$다. 두 행동의 평균이 물체를 정면으로 미는 방향이라면, 데이터에 없던 실패 행동을 출력한다.

문제는 데이터가 부정확한 것이 아니라 $p(a \mid s)$가 **다봉(multimodal)**이라는 점이다. 범주형 분포, Gaussian mixture, 잠재변수를 가진 VAE, 확산 정책 등은 여러 행동 모드를 표현할 수 있다. 어느 방법이든 실행 때는 한 모드에 일관되게 머무는지도 확인해야 한다. 매 순간 다른 모드를 고르면 경로가 지그재그로 흔들릴 수 있다.

**L1 손실로 바꾸면 다봉 문제가 사라질까?** L1의 최적 예측은 조건부 중앙값이므로 L2의 평균과는 다르다. 하지만 두 유효 행동 사이의 비유효 행동을 고를 가능성은 남는다. 여러 행동 경로를 분포로 표현하고 일관된 하나를 샘플링하는 접근이 더 직접적이다.

수치 예를 들면 같은 상태에서 유효 행동이 $a = -1$과 $a = +1$이고 두 행동이 동일한 빈도로 관측되었다고 하자. 제곱오차 $E[(a - \hat{a})^2]$를 최소화하는 예측은 $\hat{a} = E[a] = 0$이다. 그런데 0이 장애물로 직진하는 행동이라면 회귀 오차의 최적해가 과업에서는 실패다. 여러 모드를 표현하고 실행 중 한 모드를 유지하는 정책이 필요한 이유다.

평균이 최적이라는 주장은 미분으로 확인할 수 있다. 상태 $s$를 고정하고 예측 $m$에 대한 조건부 위험을 $L_s(m) = E[(a - m)^2 \mid s]$라고 두면

$$\frac{dL_s}{dm} = E[2(m-a) \mid s] = 2\{m - E[a \mid s]\}, \qquad \frac{d^2 L_s}{dm^2} = 2 > 0.$$

따라서 유일한 최소점은 $m^* = E[a \mid s]$다. 이 결과는 회귀 손실에 대해서만 최적이라는 뜻이다. 행동의 성공 여부가 비선형이면 $m^*$가 성공 행동 집합 밖에 있을 수 있다. 다차원 관절 명령에도 각 좌표의 평균이 같은 방식으로 나오므로, 두 유효 동작의 좌표별 평균이 물체와 충돌할 수 있다.

### 2.16 슬라이드 16 · 셋째 실패: 전문가와 에이전트의 관측 차이

전문가는 과거 대화에서 상대가 농구 경기를 했다는 정보를 보고 답했는데, 데이터셋에는 현재의 "잘 지내?"만 남았다고 하자. 에이전트가 볼 수 없는 과거를 이용해 전문가의 답변을 맞히라고 하면 같은 입력에 서로 다른 답이 붙는다. 모델이 크더라도 숨겨진 정보 자체를 복원할 근거가 없다.

해결책은 에이전트에도 필요한 대화 이력을 주거나, 전문가가 에이전트와 같은 정보만 보고 시연하게 하는 것이다. 로봇에서도 전문가가 손끝의 힘을 느끼는데 정책에는 영상만 주면 같은 문제가 생긴다. 이 경우 촉각 센서를 추가하거나 시연의 정보 조건을 다시 맞춰야 한다.

관측 불일치는 데이터 양만 늘려도 풀리지 않는다. 같은 현재 영상 $o$에 대해 전문가가 과거 접촉 정보 $h$를 보고 왼쪽 또는 오른쪽을 택했다면, $o$만 받는 정책은 그 차이를 구별할 수 없다. 시간에 걸친 관측을 입력하거나 촉각 신호를 추가하는 등 **결정에 필요한 정보**를 정책에 전달해야 한다.

### 2.17 슬라이드 17 · 세 실패의 진단 순서

정책이 실패하면 먼저 실행 중 본 상태가 데이터에 있는지 살핀다. 그렇지 않다면 분포 이동이다. 상태는 익숙하지만 행동 예측이 두 경로의 중간에 몰린다면 다봉 행동을 의심한다. 같은 입력에서 전문가 라벨 자체가 일관되지 않고 숨겨진 맥락이 있다면 관측 불일치일 수 있다.

손실값만으로 이 세 경우를 가리기 어렵다. 상태 임베딩이나 관측 영상을 비교하고, 행동 분포를 그려 보고, 전문가가 실제로 어떤 정보를 봤는지 확인하는 진단이 필요하다.

간단한 진단 실험을 만들 수 있다. 초기 상태를 여러 개 바꾸어 실패 시점을 기록하고, 그 시점의 영상을 훈련 데이터와 나란히 본다. 행동 예측이 평균에 몰리는지 히스토그램을 그린다. 마지막으로 전문가가 사용한 센서와 정책 입력을 표로 비교한다. 이 세 관찰이 각각 분포, 행동 모드, 정보의 문제를 좁힌다.

### 2.18 슬라이드 18 · 온라인 교정으로 넘어가기

정적 데이터셋에서의 BC는 정책이 틀려서 들어간 상태를 거의 보지 못한다. 이를 고치려면 정책을 실제 환경에서 실행하고, 그때 방문한 상태에 전문가 행동을 붙여야 한다. 데이터 수집의 주체가 전문가에서 현재 정책으로 바뀌는 셈이다.

이 방식은 새 상호작용 비용을 요구한다. 사람의 안전 개입이 필요한 로봇이나 차량에서는 무작정 실행할 수 없으므로, 어디까지 자동 제어를 허용할지와 언제 전문가에게 넘길지를 설계해야 한다.

자동차 예에서는 처음부터 실제 도로에서 실패 상태를 수집할 수 없다. 안전한 시뮬레이터나 제한된 시험 환경에서 먼저 정책을 실행하고, 위험 임계값에 도달하기 전에 사람에게 제어를 넘기는 절차가 필요하다. 온라인 교정의 핵심은 **현재 정책이 갈 법한 상태를 관측**하는 것이며, 위험을 실제로 감수해야 한다는 뜻은 아니다.

### 2.19 슬라이드 19 · DAgger의 네 단계

DAgger(Data Aggregation)는 현재 정책을 실행해 방문 상태를 모으고, 그 상태의 전문가 행동을 받아 기존 데이터에 합친 뒤 정책을 다시 학습한다. 반복 번호를 $k$, 그때의 정책을 $\pi_k$, 모은 데이터를 $\mathcal{D}_k$, $\pi_k$

의 방문 상태분포를 $d_{\pi_k}$ 라고 쓰자. $\ell(\pi(s), a^*)$ 는 예측 행동과 전문가 행동 $a^* = \pi^*(s)$ 의 차이를 재는 감독 손실이다.

$$\mathcal{D}_{k+1} = \mathcal{D}_k \cup \{(s, \pi^*(s)) : s \sim d_{\pi_k}\}, \qquad \pi_{k+1} = \arg\min_{\pi} E_{(s,a^*)\sim\mathcal{D}_{k+1}} \ell(\pi(s), a^*).$$

중요한 변화는 학습 상태 분포가 실제 정책의 방문 상태를 포함한다는 것이다. 정책이 차선 밖으로 향하는 순간의 상태에 복구 행동을 붙일 수 있다. 한계는 에이전트가 제어하는 동안에도 전문가가 상태를 보고 답할 수 있어야 한다는 점이다. DAgger 논문

**DAgger는 보상으로 학습하므로 강화학습인가?** 표준 DAgger는 정책이 방문한 상태를 수집하지만 그 상태의 학습 신호는 여전히 전문가 행동 라벨이다. 환경과 상호작용한다는 점만으로 보상 기반 RL이 되는 것은 아니다.

첫 반복의 정책 $\pi_0$ 가 차선 중앙만 알고 있다가 왼쪽으로 치우친 상태를 만들었다고 하자. 전문가가 그 상태에서 오른쪽 조향을 라벨로 붙이면 $\mathcal{D}_1$ 에 복구 사례가 추가된다. 다음 정책은 그 상태를 만났을 때 복구할 근거가 생긴다. 이 예에서 환경의 보상은 새 라벨이 아니며, **전문가가 같은 상태를 보고 답한다**는 조건이 중요하다.

앞의 제곱 지평은 전문가 분포에서만 측정한 $\epsilon$ 의 한계였다. $\ell(s, \pi) = \mathbf{1}\{\pi(s) \neq \pi^*(s)\}$ 를 행동 불일치 지표로 두고, 정책 자체의 방문분포에서 측정한 오류율을 다음처럼 정의하자.

$$\epsilon_\pi = \frac{1}{T} \sum_{t=1}^{T} E_{s\sim d_\pi^t}[\ell(s, \pi)].$$

이 값을 작게 만들면 다른 경계를 얻는다. 학습 정책의 한 행동을 전문가 행동으로 바꾸고 이후 전문가를 따른다고 할 때 기대 비용 차이가 항상 $u$ 이하라고 가정하자. 시점별로 정책의 행동을 하나씩 전문가 행동으로 교체하는 hybrid 정책들을 이어 붙이면 각 교체의 기대 영향이 최대 $uE_{d_\pi^t}\ell$ 이므로

$$J(\pi) - J(\pi^*) \leq u \sum_{t=1}^{T} E_{s\sim d_\pi^t} \ell(s, \pi) = uT\epsilon_\pi.$$

이 비교가 DAgger의 목적이다. 교정 데이터를 모으는 것만으로 $\epsilon_\pi$ 가 작아진다는 보장은 없다. 전문가 라벨의 일관성, 학습기의 성능, 수집 정책과 최종 정책의 분포 관계 같은 조건이 함께 필요하다. DAgger의 성능 분석

## 2.20 슬라이드 20 · 사람이 개입하는 DAgger

사람이 로봇의 모든 상태에 일일이 정답 행동을 입력하기 어렵다면, 로봇을 관찰하다가 실패 조짐이 보일 때 제어권을 가져와 복구 시연을 제공할 수 있다. 이런 human-gated 방식은 부분 시연을 데이터에 더한다. 사람의 개입 순간이 곧 정책의 취약한 상태를 알려 준다.

다만 빠른 시스템에서는 개입을 결정하고 실제로 조작할 때까지 지연이 생긴다. 고속 조향이나 접촉 직전의 순간에는 실수를 놓칠 수 있다. 안전 장치, 개입 지연, 어떤 상태에서 사람이 관여했는지를 결과와 함께 기록해야 한다.

개입 데이터에는 선택 편향이 있다. 사람은 쉬운 상태에서는 개입하지 않고 실패가 가까운 상태에서 주로 조종한다. 따라서 개입 구간의 행동만 무작정 섞으면 데이터의 상태 비율이 크게 달라진다. 개입 기준, 개입 전후의 관측, 기존 전문가 시연과의 혼합 비율을 함께 기록해야 결과를 해석할 수 있다.

**사람이 개입하지 않은 구간은 정답 행동이 없는가?** 그 구간에는 정책이 실행한 행동 기록이 있지만, 그것이 전문가의 정답이라는 뜻은 아니다. 사람의 개입이 없었다는 사실을 성공 라벨로 자동 해석하면 조용히 실패하는 행동까지 시연으로 취급할 수 있다.

### 2.21 슬라이드 21 · 로봇 사례를 읽는 틀

현대 로봇 모방학습은 정책 모델만으로 성립하지 않는다. 시연을 만드는 장치, 카메라와 관절 센서, 행동의 표현과 제어 주기, 성공률을 계산한 평가 절차가 모두 결과에 기여한다.

뒤의 ALOHA, ACT, Diffusion Policy와 VLA를 볼 때는 **한 시점 행동인지 긴 행동 묶음인지, 하나의 행동만 내는지 여러 모드를 표현하는지, 시연 데이터의 출처와 규모가 무엇인지**를 비교하자. 이 축이 앞의 세 실패 원인과 연결된다.

같은 80% 성공률이라도 초기 물체 위치가 늘 고정된 집기와 물체·조명·장애물이 바뀌는 집기는 난도가 다르다. 이후 사례에서는 "시연 한 개"가 얼마나 긴지, 양팔 행동을 동시에 기록했는지, 평가에서 사람이 실패를 중단했는지를 함께 살펴야 한다. 모델 구조의 기여를 알려면 데이터 수집 장치의 기여도 분리해서 읽어야 한다.

### 2.22 슬라이드 22 · ALOHA와 ACT의 결합

ALOHA는 두 로봇 팔의 세밀한 조작 시연을 비교적 저렴한 장치로 수집하려는 시스템이다. ACT (Action Chunking with Transformers)는 이렇게 모은 시연에서 미래의 여러 행동을 한 번에 예측하는 정책이다. 장치와 학습 알고리즘을 함께 설계한 점이 중요하다. 데이터 수집이 어렵다면 좋은 모델만으로 성능을 얻기 힘들다.

예를 들어 지퍼를 잡고 움직이거나 배터리를 슬롯에 넣는 일은 양팔의 협응, 접촉, 미세 위치 조정이 필요하다. 단순한 한 프레임 행동 예측은 작은 오차를 반복하므로 긴 작업에서 실패하기 쉽다. ALOHA · ACT 논문

ALOHA는 **시연을 어떻게 얻는가**에 답하고 ACT는 **얻은 시연으로 어떤 정책을 학습하는가**에 답한다. 예컨대 조작자가 두 팔로 배터리를 집어 넣는 동안 카메라 영상과 관절 궤적이 같이 저장된다. ACT는 그때의 관측에서 이어질 행동 묶음을 예측한다. 하드웨어와 알고리즘을 같은 약자로 묶어 생각하면 데이터 흐름이 보이지 않는다.

### 2.23 슬라이드 23 · 저비용 하드웨어가 바꾸는 것

장치 가격이 내려가면 연구자가 더 많은 실물 시연을 모으고 시스템을 직접 수정하기 쉬워진다. ALOHA의 하드웨어 설명은 특정 장치의 비용을 제시하지만, 모든 실험실에서 같은 가격과 성능이 보장된다는 뜻은 아니다. 카메라, 캘리브레이션, 수리와 사람의 조작 시간도 실제 비용에 들어간다.

모방학습에서는 데이터가 곧 학습의 출발점이므로, 조작자가 편하게 정확한 행동을 남길 수 있는 인터페이스가 중요하다. 고가 센서를 쓰지 않더라도 안정적인 관측과 일관된 좌표계를 확보해야 모델이 같은 행동을 재현할 수 있다.

비용이 낮아져 시연 수를 늘릴 수 있어도 관측이 흔들리면 정책이 학습할 신호는 나빠질 수 있다. 카메라가 시연마다 다른 위치를 보거나 양팔의 시계가 맞지 않으면 같은 행동에 일관된 입력을 붙일 수 없다. 데이터 수집 시스템의 품질은 시연 개수와 **동기화·캘리브레이션 정확도**를 함께 봐야 한다.

### 2.24 슬라이드 24 · ACT가 마주한 두 문제

첫 문제는 높은 제어 주기에서 반복되는 작은 오류다. 50 Hz라면 1초에 50번 행동을 고르므로 한 단계 예측이 조금만 흔들려도 접촉 작업이 망가질 수 있다. 둘째 문제는 사람 시연이 같은 작업을 서로 다른 경로로 수행한다는 다봉성이다.

슬라이드의 선행 방법 성공률 표는 세부 단계에 따라 0에 가까운 결과가 나타날 수 있음을 보여 준다. 이것을 "모든 BC가 실패한다"는 결론으로 확대하면 안 된다. 구체적인 과업, 장치, 시연 수와 비교 방법의 설정에서 긴 조작을 얼마나 안정적으로 수행하는지가 핵심이다.

50 Hz 제어에서 60단계 행동열은 명목상 1.2초에 해당한다. 하지만 매 단계마다 새로 예측하고 겹친 행동을 결합할 수도 있으므로, "60개를 출력한다"가 항상 1.2초를 눈감고 실행한다는 뜻은 아니다. **예측 길이**와 **실제 재관측 간격**을 구분해야 ACT의 피드백 능력을 정확히 이해할 수 있다.

### 2.25 슬라이드 25 · 행동을 묶어 예측하는 이유

ACT는 한 장면에서 한 행동 대신 약 60개의 미래 행동을 예측한다. 그러면 손이 물체로 접근하고 잡는 동안 매 프레임 새 결정을 내려 모드가 흔들리는 문제를 줄일 수 있다. 기본 action chunking은 묶음을 실행한 뒤 다시 관측하지만, ACT의 temporal ensembling은 매 시점 새 chunk를 예측하고 이전 chunk와 겹치는 현재 행동을 합칠 수도 있다.

행동 묶음은 완전히 열린 고리로 오래 실행할수록 교정 기회가 줄고, 너무 짧으면 한 단계 정책의 흔들림이 되살아난다. 행동 길이와 재관측 주기는 **일관성 대 피드백 속도**의 절충이다. 접촉이 급변하는 과업에서는 최신 센서 정보를 얼마나 자주 반영하는지도 함께 봐야 한다.

**행동 묶음은 오류 누적 문제를 완전히 없애나?** 아니다. 묶음 안의 예측이 틀리면 실행 중 바로 수정하지 못할 수 있다. 행동 묶음은 고주파의 작은 흔들림을 줄이지만, 긴 열린 고리 구간과 환경 변화에 대한 취약성은 별도로 관리해야 한다.

접촉이 거의 없는 팔 이동에서는 다음 30~60개 행동이 하나의 부드러운 동작으로 이어지는 편이 좋다. 반면 플러그가 구멍 입구에 닿는 순간에는 예상 밖의 힘이 생길 수 있어 빠른 재관측이 필요하다. 행동 묶음의 길이를 논할 때는 **모드의 일관성**을 얻는 이익과 **접촉 변화에 늦게 반응하는 비용**을 동시에 계산해야 한다.

ACT의 temporal ensembling에서는 서로 다른 시점에 예측한 chunk가 **현재 시점의 행동**을 여러 개 제안할 수 있다. 이를 시간 가중 평균해 chunk 경계에서 명령이 갑자기 뛰는 일을 줄인다. 따라서 예측 지평 $H = 60$이라는 숫자를 60 step 동안 반드시 관측 없이 실행한다는 뜻으로 읽으면 안 된다. 예측 지평, 새 관측을 받는 간격, 겹친 예측을 합치는 규칙은 서로 다른 설계 변수다. ALOHA · ACT 원논문

### 2.26 슬라이드 26 · 절대 관절 목표와 잠재변수

관절 각도의 작은 상대 변화량만 예측하면 한 번의 오차가 다음 목표에 더해져 드리프트가 커질 수 있다. 절대 목표 관절 위치를 출력하면 각 시점의 목표를 직접 지정해 이 누적을 줄일 수 있다. 제어기의 추종 오차는 남으므로 정확한 상태 측정과 안전 범위가 필요하다.

여러 유효 시연을 다루기 위해 ACT는 잠재변수 $z$를 둔 conditional VAE를 사용한다. 학습 때 시연 행동열 $A$와 관절 관측 $\bar{o}$를 인코딩한 $q_\phi(z \mid \bar{o}, A)$에서 $z$를 뽑고, 영상까지 포함한 관측 $o$와 함께 행동 묶음을 재구성한다. 인코더 분포가 표준정규 prior에서 너무 멀어지지 않도록 KL 항을 둔다. **원 논문의 평가 절차에서는 $z$를 prior 평균인 0으로 고정해 결정론적으로 디코딩한다.** 잠재변수가 있다는 사실만으로 실행 때 여러 모드를 무작위로 샘플한다고 해석해서는 안 된다. ALOHA · ACT 원논문

여기서 $\phi$는 시연 행동열을 잠재변수 $z$로 인코딩하는 분포 $q_\phi$의 파라미터이고, $\theta$는 관측 $o$와 $z$에서 행동열 $\widehat{A}_\theta(o, z)$를 내는 decoder의 파라미터다. $\mathcal{L}_{\text{rec}}$는 실제 행동열 $A$와 예측 행동열의 재구성 오차, $D_{\text{KL}}$은 두 확률분포의 차이를 재는 KL divergence다. $\mathcal{N}(0, I)$는 평균 0 · 단위 공분산의 정규분포이며, $\beta$는 KL 항의 가중치다. 이를 합친 학습 목표는

$$\mathcal{L}_{\text{ACT}} = E_{z\sim q_\phi(z|\bar{o},A)}[\mathcal{L}_{\text{rec}}(A, \widehat{A}_\theta(o, z))] + \beta D_{\text{KL}}(q_\phi(z \mid \bar{o}, A) \,\|\, \mathcal{N}(0, I)) \,.$$

첫 항은 시연 행동열 $A$와 예측 행동열 $\widehat{A}$를 맞춘다. 원 논문 본문은 정밀한 행동 예측을 위해 L1 재구성 손실을 사용했다고 설명한다. 둘째 항은 학습 때 본 행동열이 없는 실행 시점에도 쓸 수 있는 prior를 만든다. 실행 때 $z = 0$을 대입하는 선택과 학습 때 잠재변수로 시연 차이를 설명하려는 목적을 분리해 이해하자.

상대 명령의 드리프트를 수치로 보면 매 단계 0.01만큼 목표를 잘못 더하는 경우 50단계 뒤 누적 목표 오차는 0.5가 된다. 절대 목표는 매 단계 현재 시점의 목표를 새로 지정하므로 같은 방식으로 합산되지는 않는다. 다만 센서가 잘못된 관절 위치를 알려 주거나 하위 제어기가 목표를 못 따라가면 절대 목표도 실패할 수 있다.

**잠재변수 $z$를 두면 실행 때 반드시 다양한 행동이 나올까?** 아니다. 원 논문의 기본 평가에서는 $z = 0$으로 고정하므로 같은 관측에서의 출력은 결정론적이다. 다른 $z$를 사용하더라도 학습이 이를 무시하는 posterior collapse가 생겼다면 행동열이 비슷할 수 있다. 다양성을 주장하려면 실제 샘플별 행동열과 성공률을 평가해야 한다.

## 2.27 슬라이드 27 · ACT 구조를 입력과 출력으로 읽기

정책은 카메라 영상과 현재 관절 상태를 입력으로 받고, transformer가 이 정보를 결합해 미래의 관절 목표열을 출력한다. 영상은 물체와 공간의 배치를, 관절 상태는 현재 팔의 자세를 알려 준다. 둘 중 하나가 빠지면 "물체는 어디 있나" 또는 "팔은 지금 어디 있나"를 추측해야 한다.

출력은 한 숫자가 아니라 시간축이 있는 행동 행렬이다. 학습 손실을 읽을 때 어느 시간의 목표와 예측을 비교하는지, 패딩한 끝부분은 제외되는지, 양팔의 행동 차원이 어떻게 배열되는지 확인해야 한다. 이것이 구현에서 자주 생기는 조용한 오류다.

행동 텐서를 $B \times H \times d_a$로 쓰면 $B$는 batch 크기, $H$는 미래 길이, $d_a$는 한 시점의 행동 차원이다. 예를 들어 양팔이 각각 7개 관절 목표와 그리퍼 1개를 내면 $d_a = 16$이다. 행동 묶음 60개를 예측하면 한 예측의 출력은 $60 \times 16$이다. 이 차원을 시연 기록과 비교하면 시간축과 관절축을 바꾸는 실수를 잡기 쉽다.

## 2.28 슬라이드 28 · 50개 시연으로 만든 조작 정책

시연 50개와 높은 성공률이라는 숫자는 데이터 효율을 보여 주는 출발점이다. 그러나 이 성공률은 특정 물체, 초기 배치, 성공 판정, 반복 횟수에서 얻은 값이다. 새로운 컵, 다른 조명이나 위치에서도 같은 비율을 기대하려면 별도 평가가 필요하다.

화면의 작은 용기처럼 집기·뚜껑 열기·놓기 단계가 이어지는 작업은 단일 시점 행동보다 긴 행동 패턴이 중요하다. 성공 여부뿐 아니라 어느 단계에서 실패하는지 기록하면 action chunk의 길이와 관측 갱신 주기를 조정할 근거가 생긴다.

평가를 단계로 나누면 실패의 원인이 선명해진다. 접근까지 성공했지만 뚜껑을 여는 단계에서 실패한다면 행동 chunk의 길이보다 접촉 관측이나 그리퍼 제어가 문제일 수 있다. 반대로 매번 다른 방향으로 접근한다면 시연의 다봉성이나 잠재변수 샘플링을 살펴볼 만하다.

## 2.29 슬라이드 29 · 배터리 삽입의 정밀도

배터리를 슬롯에 넣는 작업은 집기, 목표로 이동, 방향 맞추기, 접촉 후 삽입으로 나뉜다. 마지막 삽입은 작은 자세 오차에 민감하다. 화면의 높은 성공률은 그 실험 설정에서 전체 연쇄가 잘 작동했다는 뜻이지만, 접촉력·부품 공차·초기 자세가 달라지면 난도가 달라진다.

이 사례는 정책의 평균 행동 오차만으로 성공을 설명하기 어려움을 보여 준다. 오차의 크기가 같아도 물체에 접근하는 초반인지, 슬롯에 닿는 마지막 순간인지에 따라 결과에 미치는 영향이 크게 다르다.

삽입 성공을 위해 필요한 허용 오차가 작을수록 전체 궤적의 평균 L2 오차보다 **마지막 접촉 시점의 오차**가 중요하다. 예를 들어 접근 구간에서 1cm 차이는 회복할 수 있어도 삽입 직전의 1cm 차이는 실패가 될 수 있다. 평가 지표를 최종 성공과 단계별 위치·자세 오차로 함께 나누는 이유다.

### 2.30 슬라이드 30 · Mobile ALOHA로 확장하기

탁자에 고정된 두 팔에서 이동식 플랫폼의 두 팔로 넘어가면 행동 공간이 넓어진다. 로봇은 손의 위치뿐 아니라 몸체를 어디로 움직일지 결정해야 한다. 이동이 팔의 도달 범위를 바꾸므로 주행과 조작을 따로 최적화한 정책만으로는 부족할 수 있다.

Mobile ALOHA는 전신 원격 조작으로 이동과 양팔 행동을 함께 시연한다. 데이터에는 바닥과 가구의 배치, 사람의 경로 선택, 작업 순서가 포함된다. 시스템의 장점과 한계를 보려면 새로운 방·물체·경로에서도 수행되는지 살펴야 한다. Mobile ALOHA 논문

이동식 양팔 로봇이 주방의 컵을 집는다면 베이스가 너무 멀면 팔이 닿지 않고, 너무 가까우면 팔이 움직일 공간이 없다. 따라서 베이스 이동은 단순한 선행 단계가 아니라 손의 작업 가능 영역을 바꾼다. 이동과 조작의 연계를 놓치면 같은 컵을 보더라도 어느 위치에서 집어야 할지 배울 수 없다.

### 2.31 슬라이드 31 · 원격 조작 시연의 역할

화장실 청소 장면은 사람이 조종기로 로봇을 움직이는 **시연 수집**을 보여 준다. 이 장면에서 사람이 로봇의 뒤를 따라가며 공간을 보고 조작하므로, 정책에 줄 관측이 사람의 정보와 같은지 확인해야 한다. 그렇지 않으면 앞서 본 관측 불일치가 재현된다.

이동·양팔 조작의 시연은 긴 작업을 여러 짧은 접촉 행동으로 분해하는 데도 도움이 된다. 그러나 사람의 안전 판단이나 목적 순서는 영상과 관절 로그만으로 완전히 전달되지 않을 수 있다. 목표 설명과 실패 상태까지 데이터에 담는 설계가 중요하다.

시연 영상에서 사람이 로봇 옆에서 상황을 보며 조종하는 경우, 그 사람이 본 물체나 장애물이 로봇의 카메라에는 가려질 수 있다. 시연을 학습 자료로 바꾸려면 로봇의 관측 시점에서 행동을 설명할 수 있는지 다시 확인해야 한다. 사람의 손 조작 자체보다 **사람과 로봇의 정보 차이**가 모방 정책의 한계가 될 수 있다.

### 2.32 슬라이드 32 · 자율 실행의 평가

요리 장면은 사람이 직접 조작하는 시연과 구별되는 자율 실행 사례다. 성공 장면 하나는 가능한 행동을 보여 주지만 일반적인 성공 확률이나 안전성을 알려 주지는 않는다. 다양한 초기 배치에서 반복 실행하고, 중간 실패와 사람 개입 횟수를 함께 기록해야 한다.

긴 작업에서는 초반 이동 오차가 뒤의 손 조작을 어렵게 만들 수 있다. 따라서 결과를 볼 때 최종 성공만이 아니라 접근, 집기, 운반, 놓기의 단계별 오류를 분리하면 어떤 관측과 정책 갱신이 필요한지 알 수 있다.

긴 과업의 성공률은 각 하위 단계의 성공률이 쌓여 결정된다. 접근, 집기, 운반, 놓기의 조건부 성공률이 각각 0.9이고 모두 성공해야 한다는 단순 가정이면 전체 성공률은 $0.9^4 \approx 0.66$이다. 최종 성공률이 낮아도 어느 한 단계만 나쁜 것인지 여러 단계에서 작은 실패가 누적되는 것인지 구분해야 한다.

### 2.33 슬라이드 33 · Diffusion Policy로 넘어가기

한 상태에서 여러 행동 경로가 모두 맞는다면 정책이 하나의 평균 행동만 내는 표현은 부족하다. Diffusion Policy는 조건부 생성 모델로 미래 행동열의 분포를 표현한다. 관측을 조건으로 둔 채 무작위 노이즈에서 출발해 여러 번 정제하여 하나의 행동열을 만든다.

앞의 ACT도 행동 묶음과 잠재변수로 시간적 일관성과 다양성을 다룬다. 두 방법의 공통 목적은 한 시점의 평균 행동이 아니라 **실행 가능한 행동 궤적**을 만드는 것이다. 차이는 행동 분포를 표현하고 샘플링하는 방식에 있다. Diffusion Policy 논문

ACT의 잠재변수는 한 번의 인코딩·디코딩으로 행동열을 만들고, Diffusion Policy는 초기 노이즈를 여러 번 정제해 행동열을 만든다. 둘 다 여러 유효 경로를 표현하려 하지만 추론 비용과 분포 표현

방식이 다르다. 이 비교를 할 때 모델의 복잡성뿐 아니라 로봇이 1초에 몇 번 새 행동을 요구하는지도 고려해야 한다.

### 2.34 슬라이드 34 · 세 실패 원인을 다시 연결하기

Diffusion Policy를 읽기 전, 모방학습의 세 난점을 다시 구분하자. 오류 누적은 정책이 방문하는 상태의 문제다. 다봉 시연은 같은 상태에서 여러 행동이 가능한 문제다. 관측 불일치는 결정에 필요한 정보가 입력에 없는 문제다.

확산 모델은 주로 두 번째 문제, 즉 여러 행동 경로를 하나의 평균으로 무너뜨리지 않고 표현하는 데 강점이 있다. 미래 행동을 묶어 만들면 짧은 시간의 일관성도 얻지만, 데이터에 없는 상태를 자동으로 해결하거나 숨겨진 정보를 만들어 내지는 못한다.

확산 정책도 시연에서 보지 못한 상태를 저절로 해결하지는 못한다. 예를 들어 훈련 때 컵이 항상 손잡이를 오른쪽으로 두었다면, 손잡이가 뒤로 돌아간 배치에서는 여러 행동을 샘플링해도 모두 실패할 수 있다. 행동 분포를 풍부하게 표현하는 일과 상태 coverage를 넓히는 일은 별개의 과제다.

### 2.35 슬라이드 35 · 노이즈에서 미래 행동열로

현재 관측 $o_t$에서 길이 $H$의 미래 행동열 $A_t = (a_t, \dots, a_{t+H-1})$을 예측한다. $A^{(0)}$은 깨끗한 시연 행동열이고 $A^{(k)}$는 $k$번째 노이즈 수준으로 오염시킨 행동열이다. $K$는 가장 큰 노이즈 수준, $\epsilon^{(k)}$는 수준 $k$에서 실제로 더한 잡음, $\epsilon_\theta$는 파라미터 $\theta$인 노이즈 예측망이다. 학습과 생성의 방향을 쓰면

$$\epsilon_\theta(A^{(k)}, o_t, k) \approx \epsilon^{(k)}, \qquad A^{(K)} \sim \mathcal{N}(0, I) \longrightarrow A^{(0)}.$$

학습식도 구체적으로 보자. 시연 데이터의 관측–행동열 $(o_t, A^{(0)})$을 뽑고 노이즈 수준 $k \in \{1, \dots, K\}$를 임의로 정한다. 수준 $k$의 잡음 $\epsilon^{(k)} \sim \mathcal{N}(0, \sigma_k^2 I)$를 더해 $A^{(k)} = A^{(0)} + \epsilon^{(k)}$를 만든다. $\sigma_k$는 그 수준의 노이즈 표준편차, $I$는 행동열 차원의 항등행렬이다. 한 표본의 제곱오차를 평균하면

$$\mathcal{L}(\theta) = E_{(o_t, A^{(0)}),, k, \epsilon^{(k)}} \left[ \left\| \epsilon^{(k)} - \epsilon_\theta(A^{(k)}, o_t, k) \right\|_2^2 \right].$$

왜 노이즈를 맞히는지 조건부 기댓값으로 이해할 수 있다. 오염된 열 $x = A^{(k)}$와 관측 $o_t$, 수준 $k$를 고정하고 예측값을 $f$라 하면 $E[\|\epsilon^{(k)} - f\|_2^2 \mid x, o_t, k]$의 최소점은 $f^* = E[\epsilon^{(k)} \mid x, o_t, k]$다. 이는 제곱을 전개해 $f$로 미분하면 $2\{f - E[\epsilon^{(k)} \mid x, o_t, k]\} = 0$이 되기 때문이다. 모델은 현재 오염된 열에서 제거할 잡음의 평균 방향을 학습한다. Diffusion Policy 연구

전체 행동열을 함께 생성하므로 시간상 맞지 않는 움직임을 줄일 수 있고, 서로 다른 시연 모드도 확률분포로 유지할 수 있다. 여러 정제 단계의 계산 지연은 제어 주기와 함께 고려해야 한다.

실행에서는 $A^{(K)}$에서 시작해 $k$를 줄이며 예측 노이즈를 제거한다. 생성한 열의 첫 몇 행동을 실행하고 다시 관측해 새 열을 만들면 **receding horizon** 제어가 된다. 따라서 행동열의 전체 길이 $H$와 재관측까지 실제로 실행하는 행동 수는 서로 다를 수 있다.

**무작위 노이즈에서 시작하는데 왜 행동이 무작위로 끝나지 않을까?** 학습된 모델이 관측과 노이즈 수준에 맞춰 매 단계 행동열을 시연 분포 쪽으로 정제하기 때문이다. 시작 노이즈는 여러 가능한 경로를 샘플링하는 재료이며, 학습된 정제 규칙이 없다면 유효한 행동으로 바뀌지 않는다.

### 2.36 슬라이드 36 · 결과 영상의 해석

집기, 붓기, 물체를 밀어 형태를 만드는 영상은 시각 관측에서 연속 행동을 생성해 물리적 작업을 수행할 수 있음을 보여 준다. 그러나 영상은 대표적인 성공 사례다. 과업별 성공률, 난도, 실패 유형은 논문의 반복 평가를 확인해야 한다.

확산 정책의 강점을 판단하려면 "한 번 성공했는가"보다 다양한 시작 배치에서 같은 목표를 달성하는지, 관측 잡음이나 접촉 변화에 얼마나 회복하는지, 한 번의 행동열 생성에 얼마나 걸리는지를 함께 봐야 한다. 실험 설명

영상 속 성공 장면을 평가할 때는 첫 시도가 성공했는지, 여러 후보 중 성공한 장면을 골랐는지, 목표 물체의 초기 위치를 누가 맞춰 주었는지도 확인하자. 특히 액체를 붓거나 물체를 미는 작업은 접촉 뒤의 미세한 차이가 커지므로, 반복 횟수와 초기 조건을 함께 공개해야 재현 가능한 성능이 된다.

### 2.37 슬라이드 37 · 로봇 파운데이션 모델의 조건부 정책

과업이 하나라면 $\pi(a \mid s)$로 충분할 수 있다. 여러 과업을 하나의 모델이 수행하려면 현재 관측 $s$와 함께 "셔츠를 접어라" 같은 과업 지시 $z$가 필요하다. 이를 $\pi(a \mid s, z)$라고 쓴다. 같은 옷이 눈앞에 있어도 접기와 치우기는 다른 행동을 요구한다.

지시가 텍스트인지 목표 이미지인지, 행동이 관절 명령인지 말단 위치인지에 따라 일반화의 의미도 달라진다. 로봇 파운데이션 모델은 다양한 데이터로 공통 표현을 배우려 하지만, 새 몸체나 낯선 환경에서의 성능은 별도로 평가해야 한다.

조건 $z$가 빠지면 동일한 상태에서 "컵을 치워라"와 "컵에 물을 따라라"가 상충하는 라벨이 된다. 텍스트 지시를 붙이면 적어도 두 목표를 구별할 수 있다. 하지만 문장을 이해하는 것과 손목을 몇 도 회전해야 하는지는 다른 문제다. 다양한 지시를 처리하는 능력과 물리적 조작 성공률을 따로 평가해야 한다.

### 2.38 슬라이드 38 · DROID와 환경 다양성

DROID는 가정과 연구실 등 다양한 실제 공간에서 로봇 조작 궤적을 모은 데이터셋이다. 슬라이드에 제시된 수만 episode, 수백 장면, 여러 건물과 기관의 숫자는 **규모뿐 아니라 환경의 폭**을 강조한다. 동일한 작업도 욕실·주방·사무실에서 배경과 물체 위치가 달라진다.

이런 데이터는 한 연구실의 고정 배경에 과적합하는 문제를 줄일 가능성이 있다. 하지만 장소 수가 많다는 사실만으로 모든 물체와 작업을 덮지는 못한다. 학습 장소와 평가 장소를 분리하고, 성공률을 작업 종류별로 살펴야 데이터의 실제 효과를 알 수 있다. DROID 논문

집 안의 밝은 주방에서 수집한 집기와 어두운 연구실의 집기는 물체가 같아도 영상 분포가 다르다. DROID처럼 수집 장소를 넓히는 이유는 이 변동을 데이터에 담기 위해서다. 평가할 때는 훈련에 있던 장소의 다른 물체인지, 아예 새로운 장소인지 분리해야 **장소 일반화**의 의미가 분명해진다.

### 2.39 슬라이드 39 · Open X-Embodiment와 로봇 몸체의 차이

Open X-Embodiment는 여러 연구 그룹의 로봇 데이터셋을 모아 서로 다른 로봇 몸체와 과업을 함께 학습할 수 있게 한다. 카메라 높이, 팔 길이, 관절 수, 그리퍼 형태가 다른 데이터에서 공통 구조를 찾는 것은 단일 로봇보다 어렵다.

예를 들어 한 로봇의 행동이 7개 관절 각도이고 다른 로봇은 말단 위치 명령이라면 숫자 벡터를 단순히 이어 붙일 수 없다. 각 몸체에 맞는 행동 표현이나 토큰화, 관측 정규화, 데이터 출처의 균형이 필요하다. "데이터가 100만 episode"라는 규모와 "새 몸체에 전이된다"는 성능은 서로 다른 주장이다. OXE 논문

**여러 로봇의 데이터를 한데 모으면 항상 더 좋아질까?** 아니다. 행동 단위와 좌표계가 맞지 않거나 특정 로봇의 데이터가 지나치게 많으면 음의 전이가 생길 수 있다. 공통 표현을 설계하고 로봇별 평가를 분리해야 데이터 혼합의 효과를 확인할 수 있다.

한 로봇의 "그리퍼 닫기"가 0~1의 개방도이고 다른 로봇은 힘 명령이라면 두 값을 같은 행동 차원에 넣어도 물리적 의미가 맞지 않는다. 공통 행동 표현을 만들거나 몸체별 action head를 두는 설계가

필요한 이유다. 여러 몸체의 데이터를 섞는 효과는 새 몸체에서 별도 학습 없이 되는지, 소량 적응이 필요한지까지 나누어 판단한다.

### 2.40 슬라이드 40 · $\pi_0$의 자율 수행 장면

화면의 자율 조작 장면은 시각 입력과 작업 지시에서 실제 로봇 행동까지 이어지는 모델의 목표를 보여준다. 한 번의 데모 장면은 모델이 수행할 수 있는 행동의 예시지만, 새로운 집·물체·로봇 몸체에서도 같은 수준인지 입증하려면 분리된 평가가 필요하다.

$\pi_0$는 사전학습된 시각·언어 표현을 로봇 행동 생성과 연결한다. 언어를 이해하는 능력과 정확한 접촉 조작을 하는 능력은 다르므로, 큰 VLM만 붙인다고 로봇 제어가 해결되지는 않는다. 행동 데이터와 제어 방식이 함께 중요하다. 원 논문

시연 영상에서 로봇이 셔츠를 접었다면 먼저 지시 이해, 옷의 위치 인식, 양팔의 연속 제어, 최종 성공 판정 중 무엇이 공개된 실험으로 검증되었는지 살펴야 한다. 한 번의 자율 실행은 네 요소가 특정 조건에서 함께 작동했다는 사례다. 새 종류의 옷이나 다른 탁자에서도 작동하는지는 별도 평가 항목이다.

### 2.41 슬라이드 41 · 사전학습과 과업별 후속 학습

$\pi_0$의 데이터 구성은 다양한 작업에서 넓은 기본 행동 능력을 배우는 단계와, 특정 과업의 시연으로 조정하는 단계를 구분한다. 슬라이드에는 대규모 사전학습 시간과 과업별 수십 시간 규모의 후속 데이터가 제시된다. 데이터 시간이 모델이 실제 로봇에서 연속으로 일한 시간과 같다는 뜻은 아니며 여러 로봇의 기록을 합산한 양이다.

새 과업에서 필요한 시연이 적어진다면 사전학습이 유용하다고 볼 수 있다. 이를 판단할 때는 후속 학습 데이터의 수와 질, 새 환경과 물체에 대한 시험 조건을 함께 비교해야 한다.

사전학습의 가치를 비교하려면 같은 새 과업에 두 정책을 놓고 시연량을 0, 10, 50개처럼 바꾸어 학습 곡선을 볼 수 있다. 사전학습 정책이 적은 시연에서 빨리 성공하지만 충분한 시연 뒤에는 차이가 작아질 수도 있다. 한 점의 성공률보다 **시연량에 따른 적응 곡선**이 전이의 이득을 더 분명하게 보여준다.

### 2.42 슬라이드 42 · VLM과 행동 전문가의 연결

그림의 앞부분은 카메라 영상과 언어 지시를 표현하는 VLM이고, 뒤의 action expert는 현재 행동과 노이즈를 조건으로 미래 행동열을 만든다. 서로 다른 로봇의 관절 수와 행동 차원을 다루기 위해 여러 플랫폼의 시연을 조합한다. 논문은 행동 생성을 flow matching으로 모델링한다.

슬라이드의 모델 크기, GPU 수, 학습 기간과 비용은 해당 연구의 보고 설정을 이해하기 위한 수치다. 비용은 하드웨어 임대료와 구현 조건에 따라 달라진다. 중요한 구조적 질문은 **일반 시각·언어 표현이 구체적인 로봇 행동으로 어떻게 이어지는가**, 그리고 그 접속부에 어느 정도의 로봇 데이터가 필요한가다.

**$\pi_0$의 flow matching과 Diffusion Policy는 같은 알고리즘인가?** 둘 다 노이즈에서 행동을 생성하는 계열이지만 학습 목표와 샘플링의 수학적 표현은 다르다. Diffusion Policy는 노이즈 제거 과정을, $\pi_0$는 연속 흐름의 벡터장을 학습한다. 여러 유효 행동열을 생성한다는 공통 목적만으로 업데이트 식을 같게 보면 안 된다.

동일한 장면에서도 VLM은 "빨간 컵을 집어라"의 대상과 문맥을 표현하고, action expert는 관절과 그리퍼가 시간에 따라 어떻게 움직일지 정한다. 목표 물체를 맞히고도 손이 닿지 않으면 행동 부분의 문제이고, 손 제어가 정확해도 파란 컵을 잡으면 조건 해석의 문제다. 실패를 둘로 나누면 추가 데이터가 어느 쪽에 필요한지 판단할 수 있다.

### 2.43 슬라이드 43 · 로봇 데이터와 텍스트 데이터의 규모 비교

그림은 로봇 행동 기록의 시간을 텍스트 토큰의 양과 거칠게 비교한다. 이는 로봇 학습에 쓸 수 있는 물리적 경험이 언어모델의 텍스트 사전학습 자료보다 훨씬 적다는 직관을 준다. 하지만 "시간"과 "토큰"은 같은 정보 단위가 아니다. 영상·관절·접촉 정보가 한 초에 많이 들어 있고, 서로 상관된 프레임도 많다.

따라서 크기 원만 보고 모델의 데이터 효율을 직접 비교하면 안 된다. 중요한 질문은 새로운 환경과 물체를 얼마나 덮는지, 하나의 시연에 유용한 행동 변화가 얼마나 있는지, 실패·복구 경험을 포함하는지다.

로봇 영상 1시간이 수십만 프레임이라고 해도 인접 프레임 대부분은 같은 장면이다. 반면 접촉 직전과 직후의 짧은 구간은 새로운 정보를 많이 줄 수 있다. 단순한 초 단위 규모를 넘어 **상태 다양성, 행동 변화, 실패 후 복구**가 얼마나 포함되는지 세는 방법이 필요하다.

### 2.44 슬라이드 44 · 데이터 수집 규모의 현실

여러 로봇과 사람이 동시에 시연을 모으는 장면은 대규모 로봇 데이터의 생산 비용을 드러낸다. 텍스트처럼 이미 존재하는 자료를 모으는 것과 달리 물리적 시연에는 장비, 조작자, 물체 준비, 고장 복구와 안전 관리가 필요하다.

데이터 규모를 늘릴 때는 같은 작업을 반복하는 양과 새로운 물체·장소·실패 상황을 넓히는 다양성을 구분해야 한다. 같은 탁자에서 수만 번 집기를 반복하면 그 탁자에는 강해질 수 있어도 다른 집에 바로 일반화되지는 않는다.

데이터 수집의 병목을 예로 들면 로봇 10대를 하루 8시간 운용해도 매 시도가 같은 성공 시연으로 남지는 않는다. 배터리 교체, 물체 재배치, 센서 오류 확인, 사람의 개입 시간이 들어간다. 그래서 기록 시간과 학습에 실제 사용할 수 있는 정렬된 궤적 시간은 다르다. 데이터 규모 주장에는 품질 관리 절차가 따라야 한다.

### 2.45 슬라이드 45 · 가정 환경의 긴 작업

식탁 위의 그릇과 잔을 다루는 작업은 단일 집기보다 긴 계획을 요구한다. 물체를 어떤 순서로 잡을지, 이미 놓은 물체와 충돌하지 않을지, 사람이 가까이 올 때 어떻게 멈출지를 결정해야 한다. 화면의 한 장면은 이처럼 긴 지평에서 행동과 상황이 얽히는 문제를 떠올리게 한다.

로봇 영상은 모델·조종 방식·반복 성공률을 모두 알려 주지 않는다. 자율성의 수준과 데이터 수집 절차를 확인해야 어떤 부분이 정책의 성과인지 평가할 수 있다.

식탁 정리에서 첫 컵을 잡는 행동은 다음 물체에 손이 닿을 공간을 만들거나 막는다. 따라서 개별 집기의 성공률을 높여도 전체 정리가 실패할 수 있다. 긴 작업에서는 하위 작업을 어떤 순서로 고르는지, 중간 실패 뒤에 다시 계획하는지가 추가된다. 이런 부분은 한 번의 행동 묶음 생성만으로 설명하기 어렵다.

### 2.46 슬라이드 46 · ACT-1이라는 이름을 해석할 때

화면의 ACT-1은 별도의 로봇 파운데이션 모델 이름이다. 앞서 본 **Action Chunking with Transformers(ACT)**와 약자가 같아도 동일한 알고리즘이라는 뜻은 아니다. 이름보다 공개된 입력·출력, 학습 데이터, 평가 과업을 확인해야 한다.

자료는 긴 지평 과업, 처음 보는 상황의 일반화, 손재주라는 세 목표를 제시한다. 각각은 평가가 다르다. 긴 지평은 단계 사이의 오류 축적을, 일반화는 훈련 밖 장면을, 손재주는 접촉과 정밀도를 시험한다. 한 데모로 세 목표가 모두 입증되지는 않는다.

이름이 비슷한 모델을 비교할 때는 입력이 한 장의 영상인지 여러 시점의 기록인지, 출력이 관절 행동열인지 상위 과업 명령인지 먼저 적자. 같은 ACT 표기라도 이 세 항이 다르면 성능 숫자를 직접 비교할 수 없다. 연구 이름보다 **정책이 실제로 예측하는 변수**를 읽는 습관이 중요하다.

## 2.47 슬라이드 47 · 다양성을 시각화한 데이터 지도

서로 다른 조작 과업의 행동 기록을 2차원에 투영한 그림은 데이터에 여러 종류의 행동이 있다는 직관을 준다. 점의 색은 작업 범주이고 위치는 고차원 표현의 유사성을 간략히 보여 준다. t-SNE 같은 투영은 가까운 군집의 모양을 보기 좋지만, 먼 점 사이의 거리를 원래 행동 공간의 정확한 거리로 해석하면 안 된다.

이 그림을 모델 성능 그래프로 읽어서도 안 된다. 데이터에 조립·정리·청소가 함께 있다는 사실과 새 작업을 성공적으로 수행한다는 사실 사이에는 학습·평가 단계가 필요하다.

2차원 투영에서 두 작업의 점들이 겹친다고 두 작업에 같은 정책 행동을 써도 된다는 뜻은 아니다. 투영 과정은 많은 차원을 버리고, 가까운 영상 특징과 다른 관절 목표가 공존할 수 있다. 그림은 데이터 범주의 위치를 탐색하는 도구이며, 일반화 능력은 훈련에서 분리한 과업의 성공률로 확인해야 한다.

## 2.48 슬라이드 48 · 새로운 로봇 플랫폼에서의 조작

서로 다른 두 팔과 그리퍼가 같은 장면에서 일하는 영상은 로봇 몸체 간 전이를 생각하게 한다. 하나의 정책이 여러 몸체를 제어하려면 관측뿐 아니라 행동이 각 로봇에서 무엇을 뜻하는지 맞춰야 한다. 팔 길이와 관절 제한, 그리퍼 닫힘 명령의 단위가 다르면 같은 숫자 행동을 그대로 복사할 수 없다.

실험을 평가할 때는 새 물체인지, 새 방인지, 새 몸체인지 전이 대상을 분리하자. 세 경우의 난도와 필요한 데이터가 다르다. 영상의 “fully autonomous” 표기는 사람의 실시간 조작 여부를 말하지만 학습 과정에 사람이 제공한 시연이 없었다는 뜻은 아니다.

새 플랫폼 전이의 장난감 예를 들면 관절 7개 로봇에서 배운 $[a_1, \ldots, a_7]$을 관절 6개 로봇에 그대로 보낼 수 없다. 말단 위치와 방향처럼 공통 목표를 예측하고 각 로봇의 역기구학으로 변환할 수 있지만, 그 과정에서도 도달 범위와 관절 제한은 다르다. 따라서 공통 정책의 출력 표현과 플랫폼별 변환 규칙을 함께 공개해야 한다.

## 2.49 슬라이드 49 · 모방학습의 도달 범위

모방학습은 좋은 시연이 있을 때 강한 출발점이다. 그러나 전문가가 할 수 없는 행동의 시연을 얻을 수 없고, 시연이 방문하지 않은 상태에서 스스로 시행착오를 통해 고치는 틀이 부족하다. 전문가의 평균 성과를 넘어서는 새로운 전략을 찾는 데도 보상 정보가 필요할 수 있다.

“시연보다 절대 나아질 수 없다”는 말을 수학적 불가능 정리로 읽어서는 안 된다. 잡음 있는 시연에서 평균적으로 더 안정적인 동작을 배울 수도 있다. 핵심은 **순수한 행동 복제 목적함수에는 전문가보다 더 좋은 결과를 구별하는 신호가 없다**는 것이다. 다음 장의 RL은 바로 그 결과 신호를 다룬다.

**BC 정책이 전문가보다 높은 점수를 얻었다면 모방학습의 한계가 틀린 것일까?** 가능한 현상이다. 정책이 여러 시연의 잡음을 평활화하거나 평가 표본의 우연성으로 더 좋은 점수를 받을 수 있다. 다만 BC 손실 자체는 결과 보상을 직접 최적화하지 않으므로 일관된 초과 성능의 원인을 별도로 확인해야 한다.

시연 데이터에 있는 모든 행동이 좋은 행동만은 아니다. 사람이 느리게 작업하거나 잠깐 헤맸다면 BC는 그 순간도 따라 할 수 있다. 그러나 실제 성공 시간이나 에너지 소비를 보상으로 측정할 수 있다면 더 나은 경로를 찾는 기준이 생긴다. 이런 개선에는 새 경로를 실행해 결과를 알아야 하므로 안전성과 상호작용 비용을 함께 고려해야 한다.

### 2.50 슬라이드 50 · 보상으로 배우는 단계

지금까지는 “이 상태에서 전문가가 무엇을 했는가”를 데이터로 받았다. 앞으로는 “내 행동이 어떤 결과를 냈는가”를 보상으로 받는다. 전문가가 없어도 학습할 가능성이 생기지만, 좋은 행동을 찾기 위한 탐색과 뒤늦은 보상을 앞선 행동에 연결하는 문제가 새로 생긴다.

다음 장에서는 상태·행동·전이·보상·할인율을 갖춘 Markov decision process를 정의한다. 이 틀 위에서 모방학습과 강화학습의 차이, 그리고 정책과 가치함수의 의미가 정확해진다.

예를 들어 두 조작 행동이 시연과 얼마나 비슷한지와 관계없이 하나는 컵을 안전하게 놓고 다른 하나는 떨어뜨렸다면 결과 기반 피드백은 둘을 구별한다. 다음 장에서 $r_t$와 누적 반환을 도입하는 이유가 여기에 있다. 동시에 보상을 잘못 정의하면 의도하지 않은 행동을 강화할 수 있다는 점도 처음부터 함께 기억하자.

# 3 02-1. 강화학습의 수학적 기초

모방학습은 전문가 행동이라는 목표값에서 출발했다. 이제 목표값 없이 환경의 결과를 평가할 수 있는 문제를 정의한다. **Markov decision process(MDP)**는 상태, 행동, 전이, 보상과 시간에 대한 선호를 하나의 공통 언어로 묶는다. 이후 정책경사와 Q-learning은 모두 같은 MDP를 다른 방식으로 푼다.

## 3.1 슬라이드 01 · 강화학습의 수학적 기초

강화학습은 "좋은 행동을 선택한다"는 직관만으로 알고리즘을 만들기 어렵다. 어떤 상태에서 행동을 골랐는지, 결과가 어느 확률로 나오는지, 보상을 언제 받는지를 명시해야 기대 성과를 계산할 수 있다.

이 장의 목적은 현실을 완벽하게 복제하는 수식을 만드는 것이 아니라 **서로 다른 알고리즘이 공통으로 다루는 문제를 분명히 정의하는 것**이다. 환경 모델을 실제로 알고 있어야 MDP로 쓸 수 있는 것은 아니다. 수학적 정의와 에이전트가 가진 지식을 구분하자.

예를 들어 로봇이 컵을 잡는 상황에서 물체 위치만 기록하면 팔의 속도나 손가락의 접촉 상태가 빠진다. 알고리즘이 틀렸는지 판단하기 전에 상태가 미래 결과를 설명할 수 있을 만큼 충분한지 확인해야 한다. MDP의 형식화는 현실을 추상화하면서도 **어떤 정보를 빼도 되는가**를 질문하게 만든다.

## 3.2 슬라이드 02 · 복습에서 형식화로

앞에서 전문가 시연 $(s, a^*)$로 정책을 맞혔다. 이제는 행동의 정답이 없고, 대신 상태 $s_t$에서 고른 $a_t$가 다음 상태와 보상을 만든다고 가정한다. 보상이 즉시 주어지더라도 목표는 대개 한 단계가 아니라 여러 단계의 합이다.

모방학습의 실패 원인을 복습하는 이유도 여기에 있다. 정책은 자기가 만든 상태에 들어가므로 데이터 수집 분포를 바꾼다. MDP는 그 분포 변화를 전이 $P(s_{t+1} \mid s_t, a_t)$로 명시한다.

시연 정책이 조금 오른쪽으로 움직인 결과, 다음 시점에는 전문가 기록에 거의 없던 장애물 배치가 보일 수 있다. 전이 $P(s_{t+1} \mid s_t, a_t)$를 쓰면 **행동이 다음 입력의 분포를 만든다**는 말을 확률식으로 옮길 수 있다. 이후 모든 알고리즘은 이 전이에 직접 모델을 세우거나, 표본만 이용하거나, 그 영향을 가치함수에 접어 넣는다.

## 3.3 슬라이드 03 · 평가 요소와 개념 학습

구현 과제와 시험은 다른 방식으로 같은 이해를 요구한다. 예컨대 환경에서 반환한 reward와 episode 종료 신호를 잘못 처리하면 코드가 실행돼도 Bellman 목표는 틀린다. 반대로 수식만 알고 환경 API의 반환값을 모르면 경험 데이터를 올바르게 쌓을 수 없다.

이 장에서는 상태·행동·보상·전이의 기호를 실제 코드의 변수와 대응시키며 읽자. 수학의 $s_{t+1}$과 환경이 반환하는 다음 관측이 언제 같은지 확인하는 습관이 중요하다.

하나의 환경 전이를 코드와 식에 동시에 표시해 보자. `obs`가 $s_t$, `action`이 $a_t$, `step` 뒤의 `next_obs`가 $s_{t+1}$, `reward`가 $r_t$다. 다음 관측을 현재 상태로 잘못 저장하면 Bellman 목표의 $Q(s_{t+1}, a')$ 대신 엉뚱한 값을 쓰게 된다. 표본의 시간 순서를 맞추는 일이 수식 이해의 일부다.

## 3.4 슬라이드 04 · 시험에서 MDP를 설명하는 법

MDP를 묻는 문제에는 다섯 가지 이름만 적기보다 각각의 역할을 말해야 한다. $S$는 가능한 상태, $A$는 행동, $P$는 상태 전이, $r$은 보상, $\gamma$는 시간에 따른 할인율이다. 초기 상태 분포 $p_0$와 종료 조건을 추가하면 episode가 어떻게 시작하고 끝나는지도 분명해진다.

예를 들어 로봇 보행에서 $S$를 카메라 영상이라고만 하면 속도와 접촉 상태가 빠질 수 있다. 그런 표현이 Markov인지 판단하는 것이 단순 정의 암기보다 중요한 시험과 실습 질문이다.

MDP를 정의할 때 $S$와 $A$를 적은 뒤 하나의 실제 전이를 만들어 보자. 예를 들어 상태가 "교차로에서 정지", 행동이 "우회전", 다음 상태가 "오른쪽 차선 진입", 보상이 안전하게 진입했을 때 $+1$일 수 있다. 같은 행동이 다른 교통 상황에서는 다른 전이를 만들므로 $P$는 일반적으로 확률적이다.

### 3.5 슬라이드 05 · 출석과 연속적인 학습

강화학습 개념은 앞 정의를 뒤 수식에서 반복 사용한다. 정책경사를 보려면 trajectory 확률이, Bellman 방정식을 보려면 상태·행동·보상의 조건부 관계가 필요하다. 중간에 빠진 개념이 있으면 알고리즘마다 그 정의를 다시 확인해야 한다.

학습 기록에는 실행 결과만이 아니라 어떤 전이와 반환 정의를 사용했는지도 적자. 예컨대 종료 직전의 next state 가치를 0으로 둘지 여부는 나중의 Q-learning 구현에 직접 영향을 준다.

뒤 장에서 기호가 헷갈리면 이 장의 세 구분으로 돌아오면 된다. $r_t$는 지금 관측된 한 보상이고, $G_t$는 그 시점부터의 실제 누적 반환이며, $V^{\pi}(s)$는 상태가 $s$일 때 가능한 반환의 **기대값**이다. 하나의 표본, 표본의 합, 여러 가능한 표본의 평균을 섞어 쓰면 가치함수 유도가 불분명해진다.

### 3.6 슬라이드 06 · 질문으로 확인할 수 있는 가정

"현재 상태만으로 미래를 예측할 수 있나?"는 Markov 가정을 확인하는 질문이다. "같은 행동이 항상 같은 결과를 내나?"는 전이가 결정론적인지 묻는다. "보상이 없던 앞선 행동은 어떻게 평가하나?"는 반환과 가치함수로 이어진다.

좋은 질문은 가능한 반례를 함께 제시한다. 예컨대 공의 위치만 상태에 넣으면 같은 위치에서 오른쪽으로 움직이는 공과 왼쪽으로 움직이는 공의 다음 위치가 달라진다. 속도나 이전 위치가 더 필요하다.

반례를 만들 때는 두 궤적이 같은 현재 상태로 기록되었는데 다음 결과가 과거에 따라 달라지는지 확인하자. 앞의 공 예에서는 현재 위치가 같아도 이전 위치가 다르면 속도가 달라진다. 이 반례는 "과거가 중요하니 Markov 모델을 쓰면 안 된다"가 아니라 "현재 상태에 속도 정보를 넣어야 한다"는 수정 방향을 알려 준다.

### 3.7 슬라이드 07 · 자료를 사용할 때의 순서

운영 공지와 알고리즘 참고 문헌은 역할이 다르다. 과제 실행 환경·제출 형식은 최신 공지에서, MDP의 정의와 정책·가치함수의 표준 표기는 교재에서, 특정 알고리즘의 개선 이유와 성능은 원 논문에서 확인한다.

같은 이름의 환경이라도 버전이 바뀌면 관측·보상·종료 처리가 달라질 수 있다. 실험 숫자를 다른 자료와 비교하려면 환경 버전과 평가 방법을 먼저 맞추어야 한다.

자료마다 보상을 $r(s)$, $r(s,a)$, $r(s,a,s')$로 정의할 수 있다. 셋은 같은 숫자를 쓴다는 뜻이 아니라 어느 변수까지 보상이 의존하는지를 명시한다. 식을 비교하기 전에 지금 보고 있는 보상이 행동 전의 상태 보상인지, 전이가 끝난 뒤 얻는 보상인지 맞추어야 한다.

### 3.8 슬라이드 08 · 교재와 현대 알고리즘 사이

Sutton과 Barto의 교재는 Markov 과정과 Bellman 방정식의 기반을 제공한다. CS 224R과 CS 285의 현대 강의 자료는 신경망으로 정책이나 가치를 근사할 때의 구현 선택을 보여 준다. 두 수준을 연결하면 "이론에서는 정확한 기대값, 코드에서는 유한한 rollout 표본"이라는 차이를 이해할 수 있다.

뒤의 정책경사는 trajectory의 기대값을 표본 평균으로 바꾸고, Q-learning은 Bellman 기대식을 데이터의 목표값으로 바꾼다. 어떤 근사가 들어갔는지 표시하며 읽는 것이 핵심이다.

이론의 합 $\sum_{s'} P(s' \mid s,a)V(s')$는 다음 상태의 모든 가능성을 알고 계산할 때 정확하다. 현실의 한 전이 $(s,a,r,s')$는 그 합에서 뽑힌 표본 하나다. 신경망 알고리즘의 TD 목표가 유한한 표본으로 만들어지는 이유와 그 때문에 추정 오차가 남는 이유를 이 차이에서 이해할 수 있다.

### 3.9 슬라이드 09 · 모방학습을 되짚는 이유

모방학습은 전문가 행동을 바로 알고 있을 때의 학습이다. 그 장점은 보상 설계나 탐색 없이 정책을 시작할 수 있다는 것이고, 한계는 시연이 닿지 않은 상태와 시연보다 좋은 행동을 직접 평가하기 어렵다는 것이다.

RL 형식화는 이 한계를 보상과 전이로 표현한다. 정책이 어떤 상태 분포를 만들고 그 결과로 보상을 얼마나 받는지를 목적에 넣으면, 시연의 행동 일치율과 실제 성공률이 달라지는 이유를 수학적으로 말할 수 있다.

모방 정책이 전문가와 95%의 행동 일치율을 보여도 성공률이 낮다면 두 가지를 물어야 한다. 틀린 5%가 접촉 직전 같은 중요한 시점에 몰렸는가? 첫 실수 뒤 정책이 훈련 데이터에 없는 상태로 들어갔는가? MDP는 두 질문을 보상 $r$과 방문 분포 $d_\pi$로 각각 표현하게 해 준다.

### 3.10 슬라이드 10 · 결정 문제의 최소 구성

정책 $\pi_\theta(a \mid s)$는 현재 상태에서 행동을 고른다. 그러나 "좋은 행동"은 상태와 행동만으로 정해지지 않는다. 행동이 이후 상태를 어떻게 바꾸는지와 그 상태의 보상이 있어야 평가할 수 있다.

같은 "앞으로 한 걸음"도 낭떠러지 앞에서는 나쁘고 안전한 통로에서는 좋다. 정책의 입력 상태에 위치와 주변 지형이 들어가야 하며, 목표는 한 행동의 모양이 아니라 전체 행동열이 만들어 내는 결과다.

행동 하나의 좋고 나쁨은 다음 상태의 가치에 달려 있다. 현재 보상이 $r(s,a)=0$인 두 행동 중 하나가 위험한 상태로, 다른 하나가 목표 가까이로 간다면 즉시 보상만으로는 둘을 구별하지 못한다. 전이와 미래 보상을 포함한 $Q^\pi(s,a)$를 나중에 도입하는 이유다.

### 3.11 슬라이드 11 · 행동 복제의 기준선

BC는 전문가 데이터 $\mathcal{D}$에서 $-\mathbb{E}_{(s,a)\sim\mathcal{D}} \log \pi_\theta(a \mid s)$를 최소화한다. 이 식은 데이터셋의 행동을 잘 예측하는 정책을 만든다. 보상 $r$이나 전이 $P$를 직접 사용하지 않으므로, 같은 성공을 더 효율적으로 달성하는 새 경로를 평가할 신호는 없다.

반면 RL에서는 전문가가 없어도 환경의 보상으로 정책을 비교할 수 있다. 다만 좋은 행동을 찾는 탐색 비용이 든다. 두 방식은 데이터의 종류가 다른 것이지 신경망 구조가 반드시 달라야 하는 것은 아니다.

같은 네트워크를 BC와 RL 양쪽에서 쓸 수 있어도 손실 함수의 질문은 다르다. BC는 "이 상태에서 전문가 행동이 얼마나 그럴듯한가"를 묻고, RL은 "이 정책을 실행하면 기대 반환이 얼마인가"를 묻는다. 행동의 모양이 달라져도 결과가 좋다면 RL 목표에는 유리할 수 있지만 BC 손실에는 불리할 수 있다.

### 3.12 슬라이드 12 · 모방학습의 세 실패

실행 때 다른 상태에 들어가는 분포 이동, 같은 상태에서 여러 유효 행동이 있는 다봉성, 전문가가 더 많은 정보를 본 관측 불일치는 서로 다른 실패다. 하나의 높은 훈련 정확도로 세 문제를 동시에 해결했다고 볼 수 없다.

MDP의 상태를 정할 때는 특히 관측 불일치를 다시 생각해야 한다. 전문가가 가진 과거 정보와 정책이 받는 센서가 다르면, 선택한 $s_t$가 미래의 보상과 전이를 설명하기에 충분하지 않을 수 있다.

세 실패를 MDP 관점에 붙이면 분포 이동은 정책이 바꾼 상태 방문 확률, 다봉성은 한 상태에서 여러 유효 행동, 관측 불일치는 상태 표현의 불충분함이다. 같은 실패율 수치로는 원인이 구별되지 않는다. 적절한 상태와 행동 공간을 정하는 과정 자체가 알고리즘 설계와 맞물려 있다.

### 3.13 슬라이드 13 · DAgger의 상태 분포 수정

DAgger는 현재 정책이 방문한 상태를 모으고 그곳에서 전문가 라벨을 받아 데이터셋을 늘린다. 이 과정은 정책이 만드는 분포 $d_\pi$에 학습을 맞춘다. 단순히 시연 횟수를 늘리는 것과 달리, 실제 정책이 어려워하는 상태에 데이터가 집중된다.

그러나 전문가가 개입할 수 있어야 하고, 그 판단을 받는 비용이 든다. 뒤의 RL은 전문가 라벨 대신 보상으로 결과를 평가한다는 점에서 출발 조건이 다르다.

DAgger는 분포를 고치는 방법이지만 전문가의 정답이 그 상태에서 정의되어 있어야 한다. 정책이 위험 상태에 갔을 때 전문가가 안전하게 복구할 수 없는 환경이라면 라벨을 받는 절차 자체가 성립하기 어렵다. 시뮬레이터에서 교정하거나, 안전 제약을 둔 다른 데이터 수집이 필요할 수 있다.

### 3.14 슬라이드 14 · 사람의 개입과 보상의 차이

사람이 실수를 보고 제어권을 가져와 교정하는 방식은 실행 정책의 취약 상태를 잘 포착한다. 여기서 사람의 신호는 "지금은 이렇게 움직여야 한다"는 **행동 라벨**이다. 성공·실패만 알려 주는 보상보다 직접적이지만, 매 상황에서 사람이 빠르게 반응할 수 있어야 한다.

고속 로봇에서는 개입 지연이 문제고, 사람의 피로가 시연 품질을 바꿀 수 있다. 이런 한계가 보상 기반 상호작용, 시뮬레이션과 오프라인 데이터의 활용을 고민하게 만든다.

사람이 조향을 직접 고쳐 주면 행동 라벨을 얻는다. 사람이 두 주행 영상 중 하나가 더 안전하다고 고르면 선호 신호를 얻는다. 충돌 센서가 작동하면 사건 기반 보상을 얻는다. 모두 피드백이지만 데이터 형식이 다르므로, 전문가 행동을 따라 하는 학습과 결과를 평가하는 학습을 섞어 부르지 말자.

### 3.15 슬라이드 15 · RL 문제로 넘어가는 연결

이제 행동 라벨 $\pi^*(s)$를 잠시 내려놓고, 환경의 전이와 보상만으로 정책을 평가한다. 정책이 상태 분포를 바꾸기 때문에 학습의 대상은 각 상태의 행동 오차가 아니라 정책이 만드는 **전체 궤적의 성과**다.

가장 단순한 목적은 $J(\pi) = \mathbb{E}_{\tau\sim\pi}[\sum_t r_t]$다. 기대값은 같은 정책에서도 결과가 달라질 수 있음을 나타낸다. 앞으로의 MDP 정의가 이 기대값의 확률 구조를 명시한다.

BC에서 정책의 좋고 나쁨은 보통 전문가 행동과의 거리로 측정한다. RL에서는 정책이 만든 상태 분포 자체가 목적함수에 들어간다. 예를 들어 더 많은 보상을 주는 방으로 이동한 정책은 이후에도 그 방에서 행동할 기회를 얻는다. 한 시점의 행동 확률을 조금 바꿨을 뿐인데 미래의 경험 전체가 달라지는 이유다.

### 3.16 슬라이드 16 · 누적 보상의 의미

시점 $t$부터 길이 $T$의 episode가 끝날 때까지 받는 보상을 $r_t, r_{t+1}, \ldots, r_{T-1}$이라 하자. 할인율 $\gamma \in [0, 1]$는 한 단계 뒤 보상의 가중치를 정한다. **반환** $G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}$는 현재와 미래 보상을 합친 정책의 평가량이다.

예를 들어 게임에서 당장 점수를 얻는 길이 곧 패배로 이어질 수 있다. 누적 보상이 목표라면 그 길을 피해야 한다. $\gamma$는 뒤 보상을 얼마나 반영할지 정하지만, 보상의 단위와 episode 길이도 결과에 영향을

준다.

수치 예를 들어 첫 단계에 3점, 다음 단계에 4점, 마지막에 5점을 받고 $\gamma = 0.5$라면 $G_0 = 3 + 0.5 \cdot 4 + 0.5^2 \cdot 5 = 6.25$다. 같은 세 보상도 $\gamma = 1$이면 12점이 된다. 할인율을 바꾸면 단순히 수식 모양만 바뀌는 것이 아니라 어느 시간의 결과를 정책이 더 중시하는지가 달라진다.

**보상을 매 단계 2배로 하면 최적 정책이 바뀔까?** 모든 보상을 같은 양의 양수로 곱하고 다른 제약이 없다면 각 정책의 반환도 같은 비율로 바뀌므로 최적 정책의 순서는 유지된다. 하지만 일부 보상 항만 키우거나 학습 알고리즘의 clip·정규화 설정을 그대로 두면 실제 학습 행동은 달라질 수 있다.

### 3.17 슬라이드 17 · RL 문제를 확률로 쓰기

길이 $T$의 궤적을 $\tau = (s_0, a_0, s_1, a_1, \ldots, s_T)$라 하자. $p_0(s_0)$는 초기 상태 확률, $\pi(a_t \mid s_t)$는 정책이 시점 $t$의 상태에서 행동을 뽑을 확률, $P(s_{t+1} \mid s_t, a_t)$는 환경의 다음 상태 전이 확률이다. 이 조건부 확률들을 곱하면 정책 $\pi$가 궤적 $\tau$를 만들 확률 $p_\pi(\tau)$가 된다.

$$p_\pi(\tau) = p_0(s_0) \prod_{t=0}^{T-1} \pi(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

보상도 확률적이면 $p(r_t \mid s_t, a_t)$ 항을 곱해 넣을 수 있다. 목표는 정책이 만드는 궤적에서 반환의 기대값 $J(\pi) = E_{\tau \sim p_\pi}[G_0]$을 높이는 것이다. 뒤의 정책경사는 이 식을 $\theta$에 대해 미분한다.

곱의 각 항은 한 단계의 조건부 확률에서 나온다. 먼저 $p_0(s_0)$로 시작 상태를 뽑고, 그 상태에서 $\pi(a_0 \mid s_0)$로 행동을 뽑고, $P(s_1 \mid s_0, a_0)$로 다음 상태를 뽑는다. 같은 과정을 $T$번 반복하면 chain rule에 따라 전체 궤적의 확률이 곱으로 표현된다. **전이가 Markov이고 정책이 현재 상태에만 의존한다**는 조건을 사용했기에 더 오래된 과거는 각 항에 나타나지 않는다.

**결정론적 정책에서는 행동 확률이 사라질까?** 이산 행동에서는 선택된 행동에 확률 1, 나머지에 0을 주는 특수한 정책으로 볼 수 있다. 연속 행동에서는 Dirac delta로 나타내거나 행동을 상태의 함수로 대입해 궤적의 확률을 쓴다. 정책경사의 로그확률 식을 그대로 쓰려면 보통 확률적 정책이 편리하다.

### 3.18 슬라이드 18 · MDP가 환경과 과업을 함께 담는 이유

물리적인 전이만으로는 무엇을 잘해야 하는지 정해지지 않는다. 같은 로봇과 방에서도 "빨리 이동하라"와 "에너지를 적게 쓰라"는 과업은 보상이 다르다. MDP는 전이 $P$와 보상 $r$을 함께 포함하여 **환경의 움직임과 목적**을 한 문제로 정의한다.

환경 모델을 모른다고 해서 MDP가 사라지는 것은 아니다. 실제 로봇의 마찰과 접촉 확률을 계산할 수 없어도 전이를 경험할 수 있다. 모델 프리 알고리즘은 그 경험으로 정책이나 가치를 학습한다.

같은 방에서 같은 로봇을 쓰더라도 보상이 이동 속도라면 빨리 달리는 정책, 배터리 사용량이라면 천천히 움직이는 정책이 유리할 수 있다. 전이 $P$는 두 과제에서 같고 보상 $r$만 다르다. 따라서 "환경을 배웠다"와 "과제에서 잘한다"를 구분해야 하며, 보상은 실제 의도를 충분히 담는지 별도로 검토해야 한다.

### 3.19 슬라이드 19 · 확률 과정의 기초

확률 과정은 시간에 따라 달라지는 확률변수들의 모음 $\{S_t\}_{t \in \mathcal{T}}$이다. 주사위를 한 번 던져 나온 값 하나가 아니라, 여러 시점의 상태가 이어진 무작위 경로를 다룬다. 같은 시작 상태라도 전이의 우연성 때문에 다음 경로가 달라질 수 있다.

예를 들어 공부하는 학생이 과제를 시작할지 영상을 볼지는 확률적일 수 있다. 이 단계에는 아직 에이전트의 의도적인 행동이나 보상이 없어도 된다. 상태와 다음 상태의 확률 관계만 생각하는 것이

Markov 과정의 출발점이다.

확률변수 $S_t$는 실행 전에 어떤 상태가 될지 모르는 양이고, 실제로 관측한 $s_t$는 그 한 번의 실현값이다. 같은 시작 상태에서 100번 실행해 서로 다른 경로를 얻어도 과정의 전이 규칙이 같을 수 있다. 표본 궤적 하나만 보고 전이 확률을 정확히 안다고 생각하면 안 되는 이유다.

### 3.20 슬라이드 20 · Markov 성질의 정확한 뜻

Markov 성질은 현재 상태를 알고 나면 과거 상태가 **다음 상태를 예측하는 데 추가 정보를 주지 않는다**는 조건이다.

$$P(S_{t+1} \mid S_0, \ldots, S_t) = P(S_{t+1} \mid S_t).$$

과거가 실제로 없었다는 말은 아니다. 필요한 과거 정보가 현재 상태 안에 요약되어 있어야 한다. 공의 현재 위치만 기록하면 속도를 알 수 없어 다음 위치 예측이 어려울 수 있다. 위치와 속도를 함께 상태로 두면 Markov에 가까워진다.

**과거를 모두 버려도 된다는 뜻인가?** 현재 상태가 미래 예측에 충분할 때만 그렇다. 불충분하면 이전 관측을 상태에 포함하거나, 숨겨진 상태에 대한 믿음 분포를 유지해야 한다. 상태 표현을 어떻게 정했느냐에 따라 Markov 성질의 성립 여부가 달라진다.

공의 위치가 $x_t = 0$인 두 장면을 생각하자. 한 장면은 오른쪽으로 움직이며 속도가 $+1$, 다른 장면은 왼쪽으로 움직이며 속도가 $-1$이다. 위치만 상태라면 다음 위치가 달라지므로 현재 값만으로 미래를 예측하기 어렵다. 상태를 $(x_t, v_t)$로 넓히면 두 장면이 다른 상태가 된다. **Markov 성질은 세계의 성질만이 아니라 무엇을 상태로 정했는가에도 달려 있다.**

### 3.21 슬라이드 21 · Markov chain의 두 요소

Markov 과정 또는 chain은 상태 집합 $S$와 전이 확률 $P(s' \mid s)$로 나타낼 수 있다. $P(s' \mid s)$는 현재 $s$에서 다음에 $s'$가 될 확률이며, 각 $s$에 대해 $\sum_{s'} P(s' \mid s) = 1$이어야 한다.

상태와 전이만 있으므로 아직 선택할 행동이 없다. 어떤 상태가 "좋다"는 보상도 없다. 이 단계에서 계산할 수 있는 것은 특정 상태에 도달할 확률이나 오래 머무를 확률 같은 확률적 성질이다.

두 상태 $A, B$의 예에서 $P(A \mid A) = 0.8$, $P(B \mid A) = 0.2$, $P(A \mid B) = 0.4$, $P(B \mid B) = 0.6$이라고 하자. 현재 $A$라면 두 단계 뒤 다시 $A$일 확률은 $0.8 \cdot 0.8 + 0.2 \cdot 0.4 = 0.72$다. 중간 상태 $A$와 $B$라는 두 경로의 확률을 더한 것이다. 이 계산이 전이 행렬 제곱 $P^2$의 한 원소와 같다.

### 3.22 슬라이드 22 · 공부와 과제의 Markov chain

공부, 숙제, 제출, 영상 시청을 상태로 생각하자. 화살표에 적힌 수는 현재 상태에서 다음 상태로 옮길 확률이다. 공부에서 숙제로 갈 수도, 영상을 보기 시작할 수도 있다. 한 번 영상 상태에 들어간 뒤 다시 공부로 돌아올 수도 있으므로 경로는 매번 다르다.

그림의 세 sample episode는 같은 전이 규칙에서 가능한 서로 다른 실현이다. 전이 확률은 **한 경로의 목록**이 아니라 많은 경로의 상대적 빈도를 정의한다. 제출이 흡수 상태라면 그 뒤에는 계속 제출 상태에 머물거나 episode를 끝낼 수 있다.

상태가 "공부"일 때 바로 "제출"이 보이지 않아도, 여러 단계 후 제출에 닿는 경로는 존재할 수 있다. 전이 그림의 한 화살표가 뜻하는 것은 **다음 한 단계의 확률**이다. 제출 도달 확률이나 기대 소요 시간은 가능한 여러 경로를 합쳐야 구한다. 그림에서 한 번 그려진 episode를 가장 가능성 높은 경로로 오해하지 말자.

### 3.23 슬라이드 23 · Markov Reward Process

Markov reward process(MRP)는 Markov chain에 보상 함수 $r(s)$와 할인율 $\gamma$를 더한다. 이제 상태의 좋고 나쁨을 시간에 걸쳐 계산할 수 있다. 즉 선택할 행동은 아직 없지만, 현재 상태에서 시작할 때 기대되는 미래 보상을 정의할 수 있다.

$$V(s) = E\left[\sum_{k=0}^{\infty} \gamma^k r(S_{t+k}) \mid S_t = s\right].$$

$0 \le \gamma < 1$이면 보상이 유계인 계속 과업에서 합이 잘 정의된다. 유한 episode는 $\gamma = 1$을 써도 반환이 유한할 수 있다. $\gamma$는 미래 보상 선호뿐 아니라 무한 합을 안정적으로 다루는 역할도 한다.

가치함수의 재귀식을 직접 만들 수 있다. 반환을 첫 보상과 나머지로 분리하면 $G_t = r(S_t) + \gamma G_{t+1}$이다. $S_t = s$를 조건으로 기대값을 취하고 다음 상태 $s'$에 대한 확률을 풀어 쓰면

$$V(s) = r(s) + \gamma \sum_{s'} P(s' \mid s) V(s').$$

이것이 MRP의 Bellman equation이다. 지금 받는 보상과 다음 상태의 기대 가치를 더한 식이며, 다음 상태의 가치를 안다면 현재 상태의 가치를 계산할 수 있다는 직관과 연결된다.

상태가 유한하면 각 $V(s)$를 가치 벡터 $v$, 각 상태의 즉시 보상을 벡터 $r$, 상태 간 전이확률을 행렬 $P$에 모을 수 있다. $I$는 상태 수만큼의 항등행렬이다. 앞 식은 $v = r + \gamma P v$이므로

$$(I - \gamma P) v = r, \qquad v = (I - \gamma P)^{-1} r = \sum_{k=0}^{\infty} (\gamma P)^k r.$$

$P$의 각 행 합은 1이고 $\gamma < 1$이면 $\gamma P$의 거듭제곱이 작아져 마지막 등식의 기하급수가 수렴한다. $P^k r$는 $k$단계 뒤의 보상 기대값이므로 행렬식은 처음의 반환 정의와 정확히 같은 내용을 담는다. 예를 들어 $s_1$에서 항상 $s_2$로 가고, $s_2$는 매 단계 보상 1을 주며 그곳에 머문다면 $\gamma = 0.9$에서 $V(s_2) = 1 + 0.9 V(s_2) = 10$, $V(s_1) = 0 + 0.9 V(s_2) = 9$다. 종료 상태에서 한 번만 1을 주는 과업과는 다른 설정이다.

### 3.24 슬라이드 24 · 즉시 비용과 최종 보상

공부와 과제 진행에는 작은 시간 비용, 제출에는 큰 보상이 있다고 하자. 보상을 받기 위해서는 중간의 음의 보상을 감수해야 한다. 예를 들어 지금 공부에 $-1$, 다음 과제에 $-1$, 그 뒤 제출에 100을 받고 $\gamma = 0.9$라면 반환은 $-1 + 0.9(-1) + 0.9^2(100) = 79.1$이다.

반면 영상 시청에서 계속 $-0.1$만 받고 끝나지 않는다면 할인 반환은 $-0.1/(1 - 0.9) = -1$이다. 이 간단한 계산은 즉시 보상이 더 큰 행동이 장기적으로 더 좋다는 뜻이 아님을 보여 준다. 다만 그림 자체의 확률을 사용한 **기대값**은 가능한 모든 경로를 전이 확률로 가중해 계산해야 한다.

앞 계산에서 제출 보상 100점은 두 단계 뒤에 오므로 $0.9^2$를 곱했다. 만약 제출로 갈 확률이 0.5이고 실패하면 보상을 받지 못한다면, 100점 항의 기대 기여는 $0.5 \cdot 0.9^2 \cdot 100 = 40.5$다. 보상을 받은 한 경로의 반환과 **그 경로가 일어날 확률까지 반영한 기대 가치**를 구분해야 한다.

### 3.25 슬라이드 25 · 선택을 더한 MDP

MDP는 $S, A, P, r, \gamma$의 묶음이다. 행동 $a$를 고른 뒤 전이 $P(s' \mid s, a)$가 다음 상태를 만들고 보상 $r(s, a)$ 또는 더 일반적으로 $r(s, a, s')$가 주어진다. Markov chain에서는 전이가 상태만의 함수였지만 MDP에서는 행동이 전이를 바꾼다.

어떤 책은 보상을 확률변수로 두어 $P(s', r \mid s, a)$로 합쳐 쓴다. 표기가 달라도 현재 상태와 행동을 조건으로 다음 상태와 보상의 분포가 정해진다는 뜻은 같다. 목표는 선택 가능한 정책 가운데 높은 기대 반환을 얻는 것이다.

아주 작은 MDP에서 같은 상태 $s$에 "공부"와 "영상 보기" 두 행동이 있다고 하자. 공부를 고르면 제출 상태로 갈 확률이 0.8, 영상을 고르면 0.1이라면 두 행동은 현재 상태가 같아도 미래 분포를 다르게 만든다. 행동이 없는 Markov chain은 이 차이를 표현할 수 없고, 행동별 전이 $P(s' \mid s, a)$가 필요한 이유다.

### 3.26 슬라이드 26 · 공부 예시에 행동을 넣기

이제 "공부하기", "영상 보기", "과제 시작", "제출"이 상태가 아니라 행동이 된다. 공부 중에 영상을 고르면 영상 상태로 갈 확률이 높아지고, 과제를 시작하면 숙제 상태로 갈 수 있다. 정책은 같은 상태에서 어느 행동을 얼마나 자주 고를지 정한다.

상태와 행동을 혼동하면 전이 그림을 잘못 해석한다. "숙제 중"은 현재 상황이고 "제출한다"는 선택이다. 보상은 선택 뒤의 결과를 평가한다. 이 구분을 고정하면 로봇·게임·대화 예시도 같은 표기로 옮길 수 있다.

상태와 행동을 분리하는 실용적 기준은 "지금 관측된 조건인가, 에이전트가 선택할 수 있는가"다. 이미 숙제 중이라는 사실은 상태이고, 지금 제출 버튼을 누르는 것은 행동이다. 단, 환경 설계에 따라 같은 단어도 다른 역할을 맡을 수 있으므로 이름 자체보다 **선택 가능성**과 **시간 순서**를 따져야 한다.

### 3.27 슬라이드 27 · 환경을 정의하는 식과 학습 목표

MDP는 정책이 주어졌을 때 어떤 궤적이 나올지 확률적으로 정의한다. 학습자는 전이와 보상의 결과를 관측해 $J(\pi) = E_{\tau \sim p_\pi}[\sum_t \gamma^t r_t]$를 크게 만드는 정책을 찾는다.

환경의 목적은 보상으로 표현되지만 보상 설계가 부정확하면 높은 $J$가 실제 성공을 뜻하지 않을 수 있다. 예를 들어 과제 제출 버튼만 누르면 큰 보상을 준다면 내용을 작성하지 않고 제출하는 정책도 수식상 최적일 수 있다. 목적의 측정 방법을 점검해야 한다.

보상 설계의 허점을 작은 예로 확인할 수 있다. 과제 제출 버튼에만 $+10$을 주고 내용 품질은 평가하지 않으면, 에이전트는 빈 문서를 즉시 제출해 $+10$을 얻을 수 있다. 수학적 최적화는 정의된 숫자에 충실하지만 사람이 원한 "잘 작성한 과제"와는 어긋난다. 최종 성공 지표를 보상과 독립적으로 확인해야 하는 이유다.

### 3.28 슬라이드 28 · 정책이 정의하는 행동

Markov 정책은 현재 상태만으로 행동 확률을 정한다. $\pi(a \mid s) \geq 0$이고 이산 행동에서는 $\sum_a \pi(a \mid s) = 1$이다. 주어진 MDP와 정책이 있으면 상태 전이는 정책에 대해 평균 낸 $P_\pi(s' \mid s) = \sum_a \pi(a \mid s) P(s' \mid s, a)$가 된다.

즉 하나의 정책을 고르면 MDP 위의 상태 흐름이 하나의 Markov chain으로 바뀐다. 이 관계가 정책 평가의 출발점이다. 나중에 정책을 바꾸면 방문 상태의 분포도 바뀌므로, 오래된 경험 데이터를 사용할 때 주의가 필요하다.

예를 들어 $\pi(a_1 \mid s) = 0.25$, $\pi(a_2 \mid s) = 0.75$이고 다음 상태 $s'$로 갈 확률이 각각 0.8, 0.4라면 $P_\pi(s' \mid s) = 0.25 \cdot 0.8 + 0.75 \cdot 0.4 = 0.5$다. 정책이 바뀌어 $a_1$을 더 자주 고르면 전이 규칙을 가진 환경 자체는 그대로여도 **정책 아래에서 관측되는 상태 흐름**은 달라진다.

유한 상태에서 정책을 고정하면 보상이 $r(s, a)$ 형태일 때 행동을 평균 낸 즉시 보상 $r_\pi(s) = \sum_a \pi(a \mid s) r(s, a)$와 전이 행렬 $P_\pi$를 만들 수 있다. 보상이 다음 상태에도 의존하면 그 전이까지 평균에 넣으면 된다. 할인 가치의 Bellman 식은 벡터로 $V^\pi = r_\pi + \gamma P_\pi V^\pi$다. $0 \leq \gamma < 1$이면 이를 정리해 $V^\pi = (I - \gamma P_\pi)^{-1} r_\pi$로 풀 수 있다. 표본 기반 알고리즘이 나중에 근사하는 대상은 이처럼 **정책**

**아래의 미래 보상 기대값**이다. 상태가 크거나 전이 행렬을 모르면 역행렬을 실제로 계산하지 않고 경험에서 가치를 학습한다.

**정책을 고정하면 왜 MDP가 Markov chain이 되나?** 상태 $s$에서 다음 상태로 가는 확률은 정책이 각 행동을 고를 확률과 행동별 전이 확률을 곱해 모두 더한 $P_\pi(s' \mid s)$다. 행동 선택을 이 평균 안에 포함했으므로, 상태만으로 다음 상태의 확률이 정해진다. 정책을 다시 바꾸면 다른 chain이 된다.

### 3.29 슬라이드 29 · 초기 상태 분포까지 포함하기

MDP의 정의에 $p_0(s_0)$를 넣으면 episode가 어디서 시작할지 명시된다. 출발 위치가 늘 같을 때와 무작위일 때는 같은 정책의 평균 성과가 달라진다. 환경 전이와 보상이 같아도 초기 분포가 바뀌면 학습 목표 $E_{s_0 \sim p_0}[V^\pi(s_0)]$가 달라진다.

실험에서 초기 상태를 무작위화하면 더 넓은 상태를 연습할 수 있다. 반면 평가할 때만 쉬운 시작 상태를 쓰면 성능을 과대평가한다. 학습과 평가의 초기 분포를 따로 기록해야 한다.

초기 상태가 쉬운 $s_A$와 어려운 $s_B$로 절반씩 나오고 정책의 성공 확률이 각각 1과 0이라면 평균 성공률은 0.5다. 같은 정책을 $s_A$에서만 평가하면 1로 보인다. 정책이나 환경 전이를 고치지 않아도 시작 분포만으로 수치가 크게 바뀐다. 따라서 논문의 성과를 비교할 때 초기화 규칙을 확인해야 한다.

### 3.30 슬라이드 30 · 시작 상태가 여러 개일 때

그림의 "여기서 시작" 표시는 과제를 시작하는 위치가 하나가 아님을 보여 준다. 같은 정책도 바로 숙제 상태에서 시작하면 제출까지 가는 길이 짧고, 영상 상태에서 시작하면 공부로 돌아오는 데 시간이 걸릴 수 있다.

초기 상태 분포는 단순한 구현 세부 사항이 아니다. 어떤 상황에서 좋은 정책을 원하느냐를 정의한다. $p_0$가 거의 방문하지 않는 어려운 상태에서의 정책 능력은 평균 반환에 작게 반영될 수 있으므로, 별도의 스트레스 평가가 필요하다.

평균 반환이 높은 정책이 어려운 출발점에서 약할 수 있다. 예컨대 시작의 95%가 쉬운 상태라면 드문 5%의 실패는 평균에서 작게 보인다. 서비스나 로봇의 실제 안전이 그 5%에 달려 있다면 시작 조건별 성공률을 별도로 보고해야 한다. 평균 성능과 최악 조건 성능은 다른 질문이다.

### 3.31 슬라이드 31 · 상태와 행동은 과업마다 다르다

보드게임의 상태는 바둑판의 돌 배치이고 행동은 다음 수의 위치다. 로봇의 상태는 관절 각도·속도·카메라·촉각 등이고 행동은 관절 토크나 그리퍼 명령이다. 같은 수학적 MDP라도 변수의 의미와 크기가 크게 다르다.

카메라 이미지가 상태에 포함된다고 해서 항상 전체 상태를 알 수 있는 것은 아니다. 가려진 물체, 보이지 않는 접촉력, 영상 한 장으로 알기 어려운 속도가 남는다. 상태 표현을 정할 때 "다음 상태와 보상을 예측하는 데 무엇이 빠졌는가"를 묻자.

바둑판은 돌의 배치만으로 현재 합법 수를 충분히 알 수 없는 규칙이 있을 수 있고, 로봇의 영상 한 장은 속도를 직접 보여 주지 않는다. 상태를 간단히 만들수록 학습은 쉬워질 수 있지만 중요한 정보가 빠질 위험도 커진다. 상태 표현은 입력의 크기만이 아니라 **미래 전이와 보상을 예측하는 데 충분한가**로 판단한다.

### 3.32 슬라이드 32 · 환경을 완전히 정의해도 모를 수 있다

수학적으로 MDP를 쓴다는 것은 전이와 보상이 **존재하고 일정한 규칙을 따른다**는 모델링이다. 에이전트가 그 확률표나 물리 방정식을 알고 있다는 뜻은 아니다. 실제 로봇의 접촉을 정확히 계산하지

못해도 행동을 실행해 표본 $(s_t, a_t, r_t, s_{t+1})$을 관측할 수 있다.

모델 기반 방법은 이 전이를 학습하거나 시뮬레이터를 이용해 계획한다. 모델 프리 방법은 전이를 직접 예측하기보다 표본에서 정책 또는 가치를 개선한다. 두 접근은 환경이 알려진 정도와 상호작용 비용에서 절충한다.

전이 확률표를 모르는 카드 게임도 규칙을 완전히 모른다는 이유로 MDP가 아닌 것은 아니다. 여러 번 플레이해 행동과 다음 결과의 표본을 모을 수 있다. 모델 기반 방법은 그 표본으로 결과를 예측하는 모델을 만들고, 모델 프리 방법은 결과 모델을 명시적으로 만들지 않고 가치나 정책을 갱신한다. 어느 쪽도 표본 부족의 문제에서 자유롭지는 않다.

### 3.33 슬라이드 33 · 실제 던지기의 MDP

바나나를 바구니에 넣는 로봇을 생각하자. 상태는 로봇과 물체의 위치·속도, 행동은 관절 제어, 다음 상태는 실제 물리, 보상은 바구니에 들어갔는지의 신호일 수 있다. 성공 보상이 마지막에만 주어지면 어느 관절 동작이 성공에 기여했는지 바로 알기 어렵다.

물리 전이가 알려지지 않은 상태에서 여러 번 던져 얻은 표본으로 정책을 개선할 수 있다. 그러나 실패한 던지기의 비용과 안전 문제가 있으므로 실제 로봇에서는 시뮬레이션, 시연, 안전 제약과 병행하는 경우가 많다.

던지기 직전의 각도와 속도가 조금만 달라도 착지 위치는 달라진다. 성공 보상이 마지막에만 1이면 실패 궤적 대부분의 보상은 0이다. 거리 기반 중간 보상을 넣으면 신호가 풍부해지지만, 바구니 옆을 오래 맴도는 행동이 높은 점수를 받을 수도 있다. **보상 밀도와 목표 일치** 사이의 절충을 이해하기 좋은 사례다.

### 3.34 슬라이드 34 · 환경 API와 수학 기호의 대응

환경을 초기화하면 $s_0$ 또는 초기 관측을 받고, 한 단계 진행하면 $s_{t+1}$, $r_t$, 종료 신호와 추가 정보를 받는다. 에이전트는 명령을 보낸 뒤 내부 물리 계산의 모든 세부를 몰라도 다음 전이 표본을 얻을 수 있다.

코드에서 중요한 것은 반환 형식과 종료 처리다. Gym 계열 API의 버전에 따라 초기화가 관측과 정보의 쌍을 주고, 진행 결과가 종료와 시간 제한을 따로 줄 수 있다. 시간 제한으로 잘린 episode를 진짜 종료와 혼동하면 다음 상태 가치의 부트스트랩 처리가 틀릴 수 있다.

**환경의 전이를 모르는데 Bellman 방정식을 사용할 수 있나?** 가능하다. Bellman 방정식은 기대값의 관계를 정의하고, 구현에서는 환경에서 얻은 표본으로 그 기대값을 근사한다. 전이 확률표를 직접 알아야 하는 동적 계획법과 표본 기반 TD 학습을 구분하면 된다.

구체적으로 `step(a_t)`가 `(o_{t+1}, r_t, terminated, truncated, info)`를 반환한다고 생각하자. `terminated`는 과제 자체의 종료이고 `truncated`는 시간 제한 같은 외부 중단일 수 있다. 가치 학습에서 진짜 종료 뒤의 미래 가치는 0으로 두지만, 시간만 잘린 경우에는 $V(o_{t+1})$를 이어서 추정해야 할 수 있다. 이 차이가 잘못 처리되면 모든 제한 시점의 가치를 과소평가할 수 있다.

### 3.35 슬라이드 35 · 완전 관측의 한계

MDP의 상태는 미래 예측에 필요한 정보를 충분히 담아야 한다. 실제 세계에서는 카메라가 가린 물체, 센서가 측정하지 않은 힘, 사람의 숨은 의도 때문에 관측만으로 전체 상태를 알기 어렵다. 이런 상황은 부분 관측 문제로 보는 편이 정확하다.

정책은 관측 이력 $o_{0:t}$을 사용하거나, 숨은 상태에 대한 믿음 분포 $b_t(s) = P(s_t = s \mid o_{0:t}, a_{0:t-1})$를 유지할 수 있다. 나중에 POMDP를 배우면 이 개념이 정식화된다. 현재는 관측 $o_t$와 충분한 상태 $s_t$를 무심코 같게 두지 않는 것이 중요하다.

컵이 불투명한 상자 뒤에 있을 때 카메라 영상 하나만으로 정확한 위치를 알 수 없더라도, 직전에 컵이 움직인 방향을 기억하면 추정할 수 있다. 이처럼 과거 이력을 요약한 내부 상태를 두는 정책은 한 장면만 쓰는 정책보다 정보가 많다. 단, 기억을 추가해도 센서가 전혀 제공하지 않는 정보는 추정의 불확실성이 남는다.

믿음 상태의 갱신은 두 단계다. 먼저 이전 믿음 $b_t(s)$와 행동 $a_t$로 다음 숨은 상태를 예측해 $\widetilde{b}_{t+1}(s') = \sum_s P(s' \mid s, a_t) b_t(s)$를 만든다. 다음 관측 $o_{t+1}$이 나오면 그 관측이 각 상태에서 얼마나 그럴듯한지 $O(o_{t+1} \mid s')$를 곱하고 전체 합이 1이 되도록 정규화한다. 예를 들어 컵이 왼쪽/오른쪽에 있을 사전 확률이 각각 0.5이고, 관측된 그림이 왼쪽일 가능도가 두 상태에서 0.8/0.2라면 관측 뒤 왼쪽 믿음은 $0.8 \cdot 0.5 / (0.8 \cdot 0.5 + 0.2 \cdot 0.5) = 0.8$이다. **관측을 곧 상태로 확정하지 않고 불확실성을 갱신**하는 방식이다.

### 3.36 슬라이드 36 · Ant 환경을 MDP로 쓰기

Ant 보행 과업에서는 몸통의 자세·속도와 관절 상태를 관측하고, 여러 관절의 토크 명령을 연속값으로 낸다. 환경 전이는 물리 시뮬레이터가 계산하며 보상은 전진·건강한 자세를 장려하고 큰 제어 비용은 줄인다. 초기 자세에 잡음을 주면 여러 시작 상황을 연습한다.

행동 범위가 $[-1, 1]^8$처럼 연속적이면 모든 행동을 열거해 최고 Q값을 찾기 어렵다. 이 사실은 뒤의 DDPG·TD3·SAC가 별도의 행동 정책을 학습하는 이유와 연결된다. 보상 항의 상대적 크기는 어떤 걸음을 배우는지 크게 바꿀 수 있다.

Ant의 행동 8차원을 각 축에서 세 값씩만 고른다고 해도 가능한 조합은 $3^8 = 6561$개다. 실제 행동은 연속 구간에 있으므로 경우의 수는 무한하다. DQN처럼 모든 행동의 Q를 한 번에 열거하는 방식이 그대로는 불편하고, 연속 행동을 직접 내는 actor가 필요한 이유를 수치로 확인할 수 있다.

### 3.37 슬라이드 37 · 게임에서의 상태와 보상

멀티플레이 게임에서는 화면·소리·게임 변수와 다른 플레이어의 행동이 다음 상태를 만든다. 승리와 패배는 긴 과제 끝에 알려지고, 레벨 상승이나 구조물 파괴 같은 중간 사건을 보상에 넣을 수 있다. 중간 보상은 학습을 빠르게 할 수 있지만 최종 승리와 다른 행동을 유도할 위험도 있다.

다른 사용자가 움직이므로 환경이 고정된 단일 에이전트 MDP처럼 보이지 않을 수 있다. 다른 플레이어의 전략이 학습 중 바뀐다면 전이 분포도 바뀐다. 이런 경우 상대 정책을 상태에 반영하거나 다중 에이전트 문제로 확장해야 한다.

게임에서 적이 고정된 정책으로 움직이면 내 관점에서 그 효과를 환경 전이에 넣을 수 있다. 하지만 적도 학습하며 전략을 바꾸면 같은 상태와 행동의 다음 결과 분포가 시간에 따라 변한다. 고정된 MDP의 전이를 가정한 학습기가 불안정해질 수 있으므로 **상대의 정책 변화**를 실험 설정에 명시해야 한다.

### 3.38 슬라이드 38 · 실제 로봇에서의 상태와 보상

로봇의 카메라와 관절 센서는 관측이고, 전류나 토크는 행동이 될 수 있다. 물체를 집는 과업에서 "성공"은 최종 보상으로 명확할 수 있지만 중간의 정확한 행동 라벨은 없다. 실제 전이에는 접촉, 마찰, 물체 무게와 센서 지연이 포함되어 모델링이 어렵다.

실제 로봇에서는 샘플 효율과 안전성이 특히 중요하다. 보상을 얻기 위해 수천 번 실패할 수 없다면 시뮬레이션에서 초기 학습을 하거나 시연·오프라인 데이터를 결합한다. MDP 표기는 문제를 단순하게 보여 주지만 실험 비용을 없애 주지는 않는다.

물체 집기의 성공을 1, 실패를 0으로 주는 보상은 목표가 명확하지만 로봇이 집기 전 긴 시간을 낭비해도 성공만 하면 같은 점수를 줄 수 있다. 시간 비용을 추가하면 더 빠른 정책을 찾을 수 있지만, 너무 크게 두면 성급하게 움직여 물체를 떨어뜨릴 수 있다. 실제 과업의 보상은 성공, 속도, 안전의 균형을 어떻게 수치화했는지까지 읽어야 한다.

### 3.39 슬라이드 39 · 대화의 상태와 보상

대화에서 상태는 지금까지의 대화 맥락과 사용자 요청, 행동은 생성한 응답 또는 다음 토큰열로 볼 수 있다. 사용자의 다음 메시지가 이후 상태를 바꾸므로 상호작용은 순차적이다. 사람의 선호 비교나 만족도는 보상과 연결될 수 있다.

하지만 사용자의 의도는 완전히 관측되지 않고 선호도 평가자마다 다를 수 있다. 보상 모델의 점수를 높이는 것과 실제로 도움이 되는 응답을 만드는 것을 구분해야 한다. 공개된 RLHF 사례를 이해할 때 보상 모델의 학습 데이터와 최종 평가를 함께 살피는 이유다.

대화의 한 응답을 행동으로 볼지, 토큰 하나를 행동으로 볼지에 따라 시간 간격과 보상 할당이 달라진다. 응답 전체에 마지막 선호 점수 하나만 준다면 어느 문장이 도움이 되었는지 직접 알기 어렵다. 또한 사용자 반응은 숨은 의도와 상황에 좌우되므로, 관측된 대화 기록을 완전한 상태로 가정할 때 한계가 있음을 밝혀야 한다.

### 3.40 슬라이드 40 · 용어를 한 궤적에 모으기

한 시점의 관측 $o_t$와 상태 $s_t$를 받아 정책 $\pi(a_t \mid s_t)$가 행동 $a_t$를 선택한다. 환경은 $P(s_{t+1} \mid s_t, a_t)$에 따라 다음 상태로 움직이고 보상을 낸다. 이 반복의 기록이 trajectory 또는 rollout이다. episode는 보통 종료까지의 한 궤적을 가리킨다.

제어 분야에서는 행동을 $u_t$, 상태의 구성을 $x_t$라고 쓰기도 한다. 기호가 바뀌어도 입력, 제어 명령, 환경의 변화, 평가 신호라는 역할을 유지해 읽으면 된다.

용어를 시간 순서에 놓으면 $s_0$에서 $a_0$를 선택하고 $r_0, s_1$을 받은 뒤 $a_1$을 고른다. $r_0$가 행동 전에 주어진 점수인지 행동 뒤 결과인지 표기가 자료마다 다를 수 있으므로 **시간 인덱스의 약속**을 먼저 확인하자. 이 작은 차이를 놓치면 코드에서 현재 보상과 다음 보상을 한 칸 잘못 정렬할 수 있다.

### 3.41 슬라이드 41 · 정책경사로의 연결

MDP 위의 정책을 신경망 파라미터 $\theta$로 표현하면 기대 반환 $J(\theta)$를 최대화하고 싶다. 다음 장은 $J$를 직접 미분하는 정책경사를 다룬다. 환경의 물리 전이를 미분할 수 없어도 정책이 낸 행동의 로그확률과 관측한 반환으로 기울기를 추정할 수 있다.

이 표본 기울기는 잡음이 크다. 같은 정책도 여러 궤적에서 다른 보상을 받으므로, 유용한 업데이트 방향을 안정적으로 알아내려면 충분한 표본과 분산 감소 기법이 필요하다. 정책경사 다음에 reward-to-go, baseline, actor–critic과 PPO가 이어지는 이유다.

다음 장의 첫 질문은 "환경을 미분할 수 없는데 $\nabla_\theta J$를 어떻게 구하는가"다. 앞에서 얻은 $p_\pi(\tau)$의 곱에서 정책 항만 $\theta$에 직접 의존한다면 로그미분을 통해 $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$의 합이 나온다. MDP의 전이와 궤적 확률을 먼저 쓴 이유가 바로 이 유도에 있다.

# 4 02-2. 정책경사와 REINFORCE

정책경사는 신경망 정책 $\pi_\theta$가 만드는 궤적의 기대 반환을 직접 높인다. 어려운 점은 환경의 전이를 미분할 수 없고, 표본 궤적마다 보상이 크게 다르다는 것이다. 이 장은 확률밀도의 로그미분으로 첫 문제를 풀고, 다음 장에서 다룰 분산 감소 문제를 정확히 드러낸다.

## 4.1 슬라이드 01 · 정책경사의 질문

정책 신경망의 파라미터를 $\theta$, 정책이 만든 한 궤적을 $\tau$, 그 궤적의 전체 보상을 $R(\tau)$, 궤적이 나올 확률을 $p_\theta(\tau)$라 하자. 정책의 품질은 $J(\theta) = E_{\tau\sim p_\theta}[R(\tau)]$, 즉 가능한 궤적의 반환을 발생 확률로 가중한 평균이다. 정책경사는 이 값을 크게 만드는 파라미터 변화 방향 $\nabla_\theta J$를 구한다.

정책이 바뀌면 방문 상태 분포도 바뀐다. 따라서 이미 모아 둔 상태–행동 쌍에 고정된 정답을 맞히는 보통의 지도학습보다 미분 구조가 복잡하다.

정책이 두 행동 중 하나를 고르는 단순 문제에서 좋은 행동의 선택 확률을 0.4에서 0.5로 높이면 평균 성과가 달라진다. 정책경사는 그 확률을 조절하는 파라미터를 어느 방향으로 움직일지 계산한다. 신경망의 출력과 성과 사이에 환경이 끼어 있어도 **정책이 행동을 뽑을 확률**을 알면 출발할 수 있다.

## 4.2 슬라이드 02 · 반환과 기울기

목표는 환경을 반복 실행하며 얻는 반환의 평균을 높이는 것이다. 이를 미분하려면 두 종류의 무작위성을 분리해야 한다. 정책은 행동을 샘플링하고 환경은 다음 상태와 보상을 만든다. 환경의 물리식을 모르더라도 정책이 낸 행동의 확률은 계산할 수 있다.

이 장에서 다룰 핵심은 **실제로 뽑은 행동이 좋은 결과를 냈다면 그 행동의 확률을 높이고, 나쁜 결과를 냈다면 낮추는 기울기**를 어떻게 정확히 쓰느냐다. 이 직관은 로그확률과 보상의 곱으로 나타난다.

성공한 한 episode에 큰 보상을 주는 것과, 성공할 확률이 높은 정책을 만드는 것은 다르다. 우연히 성공한 행동을 그대로 반복하면 다음에는 실패할 수 있다. 목표 $J$는 한 번의 최고 기록이 아니라 같은 정책을 여러 번 실행했을 때 얻는 **기대 반환**이므로, 표본 기울기도 여러 궤적에서 평균해야 한다.

## 4.3 슬라이드 03 · 한 궤적의 기호

시점 $t$에서 상태 $s_t$가 주어지면 $a_t \sim \pi_\theta(\cdot \mid s_t)$를 뽑고, 환경은 $s_{t+1} \sim P(\cdot \mid s_t, a_t)$를 만든다. 이 반복에서 얻은 $\tau = (s_0, a_0, r_0, \ldots, s_T)$가 rollout이다. 보상 합 $R(\tau) = \sum_{t=0}^{T-1} r_t$는 그 궤적의 성과다.

정책경사 식에서 $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$는 행동의 **로그확률이 파라미터 변화에 어떻게 반응하는지**를 나타낸다. 확률값 자체와 보상값을 혼동하지 말자.

표본에서 $r_0$가 크다고 $a_0$만 성과를 만든 것은 아니다. 나중에 실패한 $a_3$가 최종 보상을 없앴을 수도 있다. 궤적 전체의 반환을 모든 행동에 곱하는 기본형은 계산은 간단하지만 신용 할당이 거칠다. reward-to-go가 나중에 각 시점의 **이후 보상**만 남기는 이유를 이 한계를 통해 이해할 수 있다.

## 4.4 슬라이드 04 · MDP와 정책의 역할

MDP의 초기 상태와 전이 규칙은 정책을 고르기 전의 과업을 정의한다. 정책은 각 상태에서 행동 확률을 지정하므로 MDP와 합쳐져 특정 궤적의 확률을 만든다. 같은 MDP에서도 정책이 달라지면 어떤 상태를 얼마나 자주 보는지 달라진다.

목적함수의 기대값은 그 달라진 궤적 분포에 대해 계산한다. 그래서 한 번 모은 데이터에서 손실을 낮추는 것과 현재 정책의 진짜 기대 반환을 높이는 것이 정확히 같은 문제는 아니다.

예를 들어 동일한 교차로에서 정책 A가 오른쪽으로 갈 확률을 0.2, 정책 B가 0.8로 준다고 하자. 우회전 뒤의 보상이 좋다면 B는 그 결과를 더 자주 관측한다. 따라서 정책 변화는 단순히 현재 행동의 빈도뿐 아니라 **그 행동 뒤에 도달하는 상태의 데이터 분포**도 바꾼다.

### 4.5 슬라이드 05 · 확률적 정책의 의미

확률적 정책은 상태에서 하나의 행동만 고정해 내지 않고 가능한 행동의 분포를 만든다. 예컨대 두 경로 중 왼쪽을 0.7, 오른쪽을 0.3의 확률로 선택할 수 있다. 이런 무작위성은 탐색을 돕고 로그확률 기울기를 정의하게 한다.

확률이 매우 작은 행동도 샘플링될 수 있다. 그 행동의 결과가 좋았다면 정책경사 업데이트는 해당 행동의 확률을 늘리는 쪽으로 작용한다. 다만 한 번의 운 좋은 결과를 과신하지 않도록 여러 궤적의 평균이 필요하다.

확률 정책과 환경의 무작위성은 다르다. 주사위처럼 환경이 우연히 실패할 수도 있고, 정책이 탐색을 위해 일부러 덜 확실한 행동을 뽑을 수도 있다. 관측한 한 실패만으로 행동의 참 가치를 알 수 없는 이유다. 다수의 표본에서 결과를 평균하면 두 종류의 변동이 정책 개선 방향에 미치는 영향을 줄일 수 있다.

### 4.6 슬라이드 06 · 지도학습과 다른 손실

지도학습에서는 정답 $y$가 주어져 $-\log p_\theta(y \mid x)$를 바로 줄인다. 강화학습 데이터에는 $(s_t, a_t, r_t)$가 있지만, 행동 $a_t$는 현재 정책이 스스로 뽑은 것이고 라벨이 아니다. 이 행동이 좋았는지는 뒤의 보상까지 봐야 안다.

보상 합을 그대로 신경망 손실로 놓아도 환경의 샘플링과 전이 때문에 일반적인 역전파만으로 정책 파라미터의 기울기를 얻지 못한다. 다음의 로그미분 항등식은 미분할 수 없는 전이를 통과하지 않고도 기대 반환의 기울기를 표현한다.

지도학습의 정답 $y$는 모델이 출력하기 전부터 정해져 있다. 정책경사의 $a_t$는 바로 그 정책이 만든 표본이다. 정책 파라미터를 바꾸면 행동 표본의 분포도 바뀌므로, 현재 모은 행동을 영원히 고정된 정답처럼 취급할 수 없다. 이 차이가 on-policy 데이터 조건과 연결된다.

### 4.7 슬라이드 07 · 정책 손실을 바로 쓸 수 없는 이유

$\mathcal{D} = \{(s_t, a_t, r_t)\}$가 있다고 해서 각 $a_t$를 정답처럼 더 자주 내도록 학습하면, 낮은 보상을 만든 행동까지 따라 하게 된다. 반대로 보상이 큰 궤적의 행동에는 더 높은 가중치를 주고 싶다. 문제는 그 가중치를 어떤 수학적 이유로 정하느냐다.

정책경사는 확률적 정책이 만든 궤적의 기대값을 미분해 그 가중치를 유도한다. 이 유도 없이 임의로 보상과 지도학습 손실을 곱하면 원하는 목적의 기울기인지 보장하기 어렵다.

두 궤적이 같은 상태를 거쳤어도 한쪽의 결과가 좋았다는 이유만으로 그 안의 모든 행동이 더 낫다고 할 수는 없다. 보상 가중치를 둔 업데이트는 **평균적으로 기대 반환의 기울기와 일치한다**는 수학적 성질에 의존한다. 이 성질은 개별 행동에 정답 라벨을 준다는 주장보다 약하지만, 미분할 수 없는 환경에서 목적함수를 개선할 근거가 된다.

### 4.8 슬라이드 08 · 궤적 확률의 곱

유한 길이 $T$에서 궤적 확률은 초기 상태, 정책 행동, 환경 전이의 곱이다.

$$p_\theta(\tau) = p_0(s_0) \prod_{t=0}^{T-1} \pi_\theta(a_t \mid s_t) P(s_{t+1} \mid s_t, a_t).$$

기대 반환은 $J(\theta) = \int p_\theta(\tau) R(\tau)\, d\tau$로 쓸 수 있다. 이산적인 상태와 행동이면 적분 대신 모든 궤적에 대한 합이다. $\theta$는 정책에만 들어간다고 가정한다. 환경의 전이를 몰라도 정책의 확률은 계산할 수 있다.

세 단계 궤적이라면 확률은 $p_0(s_0)$와 세 정책 확률, 세 환경 전이 확률의 곱이다. 로그를 취하면 곱이 합으로 바뀌므로 정책 파라미터에 대한 미분이 각 시점의 로그확률 미분으로 분해된다. 이것이 신경망 출력의 미분을 전체 episode의 성과에 연결하는 계산의 핵심이다.

### 4.9 슬라이드 09 · 로그미분 항등식

양의 확률밀도 $p_\theta(\tau)$에 대해 $\nabla_\theta p_\theta(\tau) = p_\theta(\tau) \nabla_\theta \log p_\theta(\tau)$다. 연쇄법칙으로 $\nabla_\theta \log p = (\nabla_\theta p)/p$를 얻고 양변에 $p$를 곱하면 확인된다.

이 항등식을 기대 반환에 적용하면 다음과 같다.

$$\begin{aligned} \nabla_\theta J(\theta) &= \nabla_\theta \int p_\theta(\tau) R(\tau)\, d\tau \\ &= \int \nabla_\theta p_\theta(\tau) R(\tau)\, d\tau \\ &= E_{\tau \sim p_\theta}\left[ \nabla_\theta \log p_\theta(\tau) R(\tau) \right]. \end{aligned}$$

첫 줄에서 적분과 미분을 바꾸는 데에는 적절한 매끄러움과 적분 가능성 조건이 필요하다. 직관적으로는 전체 궤적 확률을 더 자주 만들 방향을 그 궤적의 성과로 가중한다.

직관을 확인하려면 확률이 $p$인 결과에 보상 $R_1$, 나머지에 $R_2$를 주는 두 경우를 생각하면 된다. 기대 반환은 $J = pR_1 + (1-p)R_2$이고, $p$에 대한 미분은 $R_1 - R_2$다. 높은 보상의 결과가 더 자주 나오도록 $p$를 바꾸어야 한다는 방향이 로그미분 추정량에서도 그대로 나온다.

### 4.10 슬라이드 10 · 환경 전이 항이 사라지는 이유

곱으로 쓴 궤적 확률에 로그를 취하면 합이 된다. $p_0$와 $P$가 정책 파라미터 $\theta$에 직접 의존하지 않는다면 그 로그의 $\theta$ 미분은 0이다.

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

이는 행동이 상태 분포에 영향을 주지 않는다는 뜻이 아니다. 그 영향은 정책이 만든 궤적을 표본으로 뽑는 기대값 안에 이미 들어 있다. 따라서 환경의 미분 가능한 모델 없이도 정책경사를 추정할 수 있다.

**환경을 미분하지 않는데 행동의 장기 영향이 반영될까?** 반영된다. 정책이 달라지면 샘플링되는 행동과 뒤 상태의 분포가 바뀌고, 그 궤적의 전체 보상이 로그확률 기울기를 가중한다. 미분하지 않는 것은 환경 전이 함수의 내부 계산이지, 행동이 만드는 결과를 무시하는 것이 아니다.

### 4.11 슬라이드 11 · Monte Carlo 정책경사

앞의 두 단계를 합치면 기본 정책경사가 나온다.

$$\nabla_\theta J(\theta) = E_{\tau \sim p_\theta}\left[\left(\sum_{t=0}^{T-1} \nabla_\theta \log \pi_\theta(a_t \mid s_t)\right) R(\tau)\right].$$

$N$개의 현재 정책 rollout으로 기대값을 표본 평균하면 $N^{-1}\sum_i \sum_t \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) R(\tau_i)$가 된다. 위 첨자 $i$는 $i$번째 궤적, $\widehat{\nabla J}$는 그 평균으로 추정한 기울기, $\alpha > 0$은 업데이트 크기를 정하는 학습률이다. 경사 상승은 $\theta \leftarrow \theta + \alpha \widehat{\nabla J}$로 쓰고, 손실을 최소화하는 optimizer에서는 목적에 음의 부호를 붙인다.

한 상태에서 두 행동만 가능한 예를 보자. $p = \pi_\theta(a_1) = \sigma(\theta)$이고 $\sigma(x) = 1/(1+e^{-x})$는 sigmoid 함수이며, $\pi_\theta(a_2) = 1 - p$다. 보상이 각각 1과 0이면 $J(\theta) = p$이므로 $\nabla_\theta J = p(1-p)$다. 로그확률 기울기를 보상으로 가중한 식도 $p\,\nabla_\theta \log p \cdot 1 + (1-p)\,\nabla_\theta \log(1-p) \cdot 0 = p(1-p)$를 준다.

## 4.12 슬라이드 12 · REINFORCE의 반복

REINFORCE는 현재 정책으로 여러 episode를 수집하고, 각 episode의 반환을 계산하며, 로그확률 합에 반환을 곱해 정책을 갱신한다. 갱신 뒤 정책이 바뀌므로 다음 반복은 새 정책으로 데이터를 다시 모은다.

높은 반환을 낸 궤적의 행동 확률을 늘리는 직관은 맞지만, 같은 궤적 안의 모든 행동을 동일한 전체 반환으로 가중하면 공로가 없는 행동도 함께 강화될 수 있다. 예컨대 마지막 점수와 무관한 초반 동작도 같은 점수를 받는다. 다음 장의 reward-to-go가 이 낭비를 줄인다.

한 번의 업데이트를 코드로 옮기면 `loss = -(log_probs.sum() * episode_return)` 형태가 된다. 여러 episode의 손실을 평균한 뒤 optimizer가 이를 최소화한다. `log_probs`는 행동을 뽑던 당시 정책에서 계산해 보관해야 하며, 학습 시점의 새 정책으로 다시 계산해 같은 값인 척하면 on-policy 유도와 맞지 않는다.

**반환이 음수인 궤적은 어떻게 되나?** 보상이 음수일 때는 그 궤적의 행동 로그확률을 낮추는 방향의 항이 생긴다. 하지만 보상의 기준점을 어디에 두었는지에 따라 한 표본의 부호가 달라질 수 있다. 절대적인 좋고 나쁨보다 같은 상태의 평균 성과와 비교하는 advantage가 유용한 이유다.

## 4.13 슬라이드 13 · 유도의 네 단계 복기

정책경사 유도는 네 고리로 정리된다. 첫째, $J$를 궤적 분포의 기대값으로 쓴다. 둘째, 로그미분으로 분포 미분을 기대값 안의 점수 함수로 옮긴다. 셋째, 궤적 로그확률을 정책 로그확률의 합으로 분해한다. 넷째, 기대값을 현재 정책의 표본 평균으로 근사한다.

각 고리에 가정이 있다. 보상 함수가 $\theta$에 직접 의존하지 않고, 환경 전이도 정책 파라미터를 직접 공유하지 않으며, 표본은 해당 정책에서 나와야 한다. 응용에서 이 조건이 달라지면 추정량도 바뀐다.

각 단계가 필요한 이유도 역순으로 확인할 수 있다. 표본 평균을 쓰려면 기대값 형태가 필요하고, 환경을 미분하지 않으려면 그 기대값 안에 정책의 score를 만들어야 하며, score를 시점별 행동으로 나누려면 궤적 확률의 곱이 필요하다. 이 연결을 놓치면 마지막 식만 외우고 데이터 조건을 잊기 쉽다.

## 4.14 슬라이드 14 · 장점과 두 한계

정책경사는 실제 목적 $J$의 상승 방향을 직접 겨냥할 수 있다. 연속 행동에서도 확률밀도를 정의하면 적용할 수 있다. 하지만 궤적마다 보상이 크게 달라 표본 기울기의 분산이 높고, 현재 정책 분포를 가정해 유도했으므로 오래된 정책 데이터는 그대로 재사용하기 어렵다.

"높은 반환이면 행동을 더 자주"라는 직관은 평균적으로 성립하는 미분 방향이지, 한 rollout의 각 행동이 모두 좋았다는 증명은 아니다. 표본 수와 분산 감소가 성능을 좌우한다.

연속 행동 로봇에서 정책이 Gaussian 평균과 분산을 출력한다고 하면 실제 샘플 행동의 로그밀도를 계산할 수 있다. 그러나 큰 행동 공간에서 한 궤적의 반환은 여러 관절 행동과 환경 잡음을 한꺼번에 반영한다. 기울기 식을 쓸 수 있다는 것과 적은 시도로 안정적으로 학습할 수 있다는 것은 다른 문제다.

### 4.15 슬라이드 15 · On-policy 데이터 조건

$E_{\tau\sim p_\theta}$라는 기호에는 **현재 $\theta$에서 생성한 궤적**이라는 조건이 들어 있다. 예전 정책 $\pi_{\text{old}}$의 데이터를 그대로 평균하면 다른 분포에 대한 기대값을 계산하는 셈이다. 정책이 거의 변하지 않았다면 근사 오차가 작을 수 있지만 일반적으로 편향이 생긴다.

오래된 데이터를 쓰려면 중요도 가중치 $\pi_\theta(a \mid s)/\pi_{\text{old}}(a \mid s)$로 분포 차이를 보정하거나, off-policy 학습을 위한 다른 목적을 사용한다. 이 보정 자체도 분산을 키울 수 있어 뒤의 PPO가 정책 변화를 조심스럽게 제한한다.

한 시점의 중요도 비율은 $\pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$이지만, 완전한 궤적 분포를 정확히 보정하려면 일반적으로 시점별 비율을 곱해야 한다. 지평이 길수록 곱이 아주 크거나 작아져 추정 분산이 폭증할 수 있다. PPO의 한 단계 surrogate와 ratio clipping은 이 어려움을 다루는 실용적 근사이며, 임의로 오래된 자료를 정확하게 재사용할 수 있다는 보장은 아니다.

**on-policy인데 minibatch를 여러 번 돌리면 바로 틀린가?** 자료를 모은 뒤 파라미터가 바뀌므로 엄밀하게는 새 정책의 자료가 아니다. PPO는 수집 당시의 로그확률을 저장하고 현재 정책과의 비율을 계산하며, 변화가 지나치지 않도록 surrogate를 제한해 제한된 재사용을 허용한다.

### 4.16 슬라이드 16 · 표본 기울기의 분산

기울기 추정량을 $X_i$라고 하면 $\widehat{g} = N^{-1}\sum_i X_i$다. 독립적인 표본에 대해 $E[\widehat{g}] = g$이어도 $\mathrm{Var}(\widehat{g}) = \mathrm{Var}(X_i)/N$이므로 표본이 적으면 실제 기울기와 크게 어긋날 수 있다.

특히 긴 episode의 전체 보상은 정책과 무관한 환경 잡음과 많은 행동의 결과가 섞인다. 한 번의 운 좋은 결과에 큰 반환이 붙으면 원인과 관계없는 행동의 로그확률까지 강화한다. 평균 방향의 정확도가 낮으면 업데이트마다 성과가 크게 출렁인다.

환경이 같은 행동에서 성공과 실패를 반반씩 주는 경우 한 궤적의 반환만으로 행동의 질을 판단하기 어렵다. 큰 반환을 얻은 궤적과 작은 반환을 얻은 궤적의 기울기가 서로 반대 방향일 수도 있다. 표본 평균이 참 기울기에 가까워지는 속도는 개별 추정량의 분산에 좌우되므로, **불필요한 무작위 보상 항을 제거하는 것**이 수집량을 늘리는 것만큼 중요하다.

### 4.17 슬라이드 17 · 분산을 줄이는 두 방향

첫 방법은 독립적인 rollout 수 $N$을 늘려 표본 평균의 불확실성을 줄이는 것이다. 표준오차는 대체로 $1/\sqrt{N}$ 비율로 줄어드므로 절반으로 줄이려면 약 네 배의 독립 표본이 필요하다. 실제 환경에서는 비용이 크다.

둘째 방법은 기대 기울기를 크게 바꾸지 않으면서 각 표본이 지닌 불필요한 변동을 줄이는 것이다. 과거 보상을 빼는 reward-to-go, 상태별 평균 성과를 빼는 baseline, 학습한 가치함수로 미래를 추정하는 actor–critic이 이 방향이다.

표준오차를 절반으로 낮추는 데 약 네 배의 독립 rollout이 필요하다는 계산은 데이터 비용을 정량화한다. 로봇에서 한 rollout이 2분이고 100개가 필요하다면 단순 실행 시간만 200분이다. 초기화, 충돌 복구, 사람의 감독 시간은 별도다. baseline이나 critic을 쓰는 이유는 단순히 수식을 복잡하게 만들기 위해서가 아니라 이 비용을 줄이기 위해서다.

### 4.18 슬라이드 18 · 표본 수만 늘리기 어려운 이유

한 업데이트에 더 많은 rollout을 모으면 추정이 안정되지만 로봇에서의 시간과 장비 소모, 시뮬레이터 계산량이 증가한다. 수집한 데이터를 한 번의 정책경사 단계에만 사용한다면 특히 비싸다.

또한 여러 시점을 한 episode에서 뽑았다고 모두 독립 표본이 되는 것은 아니다. 같은 궤적의 관측과 보상은 상관되어 있다. 따라서 단순히 데이터 항목 수를 크게 표시하는 것과 실제 정보량을 늘리는 것은 다를 수 있다.

하나의 100단계 episode에서 100개 시점을 얻어도, 모두 같은 실패 사건과 초기 상태를 공유한다. 독립 episode 100개의 기울기와 같은 정보량으로 보면 안 된다. 평가 곡선에도 총 환경 단계 수와 독립 시작 횟수를 모두 적어야 표본 규모를 해석할 수 있다.

### 4.19 슬라이드 19 · 더 나은 가중치의 계보

전체 반환 $R(\tau)$ 대신 시점 $t$ 이후의 보상만 더한 reward-to-go $G_t$를 쓰면 과거 보상이 현재 행동의 공로로 잘못 들어가지 않는다. 그 미래 합의 기대값을 $Q^\pi(s_t, a_t)$라고 하며, 상태 평균 $V^\pi(s_t)$를 빼면 상대적인 장점 $A^\pi = Q^\pi - V^\pi$가 된다.

$$\nabla_\theta J = E\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, Q^\pi(s_t, a_t)\right] = E\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t)\, A^\pi(s_t, a_t)\right].$$

두 번째 등식은 행동과 무관한 상태 baseline의 기대 기여가 0이라는 성질을 사용한다. $Q$와 $V$를 신경망으로 근사하면 분산은 줄 수 있지만 근사 오차가 새로 생긴다.

baseline 항의 기대 기여는 한 상태에서 직접 보일 수 있다. $b(s)$가 행동에 의존하지 않으면 $\sum_a \pi_\theta(a \mid s)\nabla_\theta \log \pi_\theta(a \mid s) b(s) = b(s)\nabla_\theta \sum_a \pi_\theta(a \mid s) = 0$이다. 그래서 평균적으로는 기울기를 바꾸지 않으면서, 행동 결과가 그 상태의 보통 성과보다 나았는지 판단하는 기준을 제공한다.

여기까지의 간결한 식은 이 장에서 정의한 **비할인** 반환 $R(\tau) = \sum_k r_k$에 대응한다. 할인 목적 $J_\gamma = E[\sum_k \gamma^k r_k]$를 그대로 미분한다면 시점 $t$의 과거 보상은 인과성으로 빠지고 남는 항은 $\sum_{k \geq t} \gamma^k r_k = \gamma^t G_t^\gamma$다. 따라서 $G_t^\gamma = \sum_{k \geq t} \gamma^{k-t} r_k$를 쓸 때는 바깥의 $\gamma^t$가 필요한 관례가 있다. 할인 목적과 코드의 reward-to-go 정의를 먼저 고정하지 않으면 할인 지수를 한 번 빠뜨리거나 두 번 곱할 수 있다.

### 4.20 슬라이드 20 · Actor–Critic으로의 연결

이 장에서 얻은 REINFORCE는 정책경사의 기본형이다. 높은 분산과 데이터 비용을 해결하려면 어떤 행동이 그 상태의 평균보다 나았는지 추정해야 한다. 다음 장의 **actor**는 정책을 바꾸고 **critic**은 상태나 행동의 미래 가치를 평가한다.

전개를 따라갈 때 정책경사의 유도 자체와 $Q$ 또는 advantage를 **어떻게 추정하는가**를 분리하자. 같은 정책경사 식도 반환 추정 방법에 따라 REINFORCE, baseline 방법, actor–critic의 모습이 된다.

예를 들어 같은 정책 기울기 $\nabla \log \pi$에 전체 episode 반환을 곱하면 기본 REINFORCE, 현재 시점 이후의 반환을 곱하면 reward-to-go, critic이 추정한 $Q - V$를 곱하면 actor–critic의 한 형태가 된다. 차이는 정책망보다 **가중치를 어떻게 추정하고 어떤 편향·분산을 감수하는가**에 있다.

# 5 03-1. 가치함수와 Actor–Critic

정책경사는 기대 반환의 기울기를 정확한 형태로 표현하지만 표본으로 계산하면 크게 흔들린다. 이 장은 행동 이전에 이미 정해진 보상을 제거하고, 미래 보상을 가치함수로 추정하며, 상태의 평균 성과를 기준으로 행동의 상대적 장점을 계산한다. 이 과정을 거치면 **정책을 바꾸는 actor와 미래를 평가하는 critic**이 자연스럽게 등장한다.

## 5.1 슬라이드 01 · Actor–Critic의 출발점

REINFORCE는 한 궤적의 전체 반환으로 모든 행동을 가중한다. 높은 보상을 얻어도 어느 행동이 공을 세웠는지 구분이 어렵고, 긴 episode가 끝날 때까지 기다려야 할 수 있다. Actor–Critic은 별도의 가치 추정기를 두어 미래 보상에 대한 더 자주, 더 안정적인 피드백을 만든다.

정책 $\pi_\theta$가 actor, 상태가치 $V_\phi$나 행동가치 $Q_\phi$가 critic이다. 둘은 역할이 다르지만 같은 경험에서 함께 학습한다. critic이 틀리면 actor의 업데이트도 잘못된 방향으로 갈 수 있다는 의존성을 기억하자.

로봇이 컵을 옮길 때 팔을 접근시키는 행동과 마지막에 손을 닫는 행동은 성공에 미치는 역할이 다르다. 전체 성공 보상만 두 행동에 똑같이 붙이면 어느 단계가 병목인지 알기 어렵다. Critic은 각 상태나 행동에서 남은 성공 가능성을 추정해, 정책에 더 국소적인 학습 신호를 주려는 장치다.

## 5.2 슬라이드 02 · 분산 감소의 세 단계

첫 단계인 reward-to-go는 현재 행동이 과거 보상에 영향을 줄 수 없다는 인과성을 이용한다. 둘째 가치함수는 한 궤적의 우연한 미래 합 대신 여러 궤적의 평균적인 미래 성과를 추정한다. 셋째 baseline은 그 상태에서 보통 얻는 성과를 빼고 **평균보다 좋은 행동인지**를 본다.

이 세 단계는 단순한 기법 모음이 아니라 같은 정책경사 식의 가중치가 어떻게 개선되는지를 보여준다. 각 단계에서 기대값이 유지되는 부분과 근사 때문에 편향이 들어올 수 있는 부분을 나눠 읽자.

세 단계의 차이를 작은 예에 적용하자. 이미 받은 10점은 지금 행동의 가중치에서 빼고, 앞으로 얻을 5점을 사용하면 reward-to-go다. 현재 상태에서 보통 4점을 얻는다면 5점의 상대 성과는 $+1$이다. 한 번 관측한 5점 대신 같은 상태·행동에서의 평균을 예측하는 것이 critic의 역할이다. 각 단계가 제거하는 잡음의 종류가 다르다.

## 5.3 슬라이드 03 · 로그미분 유도 다시 보기

정책 파라미터를 $\theta$, 한 궤적을 $\tau$, 그 궤적의 전체 반환을 $R(\tau)$, 정책이 만든 궤적 확률밀도를 $p_\theta(\tau)$라고 하자. 기대 반환 $J(\theta) = \int p_\theta(\tau)R(\tau)d\tau$를 미분할 때 $\nabla_\theta p_\theta = p_\theta \nabla_\theta \log p_\theta$를 사용하면

$$\nabla_\theta J(\theta) = E_{\tau\sim p_\theta}[\nabla_\theta \log p_\theta(\tau)R(\tau)]$$

가 된다. 이 표현은 환경의 내부 물리를 미분하지 않고 정책이 생성한 궤적을 표본으로 평균할 수 있게 한다. 단, 기대값을 표본으로 바꾸는 순간 추정 오차가 생긴다.

## 5.4 슬라이드 04 · 정책 로그확률만 남기기

궤적 확률은 초기 상태, 정책의 행동 확률, 환경 전이 확률의 곱이다. 로그를 취하면 합이 되고, 환경의 확률 법칙이 정책 파라미터 $\theta$에 직접 의존하지 않는다면 다음이 된다.

$$\nabla_\theta \log p_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

환경을 미분하지 않지만 뒤 보상의 영향은 사라지지 않는다. 행동이 이후 상태를 바꾸는 효과가 실제 rollout의 반환 $R(\tau)$에 포함되어 이 로그확률 합을 가중한다.

### 5.5 슬라이드 05 · 표본 정책경사의 구조

$N$개의 현재 정책 궤적을 모으자. 위첨자 $i$는 $i$번째 궤적, $T_i$는 그 길이, $\widehat{g}$는 표본으로 추정한 정책경사 벡터다. 각 행동의 로그확률 기울기에 해당 궤적의 반환을 곱한 뒤 평균한다.

$$\widehat{g} = \frac{1}{N} \sum_{i=1}^{N} \sum_{t=0}^{T_i - 1} \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) R(\tau_i).$$

이 추정량은 기대값의 Monte Carlo 근사다. $R(\tau_i)$가 우연한 전이와 다른 시점의 행동 결과를 모두 포함하므로, 목적과 무관한 잡음이 한 행동의 업데이트에도 들어간다. 이번 장은 그 곱에 붙는 가중치를 바꿔 분산을 줄인다.

한 episode가 성공하면 초반 탐색 행동과 마지막 집기 행동이 모두 같은 $R(\tau)$를 받는다. 성공과 상관없는 초반의 우연한 흔들림도 강화될 수 있다. 이 때문에 기울기 식이 평균적으로 맞아도 한 batch의 업데이트는 실제 성능을 악화시킬 수 있다. **unbiased estimator(불편추정량)**라는 성질은 개별 업데이트의 품질을 보장하지 않는다.

### 5.6 슬라이드 06 · On-policy와 분산의 문제

현재 정책에서 모은 데이터라면 기대 정책경사와 연결되지만 표본 수가 적으면 방향이 심하게 흔들린다. 많이 모으면 비용이 크다. 오래된 정책의 데이터를 그대로 재사용하면 분포가 달라 또 다른 오차가 생긴다.

Actor–Critic은 critic을 학습해 한 전이에서도 미래 가치를 가늠하게 한다. 이 방식은 업데이트를 더 자주 할 수 있지만 critic의 근사 오차가 정책 개선의 편향으로 돌아올 수 있다. 분산과 편향을 함께 비교해야 한다.

같은 데이터를 여러 번 업데이트하면 계산 효율은 좋아지지만 첫 업데이트 뒤부터는 데이터 생성 정책과 현재 정책이 다르다. 반대로 매번 새 rollout만 모으면 환경 사용량이 커진다. Actor–Critic의 critic은 표본을 더 효과적으로 쓰려 하지만, 정책의 데이터 분포가 바뀔 때 가치 예측도 함께 따라가야 한다.

### 5.7 슬라이드 07 · 가중치를 단계적으로 바꾸기

전체 반환은 궤적 수준의 성과다. reward-to-go는 그 행동 이후의 결과만 남긴다. $Q^\pi(s,a)$는 그 미래 합의 조건부 기대값이다. $A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$는 같은 상태에서 평균 행동보다 얼마나 나은지를 나타낸다.

뒤로 갈수록 행동의 공로를 더 구체적으로 구분한다. 단, $Q$나 $V$를 표본과 신경망으로 배워야 하므로 직접 관측한 보상과 추정한 미래 가치를 섞는 지점을 추적하자.

네 가지 가중치를 동일한 상태의 예에 대입해 보자. 전체 반환이 12, 현재까지 이미 받은 보상이 7이면 reward-to-go는 5다. 같은 상태·행동의 평균 미래 성과 $Q$가 4.5이고 그 상태의 평균 $V$가 4라면 advantage는 0.5다. 숫자는 점점 작아지지만 **무엇과 비교한 값인지**가 더 중요해진다.

### 5.8 슬라이드 08 · 먼저 필요한 점수 함수의 성질

정규화된 확률밀도 $p_\theta(x)$에서 표본 $x$를 뽑는다고 하자. 점수 함수 $\nabla_\theta \log p_\theta(x)$의 기대값은 0이다. 아래 첫 등식은 $\nabla \log p = (\nabla p)/p$, 마지막 등식은 $\int p_\theta(x)dx = 1$을 사용한다.

$$E_{x\sim p_\theta}[\nabla_\theta \log p_\theta(x)] = \int p_\theta(x) \frac{\nabla_\theta p_\theta(x)}{p_\theta(x)} dx = \nabla_\theta \int p_\theta(x) dx = \nabla_\theta 1 = 0.$$

이 식은 확률의 총합이 언제나 1이라는 단순한 사실에서 나온다. reward-to-go와 baseline의 기대값 보존을 설명하는 핵심 도구다.

두 행동의 확률이 $p$와 $1-p$라면 로그확률 기울기의 평균은 $p\nabla \log p + (1-p)\nabla \log(1-p) = \nabla p + \nabla(1-p) = 0$이다. 어떤 행동의 확률만 계속 올리고 나머지를 그대로 둘 수 없다는 확률 합의 제약이 이 식에 들어 있다. 이 항등식은 특정 보상값을 빼도 평균 정책경사가 변하지 않는 조건을 밝힐 때 쓰인다.

### 5.9 슬라이드 09 · 점수 함수의 기대값이 0인 이유

이산 행동에서는 $\sum_a \pi_\theta(a \mid s) = 1$을 미분하면 $\sum_a \nabla_\theta \pi_\theta(a \mid s) = 0$이다. 한 항을 $\pi_\theta \nabla_\theta \log \pi_\theta$로 바꾸면 $E_{a\sim\pi_\theta}[\nabla_\theta \log \pi_\theta(a \mid s)] = 0$을 얻는다.

이는 같은 상태에서 어떤 행동은 확률이 증가하고 다른 행동은 줄어야 총합 1이 유지된다는 의미다. 행동과 무관한 값을 이 기울기에 곱하면 기대값에서는 사라진다. 하지만 개별 표본에서는 0이 아니므로 분산에는 영향을 줄 수 있다.

이산 행동 두 개에서 하나의 확률이 0.7에서 0.8로 오르면 다른 하나는 0.3에서 0.2로 내려간다. 점수 함수 평균이 0이라는 말은 모든 행동의 기울기가 0이라는 뜻이 아니다. 개별 표본의 로그확률 기울기는 남아 있고, 그것에 곱하는 가중치가 어떤 행동을 더 자주 만들지 결정한다.

### 5.10 슬라이드 10 · 과거 보상은 현재 행동의 결과가 아니다

시점 $t$의 행동을 정하기 전에 이미 받은 $r_0, \ldots, r_{t-1}$은 $a_t$로 바꿀 수 없다. 그런데 전체 궤적 반환을 곱하면 이 과거 보상도 $\nabla_\theta \log \pi_\theta(a_t \mid s_t)$를 가중한다. 기대값에는 도움이 없고 표본을 흔드는 성분이다.

예를 들어 로봇이 이미 첫 구간에서 보상을 받은 뒤 마지막에 컵을 놓친다면, 마지막 행동이 앞선 성공 보상을 받는 것은 공로를 잘못 배분한 것이다. 미래 보상만 쓰는 reward-to-go가 더 자연스러운 가중치다.

시점 0에서 동전을 던져 우연히 +10 또는 -10을 받았고, 시점 1에서 현재 행동의 결과로 +1 또는 0을 받는다고 하자. 시점 1의 행동 업데이트에 전체 반환을 쓰면 동전의 $\pm 10$이 큰 잡음이 된다. 동전 결과는 이미 정해졌으므로 그 항을 빼면 현재 행동의 효과를 더 선명하게 볼 수 있다.

### 5.11 슬라이드 11 · 전체 반환을 시점별로 나누기

정책경사의 시점 $t$ 항은 $\nabla_\theta \log \pi_\theta(a_t \mid s_t) \sum_{k=0}^{T-1} r_k$다. 보상 합을 $k < t$의 과거와 $k \geq t$의 현재·미래로 나누면, 현재 행동과 무관한 과거 성분을 따로 볼 수 있다.

$$\sum_{k=0}^{T-1} r_k = \underbrace{\sum_{k=0}^{t-1} r_k}_{\text{이미 일어난 결과}} + \underbrace{\sum_{k=t}^{T-1} r_k}_{\text{현재 이후의 결과}}.$$

이 분해가 곧 reward-to-go 유도의 출발점이다. 어떤 보상이 언제 관측되는지 코드 배열의 인덱스와 맞추어 보자.

예를 들어 보상이 $r_0 = 2, r_1 = -1, r_2 = 5$이고 $t = 2$라면 전체 반환은 6이지만 현재 행동 이후의 반환은 5다. 앞의 $2 - 1 = 1$은 $a_2$보다 먼저 관측되었다. 보상 배열과 행동 배열의 시간축을 맞추어 이 분해를 코드로 구현해야 reward-to-go가 한 칸 밀리지 않는다.

### 5.12 슬라이드 12 · 과거 보상 항의 기대값

과거 기록 $h_t = (s_0, a_0, r_0, \ldots, s_t)$를 고정하면 그 시점의 과거 보상 합도 고정된다. 이때 현재 행동만 평균내면 점수 함수의 기대값이 0이므로

$$E\left[\nabla_\theta \log \pi_\theta(a_t \mid s_t) \sum_{k<t} r_k \mid h_t\right] = \left(\sum_{k<t} r_k\right) E_{a_t \sim \pi_\theta(\cdot \mid s_t)}[\nabla_\theta \log \pi_\theta(a_t \mid s_t)] = 0.$$

따라서 전체 기대값도 0이다. 여기서 필요한 것은 과거 보상과 점수 함수가 무조건 독립이라는 주장보다 **과거를 조건으로 했을 때 현재 행동의 점수 함수 평균이 0**이라는 사실이다.

조건부 기대를 쓰는 이유는 과거 보상과 현재 상태가 서로 상관될 수 있기 때문이다. 과거를 고정하지 않은 채 "과거 보상은 현재 행동과 독립"이라고 말하면 일반적으로 틀릴 수 있다. 하지만 과거 $h_t$를 고정하면 그 보상 합은 상수이고, 현재 정책의 행동 확률을 합한 점수 함수의 평균은 정확히 0이다.

### 5.13 슬라이드 13 · Reward-to-go가 남긴 것

기대값이 0인 과거 항을 빼면 시점별 가중치는 $G_t = \sum_{k=t}^{T-1} r_k$가 된다. 정책경사는 $E[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t]$로 쓸 수 있다. 할인 보상에서는 $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$를 쓰되 목적함수의 할인 관례와 일치시켜야 한다.

과거 항은 평균이 0이어도 각 rollout에서는 0이 아니므로 추정량에 잡음을 더한다. 제거하면 일반적으로 분산을 줄이는 방향이지만, 구체적 분산 크기는 항 사이의 공분산에도 영향을 받는다.

할인 목적을 엄밀히 다룰 때는 전체 목표 $E[\sum_t \gamma^t r_t]$와 각 시점에서 다시 시작한 reward-to-go $G_t$를 구분해야 한다. 같은 기호라도 외부의 $\gamma^t$ 가중치가 남는 정책경사 관례가 있다. 강의나 구현의 식을 옮길 때 할인 지수를 한 번 더 곱하거나 빼지 않도록 목적함수부터 적자.

### 5.14 슬라이드 14 · 인과성의 직관

reward-to-go는 현재 행동이 **앞으로** 받는 결과만 가중한다. 전체 궤적 반환과 기대 기울기가 같은 이유는 이미 일어난 결과의 점수 함수 기여가 평균적으로 0이기 때문이다. 이는 보상을 미래에만 주는 새로운 과업을 정의한 것이 아니라 같은 목적의 더 나은 표본 추정량을 만든 것이다.

실제 코드는 뒤에서 앞으로 반환을 누적하면 효율적이다. 마지막 보상에서 시작해 $G_t = r_t + \gamma G_{t+1}$로 계산한다. 종료 이후의 가치는 더하지 않아야 한다.

뒤에서부터 계산하는 작은 예에서 $r_0 = 1, r_1 = 2, r_2 = 3, \gamma = 0.5$라면 $G_2 = 3$, $G_1 = 2 + 0.5 \cdot 3 = 3.5$, $G_0 = 1 + 0.5 \cdot 3.5 = 2.75$다. 같은 재귀식이 critic의 TD 목표 $r_t + \gamma V(s_{t+1})$에도 등장하지만, 실제 남은 반환 대신 **추정한 다음 상태 가치**를 쓴다는 점이 다르다.

### 5.15 슬라이드 15 · 가치함수가 필요한 순간

한 궤적의 reward-to-go는 실제 미래 보상을 담지만 우연한 전이와 이후 행동의 영향을 크게 받는다. 같은 상태·행동을 여러 번 겪었을 때 평균적으로 얼마나 보상을 얻는지 추정할 수 있다면 표본의 우연성을 줄일 수 있다. 이 조건부 기대값이 가치함수다.

가치함수는 보상 함수와 다르다. 보상 $r_t$는 한 단계의 즉시 평가이고, 가치는 현재에서 시작해 미래에 받을 보상의 기대 합이다. 좋은 상태에서는 당장 보상이 0이어도 높은 가치가 가능하다.

컵을 집는 같은 상태·행동을 여러 번 실행했는데 절반은 미끄러져 실패하고 절반은 성공한다고 하자. 한 실행의 reward-to-go는 두 값 중 하나지만, $Q^\pi(s, a)$는 그 무작위 결과의 평균을 나타낸다. Critic은 표본 하나의 요동을 줄일 수 있으나 그 평균을 충분한 데이터와 적절한 함수 근사로 배워야 한다.

### 5.16 슬라이드 16 · $V^\pi$와 $Q^\pi$의 차이

상태가치 $V^\pi(s)$는 상태 $s$에서 정책 $\pi$를 따를 때의 기대 반환이다. 행동가치 $Q^\pi(s,a)$는 첫 행동 $a$를 **고정한 뒤** 이후에 $\pi$를 따를 때의 기대 반환이다.

$$V^\pi(s) = E_\pi[G_t \mid s_t = s], \qquad Q^\pi(s,a) = E_\pi[G_t \mid s_t = s, a_t = a].$$

따라서 $V^\pi(s) = E_{a\sim\pi(\cdot|s)} Q^\pi(s,a)$다. $Q^\pi(s,a) = E[r_t + \gamma V^\pi(s_{t+1}) \mid s_t = s, a_t = a]$도 성립한다. 첫 식은 행동 평균, 둘째는 한 단계 보상과 남은 미래를 분리한 Bellman 관계다.

상태 $s$에서 정책이 행동 A와 B를 각각 0.25, 0.75 확률로 고르고 $Q^\pi(s,A) = 8$, $Q^\pi(s,B) = 4$라면 $V^\pi(s) = 0.25\cdot 8 + 0.75\cdot 4 = 5$다. A의 advantage는 +3, B의 advantage는 -1이다. 두 행동 모두 미래 보상이 양수여도 **상태의 평균에 비해** 하나는 강화하고 다른 하나는 약화할 수 있다.

### 5.17 슬라이드 17 · 가치와 advantage를 한 표기로

$V^\pi(s)$는 현재 상태의 평균적 전망, $Q^\pi(s,a)$는 특정 행동을 택했을 때의 전망이다. 둘의 차이

$$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$$

가 advantage다. 양수이면 현재 정책의 평균 행동보다 낫고, 음수이면 평균보다 나쁘다. "좋다"는 절대적인 보상값이 아니라 **그 상태와 그 정책에 대한 상대적 비교**다.

Bellman 식에서는 $V^\pi(s) = E_{a,s'}[r(s,a) + \gamma V^\pi(s')]$가 된다. 여기서 기대값은 현재 정책의 행동 선택과 환경 전이 두 가지에 대해 계산한다. $Q^\pi$의 첫 행동은 이미 고정되어 있으므로 행동 평균을 다시 넣지 않는다.

위 예에서 $A^\pi(s,A) = 3$, $A^\pi(s,B) = -1$을 정책 확률로 평균하면 $0.25\cdot 3 + 0.75\cdot(-1) = 0$이다. Advantage가 절대적 점수가 아니라 같은 상태의 평균을 기준으로 정의되었기 때문에 생기는 성질이다. 정책이 바뀌면 $V^\pi$와 두 advantage도 다시 달라진다.

### 5.18 슬라이드 18 · 상태가치의 학습 목표

진짜 상태가치 $V^\pi$ 대신 파라미터 $\phi$의 신경망 $V_\phi(s)$를 학습한다. 목표 $y_t$는 실제 반환 $G_t$로 둘 수도 있고, 한 단계 TD 값 $r_t + \gamma V_{\bar\phi}(s_{t+1})$로 둘 수도 있다. 여기서 $\bar\phi$는 목표를 계산할 때 고정한 가치망 파라미터다. 종료 상태에서는 미래 가치 항을 0으로 둔다. 다음 기대값은 수집한 상태들의 분포에 대한 평균이다.

$$\mathcal{L}_V(\phi) = E\big[(V_\phi(s_t) - y_t)^2\big].$$

TD 목표는 실제 전체 궤적을 기다리지 않아도 되지만 목표에 또 다른 가치 추정치가 들어간다. 이를 **부트스트랩**이라고 한다. 목표 네트워크 $\bar\phi$를 분리할 수도 있고, 현재 네트워크의 값을 gradient가 흐르지 않는 목표로 사용할 수도 있다.

**TD 손실이 작으면 가치 추정도 정확한가?** 반드시 그렇지는 않다. 목표 $r + \gamma V_\phi(s')$ 자체가 같은 근사기의 값이어서 일관되게 틀릴 수 있다. 보상 척도, 데이터 범위와 Monte Carlo 반환에 대한 별도 비교가 도움이 된다.

TD 목표를 계산할 때는 $y_t$를 상수처럼 취급해 현재 $V_\phi(s_t)$와의 차이를 줄인다. 같은 네트워크로 다음 상태를 평가하더라도 목표 쪽 gradient를 끊지 않으면 두 값이 동시에 움직여 의도한 회귀 문제가 달라질 수 있다. 코드에서 `detach`나 target network가 어떤 역할을 하는지 이 식과 연결해 보자.

### 5.19 슬라이드 19 · 할인율의 두 역할

할인 반환 $G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$에서 $\gamma$가 1에 가까우면 먼 미래를 많이 반영하고, 작으면 가까운 보상에 집중한다. $\gamma = 0.9$라면 네 단계 뒤의 보상에는 $0.9^4 \approx 0.656$의 가중치가 붙는다.

또한 계속되는 과업에서 유계 보상의 무한합을 다루기 쉽게 한다. 예를 들어 매번 1의 보상을 받으면 $\gamma = 1$에서는 합이 발산하지만 $0 \le \gamma < 1$에서는 $1/(1-\gamma)$다. 할인율은 시간 선호와 수학적 안정성 두 관점에서 읽는다.

$\gamma = 0.99$와 0.9는 가까워 보여도 100단계 뒤 보상에 붙는 가중치는 각각 약 0.366과 0.000027이다. 긴 과업에서 할인율의 작은 차이가 학습 목표의 시간 범위를 크게 바꾼다. 에피소드가 짧은지, 중요한 보상이 몇 단계 뒤에 오는지와 함께 값을 정해야 한다.

### 5.20 슬라이드 20 · Bellman 식에 할인율 넣기

미래 반환을 $G_t = r_t + \gamma G_{t+1}$로 나누고 조건부 기대값을 취하면

$$Q^\pi(s,a) = E[r(s,a) + \gamma V^\pi(s') \mid s, a],$$
$$V^\pi(s) = E_{a\sim\pi(\cdot|s),\, s'\sim P(\cdot|s,a)}[r(s,a) + \gamma V^\pi(s')].$$

이때 즉시 보상에 붙는 가중치는 1이고 다음 상태 가치에 $\gamma$가 붙는다. 코드에서 $\gamma$를 현재 보상에도 곱하거나 종료 상태의 다음 가치를 남겨 두면 다른 목표를 학습하게 된다.

상태 $s$에서 행동 $a$를 고른 뒤 즉시 보상 2를 받고 다음 상태 가치가 5이며 $\gamma = 0.9$라면 Bellman 목표는 $2 + 0.9 \cdot 5 = 6.5$다. 현재 보상과 미래 보상을 둘 다 0.9배 하면 6.3이 되어 다른 값을 학습한다. 종료라면 뒤의 5를 넣지 않아 목표는 2다. 작은 수치로 구현의 마스크를 점검할 수 있다.

### 5.21 슬라이드 21 · Reward-to-go를 더 안정적으로 추정하기

한 번 관측한 $G_t$는 $Q^\pi(s_t, a_t)$의 표본이다. 같은 상태·행동의 미래가 매번 다르면 표본 하나는 크게 흔들린다. 여러 궤적에서 조건부 평균을 배운 $Q_\phi(s,a)$는 우연한 결과를 평활화할 수 있다.

하지만 신경망이 실제 평균을 완벽하게 알지는 못한다. 더 낮은 분산과 더 큰 근사 편향의 절충이다. 학습 데이터가 적거나 상태–행동 영역 밖에서는 $Q_\phi$가 자신 있게 틀린 값을 낼 수 있다.

Critic을 회귀시킬 때 학습 데이터가 한 행동 주변에만 있다면 다른 행동의 Q를 정확히 알 수 없다. Actor가 가장 높다고 예측된 행동을 고르면, 데이터 밖에서 우연히 크게 나온 예측을 선택할 위험이 있다. 이 문제가 뒤의 off-policy 및 offline RL에서 더 크게 드러나므로, 가치함수의 부드러운 출력이 곧 근거 있는 평가라고 생각하지 말자.

### 5.22 슬라이드 22 · $Q$가 행동의 공로를 평가한다

reward-to-go는 한 궤적의 미래 합 $\widehat{Q}_t = G_t$를 정책경사의 가중치로 쓴다. 이상적인 $Q^\pi(s_t, a_t)$는 동일한 상태와 행동에서 가능한 여러 미래를 평균한 값이다. 그 값을 알면 환경 우연성의 일부를 평균으로 없앨 수 있다.

실제로는 $Q_\phi$를 데이터에서 학습한다. 앞으로 볼 actor–critic에서 critic의 역할이 이 미래 성과 추정이다. 정책 개선이 빠를수록 정책이 바뀌어 critic이 추정해야 하는 분포도 달라진다.

한 번의 성공 궤적만 보면 마지막 손가락 동작의 $G_t$도 높다. 하지만 같은 상태·행동을 100번 반복했을 때 10번만 성공한다면 평균 $Q^\pi$는 그 우연한 한 성공보다 훨씬 낮다. 행동의 공로를 평가하려면 **그 행동을 고정한 조건에서 가능한 미래를 평균**해야 한다는 뜻이다.

### 5.23 슬라이드 23 · 진짜 $Q$를 얻는 두 방법

한 상태–행동에서 아주 많은 미래 궤적을 수집해 평균내면 $Q^\pi$에 접근할 수 있다. 하지만 실제 로봇에서는 같은 상태로 되돌리는 것부터 어렵다. 그래서 관측한 전이에서 Bellman 관계를 이용해 $Q_\phi$를 학습하는 방법을 쓴다.

가치 추정으로 정책경사를 쓰면 $\nabla_\theta J = E[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) Q^\pi(s_t, a_t)]$다. 여기서 $Q_\phi$로 대체하면 계산할 수 있는 알고리즘이 되지만, 진짜 정책경사와의 차이를 critic의 오차가 만든다.

Monte Carlo 방법은 실제 끝까지 실행한 결과를 쓰므로 목표가 직접적이지만 완료까지 기다려야 하고 분산이 크다. TD 방법은 $r + \gamma V(s')$로 이른 갱신을 하지만 미래의 추정값을 다시 목표로 쓰므로 편향이 생길 수 있다. 어느 쪽이 좋은지는 episode 길이, 보상 희소성, critic 정확도에 달려 있다.

### 5.24 슬라이드 24 · 상태 baseline을 생각하기

같은 상태에서 평균적으로 이미 높은 점수를 얻는다면 한 행동의 큰 reward-to-go가 정말 특별한 성과인지 알기 어렵다. 상태별 기준 $b(s_t)$를 빼면 행동의 상대 성과를 보게 된다.

$b(s_t)$는 현재 행동을 고르기 전에 상태만으로 결정되어야 한다. 현재 행동 $a_t$에 의존하는 값을 마음대로 빼면 기대 정책경사가 달라질 수 있다. 다음 페이지에서 이 조건을 수식으로 확인한다.

쉬운 상태에서는 어떤 행동을 해도 10점 정도를 얻고 어려운 상태에서는 1점만 얻는다고 하자. 반환 5점을 받은 행동은 쉬운 상태에서는 나쁘고 어려운 상태에서는 좋다. 전체 점수를 그대로 비교하면 이 차이를 놓친다. $b(s)$는 상태별 난도의 기준점 역할을 한다.

### 5.25 슬라이드 25 · Baseline을 빼도 기대값이 같은 증명

현재 상태 $s$를 고정하면 $b(s)$는 행동과 무관하다. 따라서

$$
\begin{aligned}
E_{a \sim \pi_\theta(\cdot \mid s)}[\nabla_\theta \log \pi_\theta(a \mid s) b(s)] &= b(s) \sum_a \pi_\theta(a \mid s) \frac{\nabla_\theta \pi_\theta(a \mid s)}{\pi_\theta(a \mid s)} \\
&= b(s) \nabla_\theta \sum_a \pi_\theta(a \mid s) = 0.
\end{aligned}
$$

연속 행동이면 합 대신 적분을 쓴다. 이 성질로 $G_t$에서 $b(s_t)$를 빼도 정책경사의 기대값은 같다. 중요한 조건은 상태가 주어졌을 때 baseline이 **그때 샘플링한 행동에 의존하지 않는 것**이다.

**actor와 critic이 파라미터를 공유해도 baseline을 뺄 수 있나?** 한 정책 파라미터 값에서 baseline이 현재 샘플 행동에 의존하지 않으면 점수 함수와 baseline의 기대 곱은 0이다. 그러나 구현에서 actor 손실 $-\log \pi_\theta(a \mid s)(G - b_\theta(s))$ 전체를 그대로 자동미분하면 $b_\theta$를 미분하는 추가 경로가 생긴다. 표준 정책경사 추정에서는 advantage를 actor 손실에서 stop-gradient 처리하고 critic은 별도 가치 손실로 학습한다.

### 5.26 슬라이드 26 · 기대값 보존과 분산 감소는 별개

어떤 상태 baseline을 빼도 기대 기울기는 같지만 아무 baseline이나 분산을 줄이는 것은 아니다. $b(s)$가 미래 보상의 규모와 동떨어져 있으면 가중치 $G_t - b(s)$의 크기가 오히려 커질 수 있다.

좋은 baseline은 그 상태에서 예상되는 반환을 잘 설명해, 행동 결과의 **예상 밖 부분**만 남긴다. 상태가치 $V^\pi(s)$가 자연스러운 후보인 이유다. 학습 초기에 가치 추정이 불안정하면 정책 학습에도 영향을 주므로 baseline 손실과 정책 반환을 함께 본다.

예를 들어 $G_t$가 대략 10 근처인데 baseline을 -100으로 두면 가중치 $G_t - b(s)$는 약 110이 된다. 기대 기울기 자체는 그대로라도 개별 표본이 더 커져 분산이 증가할 수 있다. baseline을 학습한 critic의 값으로 두고 별도 가치 손실을 확인하는 이유가 여기에 있다.

### 5.27 슬라이드 27 · 분산을 실제로 비교하는 식

추정량의 평균은 baseline 전후에 같아도 분산은 $E[X^2] - E[X]^2$의 첫 항이 달라진다. 한 시점에서 점수 함수를 $g_t = \nabla_\theta \log \pi_\theta(a_t \mid s_t)$라고 두면 표본은 $g_t(G_t - b(s_t))$가 된다.

직관적으로 $G_t - b(s_t)$가 작을수록 표본의 크기와 흔들림이 줄 가능성이 있다. 하지만 최적 baseline은 일반적으로 단순한 $E[G_t \mid s_t]$와 정확히 같지는 않다. $g_t$의 크기까지 가중한 조건부 분산 최소화 해가 존재하며, 여러 시점의 공분산도 전체 분산에 영향을 준다.

한 시점의 분산을 최소화하는 baseline을 직관적으로 구하면 $E[\|g_t\|^2(G_t - b)^2 \mid s]$를 $b$에 대해 미분한다. $-2E[\|g_t\|^2(G_t - b) \mid s] = 0$이므로 $b^*(s) = E[\|g_t\|^2 G_t \mid s] / E[\|g_t\|^2 \mid s]$다. 점수 함수의 크기가 결과와 무관할 때에만 단순 평균 $V^\pi(s)$와 같다.

### 5.28 슬라이드 28 · 왜 $V^\pi(s)$를 선택하는가

상태가치 $V^\pi(s)$는 현재 상태에서 정책을 따랐을 때의 평균 반환이다. $G_t - V^\pi(s_t)$는 실현된 미래가 그 상태의 평균보다 얼마나 좋았는지 나타낸다. 같은 보상 10을 받아도 쉬운 상태의 평균이 12라면 나쁜 결과, 어려운 상태의 평균이 2라면 좋은 결과다.

$Q^\pi(s, a)$는 특정 행동의 평균 미래 성과이고 $V^\pi(s)$는 행동을 고르기 전의 평균이다. 두 값의 차이를 사용하면 궤적 우연성과 상태 난도를 동시에 어느 정도 분리할 수 있다.

분산의 정확한 최소해가 아니어도 $V^\pi$는 **행동을 고르기 전 평균 전망**이라는 해석이 쉽고 Bellman 관계로 학습할 수 있다. 같은 상태에서 미래 반환 10을 보았을 때 평소 12라면 advantage가 음수, 평소 2라면 양수다. 보상의 절대 크기보다 상태에서의 상대 성과를 배우는 것이다.

### 5.29 슬라이드 29 · 평균보다 좋은 행동 강화하기

정책경사의 표본 가중치를 $G_t - V_\phi(s_t)$로 바꾼다. 양수면 그 행동의 로그확률을 높이고, 음수면 낮춘다. 보상값이 모두 양수라고 무조건 모든 행동의 확률을 높이는 직관보다 정확하다.

$$\widehat{g} = \frac{1}{N} \sum_{i,t} \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) \big(G_t^i - V_\phi(s_t^i)\big).$$

가치함수는 현재 상태에서 기대되는 미래를 학습해야 한다. 다른 정책의 오래된 데이터로 학습한 값이라면 현재 정책의 기준선과 달라질 수 있다.

정책 손실을 구현할 때 $V_\phi(s_t)$를 actor의 가중치로 쓰는 경로와 critic의 회귀 손실을 구분해야 한다. Baseline을 추정하는 $\phi$의 학습은 가치 목표를 맞히고, $\theta$의 학습은 advantage에 따라 행동 확률을 바꾼다. 두 손실을 한 숫자로 합칠 수 있어도 각 항이 어떤 파라미터를 갱신하는지 확인해야 한다.

### 5.30 슬라이드 30 · Advantage와 Actor–Critic

한 궤적의 $G_t$ 대신 critic의 $Q_\phi(s_t, a_t)$를 쓰면, baseline을 뺀 값 $A_\phi(s_t, a_t) = Q_\phi(s_t, a_t) - V_\phi(s_t)$를 actor의 업데이트 가중치로 쓸 수 있다. 이것이 actor–critic의 대표적인 형태다.

$$\widehat{g}_{\mathrm{AC}} = \frac{1}{N} \sum_{i,t} \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) A_\phi(s_t^i, a_t^i).$$

정책은 행동을 생성하고 critic은 그 행동이 상태 평균보다 나았는지 평가한다. critic을 배우는 과정에는 TD 부트스트랩이 들어갈 수 있으므로 낮은 분산의 대가로 가치 근사 오차를 받아들인다.

Actor는 평가자가 준 advantage가 양수인 행동을 늘리고 음수인 행동을 줄인다. 하지만 critic이 실제로 나쁜 행동에 높은 $Q_\phi$를 주면 actor는 바로 그 행동을 강화할 수 있다. 따라서 critic의 TD 손실이 낮은지만 보지 말고 실제 rollout 반환과 가치 예측이 함께 움직이는지도 살펴야 한다.

### 5.31 슬라이드 31 · 네 가지 가중치의 비교

전체 반환은 한 episode의 성과를 모든 행동에 준다. Reward-to-go는 현재 이후 보상만 남긴다. $Q_\phi(s,a)$는 같은 조건의 여러 미래를 평균하려 하고, advantage는 상태의 평균을 빼서 상대 성과를 본다.

이 계보에서 뒤의 방법이 무조건 더 정확한 것은 아니다. 실제 반환은 잡음이 크지만 관측한 사실이고, 학습된 $Q_\phi$는 부드럽지만 틀릴 수 있다. 데이터가 적거나 정책이 빠르게 바뀌면 critic의 오류가 커질 수 있다.

같은 한 궤적의 숫자로 비교해 보자. 전체 반환은 20, 현재 시점 이전 보상은 12라면 reward-to-go는 8이다. Critic의 $Q_\phi(s,a)=7$, $V_\phi(s)=6$이면 advantage는 1이다. 20→8은 인과성으로 과거를 지운 것이고, 8→7은 모델의 평균 추정, 7→1은 상태 기준선을 뺀 것이다. 마지막 숫자가 작다고 반드시 더 좋은 추정량이라는 뜻은 아니다.

### 5.32 슬라이드 32 · REINFORCE의 데이터 요구

REINFORCE는 현재 정책의 완성된 궤적을 모아 전체 반환을 계산하고 한 번의 정책 업데이트에 사용한다. 종료 보상만 있는 과업에서는 episode가 끝날 때까지 기다려야 한다. 길이가 1000인 과업이라면 한 정책 업데이트에 최소 1000개의 환경 단계가 필요할 수 있다.

이 단순함은 장점도 있다. 가치함수를 따로 학습하지 않으므로 critic의 부트스트랩 편향은 없다. 짧은 과업이나 유도·디버깅에는 좋은 기준선이며, 다른 알고리즘의 편향이 성능에 어떤 영향을 주는지 비교할 때 유용하다.

episode 길이가 1000이고 매번 전체 반환을 기다린다면 처음 10단계에서 관측한 중요한 보상도 나머지 990단계가 끝날 때까지 정책 갱신에 쓰지 못한다. Actor–Critic은 TD 목표로 중간에 갱신할 수 있지만, 미래를 완전히 관측하기 전의 가치 추정에 의존한다. 업데이트 시점과 추정의 신뢰도 사이의 절충이다.

### 5.33 슬라이드 33 · Actor와 critic의 반복

Actor–Critic은 현재 정책으로 전이를 모으고, critic을 가치 목표에 맞게 업데이트한 뒤, 그 critic의 advantage로 actor를 바꾼다. 전체 episode가 끝나지 않아도 TD 목표로 critic을 갱신할 수 있다.

다만 가치망이 새 정책의 성과를 정확히 추정하기 전에 actor를 너무 많이 움직이면 두 네트워크가 서로를 불안정하게 만들 수 있다. critic 손실, advantage의 분포와 정책 변화량을 함께 확인하는 이유다.

학습 순서를 예로 쓰면 rollout에서 $(s_t,a_t,r_t,s_{t+1})$를 모은 뒤 critic의 $r_t+\gamma V(s_{t+1})$ 목표를 계산하고, 그 예측으로 advantage를 만든 뒤 actor를 갱신한다. 다음에는 달라진 actor가 다른 전이를 모은다. Critic만 오래 갱신하거나 actor만 크게 움직이면 두 단계가 서로 다른 정책의 분포를 바라볼 수 있다.

### 5.34 슬라이드 34 · 표본 효율의 정확한 비교

그림은 길이 1000인 episode마다 갱신하는 REINFORCE와 10단계씩 모아 자주 갱신하는 Actor–Critic을 비교한다. 이는 **같은 환경 단계 수에서 더 많은 갱신 기회가 있다**는 뜻이다. 그러나 갱신 횟수가 많다고 최종 성능이 자동으로 높은 것은 아니다.

짧은 rollout에서는 미래 보상을 critic에 더 많이 의존한다. critic의 편향이 심하면 잦은 갱신이 오히려 해롭다. 샘플 효율은 특정 목표 점수에 도달하기 위해 필요한 환경 단계 수로 비교해야 한다.

"업데이트를 100번 했다"보다 "평가 반환 200에 도달하는 데 환경 전이 몇 개를 썼나"가 sample efficiency의 직접적인 비교다. 다만 같은 전이를 100번 gradient 계산에 재사용했다면 compute budget도 함께 적어야 한다. 환경 단계 수와 계산량은 서로 다른 비용이므로 한 지표로 섞지 말자.

### 5.35 슬라이드 35 · 다음 단계: 길이와 재사용

한 단계 TD는 빠르지만 critic의 추정에 크게 의존하고, 긴 실제 반환은 상대적으로 직접적이지만 분산이 크다. 다음 장의 $n$단계 반환과 GAE는 그 사이의 길이를 조절한다.

또한 현재 정책 데이터만 사용하는 한계를 완화하려고 importance sampling으로 오래된 정책의 데이터를 보정하고, PPO는 정책비의 변화를 제한하며 같은 batch에서 여러 번 갱신한다. 이때도 어느 정책이 데이터를 만들었는지를 기록해야 한다.

다음 장의 $n$단계 반환은 $n = 1$이면 빠른 TD에 가깝고, $n$이 episode 남은 길이만큼 커지면 Monte Carlo 반환에 가까워진다. GAE는 여러 길이의 TD 잔차를 감쇠 가중치로 합쳐 한 숫자의 advantage로 만든다. 길이를 조절하면 **근사 편향과 표본 분산**이 어떻게 바뀌는지 주목하자.

# 6 03-2. GAE와 PPO

Actor–Critic은 가치 추정으로 정책을 자주 갱신하지만 두 문제가 남는다. 짧은 TD 목표는 critic 오류에 민감하고, 현재 정책의 rollout을 한 번 쓰고 버리면 데이터가 비싸다. **GAE**는 advantage 추정의 길이를 조절하고, **PPO**는 수집한 batch에서 여러 번 갱신할 때 정책 변화가 지나치게 커지는 일을 완화한다.

## 6.1 슬라이드 01 · PPO가 해결하려는 문제

PPO는 정책경사, 상태가치 baseline, advantage 추정, 중요도 비율과 제한된 정책 갱신을 결합한다. 핵심은 알고리즘 이름보다 **이전 정책에서 얻은 데이터를 현재 정책의 업데이트에 얼마나 안전하게 쓸 수 있는가**다.

이미 수집한 rollout을 여러 번 사용하면 환경 상호작용당 계산량을 늘릴 수 있다. 그러나 첫 갱신 뒤 정책이 달라졌으므로 그 데이터는 더 이상 엄밀한 의미의 현재 정책 데이터가 아니다. 이 간격을 계산하고 너무 큰 이득을 억제하는 방식이 PPO의 중심이다.

로봇에서 한 rollout을 모으는 데 10분이 걸리면 같은 batch로 여러 gradient update를 하는 유인이 크다. 그러나 그 사이 정책이 크게 달라지면 수집된 행동은 새 정책의 대표 표본이 아니다. PPO는 **자료를 모으는 비용**과 **같은 자료를 반복해서 학습하는 위험** 사이의 절충으로 이해할 수 있다.

## 6.2 슬라이드 02 · 두 개선 방향

첫째, actor가 쓰는 advantage의 품질을 높인다. 한 단계 TD는 빠르지만 가치함수 오류에 민감하고, 완전한 Monte Carlo 반환은 직접적이지만 분산이 크다. $n$단계 반환과 GAE로 사이를 조절한다.

둘째, 하나의 rollout batch에서 여러 번 파라미터를 바꾼다. 중요도 비율로 이전 정책과 새 정책의 행동 확률 차이를 측정하고, 정책이 크게 바뀌었을 때 목적함수가 주는 추가 이득을 제한한다.

첫 개선은 각 표본에 붙는 숫자 $\widehat{A}_t$의 품질이고, 둘째 개선은 그 표본을 몇 번까지 쓸지다. GAE의 $\lambda$를 잘 골라도 정책이 멀리 이동하면 과거 자료 문제는 남는다. 반대로 clipping을 잘해도 critic의 값이 엉터리면 좋은 행동의 확률을 잘못 낮출 수 있다. 두 축의 로그를 따로 보는 것이 디버깅의 출발점이다.

## 6.3 슬라이드 03 · 정책경사 가중치의 계보

전체 반환, reward-to-go, $Q^\pi(s,a)$, advantage는 모두 점수 함수 $\nabla_\theta \log \pi_\theta(a \mid s)$에 곱할 성과 신호다. Advantage는 같은 상태에서 평균적인 가치 $V^\pi(s)$를 $Q^\pi(s,a)$에서 뺀 값이다. 각각 과거 보상의 잡음과 상태 난도의 영향을 다르게 제거한다.

실제 PPO에서는 $A^\pi$를 정확히 알 수 없으므로 가치망과 관측 보상으로 $\widehat{A}_t$를 만든다. 정책 목적함수의 공식이 정확해도 advantage가 심하게 잘못되면 갱신 방향이 틀릴 수 있다.

예를 들어 같은 상태의 보통 반환이 20인데 한 행동 뒤의 추정 반환이 15라면, 보상이 양수라도 advantage는 -5다. PPO는 그 행동의 확률을 낮추려 한다. 보상의 부호와 advantage의 부호를 동일하게 생각하면 clipping을 해석할 때 특히 혼동하기 쉽다.

## 6.4 슬라이드 04 · REINFORCE가 기다리는 것

REINFORCE의 기본형은 끝난 궤적의 전체 보상으로 정책을 업데이트한다. 장기 과업에서 한 episode가 길고 보상이 마지막에만 있으면 갱신까지 많은 환경 단계가 필요하다. 가치함수가 없으므로 단순하고 설명하기 좋지만 표본 분산이 클 수 있다.

PPO의 개선을 이해할 때 REINFORCE를 단순히 "옛 알고리즘"으로 보면 안 된다. 어떤 편향을 추가해서 데이터 사용을 줄였는지 비교하는 기준선이다.

최종 성공 보상만 있는 1000단계 과업에서는 앞 10단계의 행동도 episode가 끝나야 결과를 알 수 있다. REINFORCE는 가치망의 근사 오류가 없지만 긴 기다림과 큰 표본 분산을 감수한다. 그래서 actor–critic을 비교할 때 단순히 업데이트 횟수보다 같은 환경 단계에서 도달한 성과를 봐야 한다.

### 6.5 슬라이드 05 · Actor–Critic의 한 단계

Actor–Critic은 상태가치나 행동가치를 학습해 완전한 episode를 기다리지 않고 미래를 추정한다. actor는 $\widehat{A}_t$의 부호와 크기를 이용해 정책 로그확률을 갱신한다. $\widehat{A}_t > 0$이면 관측 행동의 확률을 높이고, 음수면 낮춘다.

critic의 학습 목표가 $r_t + \gamma V_\phi(s_{t+1})$처럼 자기 추정에 의존하면 갱신이 빨라지는 대신 편향이 생긴다. 다음 단계는 여러 단계의 실제 보상과 끝의 가치 추정을 섞어 이 절충을 조절한다.

현재 가치 추정이 $V(s_t) = 4$, 다음 상태 추정이 $V(s_{t+1}) = 6$, 보상이 1, $\gamma = 0.9$라면 한 단계 TD 잔차는 $\delta_t = 1 + 0.9 \cdot 6 - 4 = 2.4$다. 이 값은 "현재 행동 뒤의 결과가 가치망의 이전 전망보다 얼마나 좋았는가"를 나타낸다. 뒤의 GAE는 이런 잔차를 여러 시점에서 합친다.

### 6.6 슬라이드 06 · 자주 갱신할 수 있다는 뜻

한 episode가 1000단계인데 actor–critic이 10단계마다 가치 추정으로 갱신할 수 있다면 같은 경험에서 훨씬 많은 갱신 기회를 얻는다. 하지만 10단계 뒤의 나머지 결과는 critic에 의존한다. 업데이트 수 증가와 실제 성능 개선은 별개다.

표본 효율은 목표 점수에 도달하는 데 필요한 환경 단계 수로 평가한다. critic의 추정 오차가 크면 잦은 갱신이 오히려 학습을 불안정하게 할 수 있다.

업데이트를 자주 한다는 말은 같은 데이터를 무한히 써도 된다는 뜻이 아니다. 10단계마다 새 표본을 모은다면 데이터는 계속 현재 정책에서 나오지만, 한 10단계 batch만 계속 재사용하면 정책과 수집 분포의 차이가 커진다. **갱신 빈도**와 **한 batch의 재사용 횟수**를 구분해야 한다.

### 6.7 슬라이드 07 · 남은 두 약점

짧은 반환 추정은 분산을 낮추지만 가치망이 틀린 만큼 편향될 수 있다. 또한 현재 정책이 만든 데이터만 사용하는 순수 정책경사는 한 번 갱신 뒤 같은 batch를 그대로 재사용하기 어렵다.

GAE는 첫 약점을, 중요도 비율과 PPO 목적함수는 두 번째 약점을 다룬다. 두 기법은 서로 다른 대상을 바꾸므로 분리해 이해하면 구현 오류를 찾기도 쉽다.

GAE가 다루는 것은 "미래를 얼마나 실제 보상으로 보고, 언제 가치망의 예측으로 마무리할까"다. PPO clipping이 다루는 것은 "그 추정을 이용해 정책을 얼마나 움직일까"다. 두 질문을 한 기법으로 생각하면 GAE의 $\lambda$와 PPO의 $\epsilon$이 왜 서로 다른 하이퍼파라미터인지 이해하기 어렵다.

### 6.8 슬라이드 08 · $n$단계 반환

$r_{t+l}$은 시점 $t+l$의 보상, $\gamma$는 한 단계 할인율, $n$은 실제로 관측해 합칠 보상의 개수다. $V_\phi(s)$는 파라미터 $\phi$로 학습한 상태가치 추정치다. 모자 쓴 $\widehat{Q}_t^{(n)}$과 $\widehat{A}_t^{(n)}$은 각각 시점 $t$의 $n$단계 행동가치와 advantage **추정치**를 뜻한다.

$$\widehat{Q}_t^{(n)} = \sum_{l=0}^{n-1} \gamma^l r_{t+l} + \gamma^n V_\phi(s_{t+n}), \qquad \widehat{A}_t^{(n)} = \widehat{Q}_t^{(n)} - V_\phi(s_t).$$

$n=1$은 빠르지만 가치망을 많이 믿고, $n$이 episode 끝까지 가면 실제 반환을 많이 반영한다. 끝 상태가 진짜 종료라면 마지막 부트스트랩 항은 0이다. 어떤 $n$이 좋은지는 과업의 보상 지연과 가치망의 정확도에 따라 달라진다.

$n=2$라면 $\widehat{Q}_t^{(2)} = r_t + \gamma r_{t+1} + \gamma^2 V(s_{t+2})$다. 예를 들어 보상 1, 2와 $V(s_{t+2})=5$, $\gamma=0.9$를 쓰면 $1+1.8+4.05=6.85$다. 한 단계 목표보다 실제로 관측한 보상 하나를 더 쓰지만, 끝의 가치 추정 오차는 여전히 남는다.

**rollout이 시간 제한으로 잘리면 마지막 가치가 항상 0일까?** 진짜 종료 상태라면 뒤에 받을 보상이 없으므로 0이다. 단순히 수집 시간을 채워 잘렸고 과업은 계속된다면 다음 상태의 가치를 bootstrap해야 한다. 두 종료 원인을 같은 마스크로 처리하면 가치와 GAE가 왜곡된다.

## 6.9 슬라이드 09 · GAE의 가중 평균과 재귀

GAE의 $\lambda \in [0,1]$는 긴 반환에 줄 가중치를 정한다. $T$를 현재 rollout이 끝나는 시점, $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$를 한 단계 TD 잔차라 두자. 이 잔차를 뒤에서부터 합치면 여러 길이의 advantage를 평균한 값과 같다. 아래 재귀식의 $\widehat{A}_{t+1}$도 같은 GAE 추정치이며 끝에서는 $\widehat{A}_T = 0$으로 놓는다.

$$\widehat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l} = \delta_t + \gamma\lambda \widehat{A}_{t+1}.$$

$\lambda=0$이면 한 단계 TD advantage이고, $\lambda=1$에 가까우면 긴 실제 반환을 많이 반영한다. $\lambda$를 키울수록 가치 근사에 따른 짧은 지평 편향은 줄 수 있지만 표본 분산은 커질 수 있다. 종료와 시간 제한 처리에 따라 경계항이 달라진다. GAE 논문

**GAE를 단순한 평균으로 보면 왜 부족한가?** 각 길이의 advantage를 동일 가중으로 평균하지 않는다. $\lambda$가 긴 길이의 비중을 지수적으로 조절하고 $\gamma$가 보상의 시간 가중을 함께 정한다. 구현은 TD 잔차를 뒤에서 앞으로 재귀적으로 합치는 편이 정확하다.

두 단계만 있는 예에서 $\delta_t=2$, $\delta_{t+1}=-1$, $\gamma=0.9$, $\lambda=0.5$라면 $\widehat{A}_t = 2 + 0.9\cdot 0.5\cdot(-1) = 1.55$다. 한 단계 TD만 쓰면 2이고, 더 긴 결과를 반영하면 뒤의 나쁜 결과가 현재 행동의 평가를 낮춘다. $\lambda$는 먼 TD 잔차의 반영 정도를 조절한다.

왜 여러 길이의 반환이 TD 잔차 합으로 바뀌는지 직접 전개해 보자. $\delta_{t+j} = r_{t+j} + \gamma V(s_{t+j+1}) - V(s_{t+j})$를 $\sum_{j=0}^{n-1}\gamma^j\delta_{t+j}$에 대입하면 가운데의 $+\gamma^{j+1}V(s_{t+j+1})$와 다음 항의 $-\gamma^{j+1}V(s_{t+j+1})$가 차례로 상쇄된다. 처음의 $-V(s_t)$와 마지막의 $+\gamma^n V(s_{t+n})$만 남으므로 이 합은 정확히 $\widehat{Q}_t^{(n)} - V(s_t) = \widehat{A}_t^{(n)}$다. GAE는 길이가 다른 이 합들에 $\lambda$ 가중치를 주므로, 오래된 잔차일수록 $(\gamma\lambda)^j$가 붙는다.

남은 길이를 $H=T-t$라고 놓고, 길이 $n<H$의 가중치를 $w_n=(1-\lambda)\lambda^{n-1}$, 마지막 길이의 가중치를 $w_H=\lambda^{H-1}$로 정하자. 실제로 이 가중치의 합은 $(1-\lambda)\sum_{n=0}^{H-2}\lambda^n + \lambda^{H-1} = 1$이다. 따라서

$$\begin{aligned}\widehat{A}_t^{\mathrm{GAE}} &= (1-\lambda)\sum_{n=1}^{H-1}\lambda^{n-1}\widehat{A}_t^{(n)} + \lambda^{H-1}\widehat{A}_t^{(H)} \\ &= \sum_{j=0}^{H-1}\gamma^j\delta_{t+j}\underbrace{\sum_{n=j+1}^{H} w_n}_{=\lambda^j} = \sum_{j=0}^{H-1}(\gamma\lambda)^j\delta_{t+j}.\end{aligned}$$

내부 합 $\sum_{n=j+1}^{H} w_n = \lambda^j$는 $j$번째 잔차를 포함하는 모든 길이의 총가중치다. 시간 제한으로 rollout을 잘랐다면 마지막 $\widehat{A}_t^{(H)}$에는 $V(s_T)$ 부트스트랩이 남는다.

**$\lambda=1$이면 언제 Monte Carlo advantage와 같아지나?** 수집 구간이 진짜 종료까지 이어지고 종료

상태의 $V$를 0으로 두면 잔차 합의 중간 가치가 모두 지워져 $G_t - V(s_t)$가 된다. 단순 시간 제한에서 수집을 멈췄다면 마지막 $V(s_T)$를 남겨 부트스트랩하므로 순수 Monte Carlo 반환과 같지 않다.

### 6.10 슬라이드 10 · Advantage 추정과 데이터 재사용은 별개

GAE를 잘 계산해도 actor의 데이터가 어느 정책에서 왔는지는 변하지 않는다. 현재 정책의 rollout에 대해 한 번 갱신하고 나면 같은 상태–행동 표본은 과거 정책 분포의 자료가 된다.

반대로 중요도 비율로 분포를 보정해도 advantage가 엉터리면 학습 방향은 좋지 않다. PPO 구현에서는 가치망 목표와 GAE, 이전 행동 로그확률의 저장을 따로 검증해야 한다.

PPO 구현에서 같은 버퍼에 두 종류의 숫자가 있다. $\widehat{A}_t$는 수집된 보상과 가치 추정에서 계산하고, $\log \pi_{\text{old}}(a_t \mid s_t)$는 행동을 뽑던 순간의 정책 확률이다. 둘을 현재 정책으로 다시 계산하면 각각의 기준이 흔들린다. 특히 old log probability는 한 rollout batch의 여러 epoch 내내 고정해야 한다.

### 6.11 슬라이드 11 · 왜 현재 정책 데이터가 필요한가

기본 정책경사의 기대값은 $\tau \sim p_\theta$에 대한 것이다. 파라미터를 $\theta$에서 $\theta'$로 바꾸면 원하는 기울기는 $E_{\tau \sim p_{\theta'}}$가 되지만 손에 든 batch는 여전히 $p_\theta$에서 왔다.

정책 변화가 작으면 근사적으로 쓸 수 있어도 여러 업데이트를 반복하면 차이가 커진다. 단순히 같은 로그확률 손실을 여러 번 줄이는 것은 원래 유도의 표본 가정을 벗어난다.

예를 들어 이전 정책이 어떤 행동을 0.2 확률로 골랐는데 새 정책은 0.8 확률로 고른다면, 이전 batch에는 그 행동의 결과가 새 정책에서 필요한 빈도보다 훨씬 적다. 기존 한 표본을 네 배로 가중해 평균을 맞추려 할 수 있지만, 실제 표본 하나가 우연한 성공이었다면 그 우연도 네 배가 된다. 분포 보정과 분산 문제는 함께 생긴다.

### 6.12 슬라이드 12 · 중요도 샘플링의 기본식

분포 $q(x)$에서 뽑은 표본으로 $p(x)$의 기대값을 계산하려면 $p(x)/q(x)$를 곱한다.

$$E_{x\sim p}[f(x)] = \int q(x)\frac{p(x)}{q(x)} f(x)\,dx = E_{x\sim q}\left[\frac{p(x)}{q(x)} f(x)\right].$$

이 식이 성립하려면 $p(x) > 0$인 곳에서 $q(x) > 0$이어야 한다. 즉 이전 정책이 전혀 선택하지 않은 행동에 대해서는 데이터 재가중만으로 새 정책의 결과를 알 수 없다. 비율이 지나치게 크면 추정량의 분산도 커진다.

두 값의 장난감 예로 $q(a_1) = 0.25$, $p(a_1) = 0.5$라면 $a_1$ 표본의 비율은 2다. 반대로 $q(a_2) = 0.75$, $p(a_2) = 0.5$이면 비율은 $2/3$이다. $q$에서 얻은 표본을 이 비율로 가중하면 $p$가 더 자주 낼 행동은 키우고 덜 낼 행동은 낮춘다. $q(a_1) = 0$이면 비율 자체가 정의되지 않으므로 **없는 자료를 가중치로 만들어 낼 수는 없다.**

### 6.13 슬라이드 13 · 궤적 비율에서 행동 비율로

길이 $T$의 궤적 $\tau = (s_0, a_0, \ldots, s_T)$를 생각하자. $\pi_{\text{old}}$는 이 궤적을 수집한 정책, $\pi_\theta$는 파라미터 $\theta$로 갱신하는 정책이다. $p_{\text{old}}(\tau)$와 $p_\theta(\tau)$는 각각 두 정책 아래 같은 궤적의 확률이다. 초기 상태분포와 환경 전이확률이 같고 분모가 양수인 궤적에서는 그 항들이 비율에서 상쇄된다.

$$\frac{p_\theta(\tau)}{p_{\text{old}}(\tau)} = \prod_{t=0}^{T-1} \frac{\pi_\theta(a_t \mid s_t)}{\pi_{\text{old}}(a_t \mid s_t)}.$$

한 시점의 행동 확률비를 $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$라고 정의하자. 전체 궤적 비율은 $\prod_t \rho_t(\theta)$다. 긴 궤적에서는 이 곱의 분산이 커지므로 PPO는 시점별 $\rho_t$를 surrogate 목적에 쓴다. 보상 $r_t$와 확률비 $\rho_t$를 구별하자.

각 단계의 비율이 1.1이어도 길이 100의 궤적 비율은 $1.1^{100} \approx 13780$이다. PPO의 한 단계 비율은 전체 궤적 기대값을 정확히 보정하는 가중치가 아니라, 정책을 점진적으로 갱신하기 위한 surrogate의 요소다.

### 6.14 슬라이드 14 · 여러 갱신에서 이전 정책 고정하기

rollout 직후 행동마다 수집 정책의 로그확률 $\log \pi_{\text{old}}(a_t \mid s_t)$를 저장하고 갱신 중 고정한다. 현재 정책의 같은 행동 로그확률을 구해 $\rho_t(\theta) = \exp\{\log \pi_\theta(a_t \mid s_t) - \log \pi_{\text{old}}(a_t \mid s_t)\}$로 계산한다.

같은 batch를 여러 epoch 학습하려면 분모를 업데이트마다 새로 계산하면 안 된다. 그러면 "수집 당시 정책과 지금 정책"의 비가 아니게 된다. 중요도 비율은 계산만으로 안정성을 보장하지 않아 추가적인 갱신 제어가 필요하다.

저장된 값이 $\log \pi_{\text{old}}(a_t \mid s_t) = -1.2$이고 현재 값이 $-1.0$이라면 비율은 $\exp(0.2) \approx 1.22$다. 로그확률을 빼고 지수화하면 매우 작은 확률도 직접 나누는 것보다 계산이 안정적이다. 분모를 현재 정책으로 매번 다시 계산하면 비율이 늘 1 근처가 되어 clipping의 의미가 사라진다.

### 6.15 슬라이드 15 · 큰 중요도 비율과 정책 변화

좋은 결과를 낸 행동의 확률을 계속 높이면 $\rho_t$가 커진다. 한 batch를 반복 학습할수록 몇 표본의 큰 비율이 손실과 기울기를 지배하고, 그 표본을 낳은 상태 분포와 새 정책의 상태 분포도 멀어진다.

파라미터 거리 $\|\theta - \theta_{\text{old}}\|$는 정책 거리의 좋은 대리변수가 아닐 수 있다. 신경망에서는 작은 파라미터 이동도 출력 확률을 크게 바꿀 수 있기 때문이다. TRPO는 정책 간 KL 발산을 제한하는 신뢰영역 관점을 쓰고, PPO는 훨씬 간단한 목적함수의 clipping으로 큰 변화를 억제한다. **PPO clipping은 KL 제약을 엄밀하게 보장하지 않는다.** TRPO, PPO

실험에서는 평균 KL과 clip fraction(비율이 clip 구간 밖에 놓인 표본의 비율)을 함께 관찰한다. PPO의 clipping이 목적함수에 주는 제어와 실제 정책 변화의 크기를 나란히 확인할 수 있다.

### 6.16 슬라이드 16 · PPO를 구성하는 네 조각

PPO-Clip은 actor–critic의 가치 추정, GAE advantage, 행동 확률비, 비율 clipping을 결합한다. 수집한 batch의 표본 수를 $N$, 각 표본에서 이미 계산해 고정한 advantage를 $\widehat{A}_t$라 하자. 출발점인 surrogate $\widehat{L}(\theta) = N^{-1} \sum_t \rho_t(\theta) \widehat{A}_t$는 좋은 결과의 행동 확률을 올리고 나쁜 결과의 확률을 내린다. 같은 batch를 오래 최적화하면 비율이 과하게 변한다.

여기서 $\widehat{A}_t$는 rollout과 critic으로 미리 계산한 값으로 취급한다. actor 손실을 미분할 때 advantage 계산 그래프까지 역전파하는 구현과는 다르다. critic은 별도의 회귀 목표로 갱신한다. PPO 원논문

구현에서는 수집 당시의 로그확률과 계산된 $\widehat{A}_t$를 모두 고정한다. Actor 손실의 역전파가 critic 추정치까지 통과하지 않도록 advantage를 `detach`하고, critic은 별도 가치 손실로 학습한다.

### 6.17 슬라이드 17 · Advantage 부호에 따른 clipping 직관

Clip 폭 $\epsilon \in (0, 1)$를 정한다. $\widehat{A}_t > 0$인 행동은 비율 $\rho_t$가 커질수록 유리하지만 $\rho_t > 1 + \epsilon$에서는 목적의 추가 이득을 멈춘다. $\widehat{A}_t < 0$인 행동은 비율을 줄이는 쪽이 유리하며 $\rho_t < 1 - \epsilon$에서 추가 이득을 멈춘다.

음수의 예로 $\widehat{A}=-2$, $\rho=0.5$, $\epsilon=0.2$이면 원래 항은 $-1$이고 clip한 항은 $-1.6$이다. 최대화 목적은 둘 중 작은 $-1.6$을 택해 비율을 더 낮추는 이득을 없앤다.

양수의 예로 $\widehat{A}=2$, $\epsilon=0.2$, $\rho=1.3$이면 원래 항 2.6 대신 clip한 항 2.4를 쓴다. 같은 행동의 비율이 $\rho=0.7$이면 원래 항 1.4를 써서 확률을 다시 올릴 기울기를 남긴다.

**Advantage가 음수일 때도 비율 상한을 자를까?** 음수에서는 행동 확률을 낮추는 것이 유리하므로 비율이 $1-\epsilon$보다 작아질 때 추가 이득을 자른다. 반대로 비율이 매우 커져 나쁜 행동이 더 자주 선택된다면 손실은 계속 이를 되돌리도록 작용한다.

## 6.18 슬라이드 18 · 하나의 PPO-Clip 목적함수로 합치기

수집 당시 정책을 $\pi_{\text{old}}$, 현재 정책을 $\pi_\theta$라고 하자. $\mathcal{B}$는 $\pi_{\text{old}}$가 모은 rollout의 시점 인덱스들이며, $E_{t\in\mathcal{B}}$는 그 표본 평균이다. $\text{clip}(x,l,u)$는 $x$를 구간 $[l,u]$의 가까운 끝점으로 자르는 함수다. PPO가 **최대화**하는 actor 목적은

$$L^{\text{CLIP}}(\theta)=E_{t\in\mathcal{B}}\left[\min\left(\rho_t(\theta)\widehat{A}_t,\ \text{clip}(\rho_t(\theta),1-\epsilon,1+\epsilon)\widehat{A}_t\right)\right].$$

양의 advantage에서는 비율 상단, 음의 advantage에서는 비율 하단의 추가 이득이 잘린다. clipping 범위를 벗어난 모든 표본의 기울기가 0이 되는 것은 아니다. 예컨대 양의 advantage인데 비율이 지나치게 **작으면**, 목적은 확률을 회복시키도록 여전히 기울기를 준다. min은 과도하게 유리한 변화만 제한하는 보수적 surrogate다. PPO 원논문

**비율을 자르면 새 정책이 반드시 가까운가?** 아니다. 표본에서 목적의 유리한 방향을 자르는 장치다. 관측하지 않은 상태에서 정책이 크게 변할 수도 있고, 실제 KL이 커질 수도 있다. 실무에서는 평균 KL을 모니터링하거나 조기 중단을 더한다.

두 곱의 최솟값을 목적에 쓰므로 유리한 방향의 추가 이득이 평평해진다. 실제 확률비와 KL을 직접 제한하는 제약식은 아니어서, 학습 중 두 값은 따로 관찰해야 한다.

한 표본의 advantage를 $A$, 확률비를 $\rho$라고 줄여 쓰자. $f(\rho,A)=\min\{\rho A,\text{clip}(\rho,1-\epsilon,1+\epsilon)A\}$를 부호별로 풀면

$$f(\rho,A)=\begin{cases}A\min(\rho,1+\epsilon), & A\geq 0,\\ A\max(\rho,1-\epsilon), & A<0.\end{cases}$$

$A<0$에서 음수를 곱하면 부등호 방향이 뒤집혀 $\max(\rho,1-\epsilon)$이 나온다. $A=-2$, $\rho=1.3$처럼 나쁜 행동의 확률이 증가한 경우에는 목적이 $-2.6$으로 내려가므로 이를 되돌릴 기울기가 남는다.

## 6.19 슬라이드 19 · PPO 학습 반복의 순서

현재 정책으로 rollout을 수집하고 행동 로그확률, 보상, 상태, 종료 여부를 저장한다. critic으로 가치와 GAE를 계산한 다음 수집 정책을 $\pi_{\text{old}}$로 고정한다. 동일 batch를 여러 epoch와 minibatch에 나누어 $L^{\text{CLIP}}$를 최대화하고, 가치 손실과 정책 엔트로피 보너스를 함께 학습한다. 이후 **새 정책으로 다시 rollout**을 수집한다.

엔트로피 $H(\pi(\cdot\mid s))=-\sum_a \pi(a\mid s)\log\pi(a\mid s)$는 정책이 너무 일찍 한 행동으로 수렴하지 않도록 한다. 연속 행동에서는 해당 밀도의 미분 엔트로피를 쓴다. 가치 손실·엔트로피 계수의 스케일은 보상 스케일과 함께 점검해야 한다. 코드에서 분모의 old log probability를 매 epoch 재계산하면 알고리즘이 달라진다.

학습 순서는 rollout 수집, 가치와 GAE 계산, 이전 로그확률 고정, 여러 minibatch 갱신, 새 rollout 수집이다. 여러 epoch 동안 같은 자료를 섞어 쓰되 각 표본의 old log probability는 수집 당시 값이다.

새 자료를 모을 때만 기준 정책을 갱신한다. 이 순서가 어긋나면 코드가 실행되어도 PPO 목적의 비교 대상이 바뀐다.

### 6.20 슬라이드 20 · PPO가 널리 쓰이는 이유와 한계

같은 clipping 식이 이산 행동과 연속 행동의 정책 로그확률에 적용되고, 여러 환경을 병렬 실행해 rollout을 모으기 쉽다. 구현의 핵심도 복잡한 제약 최적화 대신 보통의 경사 최적화로 표현된다. 그래서 로봇 제어와 언어모델 학습을 포함한 여러 맥락에서 출발점으로 쓰인다.

그렇다고 기본값만으로 안정적이라는 뜻은 아니다. rollout 길이, minibatch 크기, 업데이트 epoch 수, 학습률, $\epsilon$, GAE $\lambda$, entropy 계수, 보상/advantage 정규화에 따라 결과가 바뀐다. 재현할 때는 평가 환경을 학습 환경과 분리하고 여러 seed의 평균과 분산을 함께 보고한다.

실험에서 episode 반환만 보면 정책이 왜 불안정한지 찾기 어렵다. 평균 KL이 계속 커지면 갱신 폭을, clip fraction이 지나치게 높으면 많은 표본이 더 이상 유리한 기울기를 주지 못하는지를, value loss가 높으면 advantage의 질을 확인한다. 정책 엔트로피가 갑자기 0에 가까워지면 탐색이 너무 일찍 줄었는지도 살핀다.

### 6.21 슬라이드 21 · 접촉이 많은 로봇 조립

Factory는 볼트와 너트 등 접촉이 복잡한 조립 과제를 고속 시뮬레이션으로 다룬다. 정책은 손끝 위치만 맞추는 것이 아니라 충돌, 마찰, 접촉력 변화 속에서 부품을 끼워야 한다. 보상 설계와 물리 모델, 많은 병렬 환경이 정책 최적화의 실제 성패에 영향을 준다는 사례다. 슬라이드의 시연은 PPO 계열 방법이 고차원 연속 제어에 쓰일 수 있음을 보여 주지만, 영상 하나가 현실 전이의 일반적 성공을 입증하지는 않는다. Factory 논문

**조립은 왜 단순한 위치 제어보다 어려운가?** 부품이 닿는 순간 동역학이 급격히 바뀌고, 관측 오차가 작은 힘 차이로 증폭된다. 궤적 추종만으로는 마찰과 공차를 흡수하기 어려워 피드백과 접촉 상태 추정이 필요하다.

너트를 볼트에 끼우는 상황에서 접촉 전에는 위치 오차가 중요하지만 닿은 뒤에는 힘과 미끄러짐이 중요해진다. 같은 성공 보상이라도 어느 단계에서 실패했는지에 따라 필요한 관측과 보상 항이 달라진다. Factory 사례를 읽을 때는 PPO라는 optimizer뿐 아니라 시뮬레이터의 접촉 모델과 병렬 수집 규모를 성능의 조건으로 함께 보아야 한다.

### 6.22 슬라이드 22 · SAPG와 병렬 학습

Split and Aggregate Policy Gradients는 병렬 환경을 여러 묶음으로 나눠 정책을 학습한 뒤 중요도 샘플링으로 정보를 결합한다. PPO가 병렬 환경 수를 계속 늘려도 이득이 포화될 수 있다는 문제의식에서 출발한다. 동일한 총 환경 상호작용 수라면 병렬 worker 수만 늘렸다는 이유로 표본 효율이 향상된 것은 아니다. 벽시계 시간, 총 시뮬레이션 step, 최종 성능을 분리해 비교해야 한다. SAPG 논문

병렬 worker를 10개에서 100개로 늘리면 1시간 동안 모은 표본 수는 늘 수 있지만, 총 환경 단계 100만 개를 기준으로 한 sample efficiency가 자동으로 좋아지는 것은 아니다. 더 많은 동시 실행은 정책 갱신 사이에 모인 자료가 오래된 정책에서 나올 가능성도 바꾼다. SAPG의 기여를 볼 때 **wall-clock time, 총 환경 단계 수, 최종 성능**을 따로 읽어야 한다.

### 6.23 슬라이드 23 · 사족 보행의 실제 로봇 시연

Unitree 로봇 시연은 정책의 출력이 매 제어 주기 모터 명령 또는 목표 관절값으로 이어진다는 점을 상기시킨다. 보행 학습에서는 전진 속도만 키우면 넘어져도 높은 순간 보상을 얻는 편법이 생길 수

있다. 자세 안정성, 토크와 에너지, 발 미끄러짐, 명령 추종 등을 함께 평가해야 한다. 학습은 보통 대량 병렬 시뮬레이션을 사용하고 실기에 옮길 때 센서 지연과 동역학 차이를 고려한다.

같은 전진 속도 보상을 주어도 한 정책은 안정적인 걸음으로, 다른 정책은 몸을 던지듯 움직여 점수를 얻을 수 있다. 실제 로봇에서는 모터 토크 한계, 온도, 넘어짐의 복구 비용이 중요하다. 영상 속 한 번의 성공보다 여러 지형에서의 평균 성공률과 실패 때의 안전성까지 확인해야 시뮬레이션 결과를 해석할 수 있다.

### 6.24 슬라이드 24 · 보행을 넘어서는 전신 제어

전신 제어에서는 이동과 팔 조작, 균형 유지가 결합된다. 한쪽 팔이 물체를 들면 질량중심이 바뀌고 다리의 제어 목표도 달라진다. 따라서 개별 동작의 성공 영상만 보고 "범용" 정책이라고 판단하기보다 낯선 물체·지형·외란에서 성공률을 확인해야 한다. 시뮬레이션에서 배운 제어기는 실기로 옮길 때 마찰, 무게, 지연 등을 무작위화해 여러 가능성에 견디도록 훈련하는 경우가 많다.

로봇이 물체를 들면 질량중심이 바뀌므로 같은 다리 명령도 빈손일 때와 다르게 작용한다. 전신 정책은 팔과 다리의 행동을 함께 조절해야 균형을 유지할 수 있다. 실험에서 새로운 물체 무게를 시험하지 않았다면, "한 물체를 든다"와 "다양한 물체를 안정적으로 든다"는 다른 주장이다.

### 6.25 슬라이드 25 · 조작 정책 시연을 읽는 법

로봇 조작 영상에서는 결과뿐 아니라 입력 관측, 목표, 행동 주기, 실패 후 재시도 여부를 확인해야 한다. 예를 들어 컵을 잡는 정책의 높은 성공률도 컵의 크기와 배경이 고정된 실험인지, 보지 못한 물체로 일반화하는지에 따라 의미가 다르다. RL이 담당하는 부분은 보상과 상호작용을 이용한 정책 개선이며, 비전 표현이나 초기 행동은 모방학습에서 가져올 수도 있다. 시연은 학습법의 가능성을 보여 주지만 독립 평가의 대체물은 아니다.

예를 들어 컵 집기 영상 하나가 성공했다 해도 첫 시도인지, 실패 뒤 재시도인지, 손목 카메라와 외부 카메라 중 무엇을 정책이 실제 입력으로 받았는지 확인해야 한다. 모델이 시연으로 시작한 뒤 RL로 개선했다면 각 단계의 성능을 따로 보고 어느 부분의 기여인지 구분한다. 데모와 반복 평가의 역할이 다르다.

### 6.26 슬라이드 26 · 사람 피드백과 언어모델

InstructGPT에서는 여러 후보 답변에 대한 사람의 선호 비교로 보상모델을 학습한 뒤, 언어모델 정책을 PPO로 조정했다. 상태는 지금까지의 프롬프트와 생성 토큰, 행동은 다음 토큰으로 볼 수 있다. 완료된 답변에 주어진 보상을 토큰별 정책 업데이트에 연결하되, 기준 모델과의 KL 패널티로 언어 분포가 지나치게 이탈하지 않게 한다.

로봇의 물리 보상과 달리 선호 보상은 **보상모델의 예측**이다. 정책이 이 예측의 허점을 찾으면 사람이 좋아하지 않는 문장에 높은 점수를 받을 수 있으므로 별도의 사람 평가가 중요하다. 이 사례가 모든 후속 언어모델에 동일한 학습 절차가 적용된다는 뜻은 아니다. InstructGPT 논문

InstructGPT의 PPO 단계에서 보상은 사람이 매 토큰마다 직접 준 점수가 아니라, 답변 비교 자료로 훈련한 reward model의 예측이다. 같은 답변이 짧고 정확해서 선호되었는지, 장황하지만 그럴듯해서 보상 모델이 높게 예측했는지 실제 사람 평가로 확인해야 한다. 로봇의 물리적 성공 보상과 학습 신호의 근거가 다른 사례다.

### 6.27 슬라이드 27 · 왜 다음은 off-policy인가

PPO는 한 rollout batch를 몇 번 재사용하지만, 새 정책이 수집 정책에서 너무 멀어지면 자료를 버리고 다시 모아야 한다. 서로 다른 과거 정책의 방대한 replay buffer를 자유롭게 활용하는 방식과는 다르므로 보통 on-policy 계열로 분류한다.

Q-learning과 DQN은 행동 정책과 다른 greedy 정책의 가치를 학습할 수 있다. DDPG·TD3·SAC는 연속 행동에서 replay를 활용한다. 다음 장의 핵심 질문은 **새 정책의 행동을 실험하지 않은 전이 자료만으로 어떻게 평가하고 개선할 것인가**다.

한 전이 $(s, a, r, s')$는 과거 행동 정책이 수집했어도, 그 결과 $r, s'$는 환경의 같은 전이 법칙에서 나왔다. Off-policy 방법은 이 자료를 이용해 현재와 다른 목표 정책의 가치를 추정한다. 다만 데이터가 없는 행동의 결과를 자동으로 아는 것은 아니며, Q함수의 일반화와 탐색이 중요하다.

# 7 04-1 Q-learning과 DQN

## 7.1 슬라이드 01 · Q-learning의 출발

앞 장의 정책경사는 정책 확률을 직접 미분했다. 이번에는 "이 상태에서 이 행동을 하면 앞으로 얼마나 좋은가"를 뜻하는 $Q(s,a)$를 먼저 배우고, 값이 가장 큰 행동을 고른다. 가치 추정이 정확하면 정책을 별도로 경사 갱신하지 않아도 된다. 그러나 신경망으로 근사한 값이 부정확할 때 max 연산이 어떤 오류를 만들지 계속 살펴야 한다.

두 행동 A와 B의 $Q(s,A)=8$, $Q(s,B)=5$라고 추정했다면 행동 선택은 A다. 하지만 8과 5는 관측 보상이 아니라 **현재 행동 뒤의 미래까지 합친 기대값**이다. A의 추정이 데이터 부족으로 과대평가되었다면 greedy 선택은 그 오류를 적극적으로 이용한다. 이후의 안정화 방법을 읽는 기준이 된다.

## 7.2 슬라이드 02 · 지금까지의 정책경사

Actor–critic에서는 현재 정책으로 자료를 모으고 $\widehat{A}(s,a)$를 추정하여 $\nabla_\theta \log \pi_\theta(a \mid s)\widehat{A}$로 actor를 개선했다. 여기서 advantage는 **현재 정책 평균에 비해** 그 행동이 얼마나 좋은지를 나타낸다. 정책이 좋아진 뒤에는 같은 $Q^\pi$도 달라진다. 결국 정책 평가와 정책 개선을 번갈아 수행한다.

Actor–Critic의 critic은 현재 정책이 앞으로 따를 행동까지 포함한 $Q^\pi$를 평가한다. 반면 Q-learning의 target에는 다음 상태에서의 $\max_{a'} Q$가 들어간다. 수집 정책이 어떤 행동을 했든 다음 상태에서는 가장 좋다고 추정한 행동을 택한다고 가정하는 차이가 off-policy 성질의 핵심이다.

## 7.3 슬라이드 03 · 정책 반복에서 가치 반복으로

어떤 정책의 $Q^\pi(s,a)$를 알고 있다면 $a^*(s)=\arg\max_a Q^\pi(s,a)$를 택하는 greedy 정책으로 개선할 수 있다. 이 절차를 평가와 개선으로 나누면 정책 반복이다. 가치 반복은 매 단계에서 "다음 상태에서는 이미 최선의 행동을 고른다"고 가정해 최적 가치의 고정점을 찾는다.

이산 행동이 몇 개뿐이면 모든 $a$의 가치를 비교하기 쉽다. 관절 토크처럼 연속 벡터인 행동에서는 $\arg\max_a Q(s,a)$ 자체가 고차원 최적화 문제가 된다. 이 한계가 뒤의 DDPG·TD3·SAC로 이어진다.

작은 표에서 $Q^\pi(s,A)=4$, $Q^\pi(s,B)=7$이고 현재 정책이 A를 80% 택한다면 $V^\pi(s)=0.8\cdot 4+0.2\cdot 7=4.6$이다. B를 greedy하게 선택하는 새 정책은 그 상태의 기대 가치를 높일 가능성이 있다. 단, 이후 상태에서도 정책이 바뀌므로 실제 개선을 분석하려면 평가와 개선의 전체 연결이 필요하다.

## 7.4 슬라이드 04 · $V$와 $Q$의 관계

$V^\pi(s)=E_\pi[\sum_{k=0}^{\infty}\gamma^k r_{t+k} \mid s_t=s]$, $Q^\pi(s,a)=E_\pi[\sum_{k=0}^{\infty}\gamma^k r_{t+k} \mid s_t=s, a_t=a]$라고 두자. 첫 행동까지 $\pi$를 따르면 $V^\pi(s)=E_{a\sim\pi(\cdot|s)}Q^\pi(s,a)$다. 최적 정책에서는 $V^*(s)=\max_a Q^*(s,a)$가 된다.

한 step을 떼어 내면 $Q^\pi(s,a)=E[r+\gamma V^\pi(s') \mid s,a]$다. 이 식은 다음 상태의 미지의 긴 미래를 가치함수 하나로 대체하는 **부트스트랩**의 출발점이다. $\gamma$가 없는 강의식은 $\gamma=1$인 유한 지평의 특수 경우로 읽을 수 있다.

위 예에서 현재 정책의 $V^\pi=4.6$과 최적 행동의 $\max_a Q^\pi=7$은 서로 다르다. $V^\pi$는 현재 정책이 행동을 섞어 선택하는 평균이고, 최적 가치는 최선의 선택을 반복할 때의 전망이다. 별표 $*$가 붙은 값과 특정 정책의 위첨자 $\pi$를 혼동하면 Bellman 식의 max 위치를 잘못 적기 쉽다.

## 7.5 슬라이드 05 · 모델을 요구하는 fitted value iteration

$V^*(s)=\max_a E_{s'\sim p(\cdot|s,a)}[r(s,a,s')+\gamma V^*(s')]$를 표본으로 맞추려면, 주어진 상태 $s$에서 후보 행동마다 다음 상태를 샘플링하거나 전이모델 $p$를 알아야 한다. 저장된 transition 하나는 그 상태에서

**실제로 했던 한 행동**의 결과만 알려 준다. 다른 행동의 결과까지 자동으로 제공하지 않는다.

따라서 행동 전부를 비교하는 value iteration을 모델 없는 자료에 바로 적용하기 어렵다. $Q(s,a)$를 배우면 관측된 $(s,a,r,s')$의 목표를 만들 수 있어 이 문제를 우회한다.

기록된 자료에 상태 $s$에서 A를 실행한 전이만 있다면 B를 했을 때의 $s'$를 알 수 없다. $V(s')$를 아무리 잘 학습해도 B의 전이 결과가 없으면 A와 B의 기대값을 직접 비교하기 어렵다. Fitted Q-iteration은 **관측된 현재 행동**의 Q를 회귀하고, 다음 상태의 행동은 학습된 Q로 비교한다.

### 7.6 슬라이드 06 · 최적 Bellman 방정식

$Q^*(s,a)$는 상태 $s$에서 첫 행동 $a$를 한 뒤 최적으로 행동할 때의 기대 할인 반환이다. $r$은 그 첫 행동의 보상, $s'$은 다음 상태, $a'$는 다음 상태에서 고를 수 있는 행동, $\gamma \in [0,1)$는 할인율이다. 첫 보상을 떼고 이후의 최적 선택을 남기면

$$Q^*(s,a) = E\left[ r + \gamma \max_{a'} Q^*(s',a') \mid s,a \right].$$

왼쪽은 현재 상태–행동 쌍의 전체 미래 가치, 오른쪽은 즉시 보상과 다음 상태에서 가능한 최선의 가치다. 이것은 정의에서 첫 보상을 분리하고 최적 부분문제를 남긴 식이다. 실제 transition $(s,a,r,s')$ 하나로는 기대값을 계산하지 못하므로 $r + \gamma \max_{a'} Q(s',a')$를 **표본 목표**로 사용한다.

식의 재귀를 수치로 읽어 보자. 현재 보상이 2, 다음 상태의 최대 Q가 5, $\gamma = 0.9$라면 목표는 $2 + 0.9 \cdot 5 = 6.5$다. 종료라면 미래가 없으므로 2만 남는다. 현재 Q가 4라면 6.5 쪽으로 올리는 회귀를 하지만, 5 자체가 틀린 추정이면 그 오류도 함께 전파된다.

정확한 표 형태 계산에서 이 반복이 수렴하는 이유도 볼 수 있다. $(\mathcal{T}^*Q)(s,a) = E[r + \gamma \max_b Q(s',b) \mid s,a]$를 **최적 Bellman 연산자**라 정의하자. $Q_1, Q_2$는 비교할 두 행동가치 함수다. $\|Q_1 - Q_2\|_\infty$는 모든 상태·행동에서의 최대 차이이며, 정확히 $\max_{s,a} |Q_1(s,a) - Q_2(s,a)|$다. 최댓값의 차이는 항별 최대 차이를 넘지 못하므로

$$\begin{aligned} |(\mathcal{T}^*Q_1)(s,a) - (\mathcal{T}^*Q_2)(s,a)| &\le \gamma E[\,|\max_b Q_1(s',b) - \max_b Q_2(s',b)| \mid s,a] \\ &\le \gamma \|Q_1 - Q_2\|_\infty. \end{aligned}$$

$0 \le \gamma < 1$이면 모든 $(s,a)$에 대한 최댓값 거리도 매 반복 $\gamma$배 이하로 줄어드는 **수축**이다. 그래서 정확한 Bellman 연산의 고정점은 유일하다. 신경망의 회귀·표본 오차·부분적인 상태 방문이 끼면 실제 DQN 업데이트가 이 정확한 연산과 달라지므로, 이 증명을 DQN의 무조건적 수렴 보장으로 옮기면 안 된다.

### 7.7 슬라이드 07 · Fitted Q-iteration

자료 $\mathcal{D} = \{(s_i, a_i, r_i, s_i', d_i)\}$의 $i$번째 항은 상태·행동·보상·다음 상태·종료 표시다. $Q_\phi$는 파라미터 $\phi$로 학습하는 Q망, $Q_{\bar{\phi}}$는 목표 계산 동안 고정하는 target 망, $y_i$는 회귀 목표다. 진짜 종료에는 $d_i = 1$을 두어 미래 항을 없앤다. 전이마다 $y_i = r_i + \gamma(1 - d_i) \max_{a'} Q_{\bar{\phi}}(s_i', a')$를 만들고 $\sum_i (Q_\phi(s_i,a_i) - y_i)^2$를 줄인다.

목표에는 자료 수집 정책 $\beta$의 행동 확률이 직접 들어가지 않는다. $\beta$가 택한 현재 행동의 전이를 사용하면서 **다음 상태에서는 greedy 행동**을 평가하므로 off-policy다. 단, 방문하지 않은 상태–행동 쌍까지 정확해지는 것은 아니다.

한 전이가 $r = 1$, $s'$의 target Q가 A에 4, B에 6이고 $\gamma = 0.5$라면 목표는 $1 + 0.5 \cdot 6 = 4$다. 자료에서 현재 행동이 A였는지 B였는지는 **현재 회귀할 $Q(s,a)$의 인덱스**를 정하고, 다음 상태의 최댓값은 목표 정책을 정한다. 현재 행동과 다음 행동의 역할을 섞지 말자.

### 7.8 슬라이드 08 · 세 가지 반복을 구분하기

정책 반복은 $Q^\pi$를 평가한 뒤 $\pi$를 개선한다. 모델 기반 fitted value iteration은 $V$를 맞추면서 각 후보 행동의 전이 기대값을 요구한다. fitted Q-iteration은 관측된 transition에서 $Q(s,a)$를 맞추고 정책을 $\arg\max_a Q(s,a)$로 **암묵적으로** 정의한다.

세 방법 모두 Bellman 관계를 이용하지만 필요한 자료와 최적화 대상이 다르다. 특히 "off-policy"는 자료 수집 정책과 목표 정책이 다를 수 있다는 뜻이지, 자료에 없는 행동의 가치를 틀림없이 안다는 뜻이 아니다.

Fitted Q-iteration은 현재 행동에 대해서는 기록된 전이를 쓰지만, 다음 상태의 $\max_a Q(s',a)$에서는 기록에 드문 행동까지 평가한다. 자료 coverage가 좁으면 이 최댓값이 근사 오차에 민감해진다.

### 7.9 슬라이드 09 · Q-learning의 장점과 위험

Q-learning은 고분산 정책경사를 직접 계산하지 않고 과거 transition을 다시 쓸 수 있다. 하지만 목표 $y = r + \gamma \max Q_\phi(s',a')$ 자체가 학습 중 움직이고, 학습된 $Q$를 통해 다시 자기 자신의 목표를 만든다. 함수근사 · 부트스트랩 · off-policy 자료가 겹치면 발산할 수도 있다.

이산 행동에서는 max를 열거하지만 연속 행동에서는 불가능에 가깝다. 또한 큰 $Q$가 실제 높은 반환을 보장하지 않는다. 이후 알고리즘은 **자료 상관, 목표 이동, 탐색, 최대값 편향**을 각각 줄인다.

이른바 deadly triad는 함수 근사, 부트스트랩, off-policy 데이터가 만나는 경우의 불안정을 가리킨다. 각 요소 하나만이 항상 위험하다는 뜻이 아니다. Replay는 오래된 자료를 쓰고, 부트스트랩은 현재 추정값으로 목표를 만들고, 신경망은 자료 밖에서도 값을 낸다. 세 효과가 결합하면 오류가 스스로 커질 수 있음을 구체적으로 추적해야 한다.

### 7.10 슬라이드 10 · Deep Q-Network

DQN은 영상 픽셀을 입력으로 받고 이산 행동마다 $Q_\phi(s,a)$ 하나를 출력하는 신경망이다. 화면 특징을 사람이 수작업으로 정의하지 않아도 표현과 가치를 함께 학습한다. 대표적인 Atari 실험은 여러 게임에 같은 학습 절차를 적용했다는 점에서 중요하다. 환경의 점수, 행동 반복, 관측 전처리까지 성능 비교의 일부다. DQN 원논문

입력 영상의 같은 위치에 공이 있어도 왼쪽으로 가는지 오른쪽으로 가는지에 따라 좋은 패들 이동이 다르다. DQN의 픽셀 입력은 시각 표현을 배우는 장점이 있지만, 필요한 시간 정보가 관측에 없으면 신경망을 크게 해도 해결되지 않는다. Frame stack이 붙는 이유는 CNN의 크기 문제가 아니라 상태 정보 문제다.

### 7.11 슬라이드 11 · DQN을 지탱하는 세 장치

연속된 게임 화면은 서로 비슷해 minibatch가 편향되기 쉽다. 가치망으로 만든 목표가 매 step 바뀌면 최적화가 불안정하다. greedy 행동만 고르면 아직 보지 않은 좋은 행동을 발견하기 어렵다. DQN은 각각 **experience replay**, **target network**, **$\epsilon$-greedy 탐색**으로 대응한다. 이 셋은 서로 다른 문제를 해결한다.

세 장치를 빼고 생각해 보면 역할이 선명하다. Replay가 없으면 인접 화면의 상관이 강하고, target network가 없으면 목표가 매 갱신마다 흔들리고, 탐색이 없으면 초기에 낮게 평가한 행동을 시도하지 않는다. 각각 데이터 **순서**, 학습 **목표**, 행동 **coverage**를 다룬다.

### 7.12 슬라이드 12 · 연속 transition의 상관

게임에서 공이 같은 방향으로 움직이는 100 frame을 차례로 학습하면 비슷한 화면과 행동이 계속 나타난다. 무작위 minibatch를 가정하는 SGD에서 이런 강한 상관은 기울기의 다양성을 줄이고 특정 짧은 구간에 과적합을 유도한다. 수집한 자료가 완전한 i.i.d.가 아니라는 사실은 replay를 써도 사라지지 않는다. 다만 buffer 전역에서 섞어 뽑으면 **인접 frame의 직접적 상관**을 상당히 줄일 수 있다.

공이 한동안 오른쪽으로 움직이는 100프레임을 연속 학습하면 거의 같은 예제가 100번 들어오는 셈이다. 그 직후 공이 왼쪽으로 움직일 때는 다른 패턴이 갑자기 몰려온다. Buffer에서 두 구간의 전이를 섞어 뽑으면 한 minibatch가 더 다양한 상황을 포함하지만, 특정 상태를 아예 경험하지 않았다면 replay로 만들 수는 없다.

### 7.13 슬라이드 13 · Replay buffer의 역할

전이 $(s, a, r, s', d)$를 FIFO buffer $\mathcal{B}$에 저장하고, 그 안에서 무작위 minibatch를 뽑아 여러 차례 학습한다. 경험 한 번으로 여러 업데이트를 할 수 있어 환경 상호작용당 학습량이 증가한다. 동시에 오래된 정책의 자료도 남으므로 off-policy Q-learning과 잘 맞는다.

buffer가 너무 작으면 상관 감소 효과가 약하고, 너무 크면 현재 환경과 동떨어진 오래된 분포가 많아질 수 있다. "무작위 replay = 정확한 i.i.d." 라는 등식은 성립하지 않는다.

전이 하나를 저장할 때 $s$와 $s'$의 영상, 행동, 보상, 종료 여부가 같은 시각 기준으로 맞아야 한다. 특히 frame stack이 있는 환경은 다음 상태의 stack이 한 프레임 밀리므로 버퍼에서 복원할 때 주의해야 한다. Replay는 경험의 재사용 장치인 동시에 잘못 정렬한 경험을 계속 재사용하게 만드는 장치이기도 하다.

### 7.14 슬라이드 14 · 움직이는 목표 문제

replay는 입력 표본을 섞지만 $Q_\phi$의 목표에 같은 $\phi$를 쓰면 파라미터가 변할 때 목표 $y_i$도 즉시 변한다. 고정된 정답에 회귀하는 감독학습과 다르다. 목표의 급격한 변화를 늦추는 장치가 target network다.

감독학습의 사진 라벨은 모델 파라미터가 바뀌어도 그대로다. DQN의 target $r + \gamma \max Q_\phi(s', a')$는 같은 파라미터를 바꾸면 다음 minibatch에서 값이 달라질 수 있다. 그래서 손실이 내려가는 것만으로 참 가치에 가까워진다고 단정할 수 없다. 독립적인 평가 반환과 target Q의 크기를 함께 보아야 한다.

### 7.15 슬라이드 15 · 자기 자신을 쫓는 회귀

예를 들어 진짜 값이 5인데 다음 상태 $Q_\phi(s', a')$를 우연히 8로 추정하면 현 상태의 목표도 크게 잡힌다. 현 상태의 $Q$를 올리면 다시 다른 이전 상태의 목표가 올라가는 식으로 오차가 전파될 수 있다. replay로 자료 순서를 섞어도 이 부트스트랩 고리는 그대로다. 최적화 대상과 목표 계산망을 잠시 분리해야 한다.

오류 전파의 작은 예로 $s_2$의 Q를 실제보다 3 높게 추정하면 $s_1$의 목표는 대략 $3\gamma$만큼 커질 수 있다. $s_1$이 올라가면 그 이전 $s_0$ 목표도 영향을 받는다. Target network는 이 고리의 변화 속도를 늦추지만 잘못된 숫자 자체를 자동으로 고치지는 않는다.

### 7.16 슬라이드 16 · Target network

온라인 망 $Q_\phi$는 매 minibatch 최적화하지만 target 망 $Q_{\bar{\phi}}$는 일정 주기마다 $\bar{\phi} \leftarrow \phi$로 복사한다. 그 사이 목표 $y = r + \gamma(1 - d) \max_{a'} Q_{\bar{\phi}}(s', a')$는 target 파라미터에 관해 고정된다. 소프트 업데이트 $\bar{\phi} \leftarrow (1 - \tau)\bar{\phi} + \tau\phi$도 쓰지만 원래 DQN의 주기적 복사와 구분해야 한다.

현재 망의 예측값을 목표와 맞추되 target 식으로 기울기가 흐르지 않게 하는 stop-gradient가 구현상 중요하다. 이것은 안정화 방법이지 수렴의 일반 보장은 아니다.

복사 주기가 너무 짧으면 target이 온라인 망을 거의 그대로 따라가고, 너무 길면 오래된 가치로 회귀한다. 소프트 업데이트의 $\tau$도 같은 속도를 연속적으로 조절한다. 구현에서 target 망을 optimizer에 넣지 않고 목표값 계산의 gradient를 끊어야 한다는 두 조건을 함께 확인하자.

## 7.17 슬라이드 17 · 탐색이 남는다

replay와 target network를 넣어도 한 번도 시도하지 않은 행동의 보상은 알 수 없다. 현재 $Q$ 최대 행동만 고르면 초기에 잘못 높게 추정한 행동을 계속 택할 수 있다. 수집 정책에 의도적인 탐색을 넣어 자료의 행동 다양성을 확보한다.

예를 들어 왼쪽 행동을 한 번도 시도하지 않아 Q를 낮게 초기화했다면 greedy 정책은 계속 오른쪽만 고를 수 있다. 그 결과 replay에는 오른쪽 전이만 쌓여 왼쪽 Q를 고칠 근거가 없다. 탐색은 학습 초기에 **자료를 얻을 행동**을 정하는 문제이기도 하다.

## 7.18 슬라이드 18 · $\epsilon$-greedy의 확률

$|\mathcal{A}| = m$인 이산 행동에서 확률 $\epsilon$으로 균등 무작위 행동을 택하고 나머지는 greedy 행동을 고른다. greedy 행동의 실제 선택 확률은 $(1-\epsilon) + \epsilon/m$, 다른 각 행동은 $\epsilon/m$이다. 따라서 "$\epsilon$이면 greedy를 절대 선택하지 않는다"는 해석은 틀리다.

초기에는 큰 $\epsilon$으로 탐색하고 학습이 진행되며 줄인다. 평가 때도 탐색 확률을 그대로 둘지 별도로 정의해야 비교가 공정하다. 탐색의 성공은 상태 방문 범위와 보상 구조에 좌우된다.

행동이 4개이고 $\epsilon = 0.2$라면 greedy 행동은 $0.8 + 0.2/4 = 0.85$, 나머지 각 행동은 0.05의 확률로 선택된다. 평가 정책의 $\epsilon$을 다르게 두면 학습 중 반환과 평가 반환도 달라진다.

## 7.19 슬라이드 19 · Frame skip과 frame stack

Atari에서 매 frame마다 행동을 바꾸는 것은 필요하지 않을 수 있다. 같은 행동을 몇 frame 반복하면 결정 횟수를 줄이고 보상 지연의 시간 단위를 바꾼다. 반대로 한 장의 화면은 공의 속도 방향을 알려주지 못한다. 최근 4장을 쌓으면 같은 위치에 있는 공이 왼쪽으로 오는지 오른쪽으로 가는지 구분할 수 있다.

frame skip은 **환경 step의 의미**, frame stack은 **관측 정보량**을 바꾼다. 논문 간 점수를 비교할 때 두 설정을 일치시켜야 한다.

frame skip이 4라면 정책의 한 행동은 게임 내부의 네 프레임에 영향을 준다. 그래프에 환경 step 100만이라고 쓸 때 내부 프레임 100만인지 에이전트의 결정 100만인지에 따라 실제 게임 시간이 네 배 차이 날 수 있다. Frame stack 4는 최근 네 관측을 함께 입력에 넣는 별도 조치다.

## 7.20 슬라이드 20 · 시각 자료: Atari 화면

이 페이지의 영상 예시는 DQN이 보는 입력이 "게임 규칙표"가 아니라 연속 화면임을 보여 준다. 학습기는 픽셀에서 중요한 객체 위치를 추출해야 하고, frame stack으로 운동 방향까지 추론한다. 점수만을 보상으로 쓰면 점수가 늦게 오르는 게임에서 탐색이 어렵다. 알고리즘을 이해할 때 입력 이미지, 행동 집합, 보상이 각각 무엇인지 먼저 적어 보는 것이 유용하다.

Atari 화면에서 점수를 받은 직후의 픽셀만으로는 어느 행동이 공을 맞혔는지 알기 어렵다. 여러 프레임의 움직임과 replay에 저장된 행동을 함께 봐야 한다. 시각 데모가 좋은 정책을 보여 주더라도 학습 과정의 탐색량과 표본 비용은 영상에 나타나지 않는다.

### 7.21 슬라이드 21 · DQN 구조를 다시 묶기

관측 전처리와 stack으로 상태를 만들고, $\epsilon$-greedy가 행동을 선택하고, 전이를 replay에 저장한다. 무작위 minibatch의 현재 $Q_\phi(s,a)$와 target 망이 만든 $r+\gamma \max_{a'} Q_{\bar{\phi}}(s',a')$ 사이 오차를 줄인다. 학습이 끝나면 greedy 정책 $a^*(s)=\arg\max_a Q_\phi(s,a)$로 평가한다. DQN 원논문

한 업데이트를 손으로 따라가면 다음과 같다. Buffer에서 $(s,a,r,s',d)$를 뽑고, target 망의 모든 다음 행동 Q 중 최대를 구하며, 진짜 종료면 미래 항을 지운다. 온라인 망은 **실제로 실행했던 현재 행동 $a$의 Q**만 목표에 맞춘다. 그 뒤 greedy나 탐색 정책으로 새 전이를 모은다.

### 7.22 슬라이드 22 · DQN을 개선하는 질문

기본 DQN은 작동하지만 여러 오차원이 남는다. 최대값이 낙관 편향을 만들고, 한 단계 목표는 먼 보상을 느리게 전파하며, uniform replay는 학습에 중요한 자료를 자주 보지 못할 수 있다. 평균 반환 하나로는 결과의 불확실성과 여러 가능한 미래를 표현하지 못한다. 뒤의 개선책은 각각 다른 문제를 겨냥한다.

실험 로그에서 Q값만 계속 높아지는데 평가 반환이 제자리라면 최대값 편향이나 부트스트랩 오류를 의심할 수 있다. 희소한 보상이 오래 전파되지 않는다면 $n$단계 목표가 후보이고, 중요한 실패 전이를 드물게 뽑는다면 replay 분포가 문제일 수 있다. 각 개선은 **관찰한 증상과 원인**을 연결해서 적용해야 한다.

### 7.23 슬라이드 23 · 네 가지 개선 축

Double DQN은 행동 선택과 가치 평가의 오차를 분리한다. $n$단계 목표는 보상을 더 멀리 직접 관측한다. Prioritized replay는 큰 학습 오차를 가진 transition을 더 자주 뽑는다. Distributional RL은 평균 대신 반환 분포를 예측한다. 하나를 고치면 다른 문제가 자동 해결되지는 않는다.

Double DQN은 어떤 행동을 최고로 고르는 과정의 편향을, PER은 어떤 전이를 학습 batch에 넣는 빈도를 바꾼다. $n$단계 목표는 한 목표에 실제 보상을 몇 개 넣는지, C51은 목표의 결과를 평균 숫자로 표현할지 분포로 표현할지를 바꾼다. 이름은 모두 DQN의 개선이지만 **수정하는 연산의 위치**가 다르다.

### 7.24 슬라이드 24 · 큰 Q와 큰 실제 반환은 같은가

Q망이 100을 예측한 행동을 실제 실행했을 때 평균 반환이 100에 가까워야 추정이 보정되어 있다. 학습 과정에서 Q가 증가하고 평가 반환도 증가하면 관계가 있어 보이지만, 절댓값이 맞는지는 별도 문제다. 특히 보상 변환, $\gamma$, episode 길이가 다르면 두 수치를 그대로 비교할 수 없다.

예측 Q가 20이고 실제 평가에서 얻은 평균 반환이 12라면 곧바로 8의 오류라고 말하기 전에 두 값의 정의를 맞춰야 한다. Q가 할인된 미래 보상이고 평가 지표가 할인하지 않은 episode 점수라면 단위가 다르다. 같은 초기 상태와 보상 처리로 비교한 뒤에야 Q의 calibration을 논할 수 있다.

### 7.25 슬라이드 25 · 과대평가가 드러나는 경우

Double DQN 연구는 DQN의 예측 가치와 실제 달성한 반환 사이의 차이가 일부 Atari 게임에서 커짐을 보였다. 원인은 단순히 신경망이 틀린다는 사실뿐 아니라 **큰 추정치를 고르는** max와도 관련된다. 값이 과대평가되면 정책이 실제로 좋은 행동보다 낙관적으로 보이는 행동을 택해 성능이 떨어질 수 있다. Double DQN 논문

두 행동의 실제 가치가 같을 때 추정 오차가 큰 행동일수록 우연히 최고로 보일 기회가 있다. Greedy 정책은 그 행동을 골라 과대평가된 목표를 이전 상태로 전파한다. 이런 오류는 학습 손실이 낮아도

나타날 수 있다. 손실은 **자기 목표와의 일치**를 재는 것이지 실제 반환의 정확도를 직접 재지 않기 때문이다.

### 7.26 슬라이드 26 · 최대값 편향의 간단한 유도

두 행동의 실제 가치가 모두 0이고 추정 오차가 각각 $+1, -1$ 또는 $-1, +1$로 같은 확률로 나타난다고 하자. 각 행동 추정치의 평균은 0이지만 매번 $\max(\widehat{Q}_1, \widehat{Q}_2) = 1$이다. 따라서 $E[\max_i \widehat{Q}_i] = 1 > \max_i E[\widehat{Q}_i] = 0$다.

일반적으로 최대값은 볼록 함수이므로 $E[\max_i X_i] \geq \max_i E[X_i]$다. 오차들이 독립일 필요는 없지만 완전히 같은 오차를 공유하면 이 선택 효과가 약해질 수 있다. DQN target은 같은 망이 행동 선택과 그 값 평가를 담당한다.

### 7.27 슬라이드 27 · Double DQN의 분리

표준 DQN의 다음 상태 항은 $Q_{\bar{\phi}}(s', \arg\max_{a'} Q_{\bar{\phi}}(s', a'))$다. Double DQN에서는 학습 중인 온라인 망 $Q_\phi$로 다음 행동을 고르고, 고정된 target 망 $Q_{\bar{\phi}}$로 그 행동의 값을 평가한다. $d = 1$이면 현재 전이가 종료되어 미래 가치를 0으로 만든다.

$$y^{\mathrm{DDQN}} = r + \gamma(1-d) Q_{\bar{\phi}}\Big(s', \arg\max_{a'} Q_\phi(s', a')\Big).$$

두 망의 오차가 완전히 독립인 것은 아니지만 선택과 평가가 일치해 생기는 낙관 편향을 줄인다. 이름의 "Double"이 두 개의 완전 독립 Q망을 뜻한다고 외우면 구현을 잘못 이해하기 쉽다. Double DQN 논문

다음 상태에서 온라인 망이 A를 최고로 골랐더라도 target 망은 A에 낮은 값을 줄 수 있다. Double DQN은 온라인 망의 선택을 target 망이 그대로 높게 평가하도록 강제하지 않는다. 예를 들어 온라인 Q가 A=7, B=6이고 target Q가 A=4, B=5라면 목표에는 **A의 target 값 4**가 들어간다. Target 망의 최고값 5를 쓰는 표준 DQN과 다르다.

### 7.28 슬라이드 28 · 다음 개선: 목표의 지평

한 단계 target은 다음 상태 가치에 크게 의존한다. 먼 보상은 Bellman 업데이트를 여러 번 거쳐야 이전 상태까지 전파된다. 실제 보상 몇 개를 더 관측하고 마지막에만 부트스트랩하면 초기의 부정확한 $Q$에 대한 의존이 줄어든다. 이 생각이 $n$단계 반환이다.

마지막에만 +10을 받는 5단계 경로에서 한 단계 target은 +10을 한 번에 첫 상태로 가져오지 못한다. 반대로 5단계 목표는 실제 다섯 보상을 한 번에 포함할 수 있다. 보상이 늦게 오는 과업에서 전파가 빨라지는 이유지만, 중간에 수집 정책의 행동이 들어간다는 새 가정도 생긴다.

### 7.29 슬라이드 29 · $n$단계 Q 목표

시점 $t$부터 기록된 $n$개 전이의 보상을 $r_t, \ldots, r_{t+n-1}$이라 하자. $d_{t:t+n} = 1$은 이 구간 안에서 실제 종료가 일어났다는 표시이고, 그렇지 않으면 0이다. $y_t^{(n)}$은 보상을 할인해 더한 뒤, 종료하지 않은 경우에만 마지막 상태 $s_{t+n}$의 target Q값으로 미래를 잇는 $n$단계 회귀 목표다.

$$y_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n (1 - d_{t:t+n}) \max_a Q_{\bar{\phi}}(s_{t+n}, a).$$

종료가 $n$보다 이르면 합도 종료까지만 계산하고 부트스트랩을 제거한다. $n=1$이면 원래 Q target이다. 먼 보상이 직접 들어오므로 희소 보상 전파가 빨라질 수 있다.

세 단계 보상이 $1, 0, 5$, $\gamma = 0.9$이고 세 단계 뒤 target Q가 4라면 목표는 $1 + 0.9 \cdot 0 + 0.9^2 \cdot 5 + 0.9^3 \cdot 4 = 7.966$이다. 두 번째 단계에서 진짜 종료되었다면 이후의 5와 4는 들어갈 수 없다. 구현에서는 길이 $n$만 고정해 더하기보다 종료 위치를 먼저 확인해야 한다.

### 7.30 슬라이드 30 · $n$단계의 off-policy 주의점

자료 안의 중간 행동 $a_{t+1}, \dots, a_{t+n-1}$은 수집 정책이 실행했다. 마지막에만 greedy 최댓값을 넣는 $n$단계 목표는 중간 행동도 목표 정책에서 왔다고 가정하는 완전한 off-policy Bellman target과 같지 않다. 정책 차이가 크면 편향이 생긴다. 그럼에도 실무에서는 단순한 $n$단계 target이 종종 빠른 학습을 준다. 정확한 보정을 원하면 중요도 샘플링 같은 장치가 필요하고 분산 비용이 따른다.

예를 들어 buffer의 기록 정책이 중간 단계에서 안전한 우회 행동을 골랐지만 목표 정책은 곧장 전진했을 상황이라면, 기록된 $n$단계 보상은 목표 정책의 정확한 미래가 아니다. 한 단계 Q-learning은 실제 현재 행동 뒤 다음 상태에서 바로 greedy 값을 넣지만, $n$단계는 그 사이 기록 행동을 더 따라간다. $n$을 늘릴 때 생기는 off-policy 편향을 이 경로 차이로 이해할 수 있다.

### 7.31 슬라이드 31 · 자료를 뽑는 방법도 중요하다

uniform replay에서는 buffer에 오래 쌓인 매우 쉬운 transition과 아직 예측을 크게 틀리는 transition이 같은 확률로 뽑힌다. 방문 빈도와 학습 가치가 반드시 같지 않다. Prioritized replay는 TD 오차를 중요도의 대리변수로 삼되, 큰 오차가 반드시 "유익한" 표본이라는 보장은 없음을 함께 기억해야 한다.

공이 벽에 맞아 방향을 바꾸는 드문 화면은 값 예측이 크게 틀릴 수 있고 학습에 유용할 수 있다. 반대로 센서 잡음으로 보상이 비정상적으로 튄 전이도 TD 오차가 크다. PER은 큰 오차를 **중요도 후보**로 보는 휴리스틱이므로, 이상치에 학습이 쏠리지 않는지 관찰해야 한다.

### 7.32 슬라이드 32 · Prioritized experience replay

Replay buffer에 $N$개 전이가 있다고 하자. $\delta_i = y_i - Q_\phi(s_i, a_i)$는 전이 $i$의 TD 오차, $\varepsilon_0 > 0$은 우선순위가 0이 되지 않게 하는 작은 상수다. 우선순위 $p_i = |\delta_i| + \varepsilon_0$에서 샘플 확률 $P(i) = p_i^\alpha / \sum_j p_j^\alpha$를 만들며, 지수 $\alpha = 0$이면 uniform replay다. 분포 변화의 보정 가중치는 $w_i = (1/(NP(i)))^\beta$다. $\beta = 1$은 평균 손실의 완전 보정이고, 작은 $\beta$는 큰 가중치의 분산을 줄인다.

큰 TD 오차는 드문 좋은 발견일 수도, 잡음이나 잘못된 보상일 수도 있다. 우선순위를 주기적으로 갱신하고 여러 seed로 평가해야 한다. PER 논문

두 전이의 우선순위가 9와 1이고 $\alpha = 1$이면 샘플 확률은 각각 0.9와 0.1이다. Uniform이었다면 둘 다 0.5다. 드문 두 번째 전이의 손실을 그대로 평균하면 buffer 전체의 평균과 다른 학습 목표가 되므로, 샘플 확률의 역수에 기반한 중요도 가중치를 사용한다. 그러나 그 가중치가 너무 커지면 다시 분산이 늘 수 있다.

가중치가 필요한 이유를 평균 손실에서 유도하자. buffer가 $N$개 전이를 담고 있고 원하는 손실이 $L = N^{-1} \sum_i \ell_i$라면, $i \sim P$로 뽑은 표본에 $w_i = 1/(NP(i))$를 곱했을 때

$$E_{i \sim P}[w_i \ell_i] = \sum_i P(i) \frac{\ell_i}{NP(i)} = \frac{1}{N} \sum_i \ell_i = L.$$

실제 PER의 $\beta < 1$은 이 보정을 일부만 적용해 큰 역확률 가중치의 분산을 줄인다. 이는 **buffer에서 뽑는 확률**의 보정이며, 예전 행동 정책의 자료를 현재 정책의 자료로 바꾸는 보정과는 다른 문제다.

**TD 오차가 큰 전이만 계속 뽑으면 더 빨리 배울까?** 처음 보는 전이와 작은 오차의 전이도 일정 확률로 뽑혀야 한다. 큰 오차가 관측 잡음 때문일 수도 있고, 우선순위는 마지막 갱신 뒤 오래되어 현재 오차를 반영하지 못할 수도 있다. 확률적 샘플링과 우선순위 갱신이 필요한 이유다.

### 7.33 슬라이드 33 · 평균 가치만으로 충분한가

같은 기대 반환 5라도 확정적으로 5를 받는 행동과 절반 확률로 0 또는 10을 받는 행동은 위험 수준이 다르다. 기대값 최대화라는 목표 아래 두 행동은 같아 보이지만, 결과 분포를 배우면 학습에 더 풍부한 신호가 생기고 필요할 때 위험 기준도 적용할 수 있다. 이것이 분포적 RL의 출발점이다.

위험을 싫어하는 로봇이라면 확정적으로 5를 얻는 행동을, 위험을 감수하는 게임 정책이라면 0 또는 10의 행동을 선택할 수 있다. 두 행동의 평균은 같으므로 평균 Q만으로는 이 선호를 표현하기 어렵다. 다만 **분포를 예측하는 것만으로 위험 민감 목적이 생기지는 않는다.** 어떤 분포 요약을 행동 선택에 쓸지 따로 정해야 한다.

### 7.34 슬라이드 34 · 반환 분포와 평균의 차이

$Z^\pi(s,a) = \sum_{k=0}^{\infty} \gamma^k r_{t+k}$를 상태–행동에 조건부인 **확률변수**로 보자. 보통의 $Q^\pi(s,a) = E[Z^\pi(s,a)]$는 그 평균 하나다. 확정 5와 0/10 반반은 모두 $Q = 5$지만 분산은 각각 0과 25다. 분포를 알면 여러 결과 가능성을 표현할 수 있다. 다만 표준 greedy 정책은 여전히 평균이 가장 큰 행동을 선택한다. Distributional RL 논문

Q값만으로는 분산이나 드문 큰 손실을 복원할 수 없다. C51은 가능한 반환값 구간에 확률질량을 배분해 이 정보를 일부 보존한다.

### 7.35 슬라이드 35 · 불확실성의 출처

고정한 $(s,a)$에서도 이후 환경 전이가 확률적이고, 미래 정책 행동이 확률적이며, 보상 자체도 달라질 수 있다. 이것이 **반환의 환경적 분포**다. 모델 파라미터를 잘 몰라 생기는 추정 불확실성과는 구분해야 한다. C51이 반환 분포를 예측한다고 해서 신경망의 인식론적 불확실성을 자동으로 해결하지는 않는다.

환경의 주사위 결과 때문에 보상이 달라지는 불확실성은 많은 데이터를 모아도 사라지지 않는다. 반면 데이터가 부족해 Q망의 파라미터를 잘 모르는 불확실성은 더 경험하면 줄 수 있다. C51의 반환 분포는 주로 첫 종류의 가능한 결과를 모델링한다. 두 불확실성을 혼동하면 분포 폭을 모델의 자신 없음으로 잘못 해석할 수 있다.

### 7.36 슬라이드 36 · 분포적 Bellman 관계

정책 $\pi$를 따를 때 상태 $s$와 행동 $a$ 뒤에 받을 **할인 반환의 확률분포**를 $Z^\pi(s,a)$라 하자. 그 평균이 앞서 학습한 행동가치 $Q^\pi(s,a) = E[Z^\pi(s,a)]$다. $r$은 첫 보상, $s'$은 다음 상태, $a' \sim \pi(\cdot \mid s')$는 다음 행동이다. 반환을 첫 보상과 이후 반환으로 나누면 분포의 Bellman 관계는

$$Z^\pi(s,a) \overset{D}{=} r + \gamma Z^\pi(s',a'), \qquad a' \sim \pi(\cdot \mid s').$$

$\overset{D}{=}$는 양쪽에서 뽑은 값이 늘 같다는 뜻이 아니라 **확률분포가 같다**는 뜻이다. 예를 들어 다음 상태의 반환이 0 또는 10이고 현재 보상이 2, $\gamma = 0.5$라면 현 반환 분포는 2 또는 7로 이동·축소된다. 양쪽의 평균을 취하고 기댓값의 선형성을 쓰면 $Q^\pi(s,a) = E[r + \gamma Q^\pi(s',a') \mid s,a]$가 된다.

이 예에서 다음 반환의 평균은 5이므로 현재 Q값은 $2 + 0.5 \cdot 5 = 4.5$다. 분포식은 그 평균을 만드는 두 결과 2와 7도 보존한다.

### 7.37 슬라이드 37 · C51의 고정 지지점

C51은 가능한 반환값의 범위 $[V_{\min}, V_{\max}]$에 51개 고정 원자 $z_i = V_{\min} + i\Delta z$를 놓는다. 인덱스 $i = 0, \dots, 50$이고 원자 간격은 $\Delta z = (V_{\max} - V_{\min})/50$이다. 신경망 파라미터 $\phi$가 예측하는 $p_{\phi,i}(s,a)$는 상태·행동 $(s,a)$에서 원자 $z_i$에 줄 확률이며 $\sum_i p_{\phi,i} = 1$이다. $\delta_{z_i}$를 값 $z_i$에 모든 질량이 모인 점분포라 쓰면 $Z_\phi(s,a) = \sum_i p_{\phi,i}(s,a)\delta_{z_i}$이고 평균 Q는 $\sum_i z_i p_{\phi,i}(s,a)$다.

학습 대상은 원자의 위치가 아니라 각 위치의 확률이다. 행동 선택은 이 분포들의 평균을 비교한다. C51 원논문

예를 들어 $[V_{\min}, V_{\max}] = [-10, 10]$이면 $\Delta z = 0.4$다. 원자 51개는 행동의 개수가 아니라 **반환값의 표현 해상도**다.

### 7.38 슬라이드 38 · C51 목표 분포의 투영

다음 상태 $s'$에서 target 분포의 평균이 가장 큰 행동을 $a^*$로 고른다. 그 행동의 원자 $z_i$와 확률 $p_i = p_{\bar\phi,i}(s', a^*)$를 가져와, 즉시 보상 $r$과 할인율 $\gamma$로 원자 위치를 $r + \gamma z_i$로 옮긴다. $\bar\phi$는 목표 계산 동안 고정된 target 망이다. 새 위치가 $[V_{\min}, V_{\max}]$ 밖이면 끝점으로 자르고, 두 원자 사이면 질량을 **거리 비례**로 나눈다. 이 투영 목표와 현재 예측 분포의 교차엔트로피를 줄인다.

예를 들어 지지점이 0과 10이고 옮긴 값이 3이면 질량의 70%를 0, 30%를 10에 둔다. 범위 밖을 clip하면 꼬리 정보가 사라지므로 지지 범위 설정이 중요하다.

왜 그 비율인지 확인하자. 옮겨진 값 $y = r + \gamma z_i$가 인접 지지점 $z_j \le y \le z_{j+1}$ 사이에 있고 원래 확률질량이 $p_i$라면, 투영 질량을

$$m_j = p_i \frac{z_{j+1} - y}{\Delta z}, \qquad m_{j+1} = p_i \frac{y - z_j}{\Delta z}$$

로 더한다. 두 질량의 합은 $p_i$이고, 그 평균 기여도 $m_j z_j + m_{j+1} z_{j+1} = p_i y$다. 즉 지지 구간 **안에서는** 질량 총합과 이 원자의 기대값을 함께 보존한다. $y$가 지지 구간 밖이면 끝점으로 잘라 투영하므로 원래 기대값까지 보존된다고 말할 수 없다. 여러 다음 원자의 기여를 같은 지지점에 더해 최종 목표 분포를 만든다.

투영을 식으로 계산할 때는 옮긴 원자 $y_j = \mathrm{clip}(r + \gamma z_j, V_{\min}, V_{\max})$를 격자 좌표 $b_j = (y_j - V_{\min})/\Delta z$로 바꾼다. $l = \lfloor b_j \rfloor$, $u = \lceil b_j \rceil$이면 그 원자의 확률 $p_j$를 아래 격자에 $p_j(u - b_j)$, 위 격자에 $p_j(b_j - l)$만큼 더한다. $l = u$이면 전량을 그 원자에 둔다. 이 선형 보간은 범위 **안**에서는 예컨대 3을 0과 10에 나누어 평균 3을 보존한다. 반면 15를 끝점 10으로 자르면 평균도 10으로 왜곡된다.

**C51이 분포를 예측하면 행동 선택도 위험 회피적으로 바뀌나?** 기본 C51의 greedy 행동은 각 행동 분포의 평균 $\sum_i z_i p_i(s,a)$가 가장 큰 쪽이다. 분포 전체를 학습하지만 별도의 위험 기준을 넣지 않는 한 낮은 꼬리를 특별히 피하도록 행동을 고르지는 않는다.

### 7.39 슬라이드 39 · 분포적 학습이 바꾸는 것

DQN은 $y = r + \gamma \max Q$라는 스칼라 목표를, C51은 보상에 의해 이동·축소한 **확률 분포 목표**를 맞춘다. replay, target network, 탐색 등 DQN의 주요 구성은 그대로 결합할 수 있다. 분포가 다봉형이어도 평균이 같으면 greedy 행동은 같을 수 있으므로 "분포를 예측한다 = 위험 민감 정책을 쓴다"는 등식은 틀리다.

표준 DQN의 손실은 숫자 Q와 숫자 목표의 차이를 회귀한다. C51은 현재 행동의 51개 확률과 투영한 목표 확률의 교차엔트로피를 비교한다. 그래도 탐색, replay, target 망은 여전히 필요하다. 반환 분포라는 새 표현이 데이터 coverage나 목표 이동 문제를 자동으로 없애지는 않는다.

### 7.40 슬라이드 40 · 개선들의 상호작용

Double DQN, $n$단계 목표, PER, C51은 각각 독립적으로 정의할 수 있으나 함께 쓰면 효과가 단순 합산되지는 않는다. 예를 들어 $n$단계 목표는 TD 오차의 분포를 바꾸어 PER의 샘플링 빈도도 바꾼다. 결합 알고리즘을 평가할 때는 구성요소 하나를 제거한 ablation이 필요하다.

PER과 $n$단계를 함께 쓰면 $n$단계 목표의 큰 오차를 가진 전이를 우선해서 뽑는다. C51을 쓰면 우선순위로 무엇을 오차로 측정할지도 달라질 수 있다. 한 구성요소가 다른 구성요소의 입력 분포를 바꾸므로, 각 기술의 단독 효과를 합산해 Rainbow의 성능을 예측할 수 없다.

### 7.41 슬라이드 41 · Rainbow

Rainbow는 Double DQN, multi-step, prioritized replay, distributional RL에 dueling architecture와 noisy networks까지 결합해 Atari에서 평가했다. "무지개"라는 이름은 한 핵심 공식이 아니라 여러 개선의 조합을 가리킨다. 성능을 읽을 때 어떤 전처리·환경 step 기준·평가 seed가 쓰였는지 확인해야 한다. Rainbow 논문

Rainbow의 구성요소 제거 실험을 읽으면 어느 기법이 해당 Atari 설정에서 큰 기여를 했는지 볼 수 있다. 그러나 그 순위를 다른 환경으로 그대로 옮길 수는 없다. 보상이 희소한 게임, 행동이 많은 게임, 화면 변화가 빠른 게임은 각각 병목이 다르기 때문이다. 조합의 성능과 각 요소의 일반적인 우수성을 구분하자.

### 7.42 슬라이드 42 · 연속 행동으로 넘어가기

DQN의 $a^* = \arg\max_a Q(s,a)$는 몇 개의 버튼 중 하나를 고를 때 간단하다. 로봇의 각 관절 토크가 실수라면 후보 공간이 무한해 매 step 열거할 수 없다. 다음 장은 Q망을 유지하되 $Q$를 크게 만드는 행동을 내는 actor를 별도로 학습하는 방법을 다룬다. Replay가 가능한 off-policy actor–critic의 출발점이다.

8개 관절의 토크를 각 축에서 10개 값으로만 이산화해도 행동 조합은 $10^8$개다. 매 상태에서 모두 평가하는 일은 비현실적이다. 연속 행동 actor는 $\mu_\theta(s)$로 유망한 행동 하나를 직접 제안해 최대화 문제를 근사하지만, critic이 틀리면 그 오류를 actor가 이용할 수 있다는 새 위험도 생긴다.

# 8 05-1 Off-policy Actor–Critic

## 8.1 슬라이드 01 · 연속 행동을 향하여

앞 장의 DQN은 가능한 행동을 모두 비교해 $\arg\max_a Q(s,a)$를 계산했다. 로봇 관절처럼 행동이 실수 벡터면 이 계산을 매 제어 주기 수행하기 어렵다. 이번 장은 행동을 직접 출력하는 actor와 그 행동의 가치를 평가하는 critic을 함께 배우면서 replay를 활용하는 DDPG, TD3, SAC로 이어진다.

로봇의 관절 8개에 각 10개 후보 토크만 둬도 조합은 $10^8$개다. DQN처럼 모든 후보의 Q를 계산하는 방식은 한 번의 제어 결정에도 비싸다. Actor가 좋은 행동을 직접 내도록 학습하면 실행 시 계산은 빠르지만, actor가 critic의 잘못된 높은 값을 쫓을 수 있다는 새 문제가 생긴다.

## 8.2 슬라이드 02 · 다룰 알고리즘의 관계

Q-learning의 Bellman 목표와 DQN 안정화 장치를 먼저 되짚는다. DDPG는 연속 행동의 최대화 문제를 결정적 actor로 근사하고, TD3는 DDPG의 critic 과대평가와 actor의 성급한 갱신을 줄인다. SAC는 확률적 actor와 엔트로피 목표를 도입한다. 셋 모두 데이터 수집 정책과 학습 중 평가하는 정책이 달라질 수 있는 off-policy 계열이다.

세 방법을 한 질문으로 비교하자. "다음 행동을 무엇으로 평가하는가?" DDPG는 target actor가 내는 한 행동을 critic으로 평가한다. TD3는 target 행동 주변의 잡음을 넣고 두 critic의 낮은 예측을 사용한다. SAC는 정책에서 샘플한 행동의 Q와 로그확률을 함께 사용한다. Bellman 구조는 이어지지만 목표에 들어가는 행동과 값이 달라진다.

## 8.3 슬라이드 03 · 정책 개선의 두 경로

정책경사는 $Q^\pi$가 주어졌을 때 $\nabla_\theta \log \pi_\theta(a \mid s) A^\pi(s,a)$로 확률을 조정한다. 가치 기반 접근은 $Q^\pi$가 큰 행동을 직접 선택해 개선한다. 둘 다 가치의 품질에 의존한다. $Q$가 잘못되면 actor는 잘못된 행동의 확률을 높이거나 그 행동을 직접 출력할 수 있다.

두 경로 모두 critic의 오차에 민감하지만 오류를 이용하는 방식은 다르다. 확률 정책은 높은 advantage를 가진 표본 행동의 확률을 높이고, 결정적 actor는 Q의 행동 방향 기울기를 따라 직접 움직인다. Q망이 데이터 밖에서 비현실적인 봉우리를 만들었다면 두 번째 방식은 그 봉우리를 더 빠르게 찾아갈 수 있다.

## 8.4 슬라이드 04 · Bellman 식의 재확인

$Q^\pi(s,a) = E[r + \gamma V^\pi(s') \mid s,a]$이고 $V^\pi(s') = E_{a' \sim \pi} Q^\pi(s',a')$다. 최적 정책의 경우 $V^*(s') = \max_{a'} Q^*(s',a')$로 바뀐다. 따라서 학습할 때 현재 전이의 보상과 다음 상태 가치를 합한 목표를 만든다. 연속 행동에서 어려운 지점은 방정식 자체가 아니라 $\max_{a'}$를 빠르고 안정적으로 근사하는 부분이다.

한 단계 보상이 1이고 다음 상태의 최대 Q가 4, $\gamma = 0.9$라면 최적 Bellman 목표는 4.6이다. DDPG에서 target actor가 낸 행동의 Q가 3이라면 목표는 3.7이다. 이는 actor가 모든 가능한 행동의 진짜 최댓값을 찾았다는 보장이 없기 때문이다. 계산 가능성을 얻는 대신 최대화의 근사 오차를 받아들인다.

## 8.5 슬라이드 05 · DQN의 안전장치

Replay buffer는 시간적으로 이웃한 표본의 상관을 줄이고 경험을 재사용한다. Target network는 부트스트랩 목표를 잠시 고정한다. $\epsilon$-greedy는 이산 행동의 탐색을 만든다. DDPG는 앞의 두 장치를 가져오지만 연속 행동에서 균등 무작위 행동 하나를 섞는 식의 탐색은 비효율적일 수 있어 actor 출력에 연속 잡음을 더한다.

연속 행동에서 행동 범위가 $[-1,1]^8$라면 균등 무작위로 뽑은 벡터는 actor가 현재 유망하다고 보는 행동에서 멀 수 있다. Actor 출력 주변에 작은 Gaussian 잡음을 더하면 현재 행동을 중심으로 국소 탐색을 한다. 반면 국소 잡음만 쓰면 먼 곳의 좋은 행동을 놓칠 수 있어 잡음 규모와 초기 경험의 다양성을 같이 설계한다.

### 8.6 슬라이드 06 · DQN 한 사이클

수집 정책으로 상태 $s$, 행동 $a$, 보상 $r$, 다음 상태 $s'$, 진짜 종료 표시 $d \in \{0,1\}$를 replay에 저장한다. $Q_\phi$는 학습 중인 Q망, $Q_{\bar{\phi}}$는 목표 계산 동안 고정한 target 망, $\gamma$는 할인율이다. Replay 표본마다 Bellman 목표 $y$를 만들고 $Q_\phi(s,a)$와의 차이를 줄인다.

$$y = r + \gamma(1-d)\max_{a'} Q_{\bar{\phi}}(s',a')$$

이 과정은 수집 당시 정책의 행동 확률이 목표 식에 직접 들어가지 않는다는 점에서 off-policy다. 단, 충분한 행동 coverage가 없으면 좋은 행동의 가치를 배울 표본도 없다.

Replay의 현재 행동 $a$는 수집 정책의 기록이고, 목표의 $\max_{a'}$는 다음 상태에서 Q가 큰 행동을 택한다. 이 두 선택이 다르기 때문에 과거 자료를 재사용할 수 있다.

### 8.7 슬라이드 07 · DQN 개선의 역할

Double DQN은 행동 선택과 가치 평가를 분리하고, $n$단계 목표는 실제 보상을 더 길게 반영한다. Prioritized replay는 TD 오차가 큰 자료를 더 자주 사용하고, 분포적 RL은 반환의 확률분포를 학습한다. 연속 행동 알고리즘도 비슷한 문제를 만나지만 DQN의 해법을 그대로 복사하지는 않는다.

Double DQN과 TD3는 모두 높은 Q 오류를 걱정하지만 방법은 다르다. Double DQN은 하나의 행동을 **고르는 망**과 그 행동을 **평가하는 망**을 분리한다. TD3는 연속 행동에서 actor가 고른 행동을 **두 critic의 작은 값**으로 평가한다. 이름의 유사성보다 목표식에 어느 망이 들어가는지를 봐야 한다.

### 8.8 슬라이드 08 · 최댓값은 오차를 고른다

두 행동의 참값이 모두 0이고 각각의 추정 오차가 $\pm 1$이라면 max는 양의 오차를 더 자주 선택한다. 일반적으로 $E[\max_a \widehat{Q}_a] \geq \max_a E[\widehat{Q}_a]$다. 정책이 $Q$가 큰 행동을 적극적으로 찾을수록 critic의 우연한 과대평가를 악용할 위험도 커진다. 이 현상은 연속 행동 actor가 미분을 통해 arg max를 좇을 때 더욱 중요하다.

두 행동의 실제 Q가 5인데 critic의 추정이 4와 9라면 actor는 9라고 믿는 행동 쪽으로 이동한다. 그 행동의 실제 반환을 확인하지 않고 Q만 반복 최적화하면 더 높은 오차를 찾는 셈이 될 수 있다. 연속 공간에서는 아주 좁은 잘못된 봉우리도 기울기를 통해 따라갈 수 있어 target smoothing이 중요해진다.

### 8.9 슬라이드 09 · Double DQN의 기준

Double DQN은 $a' = \arg\max_a Q_\phi(s',a)$로 행동을 고르고 $Q_{\bar{\phi}}(s',a')$로 평가한다. 두 단계가 같은 추정 오차를 완전히 공유하지 않게 해 최대값 편향을 줄인다. 연속 행동에서는 arg max 자체를 열거하지 못하므로 actor와 **두 critic의 최솟값**을 쓰는 TD3의 다른 형태로 이어진다. 두 방법의 이름에 "double"이 있어도 수식은 같지 않다. Double DQN 논문

온라인 망이 행동 A를 최고로 골랐다고 target 망까지 A를 최고로 평가할 필요는 없다. 두 망이 모두 같은 자료에서 학습해 오차가 상관되어 있어도, 선택한 값과 평가한 값을 완전히 동일한 계산에서 얻는 것보다 낙관 편향을 낮출 수 있다. TD3의 twin critic은 별도의 두 평가기를 두는 다른 구조다.

### 8.10 슬라이드 10 · 개선책의 한계도 이어진다

Replay와 target을 도입하면 표본 재사용과 목표 안정성이 좋아지지만, 정책이 아예 방문하지 않는 영역의 가치는 알 수 없다. 연속 행동에서도 actor가 높은 Q를 찾아 움직일 때 데이터 지지 밖의 critic 예측을 과신하면 실패한다. 이는 뒤의 offline RL에서 핵심 문제가 된다.

온라인 off-policy 방법은 현재 actor가 불확실한 행동을 시도해 새 데이터를 buffer에 넣을 수 있다. Offline RL에서는 이 교정 경로가 없다. 같은 Bellman 목표와 같은 신경망을 써도 자료를 더 얻을 수 있는지에 따라 critic 오류의 위험이 달라진다. 이 차이를 뒤 장에서 다시 살핀다.

### 8.11 슬라이드 11 · Q-learning의 정책은 암묵적이다

이산 Q-learning에서는 $Q_\phi$만 학습해도 $\pi(s) = \arg\max_a Q_\phi(s,a)$로 정책이 결정된다. 수집 정책은 그와 달라도 된다. 반대로 연속 행동에서는 매 순간 $Q_\phi(s,a)$를 최적화해 행동을 찾는 비용이 커서, 이 최적화 결과를 빠르게 흉내 내는 함수 $\mu_\theta(s)$를 별도로 학습한다.

이산 행동 4개라면 Q망이 길이 4의 출력을 내고 가장 큰 인덱스를 고르면 된다. 연속 행동에서는 Q망에 행동 벡터를 입력해야 하므로, 어느 벡터를 넣어야 할지 별도 문제가 된다. 결정적 actor는 상태에서 행동으로 가는 함수를 학습해 **매번 풀어야 할 최적화 문제를 빠른 함수 호출로 바꾼다.**

### 8.12 슬라이드 12 · DDPG로 넘어가는 단계

연속 행동에서는 replay 전이로 critic을 학습하고 actor가 critic의 높은 값으로 향하도록 만든다. Actor가 빠르게 움직여 자료에 드문 행동을 내면 critic의 추정 오류를 증폭할 수 있다. DDPG 이후의 TD3는 actor 갱신을 늦추고 Q 추정을 안정화하며, SAC는 확률적 actor와 엔트로피 목표를 사용한다.

### 8.13 슬라이드 13 · 연속 행동에서 전수 비교가 막힌다

10개 관절 각각에 100개 후보 토크를 두면 가능한 조합은 $100^{10}$개다. 매 step 전부 평가하는 방식은 불가능하다. 상태마다 gradient ascent로 $\max_a Q(s,a)$를 풀 수도 있지만 여러 시작점과 반복이 필요하며 비볼록 Q망의 국소 최댓값에 빠질 수 있다. actor $\mu_\theta(s)$는 상태에서 좋은 행동으로 가는 계산을 학습 시점에 압축한 함수다.

Actor는 모든 행동을 평가하지 않고 높은 Q를 낼 것으로 학습한 하나의 행동을 제안한다. 이 근사는 빠르지만 초기 정책과 critic 표면의 국소 봉우리에 민감하다.

### 8.14 슬라이드 14 · 결정적 actor로 최대화를 근사

Replay buffer에서 뽑는 상태분포를 $\mathcal{B}$, actor 파라미터를 $\theta$, critic 파라미터를 $\phi$라 하자. Critic을 고정하고 $J_{\text{actor}}(\theta) = E_{s\sim\mathcal{B}}[Q_\phi(s,\mu_\theta(s))]$를 최대화한다. 출력 $\mu_\theta(s)$가 critic의 행동 입력으로 이어지므로 자동미분으로 actor를 움직일 수 있다. DDPG 논문

예를 들어 현재 actor가 $a = 0.2$를 내고 critic이 그 근처에서 행동을 키우면 Q가 높아진다고 예측하면, actor 파라미터도 출력을 0.2보다 크게 만들도록 바뀐다. 이 기울기는 **critic이 배운 Q 표면**을 따라가는 것이지 실제 환경을 미분한 값이 아니다. Critic이 거짓 봉우리를 만들었다면 같은 계산이 정책을 악화시킨다.

### 8.15 슬라이드 15 · 결정적 정책경사의 연쇄법칙

$\nabla_a Q_\phi(s,a)$는 행동을 바꿀 때 critic 값이 변하는 방향이고, $\nabla_\theta \mu_\theta(s)$는 actor 파라미터가 행동 출력을 바꾸는 Jacobian이다. 상태 $s$를 고정해 두 기울기를 연쇄법칙으로 곱한 뒤 replay 상태분포 $\mathcal{B}$에서

평균하면

$$\nabla_\theta J_{\text{actor}} = E_{s\sim\mathcal{B}}\Big[\nabla_a Q_\phi(s,a)|_{a=\mu_\theta(s)}\,\nabla_\theta \mu_\theta(s)\Big].$$

$\nabla_a Q$는 행동을 어느 방향으로 움직이면 예측 가치가 커지는지, $\nabla_\theta \mu$는 파라미터 변화가 행동에 미치는 영향이다. PPO의 $\nabla \log \pi$ 추정과 달리 critic을 통해 직접 미분하므로 actor 기울기의 Monte Carlo 분산은 줄 수 있지만 **critic 편향**은 더 직접적으로 반영된다. Replay 상태분포로 쓰는 식은 실제 목적의 정책 상태분포와 동일하다는 일반적 보장을 뜻하지 않는다.

행동이 $m$차원이고 파라미터가 $p$개라면 $\nabla_a Q$는 길이 $m$의 벡터, actor Jacobian은 $m \times p$ 행렬이며 결과는 길이 $p$의 기울기다.

한 차원 예에서 actor가 $\mu_\theta(s) = \theta$, critic이 $Q_\phi(s,a) = -(a-2)^2$라고 예측하자. $\partial Q/\partial a = -2(a-2)$이고 $\partial \mu/\partial \theta = 1$이므로 $\theta = 0$에서 actor 기울기는 4다. 경사 상승은 행동을 2에 가깝게 움직인다. 하지만 실제 환경의 최적 행동이 2라는 증명은 아니다. **학습된 critic이 그렇게 예측했다**는 뜻이며, 데이터가 거의 없는 $a = 2$ 근처라면 이 기울기는 특히 의심해야 한다.

## 8.16 슬라이드 16 · 두 target 망의 소프트 갱신

DDPG는 critic target $\bar{\phi}$와 actor target $\bar{\theta}$를 모두 둔다. 매 step $\bar{\phi} \leftarrow \rho\bar{\phi} + (1-\rho)\phi$, $\bar{\theta} \leftarrow \rho\bar{\theta} + (1-\rho)\theta$로 천천히 따라오게 한다. $\rho$가 1에 가까울수록 목표 변화가 느리다. 다른 표기에서 $\tau = 1-\rho$를 쓰므로 코드의 숫자가 어느 방식인지 확인해야 한다.

$\rho = 0.995$라면 새 온라인 파라미터의 0.5%만 target에 반영한다. 한 번의 갱신이 target을 크게 바꾸지 않으므로 critic의 회귀 목표가 덜 흔들린다. 반면 target이 너무 늦으면 현재 actor의 행동을 오래된 critic이 평가할 수 있다. $\rho$와 $\tau$는 서로 보완 관계이므로 표기를 뒤집어 쓰지 말자.

## 8.17 슬라이드 17 · DDPG의 전체 업데이트

$\eta$는 자료 수집 때 actor 행동에 더하는 탐색 잡음이다. 전이 $(s,a,r,s',d)$를 저장한 뒤, 진짜 종료를 뜻하는 $d = 1$이면 미래 항을 제거한다. $\bar{\theta}, \bar{\phi}$는 갱신을 늦춘 target actor·critic 파라미터다. Critic의 회귀 목표는

$$y = r + \gamma(1-d)Q_{\bar{\phi}}(s', \mu_{\bar{\theta}}(s')).$$

$Q_\phi(s,a)$를 $y$에 회귀한 뒤 actor는 $Q_\phi(s, \mu_\theta(s))$를 높이고 두 target 망을 느리게 갱신한다. **실제 수집 행동의 잡음 $\eta$와 다음 상태 목표 행동**은 구분해야 한다. 종료 상태는 미래 항이 없다. DDPG 논문

탐색 잡음 $\eta$가 붙은 실제 행동 $a = \mu(s) + \eta$가 replay에 저장되고 critic의 현재 예측 $Q_\phi(s,a)$에 들어간다. 다음 상태 목표에는 target actor의 $\mu_{\bar{\theta}}(s')$가 들어간다. 두 행동을 바꾸어 쓰면 "실제로 관측한 전이"와 "앞으로 따르려는 정책"의 역할이 뒤섞인다.

## 8.18 슬라이드 18 · DQN에서 DDPG로 바뀐 자리

이산 DQN의 $\epsilon$-greedy는 연속 행동 탐색 잡음으로, $\max_a Q(s',a)$는 target actor 출력 $\mu_{\bar{\theta}}(s')$로 바뀐다. Replay와 critic target은 유지되고 actor의 정책 개선 단계가 추가된다. "DQN의 연속 버전"이라는 요약은 이 대응을 뜻하지만 결정적 actor에 대한 새 가정과 오차를 숨기지는 못한다.

같은 전이 $(s,a,r,s')$가 critic 회귀에는 쓰이지만 actor의 손실은 $Q_\phi(s, \mu_\theta(s))$를 평가한다. Replay의 $a$와 현재 actor의 $\mu_\theta(s)$가 다르기 때문이다. 이 구분이 off-policy 학습의 장점이자, 데이터에 없는 actor 행동을 critic이 잘못 평가할 수 있는 원인이다.

### 8.19 슬라이드 19 · DDPG의 실용적 한계

결정적 정책이므로 별도 탐색 잡음 없이는 거의 같은 행동만 반복한다. Critic이 우연히 높은 값을 예측하는 행동을 actor가 학습하면 오차가 증폭될 수 있다. 행동 스케일, 잡음 크기, target 갱신 속도와 학습률에 민감하다. 재현 실험에서는 같은 seed 하나의 최고점보다 여러 seed의 곡선을 살펴야 한다.

실험에서 critic 예측 Q만 계속 높고 rollout 반환이 떨어진다면 actor가 critic 오류를 이용하는지 살펴야 한다. 탐색 잡음을 너무 작게 잡으면 필요한 행동을 못 만나고, 너무 크게 잡으면 대부분 낮은 품질의 전이를 모을 수 있다. DDPG의 민감도는 하나의 학습률보다 **자료 범위와 가치 추정의 상호작용**에서 생긴다.

### 8.20 슬라이드 20 · TD3가 해결할 세 문제

TD3는 DDPG의 계산 구조를 유지하면서 세 곳을 바꾼다. **값**에서는 두 critic 중 작은 예측을 써 낙관적 목표를 줄인다. **갱신 속도**에서는 actor를 critic보다 드물게 바꿔 가치 추정이 따라올 시간을 준다. **target 행동 주변**에서는 작은 잡음을 넣어 한 좌표의 좁은 Q 봉우리에 목표가 매달리지 않게 한다.

### 8.21 슬라이드 21 · TD3의 세 안정화 장치

두 critic $Q_{\phi_1}, Q_{\phi_2}$를 따로 학습하고 느리게 갱신하는 target critic을 $Q_{\bar{\phi}_1}, Q_{\bar{\phi}_2}$라 하자. $\mu_{\bar{\theta}}$는 target actor다. $\xi$는 target 행동에만 더하는 작은 잡음, $c > 0$은 그 잡음의 절댓값 상한, $[a_{\min}, a_{\max}]$는 가능한 행동 범위다. $\mathrm{clip}(x, l, u)$는 $x$를 $[l, u]$ 안으로 자르는 함수다. Replay 전이 $(s, a, r, s', d)$의 종료 표시 $d$까지 넣으면

$$\tilde{a}' = \mathrm{clip}\big(\mu_{\bar{\theta}}(s') + \mathrm{clip}(\xi, -c, c), a_{\min}, a_{\max}\big), \qquad y = r + \gamma(1-d) \min_{i \in \{1,2\}} Q_{\bar{\phi}_i}(s', \tilde{a}').$$

Actor와 target 망은 critic보다 드물게 갱신한다.

최솟값은 과대평가를 억제하지만 지나치면 과소평가를 만들 수 있다. Target 잡음은 자료 수집용 탐색 잡음과 목적이 다르다. 전자는 **목표 Q의 날카로운 봉우리**를 완화하고 후자는 실제 새로운 상태–행동을 방문하게 한다. TD3 논문

자료 수집 잡음은 실제 환경에서 다양한 $(s, a)$ 전이를 얻기 위해 actor 행동에 붙인다. Target policy smoothing 잡음은 이미 저장된 전이의 **다음 상태 target 행동**에만 붙여 Q 목표를 계산한다. 두 잡음의 크기를 코드에서 섞으면 탐색 범위와 critic의 보수성이 동시에 바뀌어 실험 원인을 해석하기 어렵다.

### 8.22 슬라이드 22 · 두 critic이 주는 보수성

한 critic이 특정 행동을 12, 다른 critic이 5로 예측한다면 TD3는 다음 상태 목표에 5를 사용한다. 실제 가치가 6이라면 큰 과대평가를 피한다. 반대로 실제 가치가 10이라면 이 선택이 과도하게 보수적일 수 있다. 목적은 항상 낮게 예측하는 것이 아니라 actor가 우연한 큰 오류를 계속 이용하는 고리를 끊는 것이다.

두 critic의 예측이 12와 5인 상황에서 5를 택하면 낙관 오류는 막을 수 있지만 실제 가치가 10이라면 5만큼 과소평가한다. **최솟값은 참값의 하한을 증명하지 않는다.** 서로 다른 초기화와 minibatch로 학습해 오류가 완전히 같아지는 일을 줄이지만 두 망이 같은 자료를 본다는 한계는 남는다.

Double DQN과 비교하면 둘 다 큰 Q를 무비판적으로 쓰지 않지만 계산 위치가 다르다. Double DQN은 **한 온라인 망으로 행동을 고르고 target 망으로 그 행동을 평가**한다. TD3는 target actor가 낸 연속 행동을 **두 target critic에 모두 넣어 작은 평가값**을 쓴다. 이 구분은 행동 선택 오류와 가치 평가 오류가 알고리즘에서 어디에 남는지 보여 준다.

### 8.23 슬라이드 23 · TD3 학습 흐름

Replay에서 batch를 뽑아 target actor에 smoothing noise를 넣고 두 target critic의 최솟값으로 목표를 계산한다. 두 현재 critic을 매 step 갱신한다. 일정 주기마다 actor가 $Q_{\phi_1}(s, \mu_\theta(s))$를 높이도록 갱신하고, 그때 target 망들도 소프트 업데이트한다. Actor를 늦추는 이유는 부정확한 critic 위에서 정책을 너무 빨리 움직이지 않게 하기 위해서다.

구현에서는 critic을 매 step 갱신해도 actor와 target 망은 예를 들어 두 critic step마다 한 번만 갱신할 수 있다. Actor를 갱신하지 않는 step에서도 수집과 critic 학습은 계속된다. "delayed"는 환경 행동을 천천히 실행한다는 뜻이 아니라 **정책 파라미터 갱신의 빈도**를 낮춘다는 뜻이다.

### 8.24 슬라이드 24 · 성능 그래프를 해석할 때

TD3 논문의 곡선은 선택한 연속 제어 환경에서 DDPG와 다른 방법의 학습 안정성을 비교한다. 그래프의 가로축이 **환경 상호작용 수**인지 최적화 step인지 확인해야 표본 효율을 읽을 수 있다. 평균 곡선과 seed 간 분산을 함께 보아야 하며, 한 환경에서의 우세를 모든 로봇 과제의 우세로 일반화할 수 없다. TD3 논문

한 환경에서 TD3의 평균 반환이 DDPG보다 높아도 학습 시간과 표본 사용량이 같았는지 확인하자. Replay 업데이트를 더 많이 수행했다면 환경 step 기준의 효율과 계산 시간 기준의 효율이 다를 수 있다. 특히 seed에 따라 DDPG가 불안정하면 평균과 표준편차를 함께 제시해야 한 번의 운 좋은 실행을 과장하지 않는다.

### 8.25 슬라이드 25 · TD3 요약

TD3는 deterministic actor와 replay를 유지하는 DDPG의 개선형이다. Twin critic, delayed policy update, target policy smoothing이 이름의 세 핵심이다. 적용은 연속 행동에서 자연스럽지만 관측, 보상, 탐색 잡음, 행동 경계가 달라지면 하이퍼파라미터를 다시 확인해야 한다.

TD3의 장치를 표로 요약하면 현재 행동은 deterministic actor가 내고, 자료 수집 때는 탐색 잡음을 더하며, target 행동에는 별도의 smoothing noise를 더하고, 두 Q 중 작은 값으로 목표를 만든다. 이 네 위치의 행동을 각각 기록해 보면 구현에서 noise를 잘못된 위치에 넣는 실수를 예방할 수 있다.

### 8.26 슬라이드 26 · 확률적 actor라는 다른 선택

TD3의 actor는 같은 상태에 언제나 같은 평균 행동을 낸다. 환경 변화와 여러 유효 해가 있을 때는 상태별 **행동 분포**를 학습하는 접근이 유용하다. SAC는 가우시안 분포를 매개변수화해 sample을 만들고, 높은 보상과 충분한 엔트로피를 함께 추구한다.

예를 들어 로봇이 장애물을 왼쪽이나 오른쪽으로 돌아가는 두 경로가 모두 가능하다면 하나의 deterministic actor는 초기 학습에 따라 한쪽에 고착될 수 있다. 확률 정책은 여러 행동을 샘플할 수 있지만 단일 Gaussian도 두 경로를 완벽하게 표현하지는 못한다. SAC의 엔트로피는 행동 분포의 퍼짐을 장려하는 목표이지 임의의 다봉성을 자동 해결하는 장치는 아니다.

### 8.27 슬라이드 27 · 평균과 분산을 모두 배우기

상태 $s$에서 actor 파라미터 $\theta$가 출력하는 $\mu_\theta(s)$와 $\sigma_\theta(s) > 0$는 각각 Gaussian의 평균과 표준편차다. 먼저 **압축 전** 변수 $U \mid s \sim \mathcal{N}(\mu_\theta(s), \sigma_\theta(s)^2)$를 뽑는다. $\sigma$가 작으면 평균 근처에 집중하고, 크면 여러 행동을 시험한다. 행동 범위가 $[-1, 1]$이면 $a = \tanh U$로 압축한다. 따라서 실제 행동 $a$의 분포 $\pi_\theta(a \mid s)$는 Gaussian 자체가 아니다. 로그밀도를 구할 때 이 변환의 Jacobian을 반영해야 한다. SAC 논문

한 행동 좌표에서 표준정규 잡음 $\varepsilon \sim \mathcal{N}(0,1)$를 이용해 $u = \mu_\theta(s) + \sigma_\theta(s)\varepsilon$로 표본을 만들자. $p_U(u \mid s)$를 압축 전 변수 $U$의 밀도라 하면, 확률질량 보존 $\pi_\theta(a \mid s)\,da = p_U(u \mid s)\,du$에서 $\pi_\theta(a \mid s) = p_U(u \mid s)/|da/du|$다. $a = \tanh u$이고 $da/du = 1 - \tanh^2 u$이므로

$$\log \pi_\theta(a \mid s) = \log p_U(u \mid s) - \log(1 - \tanh^2 u).$$

여러 행동 차원에서는 좌표별 보정항을 합한다. $u$가 매우 크면 $a$가 $\pm 1$ 근처에 몰려 수치 오차가 커지므로, 구현은 이 항을 안정적인 형태로 계산한다. SAC 원논문

**SAC의 확률 정책은 항상 두 개의 행동 모드를 표현할까?** 일반적인 단일 Gaussian 정책은 한 중심 주변의 퍼짐을 나타낸다. 서로 떨어진 두 행동 경로를 정확히 표현하려면 mixture나 더 유연한 정책 분포가 필요할 수 있다. Entropy가 높다는 사실만으로 다봉 표현이 된다고 볼 수 없다.

### 8.28 슬라이드 28 · 최대 엔트로피 목적

여기서 $r(s_t, a_t)$는 시점 $t$의 보상, $\gamma \in [0,1)$는 할인율, $E_\pi$는 정책 $\pi$로 만든 경로에 대한 기댓값이다. 상태 $s$의 정책 엔트로피 $H(\pi(\cdot \mid s)) = E_{a\sim\pi(\cdot|s)}[-\log \pi(a \mid s)]$는 행동의 퍼짐을 측정한다. 온도 $\alpha \geq 0$로 보상과 엔트로피의 비중을 정하면

$$J_{\text{soft}}(\pi) = E_\pi\left[\sum_t \gamma^t \{r(s_t, a_t) + \alpha H(\pi(\cdot \mid s_t))\}\right]$$

를 최대화한다. $\alpha$가 크면 여러 행동을 유지하려는 압력이 강하다. 연속분포의 미분 엔트로피는 음수가 될 수 있으므로 그 숫자를 이산 엔트로피와 같은 범위로 해석하지 않는다.

두 행동이 같은 보상인데 정책이 항상 A만 고른다면 보상 항은 높아도 엔트로피는 낮다. $\alpha > 0$인 soft 목표는 B에도 일부 확률을 주는 정책을 선호할 수 있다. 이 선택은 탐색 가능성을 남기지만, 실제 배포에서 B가 위험하다면 별도의 안전 제약이 필요하다. 엔트로피는 위험의 무조건적 해답이 아니다.

한 상태에서 행동 A와 B의 soft Q가 각각 2와 0이라고 하자. 고정된 Q에 대해 $\sum_a \pi(a)Q(a) + \alpha H(\pi)$를 최대화하면 $\pi(a) \propto \exp(Q(a)/\alpha)$가 된다. $\alpha = 1$이면 A의 확률은 $e^2/(e^2+1) \approx 0.881$이고 B도 약 0.119를 받는다. $\alpha \to 0$에서는 거의 최댓값 행동만 선택하고, $\alpha$가 매우 크면 두 행동이 비슷한 확률이 된다. 이 예는 온도가 **가치 차이를 얼마나 무작위 행동으로 희석할지** 정함을 보여 준다.

Boltzmann 형태는 엔트로피 항을 미분하면 나온다. 한 상태에서 $p_a = \pi(a \mid s)$, $Q_a = Q(s,a)$라 쓰고 $\sum_a p_a = 1$을 승수 $c$로 강제한다. $\alpha > 0$일 때

$$\begin{aligned} F(p, c) &= \sum_a p_a Q_a - \alpha \sum_a p_a \log p_a + c\left(\sum_a p_a - 1\right), \\ 0 = \frac{\partial F}{\partial p_a} &= Q_a - \alpha(\log p_a + 1) + c, \\ p_a &= \frac{\exp(Q_a/\alpha)}{\sum_b \exp(Q_b/\alpha)}. \end{aligned}$$

$\alpha$로 나눈 Q 차이가 행동 확률의 비율을 정한다: $p_a/p_b = \exp\{(Q_a - Q_b)/\alpha\}$. 모든 Q에 같은 상수를 더해도 이 비율은 바뀌지 않는다. 연속 행동에서는 합 대신 적분과 기준 측도가 필요하며, 구현의 파라미터화된 actor가 항상 이 이상적 분포를 정확히 표현하는 것은 아니다.

### 8.29 슬라이드 29 · 무작위성의 효용과 비용

두 개의 서로 다른 경로가 모두 성공한다면 확률적 정책은 한 경로에 조기 고착하지 않고 둘 다 탐색할 수 있다. 환경의 마찰이 바뀌었을 때 선택지를 남겨 둔 정책이 더 빨리 적응할 가능성도 있다. 그러나

실제 제어에서 무작위 행동 자체가 안전하지 않을 수 있고, 높은 엔트로피가 항상 높은 최종 보상을 의미하지 않는다. $\alpha$와 안전 제약을 맥락에 맞게 정해야 한다.

확률적 행동이 유용한지 판단하려면 같은 평균 반환뿐 아니라 마찰이나 초기 상태가 변했을 때의 성과를 보자. 다양한 행동을 계속 시험하는 정책이 변화에 빨리 적응할 수 있지만, 관절 토크에 큰 무작위성을 유지하면 물체를 떨어뜨릴 위험도 커진다. 학습 단계의 탐색과 평가·배포 단계의 행동 선택을 구분해야 한다.

### 8.30 슬라이드 30 · Soft Bellman 식

현재 정책 $\pi$의 soft 행동가치 $Q^{\pi}_{\text{soft}}(s,a)$는 상태 $s$에서 행동 $a$를 한 뒤 받을 보상과 미래 엔트로피 보너스의 기댓값이다. 다음 상태 $s'$에서 $a' \sim \pi(\cdot \mid s')$를 뽑고 그 다양성까지 평가한 상태가치를 $V^{\pi}_{\text{soft}}(s') = E_{a' \sim \pi(\cdot|s')}[Q^{\pi}_{\text{soft}}(s',a') - \alpha \log \pi(a' \mid s')]$로 정의한다. 전이와 보상의 확률성에 대한 조건부 기댓값을 취하면

$$Q^{\pi}_{\text{soft}}(s,a) = E[r + \gamma V^{\pi}_{\text{soft}}(s') \mid s,a].$$

$-\log \pi$의 정책 평균이 엔트로피이므로 $V^{\pi}_{\text{soft}}$에는 다음 상태의 보상 전망과 행동 다양성이 함께 들어간다. $\alpha = 0$이면 엔트로피 항이 사라져 일반 정책 평가식으로 돌아간다.

### 8.31 슬라이드 31 · SAC의 critic과 actor 목표

Replay buffer $\mathcal{B}$의 한 전이를 $(s,a,r,s',d)$라 하자. $d = 1$은 종료를 뜻한다. $Q_{\phi_1}, Q_{\phi_2}$는 두 critic, $Q_{\bar{\phi}_i}$는 각 critic의 느리게 갱신하는 target 망이다. 다음 행동 $a' \sim \pi_\theta(\cdot \mid s')$를 뽑아 $y = r + \gamma(1-d)[\min_{i\in\{1,2\}} Q_{\bar{\phi}_i}(s',a') - \alpha \log \pi_\theta(a' \mid s')]$를 만든다. $y$의 경사를 끊고 두 critic은 각각 $(Q_{\phi_i}(s,a) - y)^2$를 줄인다. Actor 파라미터 $\theta$는

$$J_\pi(\theta) = E_{s\sim\mathcal{B},\, a\sim\pi_\theta}[\alpha \log \pi_\theta(a \mid s) - \min_i Q_{\phi_i}(s,a)]$$

를 최소화한다. $a = f_\theta(s,\varepsilon)$, $\varepsilon \sim \mathcal{N}(0,I)$로 재매개변수화하면 표본 행동을 통해 actor로 경사가 흐른다. 실제 SAC 구현에는 tanh 밀도 보정과 target stop-gradient가 중요하다. SAC 논문

$-Q$는 높은 가치를 선호하고 $\alpha \log \pi$는 행동 분포의 엔트로피를 유지한다. 표준정규 잡음 $\varepsilon$를 고정한 채 $f_\theta$를 미분하면 critic의 행동 기울기가 actor까지 전달된다. 로그밀도도 함께 미분하며, 앞 슬라이드의 tanh Jacobian 보정을 이 항에 적용한다.

### 8.32 슬라이드 32 · 온도 $\alpha$를 자동 조절

목표 엔트로피 $\bar{H}$ 이상을 유지한다는 제약을 두면 $\alpha \geq 0$를 그 제약의 라그랑주 승수로 볼 수 있다. 정책 엔트로피가 목표보다 낮으면 $\alpha$를 높여 퍼짐을 장려하고, 너무 높으면 낮춰 보상을 더 중시하게 한다. 실무 손실은 구현의 부호 관례에 따라 달라지므로 $-\log \pi$의 평균이 목표와 어느 방향으로 다른지 확인해야 한다.

자동 조절이 "하이퍼파라미터가 없다"는 뜻은 아니다. 목표 엔트로피의 설정, reward scale, 행동 차원이 여전히 중요하다. 자동 온도 조절 연구

목표 엔트로피가 현재 정책보다 높다면 다양성을 더 장려해야 하므로 온도 $\alpha$가 커지는 방향이 맞다. 실제 코드에서 손실의 부호가 헷갈리면 $-\log \pi$의 batch 평균을 출력하고 목표와 비교하자. 보상의 숫자 크기를 100배 키우면 같은 $\alpha$의 상대적 영향은 줄어드므로 reward scale도 함께 확인해야 한다.

부호를 제약식에서 확인할 수 있다. $H(\pi) \geq \bar{H}$를 요구하면 정책 쪽에는 $J_{\text{reward}} + \alpha(H(\pi) - \bar{H})$를 최대화하게 하고, $\alpha \geq 0$는 반대 방향으로 최소화한다. $\partial/\partial\alpha = H(\pi) - \bar{H}$이므로 $H(\pi) < \bar{H}$일

때 $\alpha$의 경사 하강은 $\alpha$를 높인다. 실제 구현은 양수를 유지하려고 $\log\alpha$를 학습하기도 한다. 목적을 **최대화하는 actor**와 승수를 **최소화하는 조절기**의 방향을 혼동하지 않으면 손실 부호를 점검할 수 있다.

### 8.33 슬라이드 33 · SAC와 TD3의 공통 기반

TD3와 SAC는 replay, target critic, 두 Q망의 작은 값이라는 기반을 공유한다. TD3의 actor는 결정적이고, 자료 수집용 잡음과 target smoothing 잡음을 따로 더하며 actor를 지연 갱신한다. SAC의 actor는 분포를 출력하고, 표본 행동의 $\alpha\log\pi$를 critic 목표와 actor 목적에 넣는다. 코드에서는 **자료 수집 행동, target 행동, actor 업데이트 행동**을 나누어 보면 각 잡음의 역할이 보인다.

### 8.34 슬라이드 34 · SAC가 유용한 조건

연속 행동에서 고차원 탐색이 필요하고 환경 step이 비쌀 때 경험을 재사용하는 SAC가 강력한 기준선이다. 그러나 보상 설계가 잘못되면 엔트로피를 넣어도 원치 않는 행동을 학습할 수 있다. 분포의 표준편차, 온도, Q 과대평가, 평가 시 평균 행동 사용 여부를 함께 점검한다. "state of the art"는 당시 비교 환경의 결과로 읽어야 하며 모든 과제에서의 보편적 순위가 아니다.

로봇의 배포 정책에서는 학습 중 샘플링한 행동을 그대로 쓸지 평균 행동을 쓸지에 따라 성능과 안전성이 달라질 수 있다. SAC가 학습 때 엔트로피를 장려했다고 평가 때도 반드시 무작위 행동을 써야 하는 것은 아니다. 논문 점수를 비교할 때 평가 행동의 선택 방식과 episode 초기화를 확인하자.

### 8.35 슬라이드 35 · TD3와 SAC 실험 비교

그래프에서 두 방법의 수익 곡선은 환경마다 다르고 seed 간 편차도 있다. 동일한 환경 step, 네트워크 규모, 평가 주기, 보상 처리 아래에서 비교해야 알고리즘 효과를 분리할 수 있다. SAC 논문 그림과 TD3 논문 그림을 나란히 놓은 경우 원래 실험 설정이 동일하다고 가정해서는 안 된다. TD3, SAC

동일한 학습 곡선이라도 첫 10만 환경 단계에서 빨리 좋아지는 방법과 마지막에 높은 점수를 얻는 방법은 장점이 다르다. 평균 곡선 아래의 변동 폭이 크면 실험 재현성이 낮을 수 있다. 목표 점수 도달 단계 수, 마지막 성능, 여러 seed의 변동을 따로 보고 판단해야 한다.

### 8.36 슬라이드 36 · 지금까지의 학습 지형

모방학습은 관측된 좋은 행동을 복제하고, on-policy 정책경사는 현재 정책 rollout으로 직접 개선하며, DQN은 이산 행동의 off-policy 가치를 학습한다. DDPG·TD3·SAC는 연속 행동에서 actor와 critic을 결합해 replay를 사용한다. 어떤 방법이든 **환경과 실제 상호작용하며 새 자료를 계속 얻는다**는 공통점이 있다. 다음의 offline RL은 이 통로가 막혔을 때의 문제를 다룬다.

지금까지의 방법을 데이터 축에 놓으면 BC는 전문가 자료, PPO는 현재 정책의 새 rollout, DQN·TD3·SAC는 replay와 새 상호작용을 함께 쓴다. Offline RL은 replay만 있고 새 상호작용이 전혀 없다. **off-policy와 offline**이 다른 개념임을 이 흐름에서 다시 확인할 수 있다.

### 8.37 슬라이드 37 · 벤치마크로 이어지는 질문

표본 효율, 최종 성능, 학습 안정성을 말하려면 비교 환경과 측정 방법을 정의해야 한다. 다음 장에서는 Atari, MuJoCo 계열, 로봇 벤치마크가 각각 무엇을 시험하고 무엇을 놓치는지 살펴본다. 동일 알고리즘도 이미지 관측과 상태 벡터 관측에서 난도가 전혀 다르다.

한 알고리즘이 Ant에서 빨리 배우고 Atari에서는 느릴 수 있다. Ant는 연속 토크와 물리 접촉을, Atari는 픽셀 관측과 이산 버튼을 요구한다. 같은 "환경 단계 100만"도 시뮬레이션 비용과 한 단계의 실제

시간은 다르다. 벤치마크 점수를 보기 전에 **관측·행동·보상·평가 예산**을 먼저 적어야 한다.

# 9 05-2 강화학습 벤치마크

## 9.1 슬라이드 01 · 벤치마크를 읽는 관점

알고리즘의 이름만으로 우열을 정할 수 없다. 어떤 관측을 받고, 어떤 행동을 하며, 어떤 보상을 최대화하고, 몇 번 환경과 상호작용했는지가 먼저 정해져야 한다. 이 장은 게임, 시뮬레이션 제어, 실제 로봇 평가가 각각 다른 능력을 측정한다는 점을 살핀다.

같은 알고리즘 A가 Atari에서 높게 나오고 B가 로봇 조작에서 높게 나온다면 두 결과는 모순이 아니다. Atari의 이산 버튼과 이미지, 로봇의 연속 관절과 접촉은 서로 다른 병목을 만든다. **무엇을 측정하는 시험인가**를 먼저 정하고, 그 과제에서의 데이터·계산 예산을 맞춘 뒤에야 성능 숫자를 비교할 수 있다.

## 9.2 슬라이드 02 · 최종 성능과 표본 효율

가로축을 환경 상호작용 수, 세로축을 독립 평가 episode의 평균 반환으로 그리자. 초반에 빨리 오르는 곡선은 같은 데이터 예산에서 좋은 **표본 효율**을 가질 수 있다. 충분히 긴 학습 뒤 높은 수준에 도달하는 곡선은 **점근 성능**이 좋다. 두 순위는 다를 수 있다.

평가 return이 단일 seed 최고값인지 여러 seed 평균인지, 평가 때 탐색 잡음을 끄는지, 환경 step과 frame을 어떻게 세는지까지 통일해야 한다. 벽시계 시간은 병렬화와 하드웨어에 영향을 받으므로 별도 축이다.

방법 A가 10만 환경 단계에서 반환 100, 방법 B가 30만 단계에서 120이라면 제한 예산이 10만인 실험에서는 A가 유용할 수 있다. 반대로 충분한 예산과 최종 점수가 목표라면 B가 나을 수 있다. 곡선 한 점만 고르지 말고 **예산별 성능**, 여러 seed의 변동, 실제 실행 시간을 따로 보자.

성공률도 표본 수 없이 숫자만 비교하면 위험하다. 20번 중 18번 성공한 정책은 90%, 17번 성공한 정책은 85%지만 차이는 단 한 번의 시도다. 초기 물체 배치를 바꾸거나 seed를 달리하면 순위가 뒤집힐 수 있다. 성공 횟수와 전체 시도 수, 초기 조건별 결과를 함께 제시하고 충분한 반복에서 불확실성 구간을 계산해야 "5%p 우수하다"는 주장에 근거가 생긴다.

독립이고 같은 성공확률 $p$를 갖는 $n$번의 평가라는 단순한 가정 아래 성공 횟수 $K$는 이항분포이고 관측 성공률은 $\widehat{p} = K/n$이다. Var는 분산, SE는 추정치의 표준오차를 뜻한다. $\mathrm{Var}(K) = np(1-p)$에서 $\mathrm{Var}(K/n) = \mathrm{Var}(K)/n^2$를 적용하고 미지의 $p$를 $\widehat{p}$로 대체하면

$$\mathrm{Var}(\widehat{p}) = \frac{p(1-p)}{n}, \qquad \widehat{\mathrm{SE}}(\widehat{p}) = \sqrt{\frac{\widehat{p}(1-\widehat{p})}{n}}.$$

$18/20 = 0.9$라면 추정 표준오차는 $\sqrt{0.9 \cdot 0.1/20} \approx 0.067$로, 90%와 85%의 차이인 0.05보다도 크다. 따라서 이 결과만으로 우열을 확신하기 어렵다. 이 근사식은 시행들이 독립이라는 전제가 있고 작은 표본이나 극단적인 성공률에서 신뢰구간으로 그대로 쓰기 부정확하다. 같은 초기 조건을 공유한 두 정책의 대응 평가라면 성공·실패가 어떻게 짝지어졌는지도 기록해야 한다.

## 9.3 슬라이드 03 · 벤치마크의 지도

연속 제어의 DMC·MuJoCo, 이산 제어의 Atari·MiniGrid, 일반화의 Procgen, 장기 로봇 조작의 RLBench·Meta-World·LIBERO, 실제 로봇 조립의 FurnitureBench는 서로 다른 병목을 드러낸다. "많은 과제"라는 표현만으로 난도를 비교할 수 없다. 보상이 촘촘한지, 관측이 픽셀인지 상태 벡터인지, 성공 판정이 자동인지 사람이 확인하는지를 구분한다.

**왜 하나의 벤치마크에서 이긴 알고리즘이 보편적으로 우수하지 않은가?** 벤치마크마다 행동 차원, 보상 지연, 관측 잡음, 시뮬레이터 속도, 데이터 예산이 다르다. 한 방법의 탐색과 함수근사 편향이 특정 환경에 잘 맞았을 수 있으므로 다른 축의 과제와 독립 seed에서 다시 평가해야 한다.

벤치마크를 축으로 나누면 관측은 상태 벡터/픽셀, 행동은 이산/연속, 보상은 조밀/희소, 자료는 온라인/고정으로 볼 수 있다. 예를 들어 Atari는 픽셀·이산 행동이고 Ant는 관절 상태·연속 행동이다. 두 환경의 점수 평균을 단순히 한 숫자로 묶으면 알고리즘의 강약이 어느 축에서 생겼는지 가려질 수 있다.

### 9.4 슬라이드 04 · Gymnasium MuJoCo와 DMC

MuJoCo는 접촉과 관절의 물리 시뮬레이션 엔진이고, Gymnasium의 MuJoCo 과제와 DeepMind Control Suite는 **그 엔진을 이용한 서로 다른 환경 모음**이다. 관절 위치·속도 같은 proprioception을 주는 설정보다 픽셀 관측은 시각 표현 학습까지 요구한다. DMC의 100K와 1M step처럼 상호작용 예산을 함께 적어야 표본 효율을 비교할 수 있다.

같은 "walker"라는 이름이라도 보상 함수, action repeat, episode 길이가 다르면 다른 과제다. DMC 원논문, Gymnasium MuJoCo 문서

같은 물리 엔진을 쓴다는 사실만으로 같은 보행 과업이 되는 것은 아니다. Walker의 목표 속도, 보상항, 종료 조건, action repeat가 바뀌면 학습 곡선도 달라진다. DMC 100K에서 얻은 수치와 MuJoCo 1M의 수치를 직접 비교하기보다, **같은 환경 정의와 단계 예산** 안에서 알고리즘을 비교해야 한다.

### 9.5 슬라이드 05 · Atari의 관측과 예산

Atari는 버튼 조합이라는 이산 행동과 화면 또는 RAM 관측을 제공한다. 화면 210×160을 84×84 등으로 전처리하고 여러 frame을 쌓는 설정이 DQN 계열 실험에 흔하다. Atari 100K는 적은 환경 상호작용에서의 성능을, 큰 예산 실험은 더 긴 학습 후 성능을 드러낸다.

학습 step이 **에뮬레이터 frame**인지 행동 결정 횟수인지, life loss를 종료로 처리하는지, no-op 시작과 sticky action을 쓰는지에 따라 결과가 달라진다. Arcade Learning Environment 원논문, Gymnasium Atari 문서

frame skip이 4이면 의사결정 10만 번은 에뮬레이터 내부 프레임 40만 개에 해당할 수 있다. 논문의 가로축이 무엇을 세는지 확인하지 않으면 표본 효율이 네 배 좋아 보이는 착시가 생긴다. Sticky action은 선택한 행동 대신 이전 행동이 일정 확률로 실행될 수 있어, 결정론적 시퀀스를 외우는 정책의 성능도 바꾼다.

### 9.6 슬라이드 06 · 로봇 벤치마크의 폭

DoorGym과 robosuite는 물체와 접촉하는 조작을, Meta-World와 RLBench는 다양한 목표의 조작을, Franka Kitchen은 연속된 하위 과제를 시험한다. 하나의 성공률을 낼 때도 물체 배치·카메라·초기 자세·훈련 때 본 목표와 새 목표를 분리해야 한다. 로봇 벤치마크는 시뮬레이션 비용과 현실 전이 가능성이 중요한 비교 축이다.

문 열기 과제는 손잡이의 위치를 맞히는 지각과 접촉 뒤 회전 방향을 제어하는 동작이 결합된다. 주방에서 여러 스위치를 순서대로 조작하는 과제는 그보다 긴 계획이 필요하다. 성공률을 비교할 때는 **부분 과업이 몇 개인지, 실패 뒤 재시도가 허용되는지**를 함께 기록해야 한다.

### 9.7 슬라이드 07 · LIBERO와 평생학습

LIBERO는 여러 로봇 조작 과제를 순차로 배우며 지식을 옮길 수 있는지 평가한다. 새 과제를 학습한 후 이전 과제의 성능이 떨어지는 **망각**과, 이전 경험 덕분에 새 과제를 빨리 배우는 **전이**를 함께 볼 수 있다. 단일 과제의 최종 성공률만으로는 이 능력을 알 수 없다. VLA 평가에서도 텍스트 지시와 시각 관측을 어떤 분포에서 제공했는지 중요하다. LIBERO 논문

과제 A를 배운 뒤 B를 배웠더니 A의 성공률이 80%에서 30%로 떨어졌다면 새 과제 성능만 좋아진 것으로는 충분하지 않다. 이전 능력을 얼마나 유지했는지와 B를 처음부터 배울 때보다 얼마나 빨랐는지를 함께 측정한다. 이것이 일반적인 단일 과제 평가와 continual learning 평가의 차이다.

## 9.8 슬라이드 08 · RoboTwin 2.0의 양팔 과제

RoboTwin 2.0은 양팔 조작 과제와 대규모 데이터 생성, 조명·배경·물체 배치 같은 도메인 무작위화를 제공한다. 한 팔이 물체를 고정하고 다른 팔이 조작하는 협응은 단일 팔보다 상태·행동 의존이 크다. 시뮬레이션에서의 높은 성공률이 현실 물체와 센서에서도 유지되는지 별도 검증해야 한다. RoboTwin 2.0 논문

양팔 조작에서 왼팔이 상자를 고정하고 오른팔이 뚜껑을 여는 동안 왼팔의 미세한 움직임은 오른팔이 느끼는 접촉을 바꾼다. 한 팔의 행동을 따로 평가하면 성공을 설명하기 어렵다. 도메인 무작위화는 색·조명만 바꾸는 것이 아니라 물체 배치와 물리 조건의 변화까지 어떤 범위로 넣었는지 확인해야 한다.

## 9.9 슬라이드 09 · SIMPLER: 현실 정책을 시뮬레이션에서 평가

현실 로봇 정책을 매번 실제 하드웨어에서 시험하는 것은 비싸고 반복하기 어렵다. SIMPLER는 현실 과제와 대응하는 시뮬레이션을 만들어 정책 간 성능의 **상대 순위**가 현실과 얼마나 맞는지 분석한다. 시뮬레이터가 실제 성공률의 절댓값을 완벽하게 예측하지 않아도 정책 선택에 유용할 수 있다. 그러나 카메라 배치나 접촉 물리가 바뀌면 순위도 뒤집힐 수 있어 real-to-sim 보정을 확인해야 한다. SIMPLER 논문

정책 A의 현실 성공률이 70%, B가 50%인데 시뮬레이션에서는 60%, 40%라고 하자. 절댓값은 각각 10%p 낮아도 순위는 맞으므로 정책 선택에는 도움이 될 수 있다. 반면 시뮬레이션의 작은 물리 오차 때문에 A와 B의 순위가 뒤집힌다면 높은 상관을 주장하기 어렵다. **절댓값 예측**과 **상대 순위 보존**은 다른 평가다.

## 9.10 슬라이드 10 · RoboArena의 분산형 실기 평가

RoboArena는 여러 기관의 실제 로봇에서 일반 조작 정책을 평가하고 선호 비교를 모은다. 다양한 실험실과 물체를 포함하면 한 연구실 장비에 맞춘 정책의 한계를 드러낼 수 있다. 반면 장비와 과제의 이질성이 커져 정책 간 비교 설계를 세심하게 해야 한다. 단순 rollout 수뿐 아니라 과제 난도, 실패 유형, 평가자의 선호 기준을 함께 기록한다. RoboArena 논문

실험실마다 카메라 위치와 로봇 보정이 다르면 같은 정책도 다른 관측을 받는다. 여러 기관 평가에서는 평균 성공률만이 아니라 기관별 성능과 실패 유형이 중요하다. 한 기관의 좋은 결과가 전체 평균을 끌어올렸는지, 대부분의 장소에서 비슷하게 성공했는지를 구분해야 **현실 일반화**를 논할 수 있다.

## 9.11 슬라이드 11 · FurnitureBench의 재현 가능한 실물 조립

FurnitureBench는 구하기 쉬운 가구 부품과 3D 프린트 구성품, 표준화된 로봇 배치로 실제 조립 실험의 재현성을 높이려 한다. 조립은 물체를 집어 옮기는 짧은 동작이 아니라 여러 부품의 순서와 접촉을 유지해야 하는 **긴 지평** 과제다. 초기 몇 단계의 성공만으로 전체 조립 성공을 대신할 수 없다. 서울대 연구진이 참여한 사례라는 점도 강의 주제와 연결된다. FurnitureBench 논문

가구 조립은 부품 A를 맞춘 뒤 B를 끼우고 나사를 조이는 등 순서가 있다. 앞 단계의 작은 위치 오차가 뒤 단계의 삽입을 막을 수 있어, 부품별 성공률이 높아도 전체 조립 성공률은 낮아진다. 서울대가 참여한 FurnitureBench 사례를 읽을 때는 **표준화된 물리 세팅**과 단계별 성공 판정을 함께 보자.

### 9.12 슬라이드 12 · HumanoidBench의 전신 제어

HumanoidBench는 팔·손·다리·몸통을 함께 쓰는 시뮬레이션 과제를 제공한다. 걷는 중 문을 열거나 물체를 조작하려면 균형과 손끝 목표가 결합되어야 한다. 단순 이동거리 보상만으로는 실제 목표를 충분히 측정하지 못한다. 신체 관절 자유도, 관측 정보, 안전 실패 기준을 함께 보고 알고리즘을 비교해야 한다. HumanoidBench 논문

문을 열며 걷는 과제에서는 팔이 손잡이를 당길 때 몸의 균형이 바뀐다. 단순 보행 정책과 단순 팔 조작 정책을 이어 붙여도 두 동작의 상호작용이 해결되지 않을 수 있다. 과제별 반환과 성공률에 더해 넘어짐, 충돌, 관절 한계 위반 같은 실패 지표를 함께 보고 전신 제어의 질을 판단한다.

### 9.13 슬라이드 13 · 과제 그림에서 평가 설계로

역도, 계단, 장애물, 주방 조작처럼 장면이 달라지면 같은 humanoid라도 필요한 지각과 제어가 달라진다. 각 과제 성공률을 평균 하나로 압축하면 특정 어려운 과제의 실패가 가려질 수 있다. 과제별 반환·성공률·실패 유형과 전체 평균을 함께 제시하고, 처음 본 지형이나 물체에서 일반화가 되는지도 살펴야 한다. HumanoidBench 프로젝트

예를 들어 네 과제 성공률이 100%, 100%, 100%, 0%이면 평균은 75%지만 마지막 과제에는 전혀 대응하지 못한다. 평균과 함께 최저 성공률이나 과제별 표를 제시해야 모델의 활용 범위가 드러난다. 특히 새로운 물체와 새로운 지형은 서로 다른 일반화 축이므로 한 수치로 합치지 말자.

# 10 06 오프라인 강화학습

## 10.1 슬라이드 01 · 데이터가 고정되면 달라지는 문제

지금까지의 off-policy 알고리즘은 과거 경험을 replay하면서도 새 정책으로 환경을 계속 방문했다. Offline RL에서는 처음 받은 데이터셋만으로 정책을 학습하고 학습 중 새 전이를 얻지 못한다. 따라서 "모르는 행동을 한 번 실행해 확인"하는 길이 막힌다. 목표는 고정 데이터가 허용하는 근거 안에서 행동을 개선하는 것이다.

병원 기록에서 어떤 처치를 시도하지 않은 환자군이 있다면 그 처치의 결과를 기록만으로 직접 알 수 없다. 로봇에서도 위험해 보여 한 번도 실행하지 않은 토크 명령의 가치를 critic이 높게 예측할 수 있다. Offline RL의 핵심은 좋은 행동을 찾는 일과 **자료가 말할 수 있는 범위**를 동시에 다루는 것이다.

## 10.2 슬라이드 02 · 이 장의 방법 지도

TD3+BC는 학습 정책을 데이터 행동 근처에 묶는다. CQL은 데이터 밖 행동의 Q값이 부당하게 높아지지 않도록 critic을 보수화한다. Filtered BC는 좋은 궤적만 모방하고, AWR은 행동의 상대적 가치를 가중치로 써 모방한다. IQL은 데이터에 없는 행동의 Q를 직접 평가하지 않고 가치와 정책을 배운다. 방법마다 **데이터 지지 밖을 다루는 위치**가 다르다.

방법들의 위치를 기억하려면 "어디에서 보수성이 들어가는가"를 보면 된다. TD3+BC는 actor의 행동을 시연 근처로 묶고, CQL은 critic의 데이터 밖 Q를 낮춘다. AWR과 IQL은 데이터 안 행동에 높은 가중치를 주어 정책을 뽑는다. 같은 고정 자료를 쓰더라도 오류를 억제하는 연산의 위치가 다르다.

## 10.3 슬라이드 03 · Offline과 online의 자료 흐름

Online RL은 자료를 수집하고 갱신한 뒤 바뀐 정책으로 다시 자료를 모은다. Offline RL은 $\mathcal{D}$가 고정되어 학습 중 행동해도 그 결과가 추가되지 않는다. 학습된 정책을 최종 평가할 때 환경에서 실행할 수는 있지만, 그 평가 자료로 다시 학습했다면 순수 offline 설정이 아니다. 실험 보고서에서 이 경계를 명확히 해야 한다.

온라인 SAC의 replay buffer가 현재 10만 전이를 담고 있어도, 다음 rollout으로 새 actor의 결과를 확인해 넣을 수 있다. Offline SAC에는 그 통로가 없다. 같은 저장 형식과 같은 손실 함수를 써도 **데이터 수집이 멈췄는가**라는 조건 하나가 학습의 위험을 바꾼다.

## 10.4 슬라이드 04 · 행동 정책과 목표 정책

$\mathcal{D} = \{(s, a, r, s', d)\}$의 행동은 알 수 없거나 혼합된 행동 정책 $\beta$에서 나왔다. 학습 목표는 새 정책 $\pi_\theta$의 $J(\pi) = E_{\tau\sim\pi}[\sum_t \gamma^t r_t]$를 높이는 것이다. 자료의 상태–행동 분포 $d^\beta(s,a)$와 새 정책의 $d^\pi(s,a)$가 다르면, $\mathcal{D}$에서 작은 예측 오차도 실제 정책 아래에서는 크게 작용할 수 있다.

"off-policy"라는 말은 수집 정책과 학습 정책이 다르다는 뜻이다. Online off-policy는 계속 새 자료를 넣을 수 있지만 offline은 그 보정 기회가 없다.

예를 들어 기록 정책 $\beta$가 어떤 상태에서 A를 99% 실행하고 B를 1%만 실행했는데 새 정책 $\pi$가 B를 90% 실행한다면, B의 가치 오차가 실제 정책 성과를 좌우한다. 고정 데이터에는 B 표본이 아주 적다. 단순히 전체 회귀 손실이 작다고 새 정책에서도 Q가 정확하다고 볼 수 없는 이유다.

## 10.5 슬라이드 05 · BC와 궤적 연결

BC는 데이터의 각 상태에서 관측된 행동을 흉내 낸다. 예를 들어 데이터에 A→B와 B→C를 성공한 부분 경로가 따로 있으면 가치 기반 offline RL은 A에서 B로 간 뒤 B에서 C로 가는 새 조합을 찾아

A→C 과제를 해결할 수 있다. 이 **stitching**은 각 부분의 전이가 데이터에 충분히 존재하고, 상태 표현이 B를 같은 상태로 인식할 때 가능하다.

나쁜 시도도 그 행동의 결과와 보상을 알려 주므로 RL에는 정보가 된다. 다만 데이터에 전혀 없는 A→C 지름길의 가치를 증명해 주지는 않는다.

부분 경로 A→B와 B→C가 각각 데이터에 충분하다면, 두 경로를 이어 A→C를 만드는 정책은 BC가 직접 복제한 한 궤적보다 좋아질 수 있다. 반대로 A→B가 한 조명 조건에서, B→C가 전혀 다른 숨은 조건에서 나온 것이라면 겉보기 상태 B가 같아도 이어 붙일 수 없다. Stitching에는 상태 표현과 전이 조건의 일관성이 필요하다.

### 10.6 슬라이드 06 · 보지 못한 행동의 함정

데이터에서 상태 $s'$의 행동이 $a_1$ 근처뿐인데 critic이 먼 $a_2$에 높은 $Q(s', a_2)$를 예측했다고 하자. 학습 손실은 데이터의 $(s', a_2)$를 검사하지 못하므로 그 큰 예측을 교정할 근거가 없다. 그런데 actor나 $\max_a Q$가 바로 $a_2$를 선택해 높은 값을 이전 상태로 전파한다.

이것은 행동의 **out-of-distribution(OOD)** 예측과 부트스트랩이 결합한 오류다. Online 방법은 $a_2$를 실행해 확인할 수 있지만 offline은 보수적 제약이나 데이터 안 행동만의 개선이 필요하다. Offline RL 개관

두 행동의 실제 Q가 A=5, B=0인데 데이터에는 A만 있다고 하자. 신경망이 B를 12로 예측해도 B의 표본이 없으면 그 예측을 벌할 supervised target이 없다. Actor가 B를 선택하면 다음 상태의 Bellman 목표에 12가 들어가 이전 상태의 Q까지 높아질 수 있다. **추정 오류의 선택과 전파**가 함께 일어난다.

### 10.7 슬라이드 07 · 언제 필요한가

로봇의 충돌, 임상 처치의 위험, 자율주행의 사고처럼 무작위 탐색이 비싸거나 안전하지 않을 때 이미 모인 기록이 유용하다. 과거 실험이나 다른 프로젝트의 자료를 다시 활용할 수도 있다. 하지만 실제 의료·운전 의사결정은 기록 누락, 선택 편향, 안전 제약이 복잡하므로 offline RL 모델 하나의 평가 점수만으로 배포할 수 없다. 이 장의 알고리즘은 **고정 자료에서 학습하는 기술적 문제**를 다룬다.

의료 기록에서 투약을 바꿔 볼 수 없다는 예는 학습 설정의 필요성을 보여 주지만, 실제 진료에서는 기록되지 않은 중증도와 담당 의사의 판단이 행동 선택에 영향을 줄 수 있다. 이런 숨은 교란이 있으면 고정 데이터의 인과적 해석도 어려워진다. Offline RL의 벤치마크 성능을 곧바로 의료 의사결정의 안전성으로 옮기면 안 되는 이유다.

### 10.8 슬라이드 08 · 보통 off-policy Q-learning을 그대로 쓰면

일반 목표 $y = r + \gamma \max_{a'} Q(s', a')$는 데이터 밖 $a'$도 최댓값 후보로 삼는다. Critic은 그 행동에서 틀릴 수 있고, 최대화는 특히 양의 오류를 택한다. 작은 과대평가가 Bellman 갱신마다 이전 상태로 퍼지면서 정책이 실제로는 형편없는 OOD 행동을 선호한다. 정적 데이터에서 가장 중요한 차이는 **새 행동을 시도해 오류를 바로잡을 수 없다는 것**이다.

보통 Q-learning의 $\max_{a'}$는 모든 행동을 후보로 두지만 데이터는 일부 행동만 보여 준다. 데이터 밖 행동의 Q가 우연히 커지면 최대값이 그 오류를 고르고, Bellman 회귀가 그 숫자를 현재 행동의 목표로 사용한다. 한 번의 잘못된 외삽이 여러 이전 상태로 퍼지는 과정을 데이터 흐름으로 그려 볼 수 있다.

### 10.9 슬라이드 09 · 예측 Q와 실제 반환의 분리

정적 데이터로 SAC를 학습할 때 critic의 예측 Q가 계속 커지는데 실제 평가 반환은 개선되지 않거나 떨어지는 그래프는 이 오류를 보여 준다. 높은 학습 Q는 정책 성능의 독립 증거가 아니다. Critic

이 자기 목표를 따라 숫자를 키웠을 수 있다. 평가 환경이 허용되는 연구에서는 rollout 반환을 따로 측정하고, 배포 전에는 데이터 coverage와 불확실성을 검토해야 한다. BEAR 연구

Critic이 예측한 평균 Q가 50에서 100으로 올라도 독립 평가 반환이 20 근처에 머무른다면, 모델 내부 수치의 상승은 성능 증거가 아니다. 벤치마크 시뮬레이터에서는 학습 완료 뒤 정책을 실행해 확인할 수 있지만 현실에서는 이런 시험 자체가 위험할 수 있다. **학습 목표의 값**과 **실제 정책 결과**를 분리해 보고해야 한다.

### 10.10 슬라이드 10 · Offline RL의 핵심 긴장

정책을 데이터 행동과 똑같이 만들면 안전하게 BC 수준에 머물 수 있지만 개선 여지가 작다. 데이터를 벗어날수록 좋은 행동을 찾을 가능성도 생기지만 Q 오차를 악용할 위험이 커진다. 핵심은 **정책 개선과 분포 이동의 균형**이다. 상태까지 새 영역으로 가면 한 단계 행동 제약만으로 위험을 모두 막을 수도 없다.

데이터와 똑같은 행동만 하면 BC의 성능에 가까워지고, 멀리 이동할수록 개선의 가능성과 불확실성이 함께 커진다. 데이터 행동 주변이라도 상태가 바뀌어 자료 밖으로 들어갈 수 있으므로 한 단계 행동 거리만으로 모든 위험을 막을 수는 없다. 정책의 **행동 이동**과 **방문 상태 이동**을 함께 생각해야 한다.

### 10.11 슬라이드 11 · 두 가지 주요 통제 위치

첫째, 정책 자체가 데이터 행동에서 멀어지지 않게 제약한다. 둘째, critic이 데이터 밖 행동에 지나치게 큰 값을 주지 않도록 훈련한다. 세 번째 길은 아예 데이터 안 행동의 반환이나 상대적 가치를 기반으로 정책을 추출하는 것이다. 이후 방법들을 "어떤 값을 낮추는가", "어떤 행동에만 높은 가중치를 주는가"로 비교하면 혼동이 줄어든다.

Actor 제약은 정책이 낼 수 있는 행동의 범위를 직접 줄이고, critic 보수화는 정책이 높은 Q를 보고 이동할 유인을 줄인다. 두 접근 모두 개선을 억제할 위험이 있으므로 제약의 강도를 조정해야 한다. 데이터 품질이 낮을 때에는 단순 BC와의 비교가 특히 중요하다.

### 10.12 슬라이드 12 · TD3+BC의 목적

고정 데이터셋 $\mathcal{D}$에는 상태–행동 쌍 $(s, a)$가 있다. $\pi_\theta(s)$는 파라미터 $\theta$인 actor의 행동, $Q_\phi(s, a)$는 파라미터 $\phi$인 critic의 가치 예측이다. TD3+BC는 데이터에서 기대한 Q를 높이되 actor 행동과 기록 행동 사이의 유클리드 제곱거리 $\|\cdot\|_2^2$를 벌점으로 준다. 두 항의 상대 비중은 $\lambda > 0$가 정한다.

$$\max_\theta E_{(s,a)\sim\mathcal{D}} \left[\lambda Q_\phi(s, \pi_\theta(s)) - \|\pi_\theta(s) - a\|_2^2\right].$$

$\lambda$가 크면 가치 개선을 더 중시하고 작으면 BC에 가까워진다. 논문의 실제 구현은 Q 크기를 정규화해 두 항의 척도를 맞춘다. 데이터 행동이 무작위적이고 매우 나쁘다면 지나친 제약이 개선을 막지만, 제약이 약하면 OOD Q 오류가 다시 나타난다. TD3+BC 논문

한 상태의 기록 행동이 $a = 0.2$이고 actor 출력이 0.8이라면 제곱거리 벌점은 $(0.8 - 0.2)^2 = 0.36$이다. Actor가 0.3으로 움직이면 벌점은 0.01로 줄지만 Q 개선이 작을 수 있다. $\lambda$는 Q의 숫자 단위에 민감하므로, 가치와 행동 거리 항의 크기를 맞추지 않으면 어느 한쪽만 손실을 지배한다.

두 항이 행동을 어느 방향으로 미는지도 직접 미분할 수 있다. 한 상태에서 $u = \pi_\theta(s)$라 놓으면 최대화할 함수와 그 행동 기울기는

$$F(u) = \lambda Q_\phi(s, u) - \|u - a_\mathcal{D}\|_2^2, \qquad \nabla_u F = \lambda \nabla_u Q_\phi(s, u) - 2(u - a_\mathcal{D}).$$

첫 항은 critic이 예측한 가치 상승 방향이고, 둘째 항은 actor를 기록 행동으로 당긴다. $Q$가 국소적으로 $Q(s, u) = cu + b$인 1차원 예에서는 정지점이 $u^* = a_\mathcal{D} + \lambda c/2$다. 따라서 $\lambda$가 같아도 critic의

기울기 $c$가 크면 데이터에서 더 멀리 이동한다. 실제 행동 경계와 비선형 Q에서는 이 정지점이 그대로 최적해가 아니며, actor 파라미터에 대한 기울기는 위 식에 $\nabla_\theta \pi_\theta(s)$를 곱한 연쇄법칙으로 얻는다.

### 10.13 슬라이드 13 · 정책 제약과 가치 보수화

정책을 데이터에 묶는 방법은 이해하기 쉽지만 모든 상태에서 적절한 거리를 정하기 어렵다. 낮은 품질 데이터와 높은 품질 데이터가 섞였다면 단일 BC 벌점은 좋은 행동으로의 이동도 제한할 수 있다. CQL은 반대로 **Q함수의 학습 목표**를 바꿔 데이터 밖 행동의 매력을 낮춘다.

가치 보수화도 무조건 낮은 값을 만들면 좋은 개선 가능성을 지워 버린다. 목표는 데이터 안 행동과 데이터 밖 행동 사이의 상대적 값에 신뢰 가능한 간격을 주는 것이다.

행동 거리의 의미는 과업마다 다르다. 관절 토크 벡터에서 0.1 차이가 작은 변화일 수 있지만 접촉 직전 그리퍼 닫힘 명령의 0.1 차이는 성공과 실패를 가를 수 있다. 단일 L2 벌점은 이런 중요도를 자동으로 알지 못한다. 정책 제약을 쓸 때 행동 정규화와 차원별 단위를 먼저 확인해야 한다.

### 10.14 슬라이드 14 · 데이터 밖 Q를 낮추는 이유

일반 Bellman 오차는 데이터 안 $(s, a)$에서만 계산된다. 그래서 데이터 밖 행동의 Q에는 직접적 벌점이 없다. CQL은 데이터 밖 후보의 높은 Q를 비용으로 만들고 데이터 안 행동의 Q를 상대적으로 유지한다. 그러면 actor가 $Q$의 허점을 찾아 멀리 이동하는 유인이 줄어든다. 단, 정말 좋은 미관측 행동까지 낮출 수 있는 보수성의 비용이 있다.

데이터에는 A 행동의 반환만 있고 B는 보지 못했다고 하자. Bellman 손실은 A의 Q를 맞추지만 B의 Q가 100이어도 직접 교정하지 못한다. CQL은 그런 후보 행동의 큰 Q에 비용을 주어 actor가 B를 고를 유인을 낮춘다. 단, B가 실제로 좋은 행동이었다면 보수성 때문에 개선 가능성을 놓칠 수 있다.

### 10.15 슬라이드 15 · CQL의 첫 목적식

먼저 $\mathcal{L}_{\text{Bellman}}(Q)$를 기록 전이에 맞춘 critic의 Bellman 회귀 손실이라 하자. $\mathcal{D}$에서 상태 $s$를 뽑고 제안분포 $\mu(a \mid s)$에서 후보 행동 $a$를 뽑는다. 후보의 높은 Q에 계수 $\alpha > 0$의 벌점을 더하면

$$\min_{Q} \; \mathcal{L}_{\text{Bellman}}(Q) + \alpha E_{s\sim\mathcal{D},\, a\sim\mu(\cdot|s)} Q(s, a)$$

로 쓸 수 있다. 최소화하므로 두 번째 항은 $\mu$가 자주 제안하는 행동의 Q를 내린다. 그러나 모든 행동을 함께 내리면 데이터 행동의 가치까지 과하게 낮아질 수 있다. 다음 단계에서 데이터 행동의 Q를 끌어올리는 상대적 항을 넣는다. CQL 논문

후보 분포 $\mu$가 데이터 밖 행동 B를 많이 뽑는다면 $E_{a\sim\mu} Q(s, a)$ 벌점은 B의 값을 낮춘다. 하지만 $\mu$가 데이터 행동만 뽑거나 중요한 OOD 행동을 놓치면 원하는 효과가 약해진다. 그래서 CQL의 구현을 볼 때는 정규화 계수뿐 아니라 **후보 행동을 어떻게 샘플했는가**를 함께 확인해야 한다.

### 10.16 슬라이드 16 · 데이터 행동과의 상대적 보수성

여기서 $E_{s,a\sim\mu}$는 실제로 $s \sim \mathcal{D}$, $a \sim \mu(\cdot \mid s)$를 차례로 뽑은 기댓값의 약기다. $E_{(s,a)\sim\mathcal{D}}$는 기록된 상태–행동 쌍의 기댓값이다. 두 분포의 Q를 비교하는 CQL 정규화 항은

$$\alpha \left( E_{s,a\sim\mu} Q(s, a) - E_{(s,a)\sim\mathcal{D}} Q(s, a) \right)$$

이다. 최소화하면 후보 행동의 Q는 내리고 **실제 데이터 행동**의 Q는 상대적으로 올린다. "모든 $(s, a)$에서 학습 Q가 참 Q보다 낮다"는 강한 보장으로 읽으면 안 된다. 논문이 분석하는 보수성은 특정

정책과 데이터 분포 아래의 기대 가치에 관한 조건부 주장이다. $\alpha$가 너무 크면 실제로 좋은 행동까지 억누를 수 있다.

예를 들어 데이터 행동 Q가 5, 데이터 밖 후보 Q가 8이면 두 평균의 차이 3이 비용으로 들어간다. critic은 이 격차를 줄이도록 갱신된다. 논문의 보수성 분석은 자료와 정책 분포에 관한 가정 아래 기대 가치에 적용되며, 실제 평가 반환도 별도로 확인해야 한다.

### 10.17 슬라이드 17 · Log-sum-exp가 나오는 이유

고정 상태 $s$에서 이산 행동의 제안분포를 $\mu_a = \mu(a \mid s)$라 쓰고 $Q_a = Q(s,a)$라 하자. $H(\mu) = -\sum_a \mu_a \log \mu_a$는 엔트로피다. $\sum_a \mu_a = 1$인 모든 분포 중 $E_{a\sim\mu} Q_a + H(\mu)$를 최대화하면

$$\max_{\mu} \{E_\mu Q + H(\mu)\} = \log \sum_a e^{Q(s,a)}.$$

Log-sum-exp는 큰 Q를 부드럽게 강조하는 max의 근사다. 연속 행동에서는 합을 전부 계산할 수 없어 정책·균등분포 등에서 행동을 샘플링해 근사한다. 후보 샘플의 분포와 밀도 보정을 무시하면 실제 구현의 목적이 달라진다.

두 행동 Q가 2와 0이면 $\log(e^2 + e^0) \approx 2.13$이다. 데이터 행동의 Q가 2이면 차이 항은 약 0.13, Q가 0이면 약 2.13이다. 같은 후보 집합에서도 기록 행동의 가치에 따라 벌점이 달라진다.

중간 유도를 쓰면 $F(\mu) = \sum_a \mu_a Q_a - \sum_a \mu_a \log \mu_a$를 $\sum_a \mu_a = 1$ 아래 최대화한다. 라그랑주 승수 $c$를 넣어 $\mu_a$로 미분하면 $Q_a - \log \mu_a - 1 + c = 0$, 따라서 $\mu_a \propto e^{Q_a}$다. 정규화하면 $\mu_a = e^{Q_a} / \sum_b e^{Q_b}$이고 이를 $F$에 대입한 값이 $\log \sum_b e^{Q_b}$다. 이 값의 $Q_a$ 미분도 $\mu_a$이므로 **log-sum-exp 항에서는 큰 Q를 가진 행동이 더 큰 하향 압력**을 받는다. 실제 CQL 정규화에서는 데이터 행동 Q를 올리는 상대적 항도 함께 작용한다. CQL 원논문

### 10.18 슬라이드 18 · CQL 학습과 조절 계수

이산 행동에서 $\mathcal{D}(a \mid s)$를 기록 행동의 조건부분포, $\alpha > 0$를 보수화 강도로 두면 Bellman 회귀에 다음 정규화 항을 더한다.

$$\mathcal{L}_{\text{CQL}} = \mathcal{L}_{\text{Bellman}} + \alpha E_{s\sim\mathcal{D}} \left[ \log \sum_a e^{Q(s,a)} - E_{a\sim\mathcal{D}(\cdot|s)} Q(s,a) \right].$$

연속 행동 SAC와 결합할 때는 적절한 행동 샘플링으로 log-sum-exp를 근사한다. $\alpha$가 크면 분포 밖 낙관을 줄이지만 정책이 너무 보수적으로 남을 수 있다. 값의 척도와 샘플 수를 함께 조절해야 한다. CQL 논문

연속 행동 공간 $\mathcal{A}$에서는 이산 합에 대응하는 $I(s) = \log \int_{\mathcal{A}} \exp\{Q(s,a)\}\, da$를 계산해야 한다. 제안 밀도 $q(a \mid s) > 0$에서 독립적인 후보 $a_1, \ldots, a_N$을 뽑는다고 하자. $q$를 곱하고 나누면 적분 안쪽에 중요도 샘플링 항등식을 적용할 수 있다.

$$\int_{\mathcal{A}} e^{Q(s,a)}\, da = E_{a\sim q} \left[ \frac{e^{Q(s,a)}}{q(a \mid s)} \right] \approx \frac{1}{N} \sum_{j=1}^{N} \frac{e^{Q(s,a_j)}}{q(a_j \mid s)}.$$

마지막 합에 로그를 취해 $I(s)$를 근사한다. 특정 행동을 자주 뽑는 제안분포 $q$를 사용하면서 $1/q$를 빼면 그 행동의 높은 Q를 과도하게 반영한다. 반대로 $q$가 중요한 영역에 거의 질량을 주지 않으면 몇 개 표본의 가중치가 커져 근사가 불안정하다. 유한 표본에서 **로그를 취한 추정량 자체는 편향**될 수 있으며, 구현의 행동 샘플링·밀도 보정·수치 안정화가 결과에 영향을 준다.

$\alpha$가 너무 작으면 OOD Q 오류를 충분히 누르지 못하고, 너무 크면 실제 개선 가능한 행동도 낮게 평가할 수 있다. Bellman 손실과 CQL 정규화 항의 값을 따로 기록하면 어느 항이 학습을 지배하는지 볼 수 있다. 학습 Q가 낮아졌다는 사실 자체가 정책 성능이 좋아졌다는 뜻은 아니다.

### 10.19 슬라이드 19 · Offline RL의 평가는 왜 까다로운가

학습 중 새 환경 상호작용을 금지해도 연구 벤치마크의 최종 평가는 시뮬레이터에서 실행할 수 있다. 실제 위험 환경에서는 그렇게 쉽게 평가할 수 없고, logged data만으로 다른 정책의 성능을 추정하는 off-policy evaluation이 필요하다. 이 추정 역시 데이터의 행동 coverage와 수집 정책 확률에 민감하다. 벤치마크의 온라인 평가 가능성과 실제 배포 전 검증 가능성을 구분한다.

학습 중 환경 호출을 금지한 실험이라도 최종 시뮬레이터 평가가 허용된다면 정책 성과를 직접 측정할 수 있다. 현실 로그만 있는 상황에서는 새 정책을 실행하지 않고 평가해야 하므로, 데이터에 거의 없는 행동에 대한 반사실적 결과를 추정해야 한다. Offline RL의 **학습 문제**와 off-policy evaluation의 **검증 문제**를 분리해 보자.

한 단계짜리 예에서 수집 정책 $\beta$가 행동 A를 0.9, B를 0.1 확률로 골랐고 새 정책 $\pi$는 B를 0.9 확률로 고른다고 하자. B의 보상 표본은 드물지만 중요도 비율 $\pi(B)/\beta(B) = 9$로 크게 가중된다. 평균을 맞추는 원리는 분명해도 B 표본 하나의 우연한 보상이나 측정 오류가 평가값을 크게 흔들 수 있다. $\beta(B) = 0$이었다면 그 행동의 실제 결과를 로그만으로 식별할 수 없다. 그래서 offline 정책의 추정 성능에는 **coverage와 불확실성**을 함께 제시해야 한다.

### 10.20 슬라이드 20 · D4RL 데이터셋

D4RL은 고정된 MuJoCo 및 다른 환경 자료를 제공해 offline 알고리즘을 같은 데이터로 비교하게 한다. random, medium, medium-replay, medium-expert 등은 수집 행동의 품질과 혼합 방식이 다르다. Expert 자료가 많을수록 BC가 강한 기준선이 되지만, 다양한 품질의 replay에는 RL의 재가중·stitching 여지가 커질 수 있다.

점수는 보통 환경별 reference score로 정규화된다. 그래서 서로 다른 환경의 raw return이나 버전이 다른 정규화 점수를 단순 합산하면 안 된다. D4RL 논문

같은 Ant 과제에서도 expert 데이터는 높은 반환 행동이 많지만 상태 범위가 좁을 수 있고, medium-replay 데이터는 학습 중 여러 수준의 행동을 포함해 stitching 가능성이 있다. 어느 쪽이 무조건 쉽다고 볼 수 없다. 알고리즘 점수를 읽을 때 데이터셋 이름 뒤에 숨은 수집 정책의 품질과 coverage를 확인해야 한다.

### 10.21 슬라이드 21 · 데이터의 질과 다양성

자료가 한 정책의 거의 동일한 성공 궤적만 담으면 모방하기는 쉬워도 대체 경로의 가치를 평가하기 어렵다. 반면 random 자료는 다양한 상태를 방문하지만 좋은 행동의 연속이 드물다. Offline RL의 실험은 데이터 크기만이 아니라 성공 비율, 행동 범위, 궤적 길이, 행동 정책의 혼합을 함께 보고해야 한다. 데이터셋의 장면 그림을 볼 때도 "얼마나 많은 transition인가"와 "어떤 영역을 덮는가"를 따로 묻는다.

성공 궤적 100개가 모두 같은 초기 위치에서 나왔고 실패 궤적 1000개가 다양한 위치에서 나왔다면, 성공 궤적만 남기는 정책은 시작 분포가 바뀔 때 약할 수 있다. 데이터 품질과 다양성은 하나의 축이 아니다. 성공률, 시작 상태의 범위, 행동 종류를 함께 보고 학습 방법을 고르자.

### 10.22 슬라이드 22 · OGBench와 목표 조건부 학습

OGBench는 주어진 목표 $g$를 달성하는 $\pi(a \mid s, g)$를 offline 데이터에서 학습하는 과제를 제공한다. 고정 데이터의 부분 경로를 연결해 보지 못한 시작–목표 조합을 해결하는 능력을 시험한다. 한 목표에

서 높은 반환을 내는 정책과 다양한 목표를 처리하는 정책의 평가는 다르다. 데이터의 goal coverage, 성공 판정과 평가 목표 분포를 함께 확인한다. OGBench 논문

목표가 B인 궤적과 목표가 C인 궤적이 따로 있을 때, 같은 중간 상태를 지나면 부분 경로를 이어 새로운 시작–목표 조합을 해결할 수 있다. 그러나 상태가 비슷해 보인다는 이유만으로 연결하면 숨겨진 물체 위치나 속도의 차이 때문에 실패할 수 있다. 목표 조건부 정책의 성과를 볼 때 **보지 못한 목표 조합**과 **새로운 환경 상태**를 따로 평가해야 한다.

### 10.23 슬라이드 23 · TD3+BC와 CQL을 비교하기

TD3+BC는 actor 손실에서 데이터 행동과의 거리 벌점을 두므로 "멀리 가지 않기"가 직접적이다. CQL은 critic이 데이터 밖 행동을 높게 평가하지 못하게 해 actor가 그 행동을 고르기 어렵게 만든다. 전자는 행동 거리 척도에, 후자는 $\alpha$와 후보 행동 샘플링에 민감하다. 두 방법 모두 데이터 밖에 정말 좋은 행동이 있어도 지나치게 보수적이면 그 가능성을 놓친다.

행동이 연속 벡터라면 TD3+BC의 거리는 관절별 단위와 정규화에 민감하다. CQL은 후보 행동의 샘플링 분포와 $\alpha$에 민감하다. 두 방법이 같은 벤치마크에서 비슷한 점수를 얻어도 서로 다른 오류를 억제한 결과일 수 있다. 데이터 범위를 바꾸어 비교하면 어느 제약이 유리한지 더 잘 드러난다.

### 10.24 슬라이드 24 · 다른 개선 경로의 질문

정책 제약과 보수적 critic도 학습 중에는 데이터 밖 행동의 Q를 어느 정도 묻는다. 만약 **오직 기록된 행동만** 가치 평가에 넣으면서 좋은 행동을 더 선호할 수 있다면 OOD 문제를 더 직접적으로 피할 수 있다. 이어지는 Filtered BC, AWR, IQL은 이 방향의 서로 다른 해법이다.

"데이터 안 행동만 쓴다"는 전략은 안전하지만 자료에 없는 새로운 행동을 직접 평가하거나 창조하지 못한다. 대신 여러 품질의 기록 행동 중 좋은 쪽을 더 자주 선택하게 만들 수 있다. 이 접근이 통하려면 좋은 행동이 **적어도 데이터 어딘가에 존재**해야 한다는 전제가 필요하다.

### 10.25 슬라이드 25 · OOD 평가가 남는 자리

TD3+BC의 actor 항 $Q(s, \pi(s))$에서 $\pi(s)$가 기록된 $a$와 완전히 같지 않으면 critic은 미관측 행동을 평가한다. CQL 역시 정규화 항과 정책 갱신에서 후보 행동의 Q를 계산한다. 벌점이 오류를 줄일 수는 있어도 그런 계산 자체가 사라지는 것은 아니다. 그래서 데이터 안 행동의 정보를 직접 이용하는 방법이 유용하다.

정책이 데이터 행동에서 0.01만큼 벗어나도 critic이 그 점을 본 적이 없다면 일반화에 의존한다. 행동 제약이나 CQL 벌점은 위험을 줄이지만 그 평가 자체를 없애지는 않는다. AWR처럼 기록 행동의 로그확률만 학습하는 방법은 actor 업데이트에서 이 질문을 피하는 대신 데이터 support 밖 개선의 폭이 작다.

### 10.26 슬라이드 26 · 보상을 쓰는 행동복제

일반 BC는 성공한 시도와 실패한 시도의 $(s, a)$를 모두 같은 가중치로 모방한다. 데이터에 보상이 있다면 어떤 시도가 좋았는지 알 수 있으므로 이를 정책 추출에 반영할 수 있다. 가장 단순한 방법은 성공 궤적을 골라 BC하는 것이다. 단, "성공"을 전체 반환 하나로 판단하면 같은 궤적 안의 나쁜 개별 행동까지 함께 모방할 수 있다.

상태 $s$에서 두 시연 행동이 있고 하나는 성공 1, 다른 하나는 실패 0이라면 BC는 둘을 비슷하게 따라 할 수 있다. 보상으로 성공한 궤적에 더 큰 가중치를 주면 행동 선택을 개선할 가능성이 생긴다. 그러나 실패한 궤적의 어떤 단계가 실제 원인인지 구분하지 못하면 좋은 중간 행동까지 버릴 수 있다.

### 10.27 슬라이드 27 · Filtered BC

각 궤적의 반환 $R(\tau) = \sum_t \gamma^t r_t$를 계산하고 상위 $K\%$ 또는 임계값 $\eta$ 이상인 궤적만 남긴다. 이어 $\max_\theta E_{(s,a)\sim\widetilde{\mathcal{D}}} \log \pi_\theta(a \mid s)$로 모방한다. 반환이 큰 궤적이 반드시 모든 상태에서 최선의 행동으로 구성되지는 않는다. 쉬운 초기 상태를 받은 궤적이 높게 평가되고 어려운 상태에서 잘 대처한 궤적이 버려질 수도 있다.

**성공 궤적만 남기면 오히려 나빠질 수도 있나?** 가능하다. 남은 자료가 매우 작아지거나 특정 초기 상태만 대표하면 배포 때 다른 상태를 다루지 못한다. 상태 난도와 운의 영향까지 고려하려면 같은 상태에서 행동의 상대적 품질을 추정해야 한다.

두 궤적 중 하나는 쉬운 시작 상태라 반환 100, 다른 하나는 어려운 시작 상태라 80을 얻었다면 100만 남기는 선택이 꼭 좋지 않다. 두 번째 정책이 어려운 상태에서 더 좋은 복구 행동을 보였을 수도 있다. 전체 궤적의 절대 반환 대신 상태별 기준을 뺀 advantage를 쓰려는 이유다.

상위 몇 퍼센트를 남길지도 자료의 질과 coverage에 따라 달라진다. 대부분의 궤적이 유용하고 소수만 센서 오류나 실패를 담았다면 상위 90%를 남겨 상태 다양성을 지킬 수 있다. 반대로 실패 궤적이 대부분이면 상위 10%가 더 깨끗한 행동 라벨을 줄 수 있지만, 그 작은 집합만으로는 드문 시작 상태를 다루기 어렵다. 또한 초반에 잘못 움직였다가 뒤에서 복구해 총반환이 높아진 궤적은 통째로 선택된다. **선택된 궤적의 모든 행동이 좋은 것은 아니다**라는 점이 다음 AWR의 전이별 가중치로 이어진다.

### 10.28 슬라이드 28 · AWR의 연속적 가중치

AWR은 각 기록 행동의 상대적 우수성을 가중치로 만든다. $R_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$는 시점 $t$ 이후의 기록 궤적 반환, $\widehat{V}(s_t)$는 그 반환에 회귀한 상태가치 추정치다. 둘의 차이 $\widehat{A}_t = R_t - \widehat{V}(s_t)$를 advantage 추정치라 한다. $\pi_\theta$는 학습할 정책, $\eta > 0$는 가중치의 온도다. 기록 전이 $(s_t, a_t) \sim \mathcal{D}$에 대해

$$\max_\theta E_{(s_t,a_t)\sim\mathcal{D}} \left[ e^{\widehat{A}_t/\eta} \log \pi_\theta(a_t \mid s_t) \right]$$

로 가중 BC를 한다. $\eta$가 작으면 높은 advantage에 가중치가 몰리고 커지면 거의 BC가 된다. 보통 폭주를 막기 위해 가중치를 clip한다. 강의 표기의 $e^{\beta \widehat{A}}$는 $\beta = 1/\eta$에 해당한다. AWR 논문

$\eta = 1$에서 advantage가 2인 행동의 가중치는 $e^2 \approx 7.39$, advantage가 -1인 행동은 $e^{-1} \approx 0.37$이다. 두 행동의 가중치 비가 약 20배가 된다. $\eta$가 작아지면 이 차이가 더 커져 소수의 표본만 학습을 지배할 수 있으므로 가중치 clipping이나 정규화가 필요하다.

### 10.29 슬라이드 29 · KL 제약에서 지수 가중치가 나오는 직관

한 상태에서 기록 행동의 분포를 $\beta(a \mid s)$, 개선할 분포를 $\pi(a \mid s)$라 하자. $Q(s,a)$는 행동가치, $V(s)$는 같은 상태의 기준 가치, $A(s,a) = Q(s,a) - V(s)$는 advantage다. $D_{\mathrm{KL}}(\pi \| \beta) = E_{a\sim\pi}[\log\{\pi(a \mid s)/\beta(a \mid s)\}]$는 새 분포가 기록 분포에서 벗어난 정도다. 이를 $\eta > 0$의 벌점으로 두고 $E_\pi Q - \eta D_{\mathrm{KL}}$를 최대화하면

$$\pi_{\mathrm{new}}(a \mid s) \propto \beta(a \mid s) \exp(Q(s,a)/\eta) = \beta(a \mid s) \exp(A(s,a)/\eta)$$

가 된다. $V(s)$는 같은 상태의 모든 행동에 공통이라 정규화 상수로 흡수된다. 그래서 데이터 행동을 advantage 지수로 재가중한 지도학습이 이 개선을 근사한다. 실제 AWR은 근사 가치와 제한된 함수 계열을 쓰므로 이 이상적인 식과 완전히 같지 않다.

같은 상태의 행동 A와 B가 기록 정책에서 각각 0.5 확률이고 Q가 4와 6이라면 $\eta = 1$의 이상적 개선 분포는 $e^4 : e^6 = 1 : e^2$ 비율이므로 B 쪽으로 기운다. 기존 확률 $\beta$가 0인 행동은 곱에 남지 않는다.

**분포를 크게 벗어나지 않으면서 기록 행동의 상대 가치로 재가중**한다는 식의 의미를 이 예에서 확인할 수 있다.

지수 가중치의 출처를 한 단계씩 계산해 보자. 고정 상태에서 $F(\pi) = \sum_a \pi_a Q_a - \eta \sum_a \pi_a \log(\pi_a/\beta_a)$를 최대화하고 $\sum_a \pi_a = 1$을 강제한다. $\pi_a$에 대한 정지 조건은 $Q_a - \eta[\log(\pi_a/\beta_a) + 1] + c = 0$이다. 이를 $\pi_a$에 대해 풀면 $\pi_a = \beta_a \exp(Q_a/\eta) \exp(c/\eta - 1)$이고, 마지막 공통인자를 정규화 상수로 흡수하면 위 식이 나온다. 상태마다 공통인 $V(s)$를 $Q$에서 빼도 행동 간 비율은 같아져 $e^{A/\eta}$를 사용할 수 있다.

**왜 KL의 방향을 $D_{\mathrm{KL}}(\pi\|\beta)$로 쓰는가?** 이 방향의 KL은 $\beta(a \mid s) = 0$인 행동에 새 정책이 양의 확률을 주면 무한대가 된다. 따라서 이상적인 해의 support가 기록 정책 안에 남는다. 반대 방향 $D_{\mathrm{KL}}(\beta\|\pi)$는 같은 최적화 해와 같은 support 제약을 주지 않는다.

## 10.30 슬라이드 30 · AWR의 장점과 경계

Actor는 **데이터에 있는 행동**의 로그확률로만 훈련하므로 critic에 임의 OOD 행동을 직접 묻지 않는다. 그러나 $R_t$는 수집 행동 정책을 따라 얻은 미래의 Monte Carlo 반환이어서 분산이 높고, AWR이 평가하는 advantage는 주로 행동 정책의 결과에 가깝다. 새로운 정책이 더 나은 미래 행동을 취할 때의 가치를 정확히 반영하기 어렵다. IQL은 데이터 행동만 평가하면서도 좋은 후속 행동의 가치를 부트스트랩하려 한다.

AWR의 actor는 기록 행동을 가중해 모방하지만 그 가중치의 근거인 Monte Carlo 반환은 수집 정책의 나머지 행동에 좌우된다. 현재 행동이 좋았어도 이후 수집 정책이 실패하면 낮은 가중치를 받을 수 있다. 정책 개선 뒤에는 더 나은 후속 행동을 할 수 있다는 가능성을 충분히 반영하지 못하는 한계다.

두 단계 예를 보자. 현재 행동 A는 보상 0을 주고 다음 상태로 가며, 그 상태의 데이터에는 보상 10인 좋은 행동과 보상 0인 나쁜 행동이 모두 있다. 수집 정책이 좋은 행동을 10%만 골랐다면 $\gamma = 1$에서 A의 수집 정책 기준 기대 반환은 $0.1 \cdot 10 + 0.9 \cdot 0 = 1$이다. 현재 행동 B는 즉시 2를 받고 종료한다고 하자. A를 실행한 자료의 90%는 반환 0을 기록하므로 AWR에서는 이 사례들이 낮게 가중된다. 드문 반환 10 사례에는 큰 지수 가중치가 붙으므로 A와 B의 **최종 가중치 순위는 온도와 표본 수에 따라 달라진다.** 그러나 다음 상태에서 좋은 기록 행동을 선택하며 이어 갈 수 있다면 A 다음의 반환은 10이 될 수 있다. AWR의 Monte Carlo 가중치와 개선 정책의 미래 가치가 다른 이유이며, IQL은 데이터 안에 있는 좋은 후속 행동을 가치 부트스트랩에 반영하려 한다. 좋은 행동의 데이터가 전혀 없다면 이 개선도 불가능하다.

## 10.31 슬라이드 31 · SARSA식 데이터 안 부트스트랩

일반 Q 목표의 $\max_{a'} Q(s', a')$는 미관측 행동을 고른다. 연속 데이터 궤적에 실제 다음 행동 $a'_{\mathcal{D}}$가 있다면 $y = r + \gamma Q_{\bar{\phi}}(s', a'_{\mathcal{D}})$로 바꿀 수 있다. 이는 데이터 밖 행동을 평가하지 않는 SARSA식 목표다. 그러나 다음 행동이 나쁜 수집 정책에서 왔다면 그 나쁜 행동의 가치까지 그대로 반영한다. 데이터 안에서 **좋은 행동 쪽으로** 개선할 장치가 필요하다.

SARSA식 목표에서 실제 다음 행동이 보상 0의 행동이라면, 데이터에 다른 좋은 행동이 있어도 그 전이의 목표는 낮게 계산된다. OOD 행동을 묻지 않는 장점과 기록 정책을 그대로 평가하는 한계가 함께 있다. IQL의 높은 expectile은 데이터 안에서 관측된 **상대적으로 좋은 행동의 미래 가치**를 대리한다.

## 10.32 슬라이드 32 · 데이터 행동 중 좋은 쪽을 고르기

이론적으로는 데이터 행동 분포 $\beta(a \mid s)$가 지지하는 행동 중 $\max Q(s, a)$를 택하고 싶다. 하지만 연속 상태에서 같은 $s$의 여러 행동을 충분히 관측하기 어렵고, 표본 최대값은 다시 오차에 민감하다. IQL은 데이터에서 관측한 $Q(s, a)$들의 상태별 **상위 방향**을 추정하는 상태 가치 $V(s)$를 학습한다. 그것이 명시적 OOD 행동 최대화의 대리 역할을 한다.

같은 상태에서 관측된 행동 Q가 $[1, 2, 9]$라면 평균 4와 최대 9 사이의 어디를 $V(s)$로 둘지 결정해야 한다. 무작정 표본 최대 9를 고르면 오류나 드문 이상치에 민감하다. Expectile은 높은 값을 더 강하게 맞추는 부드러운 회귀로 상위 방향의 가치를 얻는다. 단, 데이터에 좋은 행동이 전혀 없다면 높은 expectile도 좋은 정책을 만들어 내지는 못한다.

### 10.33 슬라이드 33 · Expectile 가치의 의미

한 상태 $s$에서 기록 행동 $a \sim \mathcal{D}(\cdot \mid s)$의 $Q(s, a)$를 확률변수로 보자. 일반 제곱오차 $\min_v E[(Q - v)^2]$의 최적값은 이 Q들의 평균이다. IQL은 상태가치 $V(s)$를 평균보다 높은 쪽에 놓기 위해 잔차 $u = Q(s, a) - V(s)$에 비대칭 가중치를 준다. $\mathbf{1}\{u < 0\}$는 $u < 0$이면 1, 아니면 0인 지시함수이고, $\tau \in (0, 1)$는 expectile 수준이다.

$$\mathcal{L}_V = E_{(s,a)\sim\mathcal{D}}\left[|\tau - \mathbf{1}\{u < 0\}|\, u^2\right]$$

를 최소화한다. $\tau > 0.5$이면 $V$보다 높은 Q에 더 큰 벌점을 주어 $V$가 데이터 행동 Q분포의 위쪽으로 올라간다. $\tau = 0.9$**가 정확한 상위 10% 분위수라는 뜻은 아니다.** Expectile은 가중 제곱오차의 해이고 quantile은 절댓값 기반의 다른 통계량이다. IQL 논문

실제로 $u \geq 0$일 때 가중치는 $\tau$, $u < 0$일 때는 $1 - \tau$다. 이 비대칭이 최적 $V(s)$를 어떻게 움직이는지는 아래 정지 조건에서 확인할 수 있다.

한 상태에서 $V(s) = v$ 하나만 최적화한다고 보면 손실을 $v$로 미분한 정지 조건은

$$\tau E[(Q - v)_+] = (1 - \tau)E[(v - Q)_+], \qquad (x)_+ = \max(x, 0)$$

이다. 왼쪽은 $v$보다 높은 Q를 **얼마나 놓쳤는가**, 오른쪽은 낮은 Q보다 $v$를 **얼마나 높게 잡았는가**의 균형이다. $\tau = 0.5$이면 두 편차의 균형점이 평균이 되고, $\tau > 0.5$이면 높은 Q를 놓친 비용이 커져 균형점이 위로 간다. 이 식은 상위 몇 퍼센트를 단순히 골라 평균내는 절차와 다르다.

$\tau = 0.9$**이면 행동의 상위 10%만 선택하거나 90th percentile을 구하는가?** 둘 다 아니다. Expectile은 높은 Q를 낮게 예측할 때 0.9, 낮은 Q를 높게 예측할 때 0.1로 **제곱오차**를 가중해 하나의 상태가치를 맞춘다. Quantile은 잔차의 크기에 선형인 pinball loss를 최소화한다. 같은 0과 10의 반반 분포에서 0.9 expectile은 9지만 0.9 quantile은 10이다. IQL의 $\tau$는 선택할 표본의 비율도 아니다.

### 10.34 슬라이드 34 · 비대칭 제곱손실 예

$\tau = 0.9$라면 $u > 0$, 즉 예측 $V$가 관측 Q보다 낮을 때 손실 가중치가 0.9다. $u < 0$이면 가중치는 0.1이다. 높은 Q값을 놓친 경우 더 크게 벌하므로 최적 $V$가 올라간다. 예를 들어 같은 상태 행동 Q가 0과 10을 같은 확률로 갖는다면 평균은 5지만 $\tau = 0.9$ expectile은 $0.1v^2 + 0.9(10 - v)^2$의 최소점 $v = 9$다. 계산은 $0.2v - 1.8(10 - v) = 0$에서 나온다.

위 정지 조건에 Q가 0과 10을 반반 갖는 분포를 대입하면 $\tau(10 - v) = (1 - \tau)v$가 되어 $v = 10\tau$다. $\tau = 0.5$에서는 평균 5, $\tau = 0.9$에서는 9가 된다. 같은 두 점 분포의 0.9 quantile은 정의 관례에 따라 10인 반면 expectile은 9다. $\tau$가 커질 때 어떻게 부드럽게 위쪽으로 움직이는지와 **quantile과 숫자가 다르다**는 점을 동시에 확인할 수 있다.

### 10.35 슬라이드 35 · IQL의 세 단계

첫째, 데이터 행동에 대한 $Q_{\bar{\phi}}(s, a)$로 상위 expectile $V_\psi(s)$를 맞춘다. 둘째, 실제 데이터 transition에 대해 $y = r + \gamma(1 - d)V_\psi(s')$로 $Q_\phi(s, a)$를 회귀한다. 셋째, $A(s, a) = Q_\phi(s, a) - V_\psi(s)$를 구해 $e^{\beta A}$를 가중치로 데이터 행동을 모방한다.

Critic과 actor의 학습 입력은 모두 데이터 안 행동이다. 그럼에도 높은 expectile이 "데이터에서 좋은 행동이 이어질 것"이라는 개선을 미래 가치에 반영한다. 데이터 지지 자체가 빈약하면 새 행동을 근거 없이 창조할 수는 없고, expectile $\tau$와 가중치 온도 $\beta$에 민감하다. IQL 논문

IQL의 Q 목표 $r + \gamma V(s')$에는 다음 상태의 임의 actor 행동이 들어가지 않는다. V는 데이터 행동 Q들에서 학습한 높은 expectile이고, actor도 기록 행동의 가중 BC로 뽑는다. 이 구조가 학습 중 OOD 행동의 Q를 직접 묻지 않게 한다. 다만 actor를 실제로 실행하면 새 상태에 갈 수 있으므로 배포 안전성까지 자동 보장되는 것은 아니다.

학습 순서에서도 차이가 있다. $V$와 $Q$의 손실은 기록된 $(s, a, r, s')$와 기록 행동의 Q만 사용하므로 actor $\pi_\theta$를 동시에 갱신할 필요가 없다. Critic을 충분히 학습한 뒤 $\exp\{\beta(Q(s,a) - V(s))\}$로 **기록 행동만** 가중해 actor를 추출할 수 있다. $\tau$는 가치가 데이터 안 Q의 어느 쪽에 가까울지, $\beta$는 추출 단계에서 좋은 행동에 얼마나 가중치를 몰아줄지를 조절한다. 두 계수의 효과를 한 가지 온도처럼 혼동하면 안 된다.

## 10.36 슬라이드 36 · 방법들의 공통 기준

TD3+BC와 CQL은 Bellman critic을 쓰되 정책 또는 가치에 제약을 둔다. Filtered BC와 AWR은 보상이 알려 주는 행동의 질을 모방 손실에 반영한다. IQL은 데이터 안 행동에 대한 가치 추정과 상위 expectile을 결합한다. 좋은 방법을 고를 때는 데이터의 품질·coverage, 연속/이산 행동, 새 상호작용 가능 여부를 함께 본다. 단일 benchmark 점수만으로 방법론을 고르면 안 된다.

방법을 선택할 때 데이터가 거의 전문가라면 BC의 단순함이 강력하고, 낮은 품질 궤적이 섞였지만 좋은 행동도 있다면 AWR·IQL의 가중이 유용할 수 있다. 데이터 support가 넓고 critic을 신뢰할 수 있다면 TD3+BC나 CQL의 가치 기반 개선을 검토할 수 있다. 이 판단은 이론적 가능성보다 실제 데이터 coverage와 독립 평가 결과에 달려 있다.

슬라이드의 IQL 비교 표는 이 차이를 구체화한다. 여러 locomotion 과제의 **표에 실린 점수 합계**에서는 10% BC가 666.2, IQL이 692.4로 단순한 방법도 경쟁력 있다. 그러나 antmaze 과제들의 합계에서는 10% BC가 134.2, IQL이 378.0이다. 미로 데이터는 목표까지 이어지는 완성 궤적만 모방하기보다 서로 다른 부분 경로의 좋은 행동을 이어 써야 할 수 있다. 이 숫자는 과제별 점수의 합계이므로 단일 환경의 성공률이나 모든 데이터셋에서의 보편적 우열로 읽으면 안 된다. IQL 원논문

## 10.37 슬라이드 37 · 고정 자료 학습의 응용

실제 응용에는 먼저 로그의 관측과 행동이 충분한지, 보상을 기록할 수 있는지, 최종 정책을 안전하게 검증할 수 있는지가 필요하다. 수집 시스템의 제약 때문에 실제 데이터는 누락, 센서 변경, 사람 개입, 정책 버전 혼합을 포함한다. 다음 사례들은 RL의 응용 가능성과 자료 재사용의 동기를 보여 주지만 **각 사례가 순수 offline RL 알고리즘으로 구현되었다는 뜻은 아니다.**

산업 로그에는 안전 관리자나 기존 제어기가 개입한 기록이 섞일 수 있다. 어떤 행동이 자동 제어의 선택이고 어느 행동이 사람의 긴급 조치인지 표시되지 않으면 행동 정책 $\beta$를 한 분포로 보기 어렵다. Offline 학습 전에 로그의 출처, 센서 변경 시점, 보상 누락을 정리해야 수식의 전제가 의미를 가진다.

## 10.38 슬라이드 38 · 로봇에서 offline 사전학습과 online 미세조정

여러 주방 과제의 기존 시연 자료로 정책이나 가치를 사전학습하면 새 목표 과제를 처음부터 탐색하지 않아도 된다. 그림의 흐름은 넓은 Bridge 자료로 사전학습하고, 새 과제 자료와 섞어 미세조정한 뒤, 실제 rollout을 더해 online 학습하는 단계다. 마지막 단계에서 새 데이터를 모으므로 전체 파이프라인은 **순수 offline**이 아니다.

혼합 비율이 너무 크면 새 과제를 충분히 반영하지 못하고, 너무 작으면 사전학습 지식이 사라질 수 있다. offline에서 online으로 넘어갈 때 critic의 분포 이동도 다시 점검해야 한다. 로봇 offline-to-online 연구

예를 들어 기존 시연으로 컵을 집는 정책을 만들고 새 물체의 시연 몇 개로 조정한 뒤 실제 로봇에서 추가 시행착오를 하면, 마지막 단계부터는 online 학습이다. “offline-to-online” 이라는 이름은 단계의 전환을 분명히 표현한다. 단계별 성능과 새 환경 step 수를 따로 보고해야 고정 자료의 기여를 알 수 있다.

### 10.39 슬라이드 39 · 핵융합 플라스마 제어

토카막의 자기 코일 전압을 조절해 뜨거운 플라스마의 모양과 위치를 유지하는 것은 고차원 고속 폐루프 제어 문제다. 그림은 시뮬레이션에서 학습한 제어 정책과 실제 TCV 장치의 센서·코일 사이의 연결을 보여 준다. 위험한 실험 장치이므로 시뮬레이터 보정과 안전 한계, 실기 검증이 필수다. 이것은 RL의 현실 제어 사례이며, 그 자체를 고정 로그만으로 훈련하는 offline RL의 전형으로 오해하지 않아야 한다. TCV 연구

플라스마 제어에서는 정책 행동이 자기 코일 전압이고, 다음 센서값에 플라스마 모양과 위치가 반영된다. 시뮬레이터에서 충분히 훈련할 수 있어도 현실 장치의 허용 전압과 안전 한계를 넘어서는 행동은 시험할 수 없다. 이 사례는 **현실의 순차 제어**를 보여 주지만 알고리즘을 고정 로그 기반 offline RL로 분류할 근거는 아니다.

### 10.40 슬라이드 40 · 건물 냉각 제어

건물 냉각에서는 실내 온도와 외부 날씨, 에너지 사용량이 상태가 되고 냉방 장치 설정이 행동이 될 수 있다. 현재 전기를 아끼려고 냉방을 지나치게 줄이면 나중에 쾌적성 위반이나 더 큰 에너지 비용이 생겨 장기 보상이 중요하다. 그림의 “실제 건물 → 보정된 에너지 모델 → 학습 → 배포” 과정은 모델 차이를 검증해야 한다는 점을 보여 준다. 안전 제약과 사람이 운영하는 제어기와의 비교가 중요하며, 특정 건물 그림만으로 알고리즘의 보편적 절감률을 주장할 수 없다. 실제 데이터센터 냉각 사례

건물 냉각에서 지금 냉방을 줄이면 전력은 바로 아끼지만 실내 온도는 뒤늦게 올라갈 수 있다. 냉방을 과도하게 켰을 때의 전기 비용과 쾌적성 위반을 같은 보상에 넣더라도 계절과 건물 사용량에 따라 균형이 달라진다. 배포 전에 실제 건물의 안전 범위와 기존 제어기 대비 성과를 별도로 검증해야 한다.

### 10.41 슬라이드 41 · 칩 회로 설계와 PrefixRL

PrefixRL에서는 회로 그래프가 상태이고, 연결을 추가·변형하는 선택이 행동이다. 합성 도구가 회로의 면적과 지연을 평가해 보상을 만든다. 두 목표 사이에 tradeoff가 있으므로 보상 가중치가 다른 설계점을 낳는다. 화면의 Q-network는 후보 연결의 장기 효과를 근사한다. 시뮬레이터나 합성 도구로 새 회로를 평가할 수 있다는 점에서 이 사례 또한 **offline 전용 방법**을 설명하는 것은 아니다. PrefixRL 논문

회로 설계에서 면적을 10% 줄이면서 지연이 20% 늘어난 후보와, 면적을 5% 줄이고 지연을 유지한 후보 중 어느 쪽이 좋은지는 제약에 따라 다르다. 합성 도구로 새 설계를 평가할 수 있으면 탐색은 계속될 수 있으므로 이 사례는 데이터 고정 여부보다 **보상 정의와 긴 설계 순서**를 설명할 때 적합하다.

### 10.42 슬라이드 42 · 전체 정리

고정 데이터에서 정책을 개선하려면 데이터 밖 행동의 가치를 과신하지 않아야 한다. TD3+BC는 행동을 기록 근처에, CQL은 가치 평가를 보수적으로, Filtered BC는 좋은 궤적에, AWR은 높은 advantage 행동에, IQL은 데이터 안 행동의 높은 expectile에 초점을 둔다.

어느 방법도 데이터가 전혀 보여 주지 않은 영역의 실제 결과를 마법처럼 알 수 없다. 성능을 판단할 때는 수집 정책의 범위, 학습 중 새 자료 유무, 독립 평가 절차를 함께 명시한다. 이 기준은 로봇·산업 제어뿐 아니라 어떤 로그 기반 의사결정 문제에도 적용된다.

마지막으로 새로운 고정 데이터 문제를 만나면 네 가지를 먼저 묻자. **누가 자료를 모았는가, 어떤 행동이 빠졌는가, 보상을 어떻게 측정했는가, 새 정책을 어떻게 독립 검증할 수 있는가?** 이 답이 있어야 BC와 가치 기반 방법의 차이, OOD 위험, 개선의 실질적 근거를 판단할 수 있다.

# 11 과제 가이드

## 11.1 과제 1 · 모방학습

이 과제는 전문가 시연에서 행동을 복제하고, 실제 rollout에서 성능을 측정하며, 정책이 방문한 상태를 전문가가 교정하는 DAgger로 확장한다. 구현의 중심은 **상태–행동 데이터가 어디서 오고 어떻게 다시 모이는가**다. 실험 결과에는 직접 얻은 수치와 실행 설정을 기록한다.

### 11.1.1 먼저 연결할 개념

- 행동 복제의 목적함수: 전문가 데이터에서 정책의 로그우도를 높인다.
- 오류 누적과 상태 분포 이동: 훈련 정확도와 rollout 성공률이 다를 수 있다.
- DAgger의 데이터 집계: 현재 정책이 방문한 상태에도 전문가 행동을 붙인다.

### 11.1.2 1. 환경 인터페이스를 먼저 확인하기

환경을 초기화하면 시작 관측이 나오고, 한 단계 진행 함수에는 행동을 넣어 다음 관측·보상·종료 여부를 받는다. 과제에서 묻는 이산 행동과 연속 행동의 차이는 정책 출력이 무엇을 뜻하는지 결정한다. 이산 행동은 후보 중 하나를 고르고, 연속 행동은 구간 안의 실수 벡터를 낸다. Ant와 다른 MuJoCo 보행 환경은 연속 관절 제어다.

한 rollout은 초기화 → 관측으로 행동 선택 → 환경 진행 → 전이 저장 → 종료까지 반복으로 만든다. 환경 API의 버전에 따라 종료와 시간 제한이 따로 반환될 수 있으므로, 스타터 코드가 실제로 어떤 형식을 기대하는지 먼저 확인한다. 관측·행동·보상 배열의 첫 차원은 시간이며, 하나의 궤적 안에서는 길이가 맞아야 한다.

**한 단계 진행에서 받은 종료 신호를 왜 따로 보관할까?** 실제 실패·성공으로 끝난 episode와 시간 제한 때문에 끊긴 rollout은 가치함수 학습에서 다르게 처리될 수 있다. 이 과제는 BC 중심이어도 평가 episode 수와 길이를 정확히 세려면 종료 원인을 기록하는 편이 좋다.

실제로 첫 episode를 수집할 때는 관측 $o_t$를 저장한 뒤 그 관측으로 고른 $a_t$를 같은 인덱스에 저장한다. 다음 단계에서 나온 $o_{t+1}$를 실수로 $a_t$와 짝지으면 정책은 한 단계 늦은 영상을 보고 행동하는 규칙을 배운다. 이 오류는 배열의 크기가 모두 맞아도 생기므로 몇 개의 전이를 시간 순서대로 직접 읽어 보는 편이 좋다.

### 11.1.3 2. 행동 복제의 데이터 흐름

전문가 시연 $N$개에서 $i$번째 상태 $s_i$와 전문가 행동 $a_i^*$를 쌍으로 추출한다. $\pi_\theta(a_i^* \mid s_i)$는 파라미터 $\theta$인 정책이 그 행동에 주는 확률 또는 밀도다. 행동 복제는 평균 음의 로그우도

$$\mathcal{L}_{\mathrm{BC}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log \pi_\theta(a_i^* \mid s_i)$$

를 최소화한다. 고정 분산 Gaussian 정책에서는 로그밀도의 $\theta$ 의존 부분이 평균 행동과 전문가 행동의 제곱거리이므로 제곱오차 최소화와 연결된다. 실제 실행에서는 정책이 자기 행동으로 훈련과 다른 상태를 만들 수 있어 rollout 반환도 평가한다.

구현은 공통 MLP 생성 → 정책의 행동·학습 함수 → 궤적 수집 유틸리티 → replay buffer → BC 에이전트 → trainer 순서로 연결하면 의존성을 따라 검증하기 쉽다. $B$를 한 batch의 시연 수, $d_s$와 $d_a$를 상태·행동 벡터 차원이라 하자. 관측 batch의 모양이 $B \times d_s$라면 정책 평균은 $B \times d_a$이어야 하고, 행동과 전문가 목표의 단위·범위가 같아야 한다.

### 11.1.4 점검할 연결

1. 환경에서 얻은 관측이 정책 입력과 같은 차원인가?
2. 정책에서 나온 행동이 환경의 행동 범위를 만족하는가?
3. replay buffer가 상태와 그때의 전문가 행동을 같은 인덱스로 보관하는가?
4. 평가할 때 탐색·훈련용 잡음을 실수로 더하지 않는가?
5. 훈련·평가의 반환을 같은 종료 조건으로 계산하는가?

연속 행동의 작은 예를 보자. 시연에서 팔의 목표 위치가 $a^{*}=0.6$인데 정책 평균이 $\mu_\theta(s)=0.2$라면 오차는 $-0.4$다. 제곱오차를 줄이면 평균은 0.6 쪽으로 이동한다. 다만 환경의 허용 범위가 $[-1,1]$인지 $[0,1]$인지에 따라 같은 출력의 물리적 의미가 달라진다. 데이터의 행동 정규화와 환경에 넘기는 행동 복원 과정을 반드시 한 쌍으로 확인한다.

**훈련 손실이 낮은데 행동이 늘 경계값에 붙는다면?** 정책 출력의 tanh와 환경의 clip을 중복 적용했거나, 정규화된 전문가 행동과 원래 단위의 정책 출력을 비교했을 수 있다. 평균 행동과 실제 환경에 전달된 행동을 각각 기록하면 어디서 포화되는지 찾을 수 있다.

### 11.1.5 3. BC 실험과 숫자의 의미

첫 실험은 Ant와 선택한 보행 환경 하나에서 BC 정책을 평가한다. 보고할 값은 각 정책 rollout 반환의 평균과 표준편차다. $K$번의 episode 반환을 $R_1,\dots,R_K$라 하면 $\bar{R}=K^{-1}\sum_k R_k$이고 표본 표준편차는 $\sqrt{\sum_k (R_k-\bar{R})^2/(K-1)}$다. 한 episode만 수집해 놓고 표준편차를 보고할 수는 없다.

평가 표본 예산이 episode 길이보다 커야 여러 rollout을 얻는다. 예컨대 최대 길이가 1000단계이고 평가 예산이 5000단계이면 대략 다섯 episode를 얻는다. 표에는 환경, 전문가 성과, BC 평균±표준편차, 평가 episode 수, 훈련 데이터 양, 네트워크 크기와 업데이트 수를 함께 적으면 비교가 공정해진다.

실행 영상을 한 장면으로 확인하는 이유는 반환 숫자가 포착하지 못하는 이상 행동을 찾기 위해서다. 로봇이 제자리에서 높은 보상을 받거나 초기 몇 단계만 걷고 넘어지는 모습을 보면 보상 함수와 종료 조건을 다시 볼 수 있다.

계산 예로 세 episode의 반환이 90, 100, 110이면 평균은 100이다. $s_R$을 이 반환들의 표본 표준편차라 쓰면 앞서 정의한 $K-1$ 분모의 식에서

$$s_R=\sqrt{\frac{(90-100)^2+(100-100)^2+(110-100)^2}{3-1}}=10.$$

`100 ± 10`이라고만 쓰면 세 번의 실행 결과인지 여러 시드 평균인지 알기 어려우므로 episode 수와 시드 수도 함께 적는다. 전문가와 BC를 비교할 때 평가 환경의 초기 상태 범위를 같게 맞추는 것도 중요하다.

### 11.1.6 4. 하이퍼파라미터 실험

하나의 요인에 대해 기본 설정을 포함한 **네 가지 이상의 값**을 비교한다. 예를 들어 시연량을 바꾸면 다른 조건인 네트워크 크기·훈련 단계·평가 episode를 고정한다. 여러 조건을 동시에 바꾸면 어느 요인이 성능 차이를 만들었는지 알기 어렵다.

가로축은 바꾼 변수, 세로축은 평가 반환의 평균으로 그리고 가능하면 표준편차나 여러 시드 결과를 표시한다. 시연량을 늘렸는데 성과가 멈춘다면 모델 용량보다 시연의 다양성이나 상태 분포 이동이 병목일 수 있다. 학습 단계를 늘렸는데 훈련 손실만 내려간다면 과적합 또는 배포 분포 차이를 의심할 수 있다.

예를 들어 시연 수를 100, 250, 500, 1000개로 늘리는 실험에서는 나머지 학습 설정을 고정해야 데이터 양의 효과를 읽을 수 있다. 대신 데이터 100개 조건에 업데이트 1000번, 데이터 1000개 조건에 업데이트 100번을 주면 두 조건은 데이터 양뿐 아니라 각 표본을 본 횟수도 달라진다. **환경 단계 수, gradient update 수, 전문가 질의 수**를 서로 다른 예산으로 기록하자.

### 11.1.7 5. DAgger의 반복

BC가 만든 초기 정책으로 환경을 실행해 현재 정책이 방문한 상태를 모은다. 반복 번호를 $k$, 현재 정책을 $\pi_k$, 그 방문 상태의 분포를 $d_{\pi_k}$라 쓰자. 전문가 정책 $\pi^*$에 상태 $s$를 입력해 교정 행동 $\pi^*(s)$를 얻고, 기존 시연 집합 $\mathcal{D}_k$에 합친다.

$$\mathcal{D}_{k+1} = \mathcal{D}_k \cup \{(s, \pi^*(s)) : s \sim d_{\pi_k}\}.$$

구현할 재라벨링 함수는 **정책이 방문한 상태**를 전문가에게 물어야 한다. 기존 전문가 데이터의 행동을 다시 읽어 붙이는 것만으로는 DAgger의 핵심인 분포 이동 교정이 생기지 않는다. 각 반복에서 추가된 시연 수와 정책 평가 반환을 함께 기록하자.

**DAgger 곡선이 매 반복 항상 올라가야 할까?** 그렇지 않다. 유한한 시연, 정책 최적화 오차, 환경의 무작위성 때문에 평가 반환은 출렁일 수 있다. 여러 반복과 가능한 경우 여러 시드의 경향을 보며 BC와 같은 조건에서 비교한다.

장난감 경로에서 BC가 중앙의 $s_0$만 보았고 실제 실행 중 왼편으로 치우친 $s_1$을 방문했다고 하자. $s_1$에서 정책 행동을 그대로 라벨로 저장하면 틀린 조향을 다시 학습할 수 있다. DAgger는 그때 전문가에게 $\pi^*(s_1)$을 물어 **복구 행동**을 붙인다. 데이터 집계 후 같은 실패가 줄어드는지 rollout에서 확인해야 한다.

**전문가 라벨을 매 단계 받을 수 없으면?** 질의 비용이 크면 위험하거나 불확실한 상태를 우선 선택해 사람에게 묻는 방식을 고려할 수 있다. 다만 그 경우 모든 방문 상태에 전문가 라벨을 붙이는 표준 DAgger와 데이터 분포가 달라지므로, 질의 기준과 횟수를 실험에 기록해야 한다.

### 11.1.8 6. BC와 DAgger 비교

두 환경에서 DAgger 반복 횟수에 따른 평균 반환 곡선을 그린다. 비교 기준선으로 BC와 전문가의 성과를 함께 표시하면 분포 이동 교정의 효과를 해석하기 쉽다. DAgger가 개선되어도 전문가까지 닿지 못한다면 전문가 라벨의 부족, 현재 정책이 방문한 상태의 다양성, 정책 표현력이나 최적화 설정을 살핀다.

곡선의 가로축이 반복 횟수이면 각 반복에서 추가한 데이터 양도 설명해야 한다. 반복 수가 같은 두 실험이라도 한 번에 수집한 상태가 다르면 사용한 전문가 피드백 예산이 다르기 때문이다.

### 11.1.9 결과를 정리하는 형식

| 항목 | 기록할 내용 | 해석할 질문 |
|---|---|---|
| BC 성능 | 환경별 평균 반환과 표준편차 | 시연 행동을 실제 실행에서 재현했나? |
| 행동 영상 | 평가 rollout의 대표 장면 | 어느 단계에서 정책이 이탈하나? |
| 변수 실험 | 네 가지 이상 설정과 동일한 평가 조건 | 데이터·모델·업데이트 중 무엇이 병목인가? |
| DAgger 곡선 | 두 과업의 반복별 평균 반환 | 교정 데이터가 분포 이동을 줄였나? |
| 비교 설명 | 전문가·BC·DAgger, 데이터 예산 | 차이를 한 요인으로 과장하지 않았나? |

보고서에는 실제 실행한 설정과 결과를 쓴다. 제출 파일은 보고서 PDF와 필요한 코드·노트북을 하나의 압축 파일에 넣는 형식이며, 원본 지침에서는 크기 제한과 영상·TensorBoard 로그 제외를 명시한다. 최종 제출 전에는 현재 공지의 파일명·용량 기준을 다시 확인한다.

## 11.2 과제 2 · 정책경사와 PPO

이 과제는 하나의 정책경사 구현을 단계별로 개선한다. 전체 궤적 반환에서 reward-to-go로 옮기고, 상태 가치 baseline을 학습한 뒤, GAE로 advantage를 추정하고, PPO-Clip으로 같은 rollout을 여러 번 조심스럽게 사용한다. 모든 비교에서 **평가 반환과 환경 상호작용 단계 수**를 함께 본다.

### 11.2.1 연결되는 개념

- 로그미분을 이용한 정책경사: 환경 전이를 미분하지 않고 기대 반환의 기울기를 구한다.
- Reward-to-go와 baseline: 과거 보상을 빼고 상태 평균 성과를 기준으로 삼는다.
- GAE: 여러 길이의 TD 추정량을 결합한다.
- PPO의 정책비: 이전 정책에서 모은 데이터를 여러 번 사용할 때 갱신을 제한한다.

### 11.2.2 1. 전체 반환과 reward-to-go

정책 파라미터를 $\theta$, 정책이 만든 궤적을 $\tau = (s_0, a_0, \dots, s_T)$라 하자. $T$는 행동 횟수, $r_k$는 시점 $k$의 보상, $\gamma \in [0,1]$는 할인율이다. 궤적 전체의 반환을 $R(\tau)$라 쓰고, 시점 $t$부터의 reward-to-go는 $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$다. $N$개의 현재 정책 궤적에서 $i$번째 궤적의 반환을 $R_i$라 할 때, 각 행동의 로그확률 기울기 $\nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i)$를 어느 성과로 가중하느냐에 따라 두 추정량이 갈린다. $\widehat{g}$는 표본 정책경사 추정치다.

$$\widehat{g}_{\text{full}} = \frac{1}{N} \sum_i \sum_t \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) R_i, \qquad \widehat{g}_{\text{rtg}} = \frac{1}{N} \sum_i \sum_t \nabla_\theta \log \pi_\theta(a_t^i \mid s_t^i) G_t^i.$$

현재 행동은 과거 보상을 바꿀 수 없으므로 reward-to-go는 불필요한 잡음을 덜 포함한다. 위 표본식은 시점별 가중치를 $G_t$로 쓰는 구현 관례를 나타낸다. 초기 상태 기준 목적을 정확히 $J(\theta) = E[\sum_{t=0}^{T-1} \gamma^t r_t]$로 두고 $G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_k$를 쓴다면, 그 목적의 정책경사에는 각 $t$ 항 바깥에 $\gamma^t$가 한 번 더 붙는다. $\gamma^t G_t = \sum_{k=t}^{T-1} \gamma^k r_k$이기 때문이다. 스타터 코드의 목적과 할인 관례를 먼저 확인하고 시간 인덱스를 일치시켜야 한다.

세 단계의 보상이 $[2, -1, 4]$이고 $\gamma = 1$이면 전체 반환은 모든 시점에서 5로 쓰이지만 reward-to-go는 차례로 $[5, 3, 4]$다. 두 번째 행동에 첫 보상 2를 붙이지 않는 차이가 드러난다. 계산한 배열을 짧은 손계산과 맞춰 본 뒤 긴 rollout에 적용하면 시간 인덱스 오류를 줄일 수 있다.

### 11.2.3 2. CartPole 비교 실험

이산 행동 CartPole에서 전체 반환과 reward-to-go, advantage 정규화의 유무를 비교한다. 작은 batch와 큰 batch의 학습 곡선을 **각각** 그리면 표본량이 분산을 얼마나 줄이는지 볼 수 있다. 과제의 두 batch 설정은 1000과 4000 환경 단계다.

정규화는 한 batch의 advantage에서 평균을 빼고 표준편차로 나누는 연산이다. 값의 상대적 순서는 유지하면서 업데이트의 크기를 바꾸지만, 모든 환경에서 항상 이득이라는 보장은 없다. 비교할 때 학습률, 네트워크 구조와 평가 방법은 고정하고, 보고서에는 실제 실행한 명령 옵션을 남긴다.

모든 학습 곡선의 가로축은 **정책 업데이트 횟수보다 누적 환경 단계 수**다. batch가 큰 방법은 한 번 업데이트 전에 더 많은 상호작용을 사용하므로 업데이트 횟수만 비교하면 표본 효율을 오해하기 쉽다.

**큰 batch의 곡선이 매끄러우면 알고리즘이 더 좋은 것일까?** 큰 batch는 기울기 표본 분산을 줄일 수 있지만 업데이트당 환경 비용이 더 든다. 같은 환경 단계 수에서의 최종 성과와 여러 시드의 변동을 비교해야 공정하다.

예를 들어 1000단계 batch로 20번 업데이트한 방법은 2만 환경 단계를 썼고, 4000단계 batch로 20번 업데이트한 방법은 8만 단계를 썼다. 같은 "20번 업데이트"가 같은 데이터 예산이 아닌 셈이다. 반환 그래프의 가로축을 환경 단계로 맞춘 뒤, 곡선의 흔들림과 목표 점수 도달 시간을 함께 해석해야 한다.

### 11.2.4 3. 상태 가치 baseline

파라미터 $\phi$인 가치망 $V_\phi(s_t)$는 상태 $s_t$에서 현재 정책을 계속 따를 때의 기대 미래 반환을 추정한다. 정책 업데이트에는 추정 advantage $A_t = G_t - V_\phi(s_t)$를 사용한다. 그 상태에서 평소보다 좋은 결과가 나온 행동은 양의 advantage, 나쁜 결과가 나온 행동은 음의 advantage를 받는다.

행동에 의존하지 않는 baseline의 기대 기여는

$$E_{a\sim\pi_\theta(\cdot|s)}[\nabla_\theta \log \pi_\theta(a \mid s) V_\phi(s)] = V_\phi(s) \nabla_\theta \sum_a \pi_\theta(a \mid s) = 0$$

이므로 정확한 기대 정책경사는 바뀌지 않는다. 다만 학습한 가치함수는 근사이고, 실제 구현에서 데이터 재사용 · 정규화 · 학습률이 분산과 편향에 영향을 줄 수 있다. 가치함수 손실의 하락과 정책 반환의 상승을 함께 봐야 한다.

상태 $s$의 보통 미래 반환이 12인데 한 행동 뒤 실제 반환이 10이면 advantage는 -2다. 보상이 양수여도 그 상태에서는 평균보다 나빴다는 뜻이다. 가치망이 $V_\phi(s) = 20$으로 크게 잘못 예측했다면 advantage는 -10이 되어 actor 갱신도 과장된다. 정책 손실과 가치 손실을 따로 관찰해야 하는 이유다.

### 11.2.5 4. HalfCheetah에서 baseline 평가

연속 제어 HalfCheetah에서는 reward-to-go 정책경사와 가치 baseline을 붙인 버전을 비교한다. 원본 실험 설정은 할인율 0.95, batch 5000, baseline 학습 단계 수와 학습률을 명시한다. baseline이 더 정확하면 advantage의 잡음이 줄 수 있지만, baseline을 지나치게 적게 또는 과하게 학습하면 정책 갱신이 흔들릴 수 있다.

보고서에는 baseline 손실 곡선과 평가 반환 곡선을 함께 그린다. baseline gradient step 수나 학습률을 줄인 추가 실험에서는 다른 설정을 고정해야 원인을 비교할 수 있다. 손실이 내려갔다고 성과가 반드시 오르지는 않으므로 두 그래프를 나란히 해석한다.

### 11.2.6 5. GAE의 계산

한 단계 TD 잔차를 $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$라고 두자. $\lambda \in [0,1]$는 긴 반환의 반영 정도, $T$는 이번 rollout이 끝나는 시점이다. $\widehat{A}_t$는 시점 $t$의 GAE 추정치이고 경계에서는 $\widehat{A}_T = 0$으로 둔다. 여러 길이의 TD 잔차를 합치면

$$\widehat{A}_t^{\mathrm{GAE}(\gamma,\lambda)} = \sum_{l=0}^{T-t-1} (\gamma\lambda)^l \delta_{t+l} = \delta_t + \gamma\lambda \widehat{A}_{t+1}.$$

마지막 등식은 뒤에서 앞으로 반복 계산할 수 있게 한다. 진짜 종료라면 다음 상태 가치를 0으로 처리하고, 시간 제한으로 잘린 궤적이라면 부트스트랩 가능 여부를 환경 계약에 맞춰 결정한다. $\lambda = 0$은 한 단계 TD에 가깝고 $\lambda = 1$은 적절한 끝 처리에서 긴 보상열에 가까워진다. 두 극단은 가치 추정 오차가 주는 편향과 표본 반환의 변동 사이를 비교하는 기준이다.

두 TD 잔차가 $\delta_t = 2$, $\delta_{t+1} = -1$이고 $\gamma = 0.9$, $\lambda = 0.95$라면 뒤에서부터 계산한 $\widehat{A}_t = 2 + 0.9 \cdot 0.95 \cdot (-1) = 1.145$다. 시간 제한 없이 진짜 종료된 뒤의 잔차는 이어 붙이지 않는다. 먼저 2~3단계짜리 인공 rollout로 이 계산을 재현하면 실제 환경의 긴 배열에서 생기는 off-by-one 오류를 찾기 쉽다.

### 11.2.7 6. HumanoidStandup의 $\lambda$ 비교

HumanoidStandup에서는 $\lambda = 0, 0.95, 1$을 같은 환경 · 네트워크 · batch · 학습률로 비교한다. 과제의 명시적인 과업 길이와 할인율을 유지해야 곡선의 차이를 $\lambda$에 연결할 수 있다. 한 그래프에 평가 반환과 누적 환경 단계를 표시하고, 어떤 값이 더 빨리 좋아졌으며 최종 성과와 변동이 어떠했는지 설명한다.

단일 실행에서 특정 $\lambda$가 더 좋아 보인다고 일반적인 우위를 주장할 수는 없다. 가치함수의 품질, 보상의 지연, 에피소드 길이에 따라 유리한 편향–분산 절충이 달라진다.

**$\lambda = 1$이면 가치함수를 전혀 쓰지 않나?** 완전한 episode의 끝까지 모두 관측하고 끝에서 부트스트랩하지 않으면 GAE의 망원합이 Monte Carlo 반환에서 현재 가치 추정치를 뺀 형태가 된다. 궤적이 중간에 잘리면 끝의 가치 추정이 남을 수 있다. 종료 처리까지 함께 봐야 한다.

### 11.2.8 7. PPO-Clip의 목적함수

수집 때의 정책을 $\pi_{\text{old}}$로 고정하고 현재 정책 $\pi_\theta$와의 행동 확률비를 $\rho_t(\theta) = \pi_\theta(a_t \mid s_t)/\pi_{\text{old}}(a_t \mid s_t)$라 하자. 이는 보상 $r_t$와 다른 양이다. $\widehat{A}_t$는 수집한 rollout에서 계산해 고정한 advantage 추정치, $E_t$는 그 rollout의 시점 표본 평균이다. $\epsilon > 0$은 허용할 확률비 범위의 반폭이고 $\text{clip}(x, l, u)$는 $x$를 $[l, u]$ 안으로 자른다. PPO-Clip은 다음 surrogate 목적을 최대화한다.

$$L^{\text{clip}}(\theta) = E_t\Big[\min\Big(\rho_t(\theta)\widehat{A}_t,\ \text{clip}(\rho_t(\theta), 1-\epsilon, 1+\epsilon)\widehat{A}_t\Big)\Big].$$

$\widehat{A}_t > 0$이면 그 행동의 확률을 과도하게 높이는 이득을 제한하고, $\widehat{A}_t < 0$이면 확률을 과도하게 낮추는 이득을 제한한다. 구현에서는 rollout 때의 이전 로그확률을 보존하고 현재 로그확률과의 차이를 지수화해 비를 계산하는 편이 안정적이다. clip은 엄격한 KL 신뢰 영역 보장이 아니므로 실제 정책 변화도 진단해야 한다.

수치로 clipping을 확인하자. $\widehat{A} = 2$, $\rho = 1.3$, $\epsilon = 0.2$라면 원래 항은 2.6, clipped 항은 2.4이므로 목적은 2.4를 쓴다. $\widehat{A} = -2$, $\rho = 0.5$라면 두 항은 -1과 -1.6이므로 -1.6을 쓴다. 두 부호에서 모두 `min`을 적용해야 한다. 구현 결과를 이 작은 예와 비교하면 부호 실수를 빨리 찾을 수 있다.

**clipping을 적용했는데도 KL이 커질 수 있나?** 그렇다. 목적함수는 관측된 표본의 유리한 추가 이득을 제한할 뿐 모든 상태의 정책 거리를 강제로 묶지 않는다. 학습 로그에 평균 KL과 clip fraction을 함께 기록하고, 너무 큰 이동이 반복되면 epoch 수나 학습률을 조정해야 한다.

### 11.2.9 8. Reacher에서 여러 번의 갱신

Reacher 과제는 GAE와 baseline을 공통으로 쓰는 정책경사 버전과 PPO 버전을 비교한다. PPO는 같은 on-policy batch를 여러 epoch와 minibatch에 걸쳐 사용한다. 원본 설정은 PPO 4 epoch, 4 minibatch를 사용한다. 데이터를 여러 번 보되 확률비와 clipping으로 정책 변화가 지나치게 커졌을 때의 이득을 제한하려는 것이다.

clip 없이 단순 정책경사 손실을 같은 batch에서 여러 번 최적화하면 첫 갱신 뒤 데이터가 이미 이전 정책의 데이터가 된다. 이후 갱신은 현재 정책 분포를 반영하지 못하고 과도한 확률 변화로 불안정해질 수 있다. 실험 결과를 설명할 때 단지 PPO의 점수뿐 아니라 같은 환경 단계 수에서의 곡선과 변동을 함께 비교한다.

### 11.2.10 구현 점검표

| 구성 | 확인할 항목 | 흔한 오류 |
|---|---|---|
| 궤적 수집 | 상태·행동·보상·종료 길이의 정렬 | 마지막 상태를 행동 배열과 같은 길이로 취급 |
| 반환 계산 | 전체 반환과 reward-to-go의 시간 인덱스 | 과거 보상이나 할인 지수를 잘못 포함 |
| 가치함수 | 예측과 목표의 batch 차원 | 정책 손실로 가치망이 우연히 역전파 |
| GAE | 역방향 재귀와 종료 마스크 | 진짜 종료 후 다음 가치를 더함 |
| PPO | 이전 로그확률의 고정, 비와 clip | 업데이트 중 이전 정책 값을 새로 계산 |
| 평가 | 누적 환경 단계와 여러 episode | 업데이트 횟수만 가로축으로 사용 |

보고서에는 CartPole의 두 batch 그래프, HalfCheetah의 baseline 손실과 반환, HumanoidStandup의 $\lambda$ 비교, Reacher의 PPO 비교와 각 질문의 해석을 담는다. 직접 실행한 명령과 설정을 기록하고, 실제 결과를 사용한다. 제출 묶음은 보고서 PDF와 필요한 코드로 구성하며 원본 지침은 용량 제한과 영상·TensorBoard 로그 제외를 명시한다. 최종 형식은 현재 공지를 기준으로 확인한다.